

Java Spring

Le socle technique des applications JEE

Ce livre apporte les éléments clés pour se repérer dans les différentes technologies utilisées dans les projets basés sur **Spring**. Il prend en compte les différences de configurations liées aux versions de **Spring (en version 4.2 au moment de l'écriture)** et se base sur des **exemples concrets d'utilisation**. Il permet au lecteur d'être très rapidement autonome sur un projet d'entreprise qui utilise Spring, que ce soit au début d'un **nouveau projet** ou pour **maintenir un projet existant** : compréhension du noyau, accès aux données, maîtrise de la couche web. Des connaissances sur le développement Java et notamment le développement d'applications web sont un prérequis indispensable pour tirer le meilleur parti possible du livre.

L'auteur présente tout d'abord les éléments simples et courants de Spring (la **configuration**, les **contextes**, les **librairies tiers**) et explique ensuite certains aspects plus complexes que l'on rencontre souvent dans les projets (**Ressources**, **Binders**, **Validateurs**, **Convertisseurs** et **Tests**). La **programmation par aspects** est expérimentée, les applications web Spring **MVC** et les **Web Services** sont présentés en détail avec les **tests unitaires associés**. L'auteur termine par une présentation de la partie Spring d'un projet généré à partir de **jHipster** afin d'expliquer une mise en œuvre très actuelle.

Tout au long des chapitres, l'auteur s'appuie sur des **exemples fonctionnels** afin de permettre l'expérimentation au plus tôt par le lecteur. Dans ce but, des éléments sont en téléchargement sur cette page.

Les chapitres du livre :

Préambule – Avant-propos – Éléments constitutifs du framework – Spring et les design patterns – Rappels sur les éléments externes à Spring – Le conteneur Spring – Configuration avancée – Les tests et Spring – Partie back de l'application – Spring dans un contexte web JSP – Intégration JSF2 – Spring en profondeur

Hervé LE MORVAN

Hervé LE MORVAN est aujourd'hui consultant DevOps pour de grands comptes de la Banque, de l'Assurance et des Telecoms. Durant ses vingt années d'expérience, il est principalement intervenu en tant que référent technique et formateur dans des équipes de Recherche et Développement ou d'Architectes et en tant que support au développement d'applications sur des missions de migration ou de modernisation de Systèmes d'Informations. À ce titre, il est intervenu dans de nombreux projets de migrations utilisant le socle Spring et connaît parfaitement les attentes des équipes dans ce domaine. C'est toute cette expérience qu'il partage volontiers tout au long des pages de ce livre.

Préambule

Lors de mes missions de consultant, j'ai souvent trouvé fastidieuses les phases d'acquisition de nouvelles technologies.

Mes missions ont le plus souvent consisté à défricher un domaine pour le mettre ensuite en place sur des projets concrets et former nos collaborateurs.

J'utilise généralement pour monter en compétence les livres, notamment ceux de la collection ENI et les sites Internet des éditeurs de logiciels ainsi que des blogs spécialisés.

Les livres m'apportent une vue globale et les sites des éclaircissements sur des points particuliers.

Pour le monde Spring, durant mes missions, j'ai eu la grande chance de recevoir une aide fondamentale d'experts Spring et Jee comme Julien Dubois, Bernard Pons, Antonio Goncalves, Florent Ramière, Nicolas Romanetti, ainsi que de beaucoup d'autres, ce qui m'a permis de mettre en place des projets complexes.

Je voulais leur rendre hommage, les remercier, car aujourd'hui encore ils m'apportent un éclairage bienveillant et m'aident à traverser les obstacles.

Les technologies Java ont atteint un tel degré de maturité que le challenge consiste surtout à assembler les bonnes briques techniques correspondant à un besoin précis et l'expérimentation et les tests de montée en charge restent le seul moyen de vérifier que l'application tient la route.

Paradoxalement, bien que le socle applicatif soit souvent complexe, l'architecture et les procédures pour construire l'application doivent pouvoir être acquises rapidement par les développeurs.

Il faut donc sans cesse ajuster la complexité pour garder les applications maintenables.

Dans le monde du Web, nous pouvons nous appuyer sur la partie back qui est relativement bien stabilisée et qui devient générique afin de pouvoir focaliser l'effort sur la partie front qui évolue encore en intégrant toujours plus de réactivité et de complexité notamment par l'utilisation de technologies basées sur le JavaScript.

La capitalisation de l'expérience a été cristallisée à travers la mise à disposition de générateurs de code comme jHipster (<https://jhipster.github.io/>) ou Celerio (www.jaxio.com/celerio.html) qui sont des outils qui structurent et harmonisent les projets. Ces outils font gagner un temps considérable lors des phases de démarrage des projets. En effet, moyennant un paramétrage relativement modeste, ils construisent un squelette applicatif complet qui permet d'avoir, très rapidement, une application fournissant à travers des IHM (interfaces homme-machine) complètes un accès aux données sous la forme de SCRUD (pour *Search, Create, Read, Update, Delete*). Il suffit alors d'adapter les applications générées pour regrouper des écrans et personnaliser le code. En contrepartie, ces outils demandent une adaptation très rapide des collaborateurs qui doivent alors remanier le code généré pour intégrer les préoccupations métiers dans un socle complexe.

Cette phase qui consiste à remanier les IHM et à ajouter le code métier dans une couche de service correspond à la phase où il y a le plus de valeur ajoutée dans un projet.

J'ai écrit ce livre avec comme ambition d'apporter des éléments pour aider nos collaborateurs à monter en compétence afin d'acquérir plus rapidement une autonomie lors de leurs interventions sur des applications pour lesquelles le socle technique est préexistant.

L'ossature de l'application est souvent mise à la disposition des équipes projets pour les applications complexes et il y a, en général, une équipe d'architectes qui met à disposition le socle technique et en assure le support.

Introduction

Les applications Java d'aujourd'hui sont de plus en plus complexes et de mieux en mieux structurées. On retrouve des applications web, des applications autonomes avec ou sans IHM et des applications mobiles. Spring est aujourd'hui principalement utilisé pour les applications web et le batch, ce dernier étant, comme vous le savez sûrement déjà, un traitement de masses ou de lots sans IHM déclenché périodiquement sur un système.

Pour les applications mobiles, nous avons souvent la partie back qui est basée sur Spring. Ce framework trouve également progressivement son chemin dans la partie front.

Ces applications sont modulaires, robustes et leurs composants sont hautement réutilisables. Vous remarquerez lors de vos interventions que l'on retrouve souvent dans une même application un empilement de briques techno-fonctionnelles qui ont été conçues à différentes époques. Cet ensemble apporte satisfaction et ne nécessite pas un refactoring qui induirait des tests de non-régression. Nous nous contentons en général de refactorer uniquement les parties de code qui se basent sur des API (*Application Programming Interface* ou interface de programmation en français) obsolètes qui ne sont plus disponibles quand nous changeons de versions les éléments du socle.

Afin de comprendre le fonctionnement de ces applications, il est donc important de connaître à la fois les façons « modernes » de programmer ces applications, mais il faut également comprendre les différents styles d'écriture et de configuration correspondants aux différentes versions de Spring car ils coexistent au sein d'un même groupe d'applications et même parfois au sein d'une même application. Vous retrouverez souvent différents niveaux et différents styles d'API dans une même application. C'est pourquoi ce livre ne se limite donc pas à la dernière génération de style utilisée actuellement, mais présente aussi les plus anciens. L'idée est de garder la configuration des applications cohérente et de n'utiliser les dernières innovations stables que pour les nouvelles applications.

Spring est très vaste, ses contributeurs ont fait un travail remarquable et il s'inscrit dans la durée. Il n'est donc pas possible de couvrir l'intégralité des possibilités offertes par ce framework dans un seul livre. Dans sa conception, Spring respecte un ensemble de bonnes pratiques qui rendront aisée l'utilisation des modules qui ne sont pas décrits dans ce livre.

Son objectif est de donner les éléments pour être autonome sur la création ou la maintenance d'une application basée sur Spring par la compréhension du noyau, l'étude de l'accès aux données et d'apporter au lecteur de ce livre la maîtrise de la couche web. Cet ouvrage n'a pas pour ambition de se substituer à la documentation officielle Spring. Elle est en effet exhaustive, mais en contrepartie elle est très volumineuse et il n'est pas facile d'y distinguer ce qu'il faut connaître de ce qu'il faut savoir, et nous ne disposons malheureusement que rarement du temps qu'il faudrait pour lire cette documentation entièrement. Ce livre s'inspire de cette documentation, des problématiques que j'ai rencontrées lors de mes missions et des blogs que j'utilise pour faire de la veille technologique. Ces blogs restent en effet, pour l'architecte, une source d'inspiration car nous pouvons considérer les blogueurs comme des chercheurs qui apportent tous les jours de nouvelles idées et de nouveaux concepts à explorer.

Ce livre a une ambition plus modeste. C'est un livre d'ingénierie, qui se concentre sur la mise en œuvre. Il est orienté vers la pratique et les cas d'utilisation pour répondre à des problématiques courantes. Il se concentre sur ce qu'il faut savoir et apportera les bases qui permettront, je l'espère, au lecteur d'approfondir ses connaissances par la suite. Il illustre les parties théoriques avec des exemples fonctionnels afin de permettre l'expérimentation au plus tôt.

L'ouvrage se concentre plus particulièrement sur les cas d'utilisation d'applications web simples basées sur Spring MVC et sur les tests, car ce sont les parties de Spring les plus complexes et les plus utilisées.

Organisation du livre

Ce livre présente tout d'abord un aperçu de l'architecture du framework. Il montre ensuite que Spring est orienté "design pattern" à travers quelques exemples. Nous présentons ensuite quelques rappels sur les éléments externes à Spring qui sont utilisés par la suite dans les exemples. Nous plongerons alors dans un premier niveau de détail sur le cœur du framework pour ensuite en explorer les recoins.

Nous aborderons ensuite la programmation orientée aspect que permet Spring. Nous étudierons la partie test des applications Spring car Spring est particulièrement puissant dans ce domaine. Nous étudierons dans un premier temps en détail la partie back des applications, puis dans un second temps, nous présenterons la partie frontale web et les web services.

Nous terminerons par l'intégration JSF2 dans les applications web Spring.

Chaque partie sera illustrée par des exemples et nous proposerons quelques exercices. Les exemples et la solution de ces exercices sont téléchargeables depuis la page Informations générales. Les expérimentations à partir de ces exemples et les exercices vous permettront, si vous le souhaitez, de vous familiariser plus rapidement avec le framework. C'est en effet à travers l'expérience que vous améliorerez votre efficacité avec ce framework.

Public visé

Toute personne intervenant sur un projet utilisant Spring peut retirer quelque chose de ce livre : le programmeur débutant sur Spring aura un panorama de ce que peut faire ce framework et pourra pratiquer pour apprendre, le tech lead et l'architecte pourront prendre quelques idées et réutiliser du contenu pour expliquer aux équipes les fondamentaux de cette technologie.

Pourquoi Spring

Spring permet principalement de faire la même chose que ce que l'on ferait avec un serveur JEE (Java dans sa version entreprise) complet comme par exemple WebSphere, WebLogic, JBoss, Glassfish, mais dans un serveur web léger comme Tomcat ou Jetty. Au niveau des fonctionnalités il est en général en avance sur JEE. Il a en effet été conçu au départ pour pallier certaines difficultés rencontrées lors de la mise en œuvre des premières versions de JEE et a ensuite progressivement creusé l'écart en évoluant plus rapidement. Il permet de s'interfacer facilement aux différents frameworks tiers via des API unifiées car il offre des façades qui simplifient le code.

Il se distingue également par sa capacité à permettre facilement de tester de façon performante les applications en embarquant dans les tests une version allégée du contexte d'exécution, alors que sur JEE il faut souvent un serveur dans son intégralité. Il n'y a que jBoss qui s'en rapproche avec les tests arquillian (<http://arquillian.org/>) ainsi que par sa mise à disposition de l'Embedded JBoss (<https://developer.jboss.org/wiki/JBossEmbeddedAS?sscc=t>).

Prérequis pour aborder sereinement la lecture de ce livre

Le framework Spring a pour principale ambition d'apporter une architecture qui permet d'offrir les fonctionnalités JEE dans un contexte de conteneur web simple et léger.

Pour bien l'utiliser, une compréhension des mécanismes JEE est donc nécessaire. Nous pouvons considérer que, idéalement, aujourd'hui en tant que consultant, pour être confiant il faut connaître à la fois Spring et JEE. Je conseille donc à ce propos l'excellent et irremplaçable livre de Jérôme Lafosse : *Développements natifs avec Java EE*, publié aux Éditions ENI, qui explique très bien et de façon très complète des concepts JEE que nous rencontrerons dans ce livre. Il apporte un regard différent et complémentaire. Il y a en effet beaucoup d'éléments communs entre Spring et JEE et vous rencontrerez de plus en plus souvent des applications hybrides.

Pour profiter pleinement de ce livre, il faut aussi, dans la mesure du possible, bien connaître la programmation Java en général et en particulier Java web dans sa version classique utilisant des servlets, des pages JSP ou JSF :

- notions de JavaBeans,
- collections,
- bases de données avec JDBC,
- notions sur la rédaction de fichiers XML.

Un ordinateur avec une connexion Internet permet de charger les exemples et les exercices disponibles depuis la page Informations générales.

Les exemples sont illustrés par des extraits de projets fonctionnels qui peuvent servir de base pour expérimenter et apprendre le framework à travers des cas réels d'utilisation.

Objectifs du livre

L'objectif du livre est de voir en détail les fondements du framework Spring. Les modules les plus utilisés sont passés en revue. L'utilisation des modules non décrits dans ce livre sera facile à aborder car Spring respecte un ensemble de bonnes pratiques.

Ce livre a été structuré de façon à donner rapidement les éléments nécessaires pour être autonome sur la création ou la maintenance d'une application basée sur Spring par la compréhension du noyau, de l'accès aux données et la maîtrise de couche web.

Introduction

Ce chapitre présente, dans un premier temps, Spring dans son contexte historique et décrit, dans un second temps, ses constituants ainsi que des éléments de configuration.

1. Historique

Historiquement le framework Spring a été conçu pour faciliter la création d'application entreprise dans un conteneur web léger comme Tomcat, c'est-à-dire sans gestion d'EJB.

Les tableaux suivants apportent un repérage temporel.

Évolution de Java Entreprise :

Année	Version	Contenu
1999	J2EE 1.2	Servlets, JSP, EJB (1.1), JMS, RMI
2001	J2EE 1.3	EJB (2.0 CMP), JCA
2003	J2EE 1.4	EJB (2.1 MDB), Web services, JAX-RPC, Deployment Specification
2006	JEE 5	EJB (3.0 annotations), JPA, Annotations, JSF, JAX-WS
2009	JEE 6	Profil web, Servlet 3.0, EJB (3.1 singletons, lite, asynchronous), CDI, JAX-RS
2013	JEE 7	JMS 2.0, JAX-RS 2.0, Servlet 3.1, API JSON, WebSockets

Évolution de Spring :

Année	Faits importants
2002	Expert One-2-One J2EE Design and Development
2004	Spring V1.0 sous licence Apache
2005	Alternative aux EJB 2.x
2006	Jolt Productivity Award
2007	Spring 2.5 avec annotations
2009	VMware achète SpringSource
2013	Pivotal : Joint venture VMware et EMC Corporation

Dates des versions principales de Spring :

Version	Date
Expert One-on-One J2EE Design and Development	Octobre 2002
Licence Apache	Juin 2003
Milestone release, 1.0	Mars 2004
1.2.6	Novembre 2005
2.0	Octobre 2006
2.5	Novembre 2007
3.0 GA	Décembre 2009
3.1 GA	Décembre 2011

3.2 GA	Décembre 2012
4.0 GA	Décembre 2013
4.1.8	Septembre 2015
4.2 GA	Juillet 2015
4.3	Non planifiée
5.0	Non planifiée

2. Utilité de Spring dans un projet

Le framework Spring est une solution légère basée sur des briques logicielles indépendantes. Il apporte des solutions techniques unifiées pour la plupart des difficultés standards rencontrées sur les projets. Il offre des implémentations pour un ensemble de design patterns (patrons de conception) comme nous le verrons dans un chapitre dédié à ce sujet. Les bibliothèques d'extension apportent également des facilités pour l'utilisation des bibliothèques logicielles les plus utilisées. Il devient alors très facile de faire des web-services, d'accéder aux données, de faire une application basée sur des enchaînements d'écrans. Les différences entre les implémentations des frameworks standards d'accès aux données, par exemple, sont lissées à travers une API unifiée.

Les projets sont alors focalisés sur la partie métier, la partie technique étant gérée par le framework Spring. Spring a été conçu à l'origine pour Java mais existe aujourd'hui pour d'autres plates-formes comme .NET, Python...

Spring est sous licence Apache 2.0. Il est en open source mais les commiteurs (les développeurs affectés au projet qui peuvent mettre à jour les sources directement soit en codant, soit en validant des propositions de modifications des contributeurs externes) sont de SpringSource. Les sources sont accessibles et des demandes d'évolutions sont possibles depuis <https://github.com>. Il est même possible de proposer des corrections de bugs ou de proposer des évolutions à travers les « change requests » (demandes de changements).

Les sources sont disponibles sur le site de Spring (<http://spring.io>) et sous GitHub : <https://github.com/spring-projects>

L'ensemble est mavenisé (avec les sources et la javadoc) sur le repository central Maven pour la version standard et sur les repositories Maven SpringSource pour les versions en cours de développement. Nous retrouverons l'emplacement des repositories en fin de chapitre. Il est possible d'utiliser Maven ou Gradle pour faire des projets Spring. Dans cet ouvrage, nous n'utilisons que les versions Maven. Le framework Spring est une solution full stack, c'est-à-dire qu'il inclut toutes les briques techniques, et permet de faire des applications complètes, clé en main. Le cœur de Spring n'a que très peu de dépendances externes. Étant modulaire, il permet de n'embarquer que les éléments nécessaires pour l'application contrairement aux solutions basées sur des serveurs « lourds » qui mettent à disposition un ensemble complet de modules. En effet, un serveur JEE doit offrir un ensemble de fonctionnalités potentiellement utilisables par les applications qu'il héberge alors que Spring n'embarque que ce qui est utilisé.

Spring est souple et permet l'utilisation progressive de ressources de plus en plus complexes en fonction de la montée en charge et de la nécessité de partager des ressources entre plusieurs applications. Il offre une alternative aux frameworks historiques tels que Struts, en réutilisant leur force tout en apportant des évolutions qui rendent la conception d'applications MVC plus simple et plus robuste.

Il permet, comme nous le verrons, d'apporter le support de la programmation orientée aspect, une aide pour l'utilisation des transactions, et facilite la persistance des données. Spring encapsule les traitements de façon non intrusive. Le code n'est, dans la plupart des cas, pas lié à Spring. Spring relie les éléments métier entre eux et leur offre un accès simple aux éléments techniques externes. C'est essentiellement un facilitateur qui offre la façon la plus simple de fournir l'environnement technique nécessaire pour obtenir une fonctionnalité centrée sur le métier.

Spring Boot peut être utilisé seul pour créer des projets de façon à se passer des conteneurs Web légers tels que Tomcat ou Jetty. Nous ne parlerons pas de Spring boot car il n'est que rarement utilisé en production. Spring boot permet pourtant de simplifier considérablement la configuration de l'environnement d'exécution d'une application Spring mais ne semble pas apporter la souplesse qui existe quand nous configurons finement le conteneur web (Jetty ou Tomcat) en complément de la configuration

"classique" de l'application. Il est très pratique pour faire des POC : *Proof of Concept* (ou preuve par l'exemple en français).

Spring peut donc être embarqué dans un conteneur web léger comme Tomcat ou Jetty, mais il peut aussi l'être dans un serveur web JEE tel que WebLogic, GlassFish ou WebSphere. Nous pouvons alors nous contenter du profil web de ces serveurs. Il est également possible d'intégrer Spring dans une application full JEE, mais un grand nombre de fonctionnalités est alors redondant et l'utilité devient plus spécifique, comme de bénéficier de l'accès via des connecteurs « propriétaires » à des systèmes externes interfaçant l'application web aux éléments du patrimoine applicatif historique comme les mainframes, les services tuxedo, les ETL, etc.

Sur un serveur avec un profil Full JEE (JEE complet), l'apport de Spring réside dans une simplification par rapport à l'utilisation des composants JEE. Il est plus simple de déployer une application web sous la forme d'un WAR plutôt que sous la forme d'un ensemble d'EAR, la configuration peut être embarquée dans l'application, l'application web peut se connecter très facilement aux ressources JEE telles que des datasources, des pools de connexion, du JMS, ou tout autre élément via JNDI. Les tests unitaires et tests d'intégrations peuvent en partie se faire dans un conteneur plus léger que le serveur entier.

Nous pouvons donc facilement déployer des applications Spring dans un serveur JEE. L'utilisation des profils depuis JEE 6 atténue les différences en offrant la possibilité de n'utiliser que les fonctionnalités web que l'on a dans un Tomcat ou un Jetty, mais dans un serveur d'application complet. Le fait d'embarquer tous les modules faisait que le serveur consommait des ressources qui n'étaient pas utilisées. Il est à noter à ce propos que le serveur jBoss a toujours apporté une alternative aux serveurs JEE classiques. Il a, en effet, proposé bien en avance les profils, une modularité hyper fine et un contrôle total du serveur.

Un autre point de vue à considérer pour la comparaison de l'utilisation de Spring par rapport aux serveurs classiques est la possibilité de faire des tests d'intégration avec Arquillian ou l'embedded jBoss, et jBoss avait, en son temps, créé le framework Seam pour pallier les manquements des EJB traditionnels. Seam a apporté des concepts qui sont aujourd'hui largement couverts par le framework Spring et il est aujourd'hui relativement possible de faire une application Spring ayant une puissance comparable à celle d'une application Seam. Les EJB 3.x convergent progressivement avec Spring mais l'utilisation de Spring reste aujourd'hui la solution standard dans la plupart des projets. Spring est parfois également utilisé conjointement aux EJB sur les serveurs d'application WebSphere car ces derniers offrent un accès facilité aux mainframes IBM qui hébergent aujourd'hui une grande part du patrimoine applicatif mondial. Il permet alors d'utiliser au mieux les couches applicatives réparties.

Spring est principalement utilisé avec Java dans sa version classique pour la conception d'applications web ou d'applications Batches mais commence à fonctionner également sur les plates-formes mobiles avec Android par exemple. Il est aussi possible de faire du Spring avec .NET avec Spring.net et une utilisation conjointe de Nhibernate qui est la version .NET de Hibernate. Ce tandem permet de réaliser des applications .NET très facilement. Le framework Spring basé sur la JVM Java peut aussi être utilisé avec Groovy grâce à un support amélioré dans Spring 4. Cette amélioration permet d'utiliser toute la puissance de Spring avec toute la modernité du code Groovy. Nous parlerons très peu de Groovy dans cet ouvrage car son utilisation n'est pas encore généralisée malgré ses éléments très novateurs.

Spring est modulaire. Il a un noyau nommé Spring core et des bibliothèques satellites indépendantes. Spring peut s'utiliser pour des applications standalone, des applications serveur ou des applications Batches. Les utilisations standalone peuvent être des applications en lignes de commande, des applications avec IHM (interfaces homme-machine) à fenêtres souvent basées sur Swing ou JavaFX.

Vue d'ensemble et sujets abordés dans l'ouvrage

1. Les modules Spring

La documentation Spring liste à ce jour une vingtaine de modules Spring regroupés par thèmes :

Le noyau central en version 1.4.7-RELEASE :

Nom du module	Utilité
spring-core	Les fondamentaux : gestion des objets et des proxies.
spring-beans	Le paramétrage et les fabriques de beans.
spring-context	Gestion du contexte, EJB, JMX, JNDI, scheduling, validation.
spring-context-support	Support ehcache, guava, mail, scheduling UI.
spring-expression	Expresssion language Spring : Spel.

L'AOP :

Nom du module	Utilité
spring-aop	AOP via les proxies.
spring-aspects	Aspects basés sur AspectJ.
spring-instrument	Agent d'instrumentation pour le bootstrapping de la JVM.
spring-instrument-tomcat	Agent d'instrumentation pour Tomcat.

Les messages :

Nom du module	Utilité
spring-messaging	Pour avoir des applications basées sur les messages.

Accès aux données :

Nom du module	Utilité
spring-jdbc	Facilités JDBC.
spring-tx	Gestion programmatique des transactions.
spring-orm	Mapping Hibernate (JPA et JDO).
spring-oxm	Mapping Object/XML comme JAXB, Castor, XMLBeans, JiBX et XStream.
spring-jms	Module Java Messaging Service et intégration spring-messaging depuis Spring Framework 4.1.

Web :

Nom du module	Utilité
spring-web	Composants web.
spring-webmvc	Web services REST et web MVC.
spring-websocket	Web sockets.
spring-webmvc-portlet	Web dans un contexte portlet.

Test :

Nom du module	Utilité
spring-test	Aide pour utiliser JUnit et TestNG avec Spring et objets mock.

2. Sujets abordés

Comme nous l'avons vu dans l'avant-propos, Spring est très vaste. Nous ne pouvons tout traiter dans un seul ouvrage. Nous nous concentrerons donc principalement sur les sujets les plus utilisés, à savoir la partie arrière des applications Batches web et souvent mobiles, ainsi que la partie avant pour laquelle nous nous limiterons aux applications web en incluant dans cette partie les web services. Nous décrirons les tests unitaires associés autant que possible. Les exemples sont issus de cas réels présents dans les projets téléchargeables depuis la page Informations générales. La programmation par aspect qui est très utilisée avec Spring est également détaillée.

Les exemples utilisent la dernière version « release » de Spring disponible au moment de l'écriture de ce livre, c'est-à-dire la version 4.1.7. Ils utilisent aussi Java 8 et sont en UTF-8. Les exemples sont mavenisés pour faciliter la manipulation. Les briques JEE correspondent à celles de JEE 1.7, c'est-à-dire JSF2, JPA2, Servlet 3...

3. Version des briques Spring utilisées dans l'ouvrage

Versions utilisées :

Module	Version	Utilité
Spring Framework	4.1.7	Cœur de Spring
Spring Web Flow	2.4.0	Moteur de flow Spring
Spring Web Services	2.2.1	Web services
Spring Security	4.0.0	Gestion de la sécurité
Spring Data Commons	1.10.0	Socle accès rapide aux données

Les projets Spring :

Projet	Version
Spring Framework	4.2.0
Spring Security	4.0.1
Spring Web Services	2.2.1

4. Version des ressources externes

Liste des principaux éléments externes :

Bibliothèque	Version
Lombok	1.16.4
JUnit	4.12
fest-assert	1.4
easymock	3.3.1
logback-classic	1.1.3
H2	1.4.187

Hibernate	4.3.10.Final
apache commons-dbcp2	1.2
apache commons-dbcp	1.2
jackson	2.2.2
jsonpath	0.8.1

5. Gestion des dépendances Maven

Les exemples utilisent le mécanisme classique pour les dépendances.

Nous n'utilisons que les versions issues de la dernière release (version identifiée comme stable) :

```
<dependencies>
<dependency>
  <groupId> org.springframework </groupId>
  <artifactId> spring-core </artifactId>
  <version> 4.1.7.RELEASE </version>
</dependency>
</dependencies>
```

L'exemple de ce chapitre montre une configuration avec tous les modules.

Pour la version standard, nous pouvons ne pas spécifier de repository car il s'agit d'une version généralisée dans les repositories Maven standards, mais nous pouvons également spécifier précisément que nous voulons la version hébergée sur les repositories Spring :

```
<repositories>
  <repository>
    <id> io.spring.repo.maven.release </id>
    <url> http://repo.spring.io/release/ </url>
    <snapshots> <enabled> false </enabled> </snapshots>
  </repository>
</repositories>
```

Lors des phases de veille technologique, il peut être parfois nécessaire d'utiliser des versions qui ne sont pas encore disponibles sur les repositories standards.

Pour la version milestone :

```
<repositories>
  <repository>
    <id> io.spring.repo.maven.milestone </id>
    <url> http://repo.spring.io/milestone/ </url>
    <snapshots> <enabled> false </enabled> </snapshots>
  </repository>
</repositories>
```

Et pour les instantanés (snapshot) :

```
<repositories>
  <repository>
    <id> io.spring.repo.maven.snapshot </id>
    <url> http://repo.spring.io/snapshot/ </url>
    <snapshots> <enabled> true </enabled> </snapshots>
  </repository>
</repositories>
```

6. Utilisation d'un BOM (Bill of Materials) Maven

Le BOM permet de s'assurer que toutes les dépendances directes et transitives utilisent les mêmes versions.

```
<dependency>
<groupId>org.springframework</groupId>
  <artifactId>spring-framework-bom</artifactId>
  <version>${spring.version}</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>
```

Un avantage supplémentaire de l'utilisation du BOM réside dans le fait que vous pouvez ne plus utiliser l'attribut `<version>`.

```
<dependencies>
  <dependency>
    <groupId> org.springframework </groupId>
    <artifactId> spring-context </artifactId>
  </dependency>
  <dependency>
    <groupId> org.springframework </groupId>
    <artifactId> spring-web </artifactId>
  </dependency>
</dependencies>
```

Nous n'utilisons pas les BOM pour cet ouvrage afin de simplifier au maximum les fichiers de configuration pom.xml.

Un chapitre dédié expliquera les bases de l'utilisation de projets mavenisés pour permettre aux personnes qui ne connaissent pas encore ce mode de fonctionnement de profiter pleinement des exemples.

Points clés

- Le framework Spring est modulaire.
- Nous utilisons Maven pour le configurer.
- Les applications web et les tests sont les deux atouts de Spring.

Introduction

Aujourd'hui, dans le monde Java, il est relativement rare d'écrire une nouvelle fonctionnalité technique. Nous assemblons plutôt des briques déjà faites en se basant sur des frameworks et des bibliothèques open source. De plus en plus, nous essayons d'implémenter des briques fonctionnelles ou métier réutilisables afin de mutualiser le plus possible les développements. Il faut donc prendre soin de ne pas réinventer la roue d'une part et d'autre part, il faut s'efforcer d'essayer de rendre son code réutilisable.

En effet, la maintenabilité du code est un sujet à prendre au sérieux car souvent c'est une équipe qui code et une autre qui fait la maintenance par la suite. Les projets s'inscrivent aussi dans la durée et sur des logiciels d'envergure développés sur plusieurs années, les développeurs ne font qu'une partie de l'application qui sera réutilisée par d'autres. Il faut à la limite considérer son code comme une partie d'un framework plus vaste et réutilisable à volonté. Le code se doit donc d'être modulaire, bien structuré et doit aussi être testable et testé. Le code doit aussi être suffisamment commenté afin de pouvoir être réutilisé ou corrigé. À ce niveau, deux écoles existent : l'une dit qu'un code bien écrit avec des noms d'API clairs et précis peut se passer de documentation, l'autre dit que tout doit être commenté et expliqué et qu'il doit être à la limite possible de générer la documentation technique du code à partir des sources. Il faut trouver un juste milieu.

En général, des modules relativement autonomes ont des points d'adhérences identifiés avec d'autres modules périphériques sous la forme d'API. Il faut au moins documenter cette API. Et s'il existe un socle applicatif, celui-ci doit être abondamment commenté pour aider les collaborateurs à s'y retrouver en début de mission. Il convient alors de faire l'API la mieux adaptée et de bien la documenter pour sa réutilisation. Tous ces éléments sont structurants et sont communs à pratiquement tous les projets.

Au fur et à mesure du temps, un ensemble de « motifs » de conceptions ont été identifiés, et de cet ensemble a émergé une panoplie de bonnes pratiques opérationnelles. Une implémentation standard de solution a été identifiée pour un grand nombre de préoccupations. Pour clarifier les objectifs relatifs à une portion de code, il est important de comprendre et de reconnaître ces motifs afin de les baliser. Un intervenant extérieur comprendra alors immédiatement les concepts en jeu, la finalité, et entrera dans le vif du sujet sans avoir à décrypter le code.

Spring a repris une partie de cet ensemble afin de proposer des solutions simples pour répondre à des problématiques parfois extrêmement complexes.

Nous allons illustrer quelques exemples mais il faut garder à l'esprit que Spring est orienté design pattern (patrons de conception) et que pratiquement toutes les parties du framework sont issues d'une réflexion simplificatrice et unificatrice basée sur l'utilisation des design patterns.

Il s'agit d'un sujet à part entière que nous ne ferons que survoler. Ce sujet est bien traité dans l'ouvrage « Design Patterns en Java » écrit par Laurent Debrauwer aux Éditions ENI. Les lecteurs qui souhaitent approfondir ce sujet peuvent consulter cet ouvrage et identifier dans Spring un exemple de chaque design pattern décrit dans cet ouvrage. Nous ne verrons donc que quelques exemples afin d'illustrer ce concept.

Le singleton en Java

Un singleton est un objet unique dans un espace donné. Il est créé lors de sa première utilisation pour être ensuite réutilisé. Nous l'utilisons quand nous souhaitons partager une ressource ou quand nous souhaitons profiter de la réentrance du code pour n'avoir qu'un exemplaire d'une séquence de traitement pour une préoccupation donnée, c'est-à-dire une méthode unique utilisable par plusieurs traitements que ce soit de façon séquentielle sur un système à un seul thread ou de façon concurrente (simultanée) grâce à la réentrance sur un système multi-threadé.

1. Préoccupation

Historiquement, quand nous avons besoin d'une action, nous l'implémentions dans l'objet qui en avait besoin. Pour un traitement identique, il y avait alors une multitude de copies de ces actions dans les diverses instanciations des objets qui l'utilisaient. Ceci était dû au fait que dans un système orienté objet, les traitements sont encapsulés dans les objets conjointement aux données.

Pour les actions qui ne se basent pas sur l'état de l'objet qui la porte, nous avons décidé de déplacer l'action dans la partie statique de la classe. En effet la partie statique est partagée pour toutes les instanciations de la classe. Il n'y a donc plus de duplication du code en mémoire.

Certains services sont séparés des données qu'ils traitent et sont alors regroupés dans des classes sans instantiation, sous forme de classes massivement statiques. On retrouve ainsi en fait un système qui sépare les données des traitements avec d'un côté des POJO, c'est-à-dire des objets sans traitement, à l'exception des mécanismes de gestion des instanciations comme les constructeurs et les accesseurs, et de l'autre côté des traitements séparés des données comme on le ferait en C. Avec ce type d'architecture, nous arrivons rapidement aux limites du système.

On élaborera alors le singleton. Le singleton est une classe qui n'est instanciée qu'une fois et qui ensuite est réutilisée par tous. Elle supporte tous les concepts de la programmation objet. Nous avons alors la simplicité d'utilisation d'une classe mais sans avoir de copies inutiles.

Les premiers singletons (avant qu'il n'y ait les énumérations) utilisaient un code qui ressemble à celui-ci :

```
class MonSingleton
    private static MonSingleton UNIQUE = null;
    private MonSingleton(){
    }
    public static synchronized MonSingleton getInstance() {

        if (UNIQUE == null) {
            UNIQUE = new MonSingleton();
        }
        return UNIQUE;
    }
}
@Log
public class Main {
    public static void main(String[] args) {
        MonSingleton s1 = MonSingleton.getInstance();
        MonSingleton s2 = MonSingleton.getInstance();

        boolean test = s1==s2;
        log.info("s1==s2:"+test);
    }
}
```

Il existe une multitude de variantes. Les plus modernes utilisent une énumération qui, par essence, est un singleton.

2. Solution Spring

Spring permet la création de bean qui sont soit des singletons (une seule instance) soit des prototypes (une instance par bean). Quand on parle d'une seule instance, on entend par là que tous les objets qui ont pour

membre un bean singleton partagent le même objet membre en mémoire. Dans la réalité, ces beans singleton définis dans la configuration Spring ne sont créés qu'une fois par contexte Spring. Ils peuvent donc être multiples dans une application qui héberge plusieurs contextes Spring.

Il faut savoir que les annotations de la JSR-330 (@Named, @Singleton) sont utilisables si le javax.inject est présent dans le classpath. Il est également possible d'utiliser les annotations Spring @Bean, @Service, @Repository, qui spécifient que l'on utilise un singleton avec des particularités inhérentes avec sa fonction.

3. Exemple

Cet exemple insiste sur le fait que le singleton est partagé entre plusieurs traitements qui peuvent s'exécuter de façon concurrente. Cela signifie que, du point de vue de la mémoire, il peut y avoir deux séquences de codes qui s'exécutent en parallèle dans une même méthode. Un des traitements peut modifier une donnée utilisée par l'autre traitement ce qui va poser des problèmes très rapidement. Il faut alors protéger les zones à risques, c'est ce que l'on appelle communément « rendre le code Thread Safe » par l'utilisation de zones synchronisées pour lesquelles seul un traitement s'exécute sur une portion de code à un instant T, pendant que les autres traitements attendent alors leur tour. Il est possible de protéger un traitement et/ou une variable avec le mot-clé `synchronized`.

En général, nous éviterons d'avoir des « états » dans les singletons mais vous en rencontrerez parfois et il faudra faire preuve de prudence.

```
@Service
class ServiceUnique {
    public int etat=0;

    public synchronized int incEtat() {
        //Opération longue
        return ++etat;
    }
    public int getEtat() {
        return etat;
    }
    public void setEtat(int etat) {
        this.etat = etat;
    }
    void afficherEtat() {
        System.out.println(etat);
    }
    @Override
    public int hashCode() {
        final int prime = 31;
        int result = 1;
        result = prime * result + etat;
        return result;
    }
    @Override
    public String toString() {
        return "ServiceUnique [etat=" + etat + "]";
    }
    @Override
    public boolean equals(Object obj) {
        ServiceUnique other = (ServiceUnique) obj;
        if (etat != other.etat)
            return false;
        return true;
    }
}

public class Main {
    public static void main(String[] args) {
        ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");

        ServiceUnique s1 =
        (ServiceUnique)context.getBean("serviceUnique");
        ServiceUnique s2 =
        (ServiceUnique)context.getBean("serviceUnique");
        s1.setEtat(3);
```

```
        System.out.println("Mêmes valeurs (equals) ?: " +
s1.equals(s2));
        System.out.println("Même référence (==)      ?: " +
(s1==s2));
        System.out.println(s1);
        System.out.println(s2);

        s1.setEtat(2);
        s2.afficherEtat();
    }
}
```

Messages dans la console :

Mêmes valeurs (equals) ?: true

Même référence (==) ?: true

ServiceUnique [etat=3]

ServiceUnique [etat=3]

Comme nous le verrons par la suite, Spring permet également de faire des singletons par rapport à un contexte d'utilisation à travers l'utilisation des scopes (portées).

Un bean de scope « session » sera par exemple unique au sein d'une session pour un utilisateur. Mais il y aura autant d'objets que de sessions.

Il existe d'autres scopes et il est également possible de créer ses propre scopes.

L'objet devient alors unique dans son scope.

L'inversion de contrôle

Avec le principe de l'IOC (*Inversion of Control*, inversion de contrôle en français), les classes implémentant les fonctions d'une librairie ne sont plus instanciées et appelées directement par l'application. Le programme indique qu'il a besoin d'une librairie et elle lui est automatiquement fournie. C'est un mécanisme externe qui met à disposition la classe via un framework ou un mécanisme de plug-ins. Par exemple, avec Spring l'application indique qu'elle a besoin d'un service avec une API précise et Spring lui fournit le meilleur candidat.

Spring permet de spécifier le service voulu, via une interface dans les cas les plus courants ou via une classe quand il n'y a pas d'interface. Il utilise pour cela généralement une fabrique d'objets qui est elle-même un design pattern pour mettre à disposition l'objet.

```
List<String> liste = new ArrayList<String>();
UtilisateurDao dao = (UtilisateurDao)
factory.getBean("utilisateurDao");
```

Une bonne habitude à prendre en général en Java est de déclarer un objet par son interface le caractérisant le mieux. Plutôt que d'instancier un objet soi-même, on laisse à Spring le soin de trouver la classe qui correspond au mieux à l'objet à instancier.

On appelle le mécanisme de l'IOC l'injection de dépendances et on utilise l'annotation `@Autowired` ou `@Inject`. Nous verrons l'utilisation et la différence entre ces deux annotations ultérieurement.

Spring instancie un proxy qui est lui-même un design pattern pour les objets injectés et nous donne un accès à l'objet définitif lors des accès.

```
public class LivreService {
@Inject
Private LivreDAO livreDao;
//implementation
[...]
```

À l'exécution, Spring trouve la classe qui correspond à l'implémentation de l'interface `LivreDao` (par exemple `LivreDaoImpl`). Il cherche alors dans son contexte un singleton qui correspond, s'il ne le trouve pas, il l'instancie, et il met sa référence dans la variable `livreDao`.

Ce découplage permet de retirer des dépendances dans les projets. La dépendance devient alors dynamique et nous n'utilisons alors que l'implémentation voulue pour une API donnée.

En prenant le contrôle de la création de l'objet, Spring peut aussi effectuer des opérations lors de l'utilisation de l'objet en ajoutant des fonctionnalités à des moments précis, comme nous le verrons au chapitre consacré à la programmation par aspects. Il est alors possible d'ajouter dynamiquement des comportements aux objets comme la transactionnalité, l'accès sécurisé aux méthodes, etc. Il est également possible de demander à Spring d'utiliser un type d'objet différent dans un contexte particulier, par exemple lors de la phase de test.

Par défaut, les beans définis dans la configuration Spring ne sont créés qu'une fois. Peu importe combien d'appels ont été faits en utilisant la méthode `getBean()`, il n'y aura toujours qu'un seul bean. En effet, par défaut, tous les beans dans Spring sont des singletons.

Façade

Une façade est par exemple une classe qui concentre les API d'un ensemble de classes en un point dans la classe façade. Spring JDBC est un exemple de façade qui masque les différences entre les implémentations de diverses bases de données via une API unifiée. Spring JDBC permet également de simplifier la gestion des exceptions liées aux bases de données. Il offre une façade qui encapsule de façon uniforme des traitements très différents d'une base à une autre, mais qui sont vus comme identiques du côté de la façade.

Un service Spring (@Service) peut être considéré comme une façade qui expose d'un côté des API métiers et de l'autre côté s'interface avec les DAO pour effectuer les opérations métier et bases de données.

Cette façade permet alors de concevoir une API qui peut être simple, testable. Le programme qui utilise la façade n'a plus besoin des dépendances car celles-ci sont déplacées dans la façade.

Fabrique

Spring utilise un modèle de fabrique (factory) pour créer des beans à l'aide d'une référence au contexte applicatif. Ces beans sont décrits dans la configuration. Une partie de la configuration est fixée à l'avance alors qu'une autre partie est déduite à partir de règles relativement complexes. Spring trouve le meilleur candidat pour un contrat d'API donné.

```
BeanFactory factory = new XmlBeanFactory(new
FileSystemResource("spring.xml"));
Triangle triangle = (Triangle) factory.getBean("triangle");
triangle.draw();
```

Spring met à disposition une batterie complète de fabriques afin de couvrir un maximum de cas d'utilisation à travers l'interface `Factory`. Cette interface a en effet dix sous-interfaces et vingt-sept implémentations dans sa dernière version.

La fabrique peut être visible dans le cas d'une application standalone (qui fonctionne seule à l'opposée d'une application web qui utilise un serveur web par exemple). Nous utilisons alors une factory qui se base sur des fichiers comme celle citée dans l'exemple. Pour une application web, la fabrique se cache dans un filtre de servlet.

Active record

Prenons comme exemple un traitement qui fait une mise à jour sur des utilisateurs de façon séquentielle, c'est-à-dire utilisateur par utilisateur. Lors du traitement sur un utilisateur, nous aurons donc en mémoire cet utilisateur accompagné de références sur tous ses objets satellites comme ses contrats, son adresse, etc.

Certains de ces objets périphériques sont alors chargés en mémoire si nous les utilisons dans le traitement, pour consolider les contrats par exemple. L'« active record » (enregistrement actif en français) représente alors l'objet qui correspond à l'enregistrement en cours et tous ses objets liés, c'est-à-dire ici la grappe d'objets qui correspond à l'utilisateur en cours d'édition ou de modification.

Une opération sur un des objets de la grappe déclenche en général la mise à jour dans la base de toute la grappe. Spring gère ces grappes d'objets avec l'aide de frameworks externes comme JPA ou Hibernate via les DAO ou les repositories afin de mettre à disposition les données à la couche service.

```
Utilisateur utilisateur = new Utilisateur();
utilisateur.setNom("CONNOR");
utilisateur.setPrenom("John");
utilisateurDao.save(utilisateur);
```

Pour les personnes qui sont familières avec les traitements en SQL, l'active record correspond à la ligne courante d'un FETCH.

Proxy

Ce design pattern est utilisé intensément par Spring pour tout ce qui concerne la programmation orientée aspect AOP et les accès à distance.

L'AOP est décrit en détail dans le chapitre Programmation orientée aspect avec Spring.

Spring ne donne pas un accès direct aux objets mais via un objet intermédiaire appelé proxy ce qui permet à Spring de modifier le comportement de l'objet d'origine.

```
interface Pojo3 {
    public void foo();
}
class SimplePojo3 implements Pojo3 {
    public void foo() {
        System.out.println("foo");
    }
}
class RetryAdvice implements AfterReturningAdvice {
    public void afterReturning(Object returnValue, Method method,
        Object[] args, Object target) throws Throwable {
        System.out.println("Après "+method.getName());
    }
}
public class MainAvecAspect {
    public static void main(String[] args) {
        ProxyFactory factory = new ProxyFactory(new SimplePojo3());
        Class<?> intf=Pojo3.class;
        factory.addInterface(intf);
        Advice advice=new RetryAdvice();
        factory.addAdvice(advice);
        factory.setExposeProxy(true);
        Pojo3 pojo = (Pojo3) factory.getProxy();
        pojo.foo();
    }
}
```

Comme nous le verrons par la suite, il existe deux types de proxy. Pour les objets référencés par une interface, on utilisera un proxy classique Java et pour les objets n'ayant pas d'interface, nous utiliserons des proxies basés sur CGLIB. Ces derniers sont moins pratiques à utiliser.

Comme nous le verrons en détail par la suite, le proxy permet plusieurs choses intéressantes. Nous pouvons parfois différer l'instanciation de l'objet référencé par le proxy. Cette instanciation ne se fait que lors du premier accès aux propriétés de l'objet, via un appel de méthode sur le proxy. Cela permet d'injecter des objets qui ne sont pas encore disponibles mais qui le seront quand nous en aurons besoin.

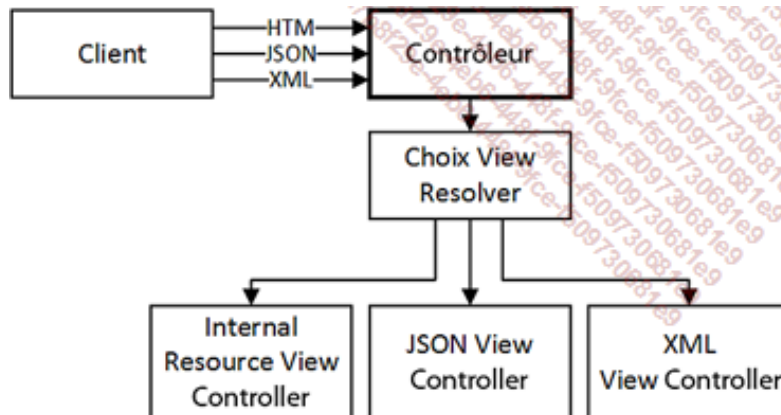
Dans un même ordre d'idée, JPA et Hibernate utilisent aussi les proxies. Il est en effet possible, quand on a un objet lié à d'autres objets par des relations Many2One, One2One, etc., de demander que lors du chargement de l'objet principal depuis la base de données, on n'ait que des références vers les objets périphériques qui ne se chargeront que si on y accède. Ceci permet de n'avoir en mémoire que les objets modifiés.

Modèle Vue Contrôleur (MVC)

Spring MVC permet de faire simplement des applications web en séparant les trois éléments principaux :

Le modèle	les données.
La vue	ce qui est affiché.
Le contrôleur	le traitement sur les données et l'enchaînement des vues.

Spring délègue le choix de la vue à un ViewResolver qui est, lui-même, un design pattern.



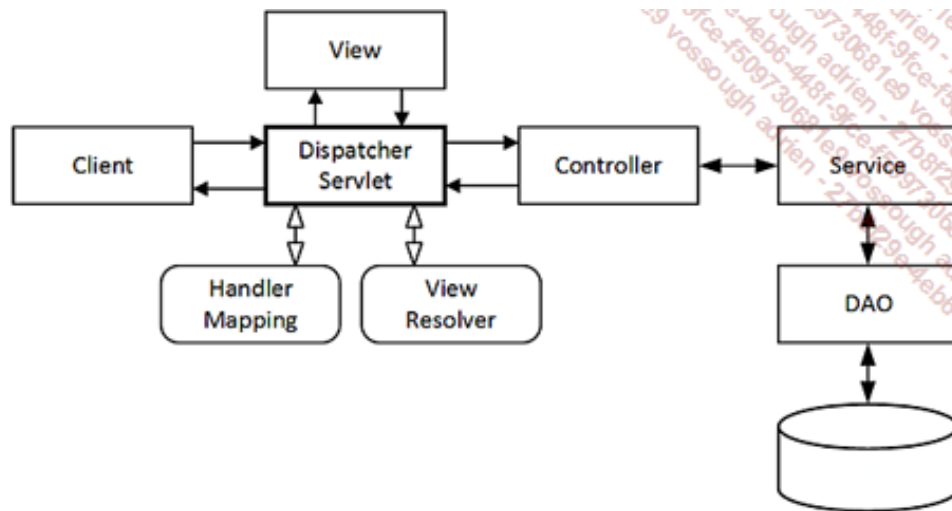
Par exemple, le contrôleur suivant sur la sollicitation de l'URL avec `/bonjour` demandera l'affichage de la page `bonjour.jsp` en lui fournissant la date pour que la page l'affiche :

```
@Controller
public class BonjourController {
    @RequestMapping("/bonjour")
    public ModelAndView bonjour() {
        ModelAndView mav = new ModelAndView();
        mav.setViewName("bonjour");
        mav.addObject("date", new Date());
        return mav;
    }
}
```

Le contrôleur de partie frontale (Front Controller)

Dans Spring MVC, Spring fournit un `DispatcherServlet` pour s'assurer qu'une requête entrante est correctement dispatchée au bon contrôleur. C'est un exemple de contrôleur de partie frontale.

La classe `org.springframework.web.servlet.DispatcherServlet` délègue les requêtes à des contrôleurs.



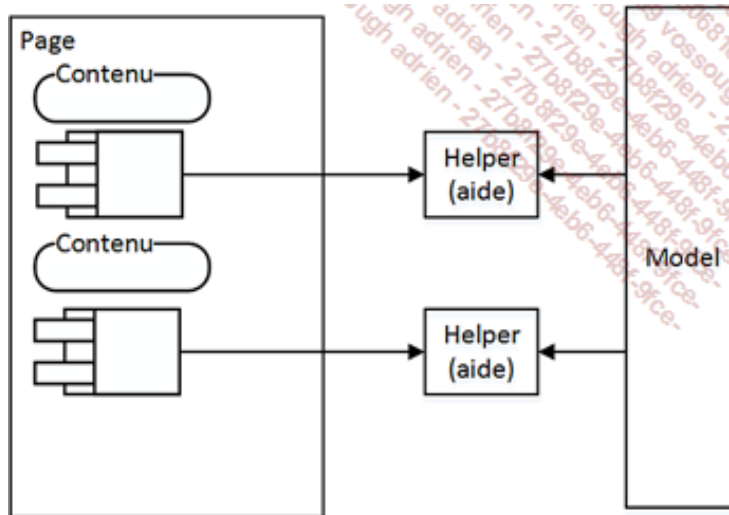
Extrait d'un fichier `web.xml` pour la déclaration du `DispatcherServlet` :

```
<servlet>
  <servlet-name>HelloWeb</servlet-name>
  <servlet-class>
    org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>
```

Les assistants de vue (View Helper)

Spring MVC utilise des assistants de vue sous la forme de balises JSP.

Ces balises JSP personnalisées aident à simplifier le design des pages via l'utilisation de TLD par exemple.



```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<taglib xmlns="http://java.sun.com/xml/ns/j2ee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
web-jsptaglibrary_2_0.xsd"
        version="2.0" >
<tlib-version>1.0</tlib-version>
<jsp-version>2.0</jsp-version>
<short-name>mesTags</short-name>
<description>
    Librairies de tags personnalisés
</description>
<tag>
    <name>myTag</name>
    <tag-class>fr.eni.tags.mesTag</tag-class>
    <body-content>JSP</body-content>
    <attribute>
        <name>myAttribut</name>
        <required>yes</required>
    </attribute>
</tag>
</taglib>
<%@ taglib uri="/aides" prefix="aides" %>
<aides:monTag monAttribut="bla bla bla">
    Contenu du texte....
</aides:monTag>
```

Les templates

On utilise des templates (modèles) pour tous les codes génériques.

Spring utilise massivement ces templates. Voici quelques exemples :

Template	Module	Utilité
RepeatTemplate	Spring Batch	Implémentation de RepeatOperations
JdbcTemplate	Spring	Opération courante JDBC
QueryDslJdbcTemplate	Spring Data	Support QueryDsl
NamedParameterJdbcTemplate	Spring	JdbcTemplate mais avec des paramètres nommés
TransactionTemplate	Spring	Gestion générique des transactions
HibernateTemplate	Spring	Opérations courantes sur Hibernate
JdoTemplate	Spring	JDO générique
JpaTemplate	Spring	Opération JPA
JpaTransactionManager	Spring	Transactions JPA
RestTemplate	Spring	Web service REST
SimpMessagingTemplate	Spring	Message (par exemple JMS)

Ce livre présente des exemples pour l'utilisation de certains de ces templates.

Les templates sont une grande force pour Spring car ils simplifient le code. Ils masquent la complexité et il est même possible d'interfacer un système exotique en étendant un template Spring et d'offrir une interface standard aux développeurs.

Stratégie

L'injection de dépendance (DI) est un design pattern de stratégie. En effet, chaque fois que vous voulez mettre en place une logique d'interchangeabilité entre beans, vous trouverez une interface qui correspond à un constructeur ou une méthode setter appropriée sur la classe d'accueil pour câbler votre propre implémentation de cette interface. Pour une même interface, nous pouvons demander à Spring d'injecter un objet implémentant l'interface et choisir ainsi un objet ou un autre ayant les mêmes API de traitement mais avec des API ayant des comportements différents. Par exemple pour une liste qui a pour interface List on pourra choisir entre les implémentations suivantes : `AbstractList`, `AbstractSequentialList`, `ArrayList`, `AttributeList`, `CopyOnWriteArrayList`, `LinkedList`, `RoleList`, `RoleUnresolvedList`, `Stack`, `Vector`, en fonction du comportement que l'on souhaitera avoir.

Points clés

- Le framework Spring est massivement orienté design pattern.
- Il faut identifier les design patterns pour les signaler dans les commentaires du code.
- Avant de coder quelque chose, il faut voir s'il s'agit d'un design pattern et réutiliser ou spécialiser du code qui implémente déjà ce patron de conception.
- Coder avec l'esprit design pattern permet à son code d'être plus facilement réutilisable.

Codage equals et hashCode

L'égalité entre objets est importante dans Spring, notamment pour les architectures utilisant Spring en conjonction avec un mapping objet-relationnel (en anglais *Object-Relational Mapping* ou ORM) comme Hibernate et JPA.

Il y a en effet, comme nous le détaillerons par la suite, dans Spring, Hibernate et JPA des notions de proxies. Quand nous aurons deux objets en mémoire, nous devons faire attention à ne pas comparer un proxy avec l'objet qu'il proxyfie.

De même, deux objets pourront être identiques d'un point de vue métier alors qu'ils sont présents dans deux objets distincts en mémoire. Nous verrons que cet aspect est crucial lors de l'utilisation de collections.

Une définition selon Java nous présente ce que l'on attend des méthodes `equals` et `hashCode`. À partir de cette définition, nous verrons comment adapter ce concept pour une utilisation dans une architecture qui utilise massivement les proxies et les listes comme celle de Spring, JPA et Hibernate.

Dans ce chapitre, nous dirons le "equals" pour désigner la méthode `equals` et le "hashCode" pour désigner la méthode `hashCode`.

Une traduction approximative de la documentation officielle pour `equals` et `hashCode` est présentée ci-dessous.

equals

```
public boolean equals(Object obj)
```

Indique si un objet est égal à celui passé en paramètre.

La méthode `equals` implémente une relation d'équivalence entre deux références d'objets non nulles.

Elle est réflexive : pour toute référence d'une valeur non nulle `x`, `x.equals(x)` doit retourner `true`.

Elle est symétrique : pour toute référence d'une valeur non nulle `x` et `y`, `x.equals(y)` retourne `true` si et seulement si `y.equals(x)` retourne `true`.

Elle est transitive : pour toute référence d'une valeur non nulle `x`, `y` et `z`, si `x.equals(y)` retourne `true` et si `y.equals(z)` retourne `true` alors `x.equals(z)` doit retourner `true`.

Elle est consistante : pour toute référence d'une valeur non nulle `x` et `y`, de multiples invocations de `x.equals(y)` retournent toujours `true` ou de façon consistante toujours `false`, si aucune information fournie utilisée dans les comparaisons d'égalité sur les objets n'est modifiée.

Pour une référence non nulle sur une valeur `x`, `x.equals(null)` doit retourner `false`.

La méthode `equals` pour la classe `Object` implémente la relation d'équivalence la plus discriminante possible sur les objets ; cela consiste à ce que pour toutes les valeurs de référence non nulles `x` et `y`, cette méthode renvoie `true` si et seulement si `x` et `y` se réfèrent au même objet (`x == y` a la valeur `true`).

Notez qu'il est généralement nécessaire de redéfinir la méthode `hashCode` chaque fois que la méthode `equals` est surchargée, de manière à maintenir le contrat général pour la méthode `hashCode`, qui stipule que les objets égaux doivent avoir des codes de hachage égaux.

Paramètres :

`obj` : la référence de l'objet avec lequel comparer.

Valeur retournée :

`true` si cet objet est le même que celui transmis par l'argument `obj` ; `false` sinon.

Voir aussi :

`hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Retourne une valeur de code de hachage pour l'objet.

Cette méthode fournit une prise en charge au bénéfice des tables de hachage telles que celles fournies par `HashMap`.

Le contrat général de `hashCode` est : chaque fois qu'il est appelé sur le même objet plus d'une fois lors d'une exécution d'une application Java, la méthode `hashCode` doit toujours retourner le même entier, tant qu'aucune information utilisée dans les comparaisons sur l'objet n'est modifiée.

Cet entier n'a pas besoin de rester cohérent d'une exécution d'une application à une autre exécution de la même application.

Si deux objets sont égaux selon la méthode `equals (Object)`, alors l'appel de la méthode `hashCode` sur chacun des deux objets doit produire le même résultat sous forme d'entier.

Dans le cas où deux objets sont inégaux selon la méthode `equals (java.lang.Object)`, la méthode `hashCode` appelée sur chacun de ces deux objets peut produire des résultats distincts. Toutefois, le programmeur doit être conscient que la production de résultats sous la forme d'entiers distincts pour les objets inégaux peut améliorer la performance des tables de hachage.

Autant que possible, la méthode `hashCode` définie par la classe `Object` ne retourne des entiers distincts que pour des objets distincts. Elle est généralement mise en œuvre par la conversion de l'adresse interne de l'objet en un entier, mais cette technique de mise en œuvre n'est pas requise par le langage de programmation Java.

Valeur retournée :

Une valeur de hashcode pour cet objet.

Voir aussi :

```
equals (java.lang.Object),
```

```
System.identityHashCode (java.lang.Object)
```

1. Description de la problématique

Par défaut, Java se base sur les identifiants mémoires pour faire les opérations de `equals` et `hashCode`. Cela fonctionne bien pour des cas simples. Cependant le codage du `equals` et du `hashCode` est problématique, notamment dans des environnements basés sur des proxies comme Spring, Hibernate et JPA.

En effet, la ou les colonnes qui portent les clés primaires et qui sont mappées dans l'objet dans la variable portant l'annotation `@Id` ou composite (`@IdClass` ou `@EmbeddedId`) ne sont renseignées qu'après la sauvegarde dans la base.

Tous les systèmes basés sur des collections utilisent le `equals` et le `hashCode` pour voir si l'élément existe dans la collection.

Pour les proxies, il faut savoir que le proxy sur les objets surcharge le `equals` et le `hashCode`. Il faudra utiliser nativement `AspectJ` si on souhaite intercepter ces méthodes.

Pour les proxies liés au `lazy loading` (chargement à la demande), il faut éviter de comparer les références des objets ainsi que les types en se concentrant sur le contenu, ce qui force le chargement dans le cas de JPA et d'Hibernate ou l'instanciation et l'injection de l'objet pour Spring.

2. Mise en œuvre

La solution la plus raisonnable est d'utiliser une clé métier basée sur les données réelles, mais cela est assez difficile à maintenir. Nous utilisons, en effet, souvent une clé primaire séquentielle indépendante des clés métiers qui se distinguent au mieux par leur unicité. Pour la comparaison, nous comparons les valeurs et pour le hashcode, nous calculons le hashcode des variables et nous combinons ces valeurs pour obtenir un hashcode global. Les choses se compliquent si nous faisons intervenir les objets liés dans la comparaison. Cette préoccupation de comparaison et de hashcode se retrouve souvent dans une partie de code très générique et répétitive à faible valeur ajoutée, comme la couche domaine.

Une solution a été proposée sur les forums ici : <https://forum.hibernate.org/viewtopic.php?f=1&t=928172&start=105>

```
import java.io.Serializable;
import java.rmi.dgc.VMID;
//pris ici : https://forum.hibernate.org/viewtopic.php?f=1&t=928172&start=105
public class ModelElabore implements Serializable {

    private volatile VMID vmid;
    private Long id;

    public Long getId() {
        return id;
    }
    private void setId(Long id) {
        this.id = id;
    }

    public boolean equals(Object obj) {
        final boolean returner;
        if (obj instanceof ModelElabore) {
            return getVmid().equals(((ModelElabore)obj).getVmid());
        } else {
            returner = false;
        }
        return returner;
    }

    public int hashCode() {
        return getVmid().hashCode();
    }

    private Object getVmid() {
        if (vmid != null || vmid == null && id == null) {
            if (vmid == null) { //Avoid the performance impact
                                //of synchronized if we can
                synchronized(this) {
                    if (vmid == null)
                        vmid = new VMID();
                }
            }
            return vmid;
        }
        return id;
    }
}
```

Avec l'utilisation de la classe `VMID()`, nous détournons l'API Java qui permet de créer un id unique pour toutes les machines virtuelles Java. Cette API est utilisée habituellement pour gérer les garbage collectors (ramasse-miettes) distribués afin d'identifier les machines virtuelles clientes.

Nous sommes alors assurés d'avoir un identifiant unique. Il est également possible d'utiliser une API de la librairie commons d'Apache, à savoir l'API `org.apache.commons.id.uuid`, mais autant utiliser l'API de Java si elle est disponible.

On pourrait trouver cette solution trop puissante mais elle fonctionne dans une majorité de cas. Le générateur Celerio de Jaxio utilise une technique similaire.

Projet Lombok

La librairie du projet Lombok (<https://projectlombok.org/>) simplifie beaucoup la conception des POC car elle minimise le code, mais elle pose parfois des problèmes sur certaines machines. J'ai décidé de l'utiliser pour les exemples de code afin de réduire la taille du code. Il est très facile de l'enlever et d'utiliser les facilités de génération de code de son IDE (*Integrated Development Environment* ou environnement de développement intégré en français), c'est-à-dire en général Eclipse ou IntelliJ Idea, et d'ajouter le cas échéant la déclaration du logger.

En effet, grâce à cette librairie nous pouvons utiliser des annotations qui injectent du code à la volée pour effectuer des tâches courantes comme les logs, les accesseurs, les constructeurs et bien d'autres choses utiles.

Pour utiliser la librairie du projet Lombok, il faut ajouter la dépendance dans le fichier pom.xml :

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.16.4</version>
</dependency>
```

Si on utilise Eclipse, il faut ensuite patcher son environnement Eclipse, comme l'indique le site du projet Lombok. Pour cela, lancez le JAR de la librairie depuis la ligne de commande et sélectionnez son Eclipse à patcher. Sous IntelliJ Idea, il suffit d'installer le plug-in Project Lombok.

Cette manipulation est nécessaire pour que la complétion automatique fonctionne et pour que l'outil de développement ne considère pas comme un bug les méthodes qui ne sont pas déclarées mais qui sont injectées par le projet Lombok.

Nous n'utilisons pas toutes les possibilités de la librairie afin de garder le code aussi propre que possible, mais nous les présentons à titre indicatif :

Annotation	Utilité
@NonNull	Mini-validateur.
@Cleanup	Ferme automatiquement les ressources comme les fichiers.
@Getter / @Setter	Ajoute les accesseurs.
@ToString	Fait une sortie propre.
@EqualsAndHashCode	Ajoute le equals et le hashCode (à éviter avec Hibernate et JPA).
@NoArgsConstructor	Ajoute un constructeur sans argument.
@RequiredArgsConstructor	Ajoute un constructeur avec des arguments.
@AllArgsConstructor	Ajoute un constructeur avec tous les arguments.
@Data	Raccourcis pour @ToString, @EqualsAndHashCode, @Getter sur tous les champs, @Setter sur les champs non finaux, et @RequiredArgsConstructor.
@Value	Gestion des classes immutables.
@Builder	Permet de fabriquer simplement des builders.
@SneakyThrows	Manipulation d'exceptions (à éviter).
@Synchronized	Simplifie les synchronisations.
@Getter(lazy=true)	Gère une sorte de cache sur un champ (à éviter).
@Log	Logger générique, existe pour chaque implémentation :

	@CommonsLog, @Log, @Log4j, @Log4j2, @Slf4j, @XSlf4j.
--	---

Pour chaque annotation, il y a des annotations secondaires. Nous utilisons principalement : @Slf4j, @Getter, @Setter, @ToString, @NoArgsConstructor et @AllArgsConstructor.

L'utilisation de ces annotations fait gagner beaucoup de place dans les exemples du livre. Si vous rencontrez des problèmes liés à votre machine, n'hésitez pas à ne pas utiliser ce framework. Il est utilisable en production si on s'assure que tout a été testé sur la plate-forme cible.

Vous rencontrerez, parfois, dans certains frameworks, du code avec tous les membres publics pour éviter les accesseurs. J'ai préféré garder les membres privés et utiliser lombok.

Les systèmes de log

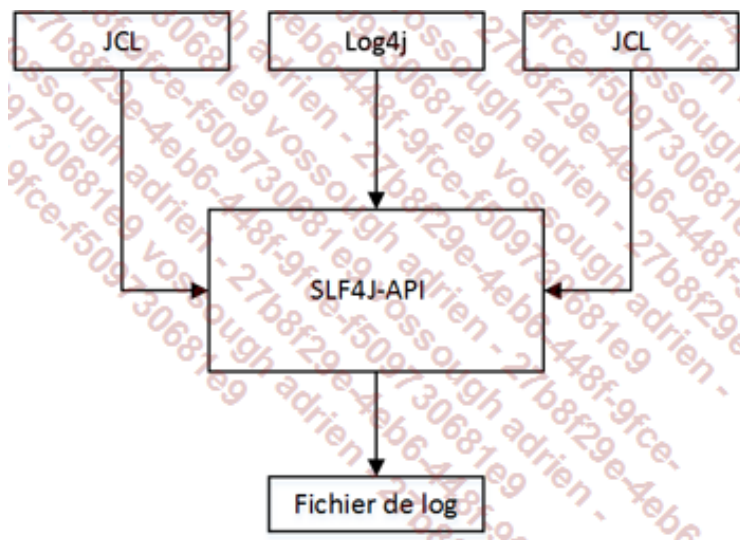
Log4j, SLF4J et Logback

Nous avons utilisé durant de longues années, traditionnellement, le système de log Log4j pour la journalisation des messages d'information, de debug et d'erreur. Cependant, la nouvelle librairie Logback (<http://logback.qos.ch/>) est bien plus puissante que Log4j.

Historiquement, la JCL (*Jakarta Common Logging*) fut une des premières API concurrentes mais elle restait très restreinte. Vers 2006, de gros projets comme Hibernate adoptèrent l'API SLF4J (<http://www.slf4j.org/>) ce qui la popularisa. Cependant, malgré de bons débuts, SLF4J n'a pas totalement éclipsé Log4j.

Pourtant, grâce à SLF4J, il est possible de centraliser dans un seul ensemble de fichiers de logs les appels qui sont faits par les différentes API de journalisation comme JCL, Log4j, JUL (<http://docs.oracle.com/javase/8/docs/api/java/util/logging/package-summary.html>)...

Les logs issus des appels aux API de logs sont alors regroupés et il devient possible de choisir quelle implémentation de journalisation on souhaite utiliser. Contrairement à JCL qui charge les librairies à l'exécution avec le classloader, SLF4J choisit l'implémentation cible à partir du package correspondant au pont logiciel entre types d'API.



L'API Logback est aujourd'hui une des meilleures API de journalisation. Elle offre en effet beaucoup de souplesse pour faire des logs. Il est possible également d'associer Logback à SLF4J et à Lombok :

```
package fr.eni;
import lombok.extern.slf4j.Slf4j;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
@Slf4j
public class LesLogs {
    private static final Logger slf4jLogger = LoggerFactory
        .getLogger(LesLogs.class);
    public static void main(String[] args) {
        slf4jLogger.trace("Bonjour à vous!");
        String nom = "John CONNOR";
        slf4jLogger.debug("Hi, {}", nom);
        slf4jLogger.info("Log de type info.");
        slf4jLogger.warn("Log de type warn.");
        slf4jLogger.error("Log de type error.");
        log.info("Log de type info via lombok, {}", nom);
    }
}
```

Cet ouvrage utilise, dans la mesure du possible, les logs Logback.

Nous avons dû nous limiter pour ce chapitre mais Logback fonctionne très bien dans une stack ELK (*Elasticsearch-Logstash-Kibana*). Dans la mesure du possible, il faut garder à l'esprit que nos logs peuvent servir à aider les personnes chargées de l'exploitation et de la production pour nos applications en environnement cible. Il faudra donc journaliser un maximum de données exploitables.

Bases de données H2

1. Description de la problématique

Il existe aujourd'hui beaucoup de bases de données relationnelles, dans le monde SQL, qui s'inscrivent dans des contextes d'utilisations très variés. Les versions commerciales les plus connues sont Oracle, DB2, SQL Server et Sybase. Il existe d'autres bases de données libres comme Derby, Firebird, HSQLDB, H2, Ingres, MariaDB, MySQL et PostgreSQL, pour ne citer que les plus connues.

Nous utiliserons souvent HSQLDB et H2 pour leurs possibilités d'utilisation très étendues. Cela nous permet en effet d'avoir la base de données la plus light (légère) possible pour faire une application légère et réaliser des tests. Ces bases peuvent s'exécuter en mémoire ou avec des fichiers, elles offrent des consoles pour effectuer des opérations sur la base et sont très légères en termes d'occupation mémoire ou d'espace disque.

Nous travaillerons de préférence avec H2 qui apporte quelques avantages précieux comme le support de compatibilité SQL avec Oracle, DB2 et MySQL. Spring utilise en natif HSQLB pour nous aider à faire nos tests unitaires, mais nous verrons comment utiliser H2 à la place.

2. Mise en œuvre

a. Installation

Téléchargez la base de données à l'adresse <http://www.h2database.com/> et installez-la.

La console texte ou web est disponible via le lien dans le menu **Démarrer** ou en ligne de commande.

La documentation sur le site de la distribution est très complète.

b. Configuration du POM

Pour pouvoir utiliser H2 dans un projet mavenisé, il faut ajouter la dépendance suivante dans le fichier pom.xml.

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <version>1.4.187</version>
</dependency>
```

Les versions évoluent souvent. Il faut utiliser une version récente car d'anciennes versions géraient une partie des codes d'erreur SQL.

H2 nous permet de tester tout un système qui utilise une base complexe mais sans cette base, en l'émulant en mémoire.

c. Connexion à la base de données en utilisant JDBC

Nous pouvons nous connecter à la base de données, depuis le code Java, via le code suivant :

```
public class ConnectH2 {
    public static void main(String... args) throws Exception {
        //On efface la base
        DeleteDbFiles.execute("~", "test", true);
        //Connexion
        Class.forName("org.h2.Driver");
```

```

        Connection connection =
DriverManager.getConnection("jdbc:h2:~/test");
        Statement statement = connection.createStatement();
        //Création table
        statement.execute("create table test(id int primary key,
name varchar(255))");
        //Insertion d'une ligne
        statement.execute("insert into test values(1, 'ENI
Editions')");
        //Lecture de la ligne
        ResultSet rs;
        //Affichage
        rs = statement.executeQuery("select * from test");
        while (rs.next()) {
            System.out.println(rs.getString("name"));
        }
        //Fermeture Statement
        statement.close();
        //Fermeture base
        connection.close();
    }

```

Dans cet exemple de code, nous commençons par effacer les traces qui resteraient de la dernière utilisation de cette base par la commande `DeleteDbFiles.execute` mais en général nous n'aurons pas à faire cette opération.

Nous chargeons ensuite le driver et nous créons une connexion.

Nous créons ensuite une table et nous insérons une ligne que nous lisons ensuite pour l'afficher.

Nous fermons alors la connexion à la base de données.

d. Utiliser un listener de servlet pour démarrer et arrêter la base

Nous pouvons configurer le fichier `web.xml` du projet web pour gérer la base :

```

<web-app>
  <display-name>Archetype Created Web Application</display-name>
  <context-param>
    <param-name>db.url</param-name>
    <param-value>jdbc:h2:~/test</param-value>
  </context-param>
  <context-param>
    <param-name>db.user</param-name>
    <param-value>sa</param-value>
  </context-param>
  <context-param>
    <param-name>db.password</param-name>
    <param-value></param-value>
  </context-param>
  <context-param>
    <param-name>db.tcpServer</param-name>
    <param-value>-tcpAllowOthers</param-value>
  </context-param>
  <listener>
    <listener-class>org.h2.server.web.DbStarter</listener-class>
  </listener>
</web-app>

```

Ceci permet de facilement avoir une base de données dans un site web Java.

Nous utilisons peu cette possibilité car nous laisserons généralement la gestion de la base de données à Hibernate ou JPA. De même, comme nous le verrons dans le chapitre sur les tests, les librairies de tests unitaires prennent également en charge la gestion de la base de données H2.

Projets Maven

Le framework Java offre de base un JDK très fourni en API ce qui permet de créer facilement des librairies et de les partager. De nombreuses librairies ont vu le jour.

Ces librairies utilisent d'autres librairies et nous nous retrouvons avec des dépendances. Les choses se compliquent quand nous avons par exemple deux librairies qui utilisent toutes deux d'autres librairies, mais dans des versions différentes. Le jeu consiste alors à trouver la bonne combinaison. L'ordre de chargement importe beaucoup aussi, car dans les serveurs JEE les librairies du serveur sont prioritaires sur les librairies de l'application.

Il est bien sûr possible d'inverser la priorité en chargeant en priorité les librairies de l'application avant celles du serveur, mais nous perdons alors le support du fournisseur du serveur d'application. Nous avons généralement un répertoire pour les librairies du serveur et un répertoire pour les librairies de chaque projet.

1. Description de la problématique

Au-delà d'un certain nombre de projets, une gestion des dépendances devient rapidement indispensable. De nombreux outils de gestion de dépendances essaient de répondre à ces problématiques. Nous utilisons au départ des makefiles qui furent remplacés par Ant (<http://ant.apache.org/> : un makefile évolué) qui fut à son tour remplacé par Maven. Gradle (<https://gradle.org/>) essaie aujourd'hui de surpasser Maven mais il n'est pas encore assez démocratisé. Malgré le fait que le framework Spring utilise Gradle pour ses dépendances, nous utiliserons Maven pour les exemples du livre afin de rester en phase avec le gestionnaire de dépendance dominant.

2. Mise en œuvre

Maven est compatible entre Windows, Mac et Unix (Linux). Il permet comme un makefile de produire des binaires et accessoirement de les packager à partir d'un ensemble de fichiers sources. Il utilise un concept nommé POM (*Project Object Model*) qui décrit un projet modulaire et les étapes nécessaires pour sa compilation, son test, son packaging, son installation et son déploiement.

Maven centralise dans des espaces de stockage partagés, appelés repositories publics sur Internet, la majorité des versions des projets open source qui sont en dépendance. Il met à disposition de notre projet, via un dépôt local, une copie des fichiers nécessaires à partir du nom de la dépendance et de sa version et indique au projet où trouver ses dépendances sur le dépôt local.

Il est également possible de sauvegarder les versions de ses propres modules dans le dépôt local afin de les partager entre nos différents projets.

Les projets et les sous-projets sont configurés via le fichier pom.xml qui se trouve dans le répertoire racine du projet. Ce fichier décrit les dépendances ainsi que des informations sur le projet. Il est possible de faire une hiérarchie de configuration POM, les propriétés des POM parents dans la hiérarchie étant alors surchargées dans les POM enfants. Il est également possible d'externaliser certains éléments liés au paramétrage afin de pouvoir personnaliser la configuration en fonction des environnements d'exécution de l'application.

Afin de simplifier le propos, nous ne verrons que l'utilisation simplifiée qui répond à la problématique d'illustration des possibilités de Spring. Nous n'utiliserons pas le mécanisme des POM parents afin de donner de la visibilité et une facilité de compilation et de tests des exemples.

3. Installation de Maven sous Windows

Ce chapitre décrit l'installation sur Windows. Pour les autres plates-formes, il faut suivre les recommandations d'installation correspondant à votre système sur la page décrivant l'installation de Maven.

→ Téléchargez et installez Maven comme indiqué sur le site Apache de Maven : <http://maven.apache.org/download.cgi>

- Éditez le fichier de configuration Maven settings.xml si vous voulez changer l'emplacement du repository local. Par défaut, Maven positionne le repository dans un répertoire .m2 dans le répertoire correspondant à votre utilisateur. Il faut parfois le faire pointer vers un espace disque plus grand car il devient vite volumineux.
- Faites pointer la variable M2_HOME et M3_HOME vers le répertoire d'installation de Maven.
- Modifiez le path pour qu'il ait le chemin %M3_HOME%\bin.
- Mettez dans MAVEN_OPTS les paramètres JVM, par exemple -Xms256m -Xmx512m pour Java 7.
- Ouvrez une fenêtre DOS ou Git Bash pour vérifier que la commande Maven répond bien : `mvn --version`

Comme nous l'avons évoqué, Maven utilise pour la configuration du package un fichier pom.xml. Ce dernier décrit les dépendances et la façon de compiler, de tester et de packager l'application.

Nous verrons le contenu de ce fichier un peu plus loin.

4. Utilisation d'un archétype Maven

Maven peut aussi être utilisé pour créer un squelette applicatif très simplement.

Nous pouvons par exemple créer un projet simple avec la commande suivante :

```
mvn archetype:generate -DgroupId=simple -DartifactId=simple
-DpackageName=fr.eni -Dversion=1.0-SNAPSHOT
```

Maven est alors en mode interactif et pose quelques questions auxquelles il suffit de répondre en validant par la valeur par défaut.

Maven crée alors un projet "hello world" très simple avec le fichier pom.xml associé.

La commande suivante le compile et lance le programme :

```
$ mvn exec:java -Dexec.mainClass="simple.App"
[INFO] Scanning for projects...
[INFO]
[INFO] -----
[INFO] Building chap03-simple 1.0-SNAPSHOT
[INFO] -----
[INFO]
[INFO] --- exec-maven-plugin:1.4.0:java (default-cli) @ chap04-simple ---
Hello World!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.420 s
[INFO] Finished at: 2015-07-21T13:31:25+02:00
[INFO] Final Memory: 10M/309M
[INFO] -----
```

La commande `mvn site` génère une documentation complète du projet Maven dans le répertoire target/site.

5. Contenu du fichier pom.xml dans les cas simples utilisés dans ce livre

Voici le contenu du fichier pom.xml de l'application simple que nous venons de générer, ajusté avec quelques informations supplémentaires :

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>simple</groupId>
  <artifactId>chap05-simple</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <name>chap04-simple</name>
  <url>http://maven.apache.org</url>
  <properties>
    <java-version>1.8</java-version>
    <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
    <org.springframework-
version>4.2.0.RELEASE</org.springframework-version>
    <spring.version>4.2.0.RELEASE</spring.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.12</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
  <build>
    <finalName>${project.artifactId}</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>2.3.2</version>
        <configuration>
          <source>${java-version}</source>
          <target>${java-version}</target>
        </configuration>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>2.18.1</version>
        <configuration>
          <includes>
            <include>/**/*.Tests.java</include>
          </includes>
          <excludes>
            <exclude>/**/*.Abstract*.java</exclude>
          </excludes>
          <junitArtifactName>junit:junit</junitArtifactName>
          <argLine>-Xmx512m</argLine>
        </configuration>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-dependency-plugin</artifactId>
        <executions>
          <execution>
            <id>install</id>
            <phase>install</phase>
            <goals>
              <goal>sources</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>

```

Par rapport au projet généré, nous ajoutons le fait que nous souhaitons utiliser Java 8 et l'UTF-8 (en gras dans le code ci-dessus). Nous indiquons aussi que nous utilisons Spring dans le projet.

Points clés

- Le framework Spring est massivement orienté design pattern.
- Il faut identifier les design patterns pour les signaler dans les commentaires du code.
- Les méthodes `equals` et `hashCode` doivent être adaptées pour une utilisation avec Spring, Hibernate et JPA.
- Nous utilisons le projet Lombok pour économiser du code dans les exemples.
- Nous utilisons Logback pour avoir de bons logs.
- La base H2 est retenue dans les exemples pour sa simplicité d'utilisation.
- Nous utilisons Maven dans sa forme la plus simple pour les exemples.

Introduction

Ce chapitre présente une utilisation simpliste de Spring afin que nous ayons un aperçu de Spring sans nous perdre dans les détails. Nous verrons par la suite des éclairages sur des parties qu'il vaut mieux connaître pour bien se repérer dans une application plus complexe. L'exemple qui l'illustre permet déjà d'expérimenter une grande partie des problématiques relatives à l'utilisation de ce framework.

Le principal intérêt de Spring est l'instanciation et la mise à disposition automatisée de beans. Ces objets peuvent être de deux types : des singletons, comme nous l'avons déjà évoqué, mais aussi des objets « dupliqués » nommés « prototypes ».

L'objet `singleton` est un objet partagé instancié une seule fois dans un même conteneur Spring et dont l'utilisation est partagée. Spring gère en interne une liste des singletons instanciés. Si un bean membre d'un bean est à injecter et qu'il existe déjà, Spring copie sa référence, sinon il l'instancie et le met dans sa liste. Il l'injecte alors automatiquement.

Un objet `prototype` est un objet qui est instancié à chaque fois que Spring l'injecte dans le membre d'un objet Spring. Tous les mécanismes de facilitation offerts par Spring sont disponibles pour les objets `prototype` mais l'utilisation de ce type d'objets est relativement rare.

Spring permet également de contrôler le cycle de vie et offre la possibilité d'intercepter les appels des méthodes des objets managés afin d'en prendre le contrôle.

Il existe deux façons de configurer les beans. Ces typologies de configuration sont interchangeables et il est possible de les mixer : la configuration peut être implicite ou explicite. Dans le premier cas, Spring découvre les beans au lancement du conteneur en parcourant les classes. Dans le second cas, Spring ne prend en compte que l'emplacement et la fonction exacts des beans ainsi déclarés.

Les contextes de Spring

L'espace mémoire dans lequel Spring range ses éléments de configurations et les références sur les objets gérés se nomme le contexte. Si nous souhaitons faire des opérations en utilisant Spring, il nous faut obligatoirement un contexte.

Il existe deux aspects complémentaires : le premier est la configuration des beans qui indique quels objets Java sont à considérer comme des beans Spring et le second concerne l'injection de ces beans dans les variables de classes ou de méthodes, c'est-à-dire l'utilisation des beans dans le projet. Ces notions se « mélangent » à un moment donné, car nous pouvons référencer des beans dans d'autres beans dans la configuration.

Il faut garder à l'esprit qu'il existe bien un aspect déclaration séparé d'un aspect utilisation, comme nous le verrons en détail par la suite.

Configuration des beans

Comme nous l'avons évoqué, la configuration des beans Spring peut être implicite ou explicite. En mode implicite, Spring reconnaît de lui-même les classes Java à ajouter à la liste des beans à partir d'un balisage sur les candidats. En mode explicite, nous indiquons à Spring le nom et l'emplacement exacts des beans. Le mode explicite est plus sécurisé mais il est plus long à configurer. C'est pourquoi il est rarement utilisé. Les beans sont configurables par l'utilisation de fichiers XML, d'annotations ou directement en Java.

Possibilité de déclaration des beans

Type	Mode	Utilisation
Fichiers XML	Explicite	Beans personnalisés de façon externe pour une variation par environnement.
Code Java	Implicite et explicite	Beans de configurations statiques.
Annotations	Implicite	Beans subissant peu de changements structurels.

Nous pouvons donc par ce biais déclarer un arbre d'objets contenant des objets singletons et des objets prototypes. Nous verrons qu'il est aussi possible d'avoir un fichier spécial pour les tests unitaires qui complète alors le fichier principal.

La configuration par XML présente des difficultés liées à l'utilisation de configuration au format texte. Spring doit en effet faire des conversions du texte vers le type cible. Pour indiquer un nombre, nous le mettons dans une chaîne, qui est ensuite convertie. Pour indiquer le nom d'une variable ou d'un paramètre, nous utilisons aussi du texte. Les problèmes ne se voient alors qu'à l'exécution alors qu'en configurant en Java, les problèmes se révèlent directement lors de la phase de compilation.

Nous utiliserons donc en priorité la configuration Java. Les exemples montrent les deux types afin de les présenter.

1. Fichier XML

Les applications simples ou anciennes sont configurées directement dans un ou plusieurs fichiers XML organisés de façon hiérarchique. Nous avons en général un fichier XML principal par conteneur Spring, accompagné de fichiers XML périphériques.

L'exemple suivant montre comment déclarer un bean `monService` qui correspond à la classe d'implémentation `monPackage.MonServiceImpl`.

```
<Beans>
  <Bean id="monService" class="monPackage.MonServiceImpl"/>
</Beans>
```

Si l'interface de la classe du bean est `monPackage.MonService`, nous indiquerons à Spring que nous voulons créer un singleton nommé `monService`. Ce singleton est alors injectable dans un autre bean, c'est-à-dire que nous pouvons demander à Spring de recopier la référence sur cet objet dans un membre d'autre objet de notre choix :

```
<bean id="monConteneur" class="monPackage.MonConteneurImpl">
  <property name="service" ref="monService" />
</bean>
```

Ce nouveau singleton `monConteneur` qui a pour implémentation `MonConteneurImpl` contiendra comme membre l'objet `service` qui contiendra le singleton précédemment déclaré.

Nous pouvons indiquer à Spring de découvrir automatiquement les classes Java candidates pour devenir des beans Spring en indiquant dans le fichier XML la liste des packages à scanner :


```
<context:component-scan base-package="monPackage" />
```

Il est possible d'utiliser une expression régulière pour ne pas avoir à lister à la main les packages. Le fonctionnement est alors dépendant de conventions de nommage qu'il faudra bien documenter. En cas d'erreur, nous pouvons avoir des beans Spring qui n'étaient pas candidats, par exemple des beans destinés aux tests et qui se retrouvent tout de même en production. À ce niveau, la confiance indique qu'il faudra se doter de moyen de contrôle afin d'éviter certains abus.

2. Annotations pour configurer l'application

Cette évolution récente de Spring permet d'avoir une façon plus souple. Nous pouvons déclarer la configuration des beans Spring d'une application directement en Java.

Nous aurons souvent, en complément de la configuration Java, un fichier de configuration XML, mais ce dernier sera considérablement allégé. Nous veillerons à regrouper, autant que possible, les éléments de configuration dans un même package.

L'utilisation des annotations pour configurer l'équivalent de l'exemple précédent nous donne le code suivant :

```
package monPackage;
@Configuration
public class AppConfig {
    @Bean
    public MonService myService() {
        return new MonServiceImpl();
    }
}
```

Cette déclaration indique à Spring que le bean ayant l'id `monService` a comme classe d'implémentation la classe `MonServiceImpl` du package `monPackage`. Nous qualifions le type d'objet retourné par son interface. Ceci précise que tout objet déclaré par cette interface peut recevoir par injection ce service.

Nous contrôlons également le processus d'instanciation de l'objet, ce qui nous permet de personnaliser cette création de l'objet. Nous voyons donc que cette façon de procéder est bien plus puissante que la simple utilisation des XML. Nous pouvons aussi, en cas de problème, tracer et déboguer le processus de création du contexte applicatif.

Liste des principales annotations pour configurer les beans à partir des annotations :

Annotation	Effets
@Configuration sur la classe	La classe annotée @Configuration correspond à une usine à beans (design pattern factory).
@Bean sur la méthode	La méthode annotée @Bean remplace l'élément de configuration XML <Beans/> du fichier de configuration XML.
@ComponentScan	Nous pouvons utiliser l'annotation @ComponentScan en conjonction avec l'annotation @Configuration pour indiquer à Spring qu'il doit scanner les packages en ayant pour base la classe qui porte l'annotation @Component.

L'annotation @ComponentScan :

```
@ComponentScan(basePackages={"monPackage"});
```

Nous pouvons affiner pour spécifier les packages en paramètre de l'annotation :

```
@ComponentScan(basePackages={"package1", "package2"})
```

3. Configurer les beans applicatifs implicitement

Nous pouvons donc nous baser sur une configuration XML ou Java pour configurer les beans automatiquement en utilisant une facilité de Spring qui permet de découvrir les beans en se basant sur un package de base et d'expressions régulières.

La déclaration implicite est la façon la plus simple de configurer les beans applicatifs. Elle permet de ne pas les déclarer un par un. Nous spécifions dans le fichier de configuration XML, ou dans la configuration Java, une règle basée sur une expression régulière pour identifier des beans Spring balisés par des annotations.

L'annotation associée au bean le plus simple est `@Component`. Il est possible d'utiliser la nouvelle implémentation de la JSR-330 : `javax.inject` qui apporte des annotations dont le fonctionnement est proche de celui de Spring car Spring s'est interfacé avec ces nouvelles librairies mais ces annotations sont moins avancées que celles de Spring.

Nous choisirons dans les cas simples les annotations de la JSR-330 `@Inject` et nous utiliserons l'annotation Spring `@Autowired` pour les projets complexes car l'attribut `required` n'existe pas pour `@Inject`.

Annotation	Utilité
@Component	Annotation générique pour tous les beans.
@Repository	Bean de type DAO pour la couche de persistance.
@Service	Bean de type Service pour la couche service.
@Controller	Contrôleur MVC pour la couche de présentation SpringMVC.

Il existe aussi d'autres annotations qui indiquent si le bean est sujet à l'AOP ou à des préoccupations de transactionnalités.

Utilisation des beans : injection par setters et constructeurs

Le bean devient disponible pour l'application une fois que le bean est déclaré dans la configuration. Nous pouvons alors l'injecter comme membre de notre classe.

Spring ne peut injecter des dépendances que dans un bean Spring.

➤ Les connaisseurs vous diront qu'il est possible, en paramétrant la JVM, d'injecter les beans où on le souhaite, en modifiant le class loader, mais dans un contexte d'utilisation normal, nous nous limiterons aux beans Spring.

Le premier bean de notre application sera donc obligatoirement instancié par une factory Spring qui chargera le contexte et appliquera la configuration à ce bean.

Spring crée l'objet avec le constructeur par défaut et appelle ensuite les setters pour injecter les références des objets à injecter. Si le constructeur par défaut n'est pas disponible, Spring utilise alors un constructeur avec arguments ce qui permet aussi de renseigner les dépendances. Cette possibilité est utilisée pour les API externes à Spring qui ne disposent pas de constructeurs par défaut comme les pools de connexion pour lesquels nous devons passer la datasource en paramètre de constructeur.

L'annotation `@Autowired` est mise sur le setter de la variable. Il est possible de spécifier que la résolution peut être différée au démarrage via le paramètre `required=false` :

```
@Autowired(required=false)
```

Ceci permet de préciser à Spring que le bean à mapper peut ne pas être disponible au démarrage.

Dans le cas pour lequel nous n'aurions pas besoin d'utiliser le chargement différé, il est aussi possible d'utiliser le `@Inject` de la JSR-330.

1. Mapping via le constructeur en XML

On utilise l'élément `<constructor-arg>` ou la technologie `c-namespace` plus compacte introduite dans Spring 3.

```
<Bean id="disquette" class="lecteur.DISQUETTE">
  <constructor-arg ref="format" />
</Bean>
```

ou :

```
<Bean id=" disquette " class="lecteur.DISQUETTE"
c:cd-ref="format" />
```

Pour le « `c-namespace` », il est possible de déterminer le nom de la variable du constructeur par son nom ou par son indice avec un indice implicite s'il n'y a qu'un argument.

Syntaxe	Déterminant
<code>c:_nomdelavariante="valeur"</code>	un nom
<code>c:_0="valeur"</code>	un indice
<code>c:_="valeur"</code>	Implicite car il y a un seul paramètre

Spring traduit automatiquement du type `String` vers le type possible détecté dans la liste des constructeurs disponibles car la valeur est en `string` dans le fichier de configuration. Le transtypage de type est risqué car il peut être la source de nombreuses erreurs, c'est pourquoi l'équivalent en Java de la configuration XML est

beaucoup plus fiable.

Il est possible d'avoir des mappings avec des constructeurs ayant des paramètres de type collection. Le bean peut être instancié et ses valeurs positionnées de façon automatique. Ceci est très pratique pour, par exemple, créer une datasource préconfigurée.

```
<Bean id="dataSource"
  class="org.apache.commons.dbcp.BasicDataSource" destroy-
method="close"
  p:driverClassName="oracle.jdbc.driver.OracleDriver"
  p:url="jdbc:oracle:thin:@localhost:1521:xe"
  p:username="MonUtilisateurDeTest"
  p:password="LeMotDePasseDeMonUtilisateurDeTest"
/>
```

2. Comparaison des méthodes d'instanciation

Avantages	Par setter	Par constructeur	Par champ
JavaBean	Oui	Non	Non
Héritage	Oui	Non	Non
Clarté	Haute	Moyenne	Basse
Souplesse	Haute	Basse	Haute
Concision	Basse	Haute	Très haute
Faiblesse	Tests unitaires	Aucune	Aucune

➤ On utilisera l'injection par setters dans la majorité des cas. Certaines instanciations nécessitent des paramètres dans les constructeurs, on utilise alors l'injection par constructeurs. L'injection par champs est à éviter car elle est moins maintenable mais elle est parfois nécessaire pour certains frameworks techniques et se fait alors explicitement dans le fichier XML pour plus de clarté.

Contrôle du cycle de vie : construction et destruction

Spring permet d'intercepter la création et la description des objets Spring.

Il n'est pas possible d'utiliser les beans injectés directement dans le constructeur car à ce moment précis, Spring n'a pas encore instancié les objets contenus dans les membres de la classe. De même, l'appel à la méthode `Finally` n'est pas simple car elle est appelée par le garbage collector. Spring donne la possibilité de marquer des méthodes pour qu'elles soient appelées à la création et à la destruction de l'objet. Il existe plusieurs mécanismes. Voici le plus récent.

Annotation	Moment de l'appel
@PostConstruct	Après le constructeur, avec les beans initialisés.
@PreDestroy	Avant la destruction.

Il est aussi possible d'utiliser les aspects de la programmation orientée aspect (AOP) pour effectuer des actions lors de la création et la destruction de l'objet.

Nous utilisons avant ces annotations des méthodes paramétrées dans la configuration avec les paramètres `init-method` et `destroy-method` comme dans l'exemple suivant :

```
<bean id="helloWorld"
  class="com.tutorialspoint.HelloWorld"
  init-method="init" destroy-method="destroy">
  <property name="message" value="Bonjour!"/>
</bean>
```

Cette façon de procéder fonctionne toujours mais il est plus simple d'utiliser les annotations.

Exemple illustrant les mappings standards

Cet exemple présente, pas à pas, la création d'un projet simple. Il est présent dans les exemples téléchargeables depuis la page Informations générales. Ce chapitre peut paraître un peu complexe aux débutants de prime abord, mais pourra être réutilisé pour expérimenter.

1. Le projet Maven

→ Générez un projet Maven vide avec l'archétype suivant :

```
mvn archetype:generate -DgroupId=fr.eni.editions
-DartifactId=mappingApp -DarchetypeArtifactId=maven-archetype-quickstart
-DinteractiveMode=false
```

→ Importez le fichier Maven dans Eclipse ou IntelliJ Idea (ou dans votre outil de développement préféré).

→ Afin de séparer les fichiers de configuration des fichiers de code, il faut créer les répertoires de sources :

- src/main/resources
- src/test/resources

Nous mettrons dans ces répertoires les fichiers de configuration de Spring et du système de log.

2. Fichier de configuration de Spring

Nous allons créer un fichier Spring nommé applicationContext.xml et le placer dans le répertoire src/main/resources/Spring/.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans default-lazy-init="true"
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:jdbc="http://www.springframework.org/schema/jdbc"
xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-4.1.xsd
    http://www.springframework.org/schema/jdbc
    http://www.springframework.org/schema/jdbc/spring-jdbc-4.1.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context-4.1.xsd">
    <context:component-scan base-package="fr.eni" />
</beans>
```

Nous indiquons dans ce fichier que les beans annotés à prendre en considération sont dans le package "fr.eni" et dans tous ses sous-packages.

3. Dépendance Spring Core et système de log

Spring est autonome au niveau des dépendances à l'exception du framework de log de façon à pouvoir utiliser le système de log du projet embarquant le moteur Spring.

Spring utilise par défaut le système commons-logging du Spring Core. De façon à avoir son propre système de logging, il faut exclure le common-logging pour le remplacer par Logback car il est en effet, comme nous l'avons vu dans le chapitre sur le système de log, bien plus pratique et bien plus puissant que Log4j.

Nous utiliserons SLF4J (<http://www.slf4j.org/>) pour s'interfacer avec Logback dans les versions suivantes :

- SLF4J en version 1.4.8.
- Logback en version 1.1.12.
- Spring Core en version 4.2.2.RELEASE.

Ce qui donne les dépendances suivantes dans le fichier pom.xml :

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <version>4.2.2.RELEASE</version>
  <exclusions>
    <exclusion>
      <groupId>commons-logging</groupId>
      <artifactId>commons-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

4. Dépendance vers les bibliothèques de tests unitaires

Nous verrons les spécificités des tests unitaires et des tests d'intégration dans un chapitre dédié. En effet Spring est très pratique pour faire des tests. Nous utiliserons JUnit en conjonction de org.easymock pour cet exemple. Nous indiquons, en utilisant le scope `<test>`, que les dépendances de ce scope ne seront présentes que lors des tests.

Extrait du fichier pom.xml :

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.12</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.easymock</groupId>
  <artifactId>fest-assert</artifactId>
  <version>1.4</version>
</dependency>
```

Les artefacts Spring-test et Spring-context sont également à intégrer en scope test. Ils sont en effet spécifiques aux tests :

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-test</artifactId>
  <version>4.2.0.RELEASE</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>4.2.0.RELEASE</version>
  <scope>test</scope>
</dependency>
```

5. Exemple illustrant l'utilisation des logs

Nous testons cette configuration en ajoutant de simples logs dans la classe de test `AppTest` :

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(locations = {
  "classpath*:Spring/applicationContext-test.xml" })
```

```

public class AppTest
[...]
```

```

    @Test
    public void testApp(){
        logger.info("Basic info Spring Test");
        logger.debug("Basic debug Spring Test");
        simpleService.printHello();
        assertThat( true );
    }

```

Il ne nous reste plus qu'à modifier le fichier de configuration des logs logback.xml dans le fichier src/tests/resources :

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>
[...]
```

```

    <logger name="fr.eni.editions" additivity="false">
        <level value="DEBUG" />
        <appender-ref ref="consoleAppender" />
    </logger>
</configuration>

```

6. Fichier de configuration spécifique pour les tests

Nous allons créer un fichier de configuration Spring applicationContext-test.xml pour les tests dans /src/test/resources/Spring.

```

<?xml version="1.0" encoding="UTF-8"?>
<Beans default-lazy-init="true"
xmlns="http://www.Springframework.org/schema/Beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.Springframework.org/schema/context"
xmlns:jdbc="http://www.Springframework.org/schema/jdbc"
xsi:schemaLocation="
    http://www.Springframework.org/schema/Beans
http://www.Springframework.org/schema/Beans/Spring-Beans-4.1.xsd
    http://www.Springframework.org/schema/jdbc
http://www.Springframework.org/schema/jdbc/Spring-jdbc-4.1.xsd
    http://www.Springframework.org/schema/context
http://www.Springframework.org/schema/context/Spring-context-4.1.xsd">
    <import resource="classpath*:Spring/applicationContext.xml" />
</Beans>

```

Ce fichier ressemble au fichier principal. Nous nous contentons pour le moment d'importer le fichier principal. Nous pourrions ajouter dans ce fichier les éléments de configuration spécifiques aux tests.

La trace suivante sera générée lors de l'exécution du programme :

```
15:45:56.278 [main] INFO fr.eni.editions.AppTest - Basic Spring Test
```

Il est possible de voir l'intégralité des logs de debug de Spring en décommentant l'appender des logs Spring : le fichier de log généré fait alors plus de cent lignes.

Un bon exercice consistera à lire en détail ce log.

Il est possible de charger la javadoc et les sources des dépendances Maven depuis la configuration Maven du projet. Nous pouvons alors suivre pas à pas l'exécution des méthodes du moteur Spring pour en découvrir le fonctionnement interne. Ceci est un très bon exercice pour voir une partie du fonctionnement interne du framework.

 Si cela vous intéresse, vous pouvez télécharger les sources de Spring pour en faire un projet indépendant que vous mettrez en dépendance de votre projet. Ceci vous permettra alors de fouiller le code, de mettre des points d'arrêt, etc.

Points clés

- Le framework est configurable en Java ou via des fichiers XML.
- Nous utiliserons de préférence la configuration Java pour les nouveaux projets.
- Les beans sont injectés grâce aux constructeurs ou par les setters.
- Nous pouvons améliorer le système de log par défaut.
- Il est très facile de faire des tests avec Spring.

Introduction

Nous traitons dans ce chapitre des éléments plus complexes de Spring. Nous commencerons par les ressources, nous continuerons par les convertisseurs et les formateurs, et nous terminerons par les bindeurs et les validateurs.

Ces éléments sont utilisés dans un contexte d'application standalone et dans un contexte Web.

Fichier XML et fichiers properties

Nous pouvons utiliser des fichiers `.properties` afin de substituer des valeurs dans le fichier de configuration XML. Cela permet de personnaliser la configuration en fonction d'éléments externes.

Nous utilisons pour cela la classe `PropertyPlaceholderConfigurer` dans le fichier de configuration Spring :

```
<bean
  class="org.springframework.beans.factory.config.PropertyPlacehold
erConfigurer">
  <property name="location" value="jdbc.properties" />
</bean>
```

Il est possible de spécifier plusieurs fichiers.

Spring remplacera alors les expressions entre accolades en cherchant les valeurs de substitution des propriétés dans les fichiers `.properties` signalés.

Par exemple, nous pourrions avoir pour un bean de datasource :

```
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource"
  destroy-method="close">
  <property name="driverClassName" value="${jdbc.driverClass}" />
  <property name="url" value="${jdbc.url}" />
  <property name="username" value="${jdbc.username}" />
  <property name="password" value="${jdbc.password}" />
</bean>
```

Avec le fichier `jdbc.properties` associé :

```
jdbc.driverClass=org.h2.Driver
jdbc.url=jdbc:h2:~/test5
jdbc.username=sa
jdbc.password=
```

Cette façon de procéder est très pratique et permet d'utiliser un jeu de fichiers de propriétés en fonction des environnements.

Les fichiers de ressources

Une ressource est, par exemple, un fichier texte. Nous verrons dans un premier temps l'utilisation normale des fichiers de ressources en Java. Nous verrons ensuite comment charger un fichier de ressources avec Spring via le contexte Spring. Nous verrons, enfin, comment utiliser le `ResourceLoaderAware`.

1. Les fichiers de ressources standards

Les fichiers et les URL désignant un fichier peuvent être lus de façon standard comme dans cet exemple :

```
@Log
public class Ressources1 {
    public static void traiterRessource(Resource resource) {
        try {
            InputStream is = resource.getInputStream();
            BufferedReader br = new BufferedReader(new
InputStreamReader(is));
            String ligne;
            while ((ligne = br.readLine()) != null) {
                log.info(ligne);
            }
            br.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) throws IOException {
        Resource r0 = new
FileSystemResource("src/main/resources/fr/eni/editions/ressource.txt");
        traiterRessource(r0);

        String
fichier=System.getProperty("user.dir")+"/src/main/resources/fr/eni/
editions/ressource.txt";
        log.info("Fichier:"+fichier);

        Resource r1 = new UrlResource("file:"+fichier);
        traiterRessource(r1);
        Resource r2 = new
ClassPathResource("fr/eni/editions/ressource.txt");
        traiterRessource(r2);
    }
}
```

Nous avons une méthode `traiterRessource` dans la classe `Ressources1` avec comme paramètre `resource` de type `Resource`. Cette méthode traite la ressource et sera reprise dans les exemples suivants.

La première partie ouvre le fichier de l'exemple avec un `FileSystemResource` puis le traite par la méthode `traiterRessource`.

La seconde partie de l'exemple ouvre le fichier de l'exemple via un `UrlResource`, puis le traite par la méthode `traiterRessource`.

Le premier est donc un fichier et le second une URL qui pointe sur un fichier.

La classe `UrlResource` permet aussi d'agir sur des ressources d'autres types comme des flux HTTP, etc.

2. Les fichiers de ressources chargés via le contexte

Nous pouvons utiliser les facilités de Spring pour charger des ressources.

Nous réutilisons le traitement de ressources du premier exemple.

```

@Log
public class Ressources2 {

    public static void traiterRessource(Resource resource) {
[...]
```

```

    }

    public static void main(String[] args) {
        ApplicationContext ctx = new
ClassPathXmlApplicationContext(
        new String[] { "applicationContext.xml" });
        String rep=System.getProperty("user.dir");
        log.info("Rep courant:"+rep);

        Resource r0 = ctx

.getResource("file:"+rep+"/src/main/resources/fr/eni/editions/ressource.txt");
        traiterRessource(r0);
        Resource r1 = ctx

.getResource("classpath:fr/eni/editions/ressource.txt");
        traiterRessource(r1);
        Resource r2 = ctx

.getResource("url:http://echo.jsonest.com/key/value/one/two");
        traiterRessource(r2);

    }

```

La méthode `getResource` du `ApplicationContext` permet de charger tout type de ressources en spécifiant le protocole File ou HTTP. Nous utilisons la même méthode quel que soit le type de ressources, le code est plus simple.

3. Les fichiers de ressources chargés via un service `ResourceLoaderAware`

Nous pouvons utiliser les possibilités de Spring pour charger des ressources et les exposer dans un service. Le service implémentant `ResourceLoaderAware` expose alors deux méthodes :

```
public void setResourceLoader(ResourceLoader resourceLoader)
```

et

```
public Resource getResource(String location)
```

Spring injecte automatiquement la ressource dans le bean et la met à disposition via la méthode `getResource`.

Nous réutilisons le traitement de ressources du premier exemple pour traiter la ressource.

a. Le programme principal

```

@Log
public class Ressources3 {

    public static void traiterRessource(Resource resource) {
[...]
```

```

    }

    public static void main(String[] args) {
        ApplicationContext ctx = new
ClassPathXmlApplicationContext(
        new String[] { "applicationContext.xml" });

```

```

    RessourceHelperService svc =
(RessourceHelperService)ctx.getBean("ressourceHelperService");

    String rep=System.getProperty("user.dir");
    log.info("Rep courant:"+rep);

    Resource r0 = svc.getResource("file:"+rep+"/src/main/resources/
fr/eni/editions/ressource.txt");
    traiterRessource(r0);
    Resource r1 = svc

    .getResource("classpath:fr/eni/editions/ressource.txt");
    traiterRessource(r1);
    Resource r2 = svc

    .getResource("url:http://echo.jsonest.com/key/value/one/two");
    traiterRessource(r2);
}

```

La création du bean par la fabrique Spring déclenchera le chargement automatique de la ressource.



Le site <http://echo.jsonest.com> est très pratique pour expérimenter du JSON.

b. Le service ResourceLoaderAware

```

public class RessourceHelperService implements
ResourceLoaderAware
    private ResourceLoader resourceLoader;
    public void setResourceLoader(ResourceLoader resourceLoader) {
        this.resourceLoader = resourceLoader;
    }
    public Resource getResource(String location){
        return resourceLoader.getResource(location);
    }
}

```

Ce service Spring permet de centraliser dans un Singleton le chargement de la ressource. Il n'est ainsi chargé qu'une fois au lancement de l'application.

Convertisseurs et formateurs

Spring manipule souvent les valeurs à travers l'utilisation des chaînes de caractères. C'est le cas quand nous avons une configuration dans un fichier texte, par exemple un fichier XML. Ce principe est également utilisé quand Spring échange des données entre les différentes parties d'une application comme dans le cas de Spring MVC pour lequel les données en provenance et à destination des vues, que ce soient des pages JSP ou des pages JSF, circulent au format texte.

1. Built-in converters

Un built-in converter est un convertisseur intégré. Nous utilisons la classe `SpringGenericConversionService` pour disposer des convertisseurs standards. Cette classe permet de disposer d'un service de conversion qu'il est possible d'augmenter par des convertisseurs personnalisés.

L'exemple suivant montre comment utiliser un `DefaultConversionService` qui implémente la classe généraliste `GenericConversionService`.

Déclaration du convertisseur générique :

```
GenericConversionService cs = new DefaultConversionService();
```

a. Pour les types standards

Pour les types standards, nous utiliserons les convertisseurs disponibles.

L'exemple suivant présente la conversion suivie du test unitaire associé :

```
Boolean boolean = cs.convert("true", Boolean.class);
assertTrue(boolean);

boolean = cs.convert("no", Boolean.class);
assertFalse(boolean);

Character car = null;
car = cs.convert("Z", Character.class);
assertEquals(new Character('Z'), car);

try {
    car = cs.convert("Exception", Character.class);
    fail("Devrait lever une exception car la chaîne n'est pas un caractère !");
} catch (ConversionFailedException aExp) {
    assertEquals(aExp.getMessage(), "Failed to convert from type java.lang.String to type java.lang.Character");
}

Integer entier = cs.convert("43210", Integer.class);
assertEquals(new Integer(43210), entier);

Float flottant = cs.convert("123.456f", Float.class);
assertEquals(new Float(123.456f), flottant);
```

b. Pour les tableaux et les listes

Pour les tableaux et les listes, nous pouvons utiliser directement le `String[].class` ou le `List.class` au niveau du paramètre `Class<T> targetType` du convertisseur :

```
String[] tabChaine = cs.convert("Rond,Carré,Triangle",
                                String[].class);
assertEquals("Rond", tabChaine[0]);
assertEquals("Carré", tabChaine[1]);
assertEquals("Triangle", tabChaine[2]);
```



```
List<String> lstChaine = cs.convert(
    "Rond,Carré,Triangle", List.class);
assertEquals("Rond", lstChaine.get(0));
assertEquals("Carré", lstChaine.get(1));
assertEquals("Triangle", lstChaine.get(2));
```

Nous pouvons aussi utiliser un tableau de chaîne qui donne le même résultat :

```
lstChaine = conversionService.convert(
    new String[]{"Rond","Carré","Triangle"}, List.class);
```

La conversion est réversible :

```
String formes = cs.convert(
    new String[]{"Rond","Carré","Triangle"}, String.class);
assertEquals("Rond,Carré,Triangle", formes);
```

c. Pour les énumérations

Pour l'énumération suivante :

```
public enum MonEnum {
    UN,
    DEUX,
    TROIS
}
```

nous aurons :

```
MonEnum monEnum = conversionService.convert("DEUX",
MonEnum.class);
assertEquals(monEnum, MonEnum.DEUX);
```

d. Pour les objets en général

Si nous avons un utilisateur :

```
public class Utilisateur {
    [accesseurs]
    private String nom;
    private String prenom;
}
```

nous pouvons fabriquer un convertisseur sur mesure qui convertit notre objet en String et réciproquement :

```
public class StringToUtilisateurConverter implements
Converter<String, Utilisateur>{
    @Override
    public Utilisateur convert(String UtilisateurAsString) {
        if (UtilisateurAsString == null){
            throw new
ConversionFailedException(TypeDescriptor.valueOf(String.class),
TypeDescriptor.valueOf(String.class),
UtilisateurAsString, null);
        }
        String[] tempArray = UtilisateurAsString.split(",");
        Utilisateur article = new
Utilisateur(Integer.parseInt(tempArray[0]), tempArray[1],
tempArray[2]);
        return article;
    }
}
```

```
}  
}
```

et :

```
public class UtilisateurToStringConverter implements  
    Converter<Utilisateur, String> {  
    @Override  
    public String convert(Utilisateur article) {  
  
        if (article == null) {  
            throw new ConversionFailedException(  
                TypeDescriptor.valueOf(Utilisateur.class),  
                TypeDescriptor.valueOf(String.class), article, null);  
        }  
        StringBuilder builder = new StringBuilder();  
        builder.append(article.getId());  
        builder.append(",");  
        builder.append(article.getNom());  
        builder.append(",");  
        builder.append(article.getPrenom());  
        return builder.toString();  
    }  
}
```

Il faut enregistrer le convertisseur pour l'utiliser :

```
private static void testToObject(GenericConversionService  
conversionService) {  
    conversionService.addConverter(new  
UtilisateurToStringConverter());  
    Utilisateur utilObject = new Utilisateur(2, "CONNOR","John");  
    String utilAsString = conversionService.convert(  
        new Utilisateur[] { utilObject }, String.class);  
    System.out.println("Utilisateur -->" + utilAsString);  
}
```

2. Convertir un Array en Collection et String

Il est possible de convertir directement un Array en Collection et en String, comme le montre l'exemple suivant :

```
@Log  
public class LesConversions2 {  
    public static void main(String[] args) {  
        GenericConversionService conversionService =  
ConversionServiceFactory.createDefaultConversionService();  
        testToCollection(conversionService);  
        testToString(conversionService);  
        testToObject(conversionService);  
    }  
  
    private static void testToCollection (GenericConversionService  
conversionService){  
        @SuppressWarnings("unchecked")  
        List<String> listOfTerm = conversionService.convert(  
            new String[]{"t100","t800","tx"}, List.class);  
        for (String term : listOfTerm){  
            System.out.println("Terminator is " + term);  
        }  
    }  
  
    private static void testToString(GenericConversionService  
conversionService){  
  
        String terms = conversionService.convert(  

```

```

        new String[]{"t100","t800","tx"}, String.class);
        System.out.println("Terminator is " + terms);
    }

    private static void testToObject(GenericConversionService
conversionService){

        conversionService.addConverter(new
StringToUtilisateurConverter());

        Utilisateur utilisateur = conversionService.convert(
            new String[]{"5,John,REESE"}, Utilisateur.class);
        System.out.println("Le nom est " + utilisateur.getNom());
        System.out.println("Le prénom est " +
utilisateur.getPrenom());
    }
} Built In For Collection Type Test

```

3. Converter Factory

Nous pouvons aussi utiliser des factories qui nous retourneront des convertisseurs.

a. Classe StringToUtilisateurConverterFactory

Nous pouvons par exemple avoir une factory pour le convertisseur pour la classe `Utilisateur` :

```

public class StringToUtilisateurConverterFactory implements
    ConverterFactory<String, Utilisateur> {
    @SuppressWarnings("unchecked")
    @Override
    public <T extends Utilisateur> Converter<String, T>
getConverter(
        Class<T> arg0) {
        return (Converter<String, T>) new
StringToUtilisateurConverter();
    }
}

```

b. Classe UtilisateurToStringConverterFactory

Et réciproquement :

```

public class UtilisateurToStringConverterFactory implements
    ConverterFactory<Utilisateur, String> {
    @Override
    public <T extends String> Converter<Utilisateur, T>
getConverter(
        Class<T> arg0) {
        return (Converter<Utilisateur, T>) new
UtilisateurToStringConverter();
    }
}

```

c. Classe LesConversions5

Ce qui nous donne l'exemple suivant pour illustrer l'utilisation des factories de convertisseurs :

```

public class LesConversions5 {
    public static void main(String[] args) {
        GenericConversionService conversionService = new
GenericConversionService();
        conversionService
            .addConverterFactory(new
UtilisateurToStringConverterFactory());
        conversionService
            .addConverterFactory(new
StringToUtilisateurConverterFactory());
    }
}

```

```

        String utilisateurAsString = "7,Sarah,CONNOR";
        Utilisateur utilisateur =
conversionService.convert(utilisateurAsString,
        Utilisateur.class);

        System.out.println("Le nom est " + utilisateur.getNom());
        System.out.println("Le prénom est " +
utilisateur.getPrenom());
        utilisateurAsString =
conversionService.convert(utilisateur, String.class);
        System.out.println("Utilisateur sous forme de chaîne [" +
utilisateurAsString + "]");
    }

```

4. Les convertisseurs et les formateurs par défaut

Il existe une panoplie de formateurs et de convertisseurs qui sont disponibles nativement dans Spring.

L'exemple suivant illustre leur utilisation.

a. Classe LesConversions6

Il est possible d'utiliser les formateurs pour les dates et les nombres, comme le montre l'exemple présenté ci-dessous :

```

@Log
public class LesConversions6 {
    public static void main(String[] args) throws Exception {
        testDateFormatter();
        testNumberFormatter();
    }
    private static void testDateFormatter() {
        Formatter dateFormatter = new DateFormatter();
        String dateAsString = dateFormatter.print(new Date(),
Locale.FRANCE);
        System.out.println("Date en France : " + dateAsString);
    }
    private static void testNumberFormatter() throws Exception {
        NumberFormatter doubleFormatter = new NumberFormatter();
        doubleFormatter.setPattern("#####.###");
        String number = doubleFormatter.print(new
Double(12345.6789d),
        Locale.FRANCE);
        System.out.println("Nombre : " + number);
    }
}

```

5. Les formateurs personnalisés

Nous pouvons fabriquer des formateurs personnalisés. Nous avons pris comme exemple le formateur de numéro de carte de crédit. Cet exemple est assez courant mais il illustre les possibilités de formatages.

a. Classe CarteDeCredit

Considérons une carte de crédit constituée de quatre quadruplets :

```

@Data
@AllArgsConstructor
@NoArgsConstructor
public class CarteDeCredit {
    private int premierQuadruplet;
    private int secondQuadruplet;
    private int troisiemeQuadruplet;
    private int quatriemeQuadruplet;
}

```

b. Classe CarteDeCreditParser

Voici un exemple de parseur qui décompose les constituants d'un numéro de carte de crédit :

```
public class CarteDeCreditParser implements Parser {
    @Override
    public CarteDeCredit parse(String cc, Locale locale)
        throws ParseException {
        String caracteres[] = cc.split("-");
        if (caracteres == null || caracteres.length != 4) {
            throw new
org.springframework.expression.ParseException(-1,
                "Invalid format");
        }
        CarteDeCredit c = new CarteDeCredit();
        c.setPremierQuadruplet(Integer.parseInt(caracteres[0]));
        c.setSecondQuadruplet(Integer.parseInt(caracteres[1]));
        c.setTroisiemeQuadruplet(Integer.parseInt(caracteres[2]));
        c.setQuatriemeQuadruplet(Integer.parseInt(caracteres[3]));
        return c;
    }
}
```

c. Classe CarteDeCreditPrinter

Voici le code d'un afficheur simplifié qui affiche le numéro de carte de crédit. L'utilisation de la bibliothèque Projet Lombok simplifie la mise en forme grâce à l'annotation `@Data` sur la classe qui apporte la méthode `toString` améliorée.

```
public class CarteDeCreditPrinter implements Printer {

    @Override
    public String print(Object cc, Locale locale) {
        CarteDeCredit objet=(CarteDeCredit)cc;

        return objet.toString();
    }
}
```

d. Classe CarteDeCreditFormatter

Un formateur est à la fois un `Printer` et un `Parser`. Il faut alors fournir les deux implémentations des méthodes concernées comme dans l'exemple suivant :

```
public class CarteDeCreditFormatter implements Formatter {
    private Parser parser;
    private Printer printer;
    public CarteDeCreditFormatter(Parser parser, Printer printer)
    {
        this.parser = parser;
        this.printer = printer;
    }
    @Override
    public CarteDeCredit parse(String cc, Locale l)
        throws ParseException {
        return (CarteDeCredit) parser.parse(cc, l);
    }
    @Override
    public String print(Object cc, Locale l) {
        return printer.print(cc, l);
    }
}
```

e. Classe LesConversions7

L'exemple suivant montre comment utiliser les formateurs et convertisseurs :

```
@Log
public class LesConversions7 {
    public static void main(String[] args) {
        FormattingConversionService service = new
FormattingConversionService();
        CarteDeCreditParser parser = new CarteDeCreditParser();
        CarteDeCreditPrinter printer = new CarteDeCreditPrinter();
        service.addFormatterForFieldType(CarteDeCredit.class,
printer, parser);
        test1(service);
        test2(service);
    }

    private static void test1(FormattingConversionService service)
{
        String cc = "1111-2222-3333-4444";
        CarteDeCredit o = (CarteDeCredit) service.convert(
            cc, CarteDeCredit.class);
        log.info(o.toString());
    }

    private static void test2(FormattingConversionService service)
{
        CarteDeCredit o = new CarteDeCredit(1111, 2222, 3333,
4444);
        String cc = service.convert(o, String.class);
        log.info("Le no de la carte de crédit est :" + cc);
    }
}
```

Les BeanWrappers, le binding et les validateurs

Les wrappers, le binding et la validation sont utilisés conjointement. On rencontre beaucoup ces utilisations dans les applications Spring MVC.

1. Classe LesBeanWrappers

Cet exemple montre les possibilités de la classe `BeanWrapperImpl` :

```
@Log
public class LesBeanWrappers {
    public static void main(String[] args) {
        //Utilisation du BeanWrapper
        Utilisateur utilisateur = new Utilisateur();
        BeanWrapper bw = new BeanWrapperImpl(utilisateur);
        bw.setPropertyValue("nom", "John");
        bw.setPropertyValue("prenom", "DO");
        bw.setPropertyValue("id", "200");
        final int id=200;
        log.info("Vérification :"+(id==utilisateur.getId())+
"Utilisateur="+utilisateur);
        try {
            bw.setPropertyValue("id", "pas numérique");
        } catch (TypeMismatchException e) {
            log.info("Err binding :"+e.getMessage());
        }

        //Un peu plus complexe
        log.info("MutablePropertyValues:");
        MutablePropertyValues values = new
MutablePropertyValues();
        values.addPropertyValue("nom", "DO");
        values.addPropertyValue("prenom", "Jane");
        //values.addPropertyValue("id", "2100");
        DataBinder dataBinder = new DataBinder(utilisateur, "John");
        dataBinder.setAllowedFields(new String [] {"nom", "prenom"});
        dataBinder.setValidator(new UtilisateurValidator());
        dataBinder.bind(values);
        dataBinder.validate();
        log.info("allErrors:");
        List<ObjectError> allErrors =
dataBinder.getBindingResult().getAllErrors();
        for (Object object : allErrors) {
            ObjectError error = (ObjectError) object;
            log.info(error.toString());
        }
    }
}
```

2. Classe UtilisateurValidator

Les validateurs sont utilisés pour vérifier la validité des données.

```
public class UtilisateurValidator implements Validator {
    public boolean supports(Class clazz) {
        return Utilisateur.class.equals(clazz);
    }
    public void validate(Object obj, Errors e) {
        ValidationUtils.rejectIfEmpty(e, "nom", "nom.vide");
        Utilisateur p = (Utilisateur) obj;
        if (p.getNom().contains("s")) {
            e.rejectValue("nom", "contient.s");
        } else if (p.getId() < 0) {
            e.rejectValue("id", "id.neg");
        }
    }
}
```

} }

Points clés

- Les convertisseurs, bindeurs, BeanWrappers et validateurs sont simples à utiliser avec Spring.
- Il faut raisonner en terme global afin de réutiliser le plus possible ces outils de conversion.
- Il faut utiliser les possibilités d'externaliser des valeurs dans des fichiers `.properties` pour personnaliser la configuration Spring en fonction d'éléments externes.

Introduction

L'AOP (programmation orientée aspect) ajoute des notions de transversalités qui ne sont pas disponibles dans la programmation orientée objet (POO). Cette transversalité est centrée sur les aspects, qui permettent d'ajouter un traitement spécifique quand un type d'évènement s'exécute sur un ensemble d'objets. Il est alors possible d'ajouter des préoccupations telles que la gestion des transactions qui est transverse à un ensemble d'objets. L'AOP est indépendante du noyau Spring. On ne l'ajoute que si elle est nécessaire.

Comme pour la configuration du contexte, il est possible de définir des aspects sous la forme de fichiers XML ou d'annotations ou d'un mix des deux. La gestion des transactions est gérée par Spring via les `@Transactional`, cette annotation est un aspect particulier géré directement par Spring.

Les aspects sont généralement mis dans le socle technique et répondent à des préoccupations techniques comme par exemple l'instrumentation du code. Ils ne sont que très rarement exploités pour des préoccupations métiers car ce type de programmation est complexe au niveau de la maintenance. Le code des traitements AOP doit donc être soigné et très bien commenté.

Les concepts d'AOP

Spring utilise la terminologie standard pour nommer ses éléments relatifs à l'AOP même si elle n'est pas très intuitive.

- Aspect : modularisation d'une préoccupation qui concerne un ensemble classes de façon transverse.
- Point de jonction (Joinpoint) : une étape au cours de l'exécution d'un programme, comme l'exécution d'une méthode ou le traitement d'une exception. Dans Spring AOP, un point de jonction représente toujours une exécution de méthode.
- Greffon (Advice) : les actions d'interception prises par un aspect particulier à un point de jonction. Les différents types d'actions sont « autour », « avant » et « après ». Il est possible de chaîner ces intercepteurs.
- Coupe (Pointcut) : un prédicat qui permet de sélectionner des points de jonction à travers une expression qui indique par exemple le nom d'une méthode. Spring utilise le langage de AspectJ par défaut.
- Introduction : permet d'injecter dynamiquement des champs et des méthodes supplémentaires dans un objet. Spring AOP vous permet aussi d'introduire dynamiquement de nouvelles interfaces avec leurs implémentations. Une injection est connue comme une déclaration « inter-types » dans la communauté AspectJ.
- Objet ciblé (Target object) : objet ciblé par un ou plusieurs aspects. On parle aussi d'objet aspecté. Spring utilise toujours des proxies pour les objets aspectés.
- AOP proxy : un objet créé par le cadre AOP afin de mettre en œuvre les contrats d'aspect, un proxy AOP sera un proxy dynamique JDK ou un proxy CGLIB.
- Tissage d'aspect (Weaving) : lie les aspects avec d'autres types ou d'objets application pour créer un objet aspecté. Cela peut être fait au moment de la compilation avec AspectJ, au chargement, ou à l'exécution. Spring AOP fait le tissage à l'exécution.

Types d'actions d'interception :

- Avant (Before) : advice qui s'exécute avant un point de jonction mais qui n'a pas la capacité d'empêcher le déroulement de l'exécution de la fin de la méthode, à moins qu'il ne lève une exception.
- Après retour (After returning) : advice à exécuter après un joinpoint pour une méthode qui se termine sans lever une exception.
- Après une levée d'exception (After throwing) : advice à exécuter si une méthode se termine en levant une exception.
- Après (After) : advice à exécuter indépendamment du fait qu'il y ait eu ou non une levée d'exception.
- Autour des advices (Around) : advice qui entoure un point de jonction comme un appel de méthode. Ceci est le type d'advice le plus puissant. L'advice Around peut effectuer un comportement personnalisé avant et après l'invocation de méthode.

Il permet de faire une action avant la méthode interceptée, il permet de remplacer le code de la méthode interceptée si on le souhaite, il permet aussi de faire une action après l'appel de la méthode interceptée (ou remplacée) et peut même intercepter les exceptions et ne pas les transmettre à l'appelant.

 Il faut utiliser le type d'interception le plus simple par rapport à la fonctionnalité voulue. Les valeurs de retours des méthodes des aspects doivent être simples.

 Les objets aspectés doivent garder leur indépendance et ne doivent pas par conséquent voir les aspects pour rester dans l'esprit non invasif de Spring.

Limites de Spring AOP et utilisation d'AspectJ

Spring AOP se limite à l'exécution de méthodes sur des points de jonction, ce qui suffit dans une grande majorité des cas d'utilisation. L'interception sur le changement d'état des champs des objets n'est, par exemple, pas prise en compte. On utilisera plutôt directement AspectJ pour une utilisation avancée.

Le support @AspectJ dans Spring

On utilise ce support pour réutiliser l'interprétation des annotations de AspectJ pour la recherche et la détection des méthodes candidates pour un aspect. Seul le support des annotations est utilisé. Spring n'utilise pas le tisseur d'aspect d'AspectJ.

Il est tout à fait possible d'utiliser AspectJ en entier en conjonction de Spring pour des cas complexes pour lesquels Spring ne suffirait pas.

1. Activation du support

La librairie AspectJ aspectjweaver.jar doit être dans le classpath.

```
<dependency>
  <groupId>aopalliance</groupId>
  <artifactId>aopalliance</artifactId>
  <version>1.0</version>
</dependency>
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
  <version>1.6.8</version>
</dependency>
```

2. Activation de @AspectJ avec configuration XML

Pour activer AspectJ support avec une configuration basée sur XML, il faut utiliser l'élément `AOPaspectj-autoproxy`.

Ajoutez cet élément dans la configuration et vérifiez qu'il y a bien la définition de l'AOP :

```
<beans default-lazy-init="true"
xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:jdbc="http://www.springframework.org/schema/jdbc"
  xmlns:aop="http://www.springframework.org/schema/aop"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-4.1.xsd
    http://www.springframework.org/schema/jdbc
http://www.springframework.org/schema/jdbc/spring-jdbc-4.1.xsd
    http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-4.1.xsd
    http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
  <aop:aspectj-autoproxy/>
  <import resource="classpath*:spring/applicationContext.xml" />
  <context:component-scan base-package="fr.eni.editions" />
</beans>
```

3. Déclaration d'un aspect

Une fois @AspectJ activé, Spring AOP détecte automatiquement toutes les classes aspectées. Il est possible de déclarer l'aspect pour une classe par une annotation @Aspect (org.aspectj.lang.annotation.Aspect) ou dans le fichier XML.

La déclaration des aspects dans le fichier de configuration XML n'est pas abordée car elle est massivement remplacée par les annotations. Le bean correspondant à la classe aspectée est par défaut un singleton. L'objet peut contenir des méthodes et des variables ainsi que des pointcuts, des advices et des déclarations inter-types.

- L'annotation `@Aspect` ne suffit pas toujours pour trouver le bean dans le classpath. Il faut parfois ajouter un complément avec l'annotation `@Component`.

Les classes annotées avec `@Aspect` peuvent avoir des méthodes et des champs comme toutes les autres classes, mais il n'est pas possible d'aspecter un aspect.

4. Déclaration d'un pointcut

Spring permet l'interception d'un ensemble de méthodes à partir de leur nom en détectant des points communs dans la typologie de leurs signatures.

```
@Pointcut("execution(* voir*(..))")
public void aspectjLoadTimeWeavingExamples() {
```

Le pointcut est appelé avant la méthode interceptée et peut modifier puis retourner l'objet retourné par la méthode interceptée.

- AspectJ propose d'autres pointcuts qui ne sont pas pris en charge dans Spring : `call`, `get`, `set`, `preinitialization`, `staticinitialization`, `initialization`, `handler`, `adviceexecution`, `withincode`, `cflow`, `cflowbelow`, `if`, `@this`, et `@withincode`. Pour les utiliser, il faut basculer vers une utilisation complète de AspectJ.

Les verbes suivants indiquent le type de coupe :

execute	se base sur le nom de la méthode.
within	se base sur le type de la méthode.
this	se base sur la référence du proxy de l'objet.
target	se base sur le type de l'objet référencé par le proxy.
args	se base sur le type des arguments de la méthode.

Il est également possible de limiter les candidats avec des critères :

@target	filtre sur la présence d'une annotation de ce type sur l'objet.
@args	filtre sur la présence d'une annotation de ce type sur les paramètres de la méthode.
@within	filtre sur la présence d'une annotation de ce type sur la méthode.
@annotation	filtre sur le type de la méthode ciblée par l'annotation.

On ne peut intercepter que des méthodes publiques. Les méthodes `private` et `protected` ne sont pas prises en compte pour les aspects.

Il est possible de combiner des points de jonction via l'utilisation des opérateurs `&&` (et), `||` (ou), ainsi que `!` (négation).

```
@Pointcut("execution(* entendre*(..))")
public void aspectjLoadTimeWeavingExamples2() {
```

L'exécution de toutes les méthodes publiques :

```
execution(public * *(..))
```

Exécution de toutes les méthodes commençant par set :

```
execution(* set*(..))
```

Exécution de toutes les méthodes qui sont définies dans l'interface ClientService :

```
execution(* com.eni.ClientService.*(..))
```

Exécution de toutes les méthodes qui sont dans un package donné :

```
execution(* com.eni.*.*(..))
```

Exécution de toutes les méthodes qui sont dans un package ou un sous-package donné :

```
execution(* com.eni..*.*(..))
```

Tous les points d'actions qui sont dans le package service :

```
within(com.eni.service.*)
```

Tous les points d'actions qui sont dans le package service et ses sous-packages :

```
within(com.eni.service..*)
```

5. Déclaration des greffons simples

Pour les greffons simples, il est possible d'utiliser les annotations suivantes :

@Before	Le code s'exécutera avant l'appel de la méthode.
@AfterReturning	Le code s'exécutera après l'appel de la méthode.
@AfterThrowing	Le code s'exécutera après l'appel à la gestion d'exception.
@After	Le code s'exécutera après l'appel à finally.
@Around	Le code encapsule l'appel à la méthode interceptée.

@Before est utilisé par exemple si on cherche le mot "demarrer" dans la signature pour exécuter un traitement avant l'appel à la méthode.

Sur une turbine :

```
@Before("execution(*  
fr.eni.editions.beans.Bateau.demarrerTurbine(..))")  
public void verifierEssenceTurbine() {  
    final Logger logger =  
        LoggerFactory.getLogger(VerifierAvant.class);  
    logger.info("avant démarrer turbine!");  
}
```

Sur tous les moyens de propulsion :

```
@Before("execution(* demarrer*(..))")  
public void verifierEssence() {  
    System.out.println("avant démarrer tout court!");  
}
```

@After est utilisé par exemple si on cherche le mot "demarrer" dans la signature pour exécuter un traitement après l'appel à la méthode.

Sur tous les moyens de propulsion :

```
@After("execution(* demarrer*(..))")
public void verifierVitesse() {
    System.out.println("Après démarrage!");
}
```

Il est aussi possible de retourner la valeur de retour de la méthode interceptée :

```
@After("execution(* demarrer*(..))")
public Object verifierEssence(Object retVal) {
    System.out.println("après démarrer tout court!");
    return (retVal);
}
```

@AfterThrowing est utilisé pour avoir un traitement après une exception.

```
@Aspect
public class AfterThrowingExample {
    @AfterThrowing("fr.eni.MonException()")
    public void doRecoveryActions() {
        // ...
    }
}
```

Il est possible de récupérer le type de l'exception pour l'exploiter :

```
@Aspect
public class AfterThrowingExample2 {
    @AfterThrowing(
        pointcut="fr.eni.MonException()",throwing="ex")
    public void doRecoveryActions(DataAccessException ex) {
        // ...
    }
}
```

@After est utilisé pour avoir du code après le finally.

```
@Aspect
public class AfterFinallyExample {
    @After("fr.eni.MonException()")
    public void doReleaseLock() {
        // ...
    }
}
```

@Around intercepte totalement un appel de méthode.

La méthode d'interception a accès aux paramètres d'appel de la méthode interceptée.

```
@Around("execution(* demarrer*(..))")
public Object generique(ProceedingJoinPoint joinPoint) throws
    Throwable {
    final Logger logger =
    LoggerFactory.getLogger(joinPoint.getSignature().getDeclaringType
    ());
    logger.info("Interception demarrer:");
    logger.info("method : " + joinPoint.getSignature().getName());
    logger.info("arguments : " +
    Arrays.toString(joinPoint.getArgs()));
    logger.info("Around before is running!");
    Object ret = joinPoint.proceed();
    logger.info("Around after is running!");
    logger.info("Return " + ret.toString());
    return ret;
}
```

6. Les types génériques

Spring AOP gère les types génériques. On peut souhaiter n'intercepter que certains types. On filtre alors sur les types.

Par exemple pour ne filtrer que sur MyType pour l'interface suivante :

```
interface Sample<T> {  
    void sampleGenericMethod(T param);  
    void sampleGenericCollectionMethod(Collection<T> param);  
}
```

On filtre sur le type :

```
@Before("execution(* ..Sample+.sampleGenericMethod(*)) &&  
args(param)")  
public void beforeSampleMethod(MyType param) {  
    // Advice implementation  
}
```

Pour les collections génériques, c'est plus compliqué. On ne peut pas faire ce type de code :

```
@Before("execution(* ..Sample+.sampleGenericCollectionMethod(*))  
&& args(param)")  
public void beforeSampleMethod(Collection<MyType> param) {  
    // Advice implementation  
}
```

Il faut passer par ce contournement :

```
@Before("execution(* ..Sample+.sampleGenericCollectionMethod(*))  
&& args(param)")  
public void beforeSampleMethod2(Collection<?> param) {  
    // Advice implementation  
}
```

et valider pour chaque type si nous détournons ou non.

7. Détermination de noms d'arguments

Avant Java 8, il n'était pas possible de récupérer simplement le nom des paramètres d'une méthode. Spring AOP, qui reste compatible avec Java 7, utilise des stratégies pour déterminer les noms de paramètres. La façon la plus simple est d'avoir le même nombre et le même ordre pour les paramètres entre la méthode interceptée et la méthode d'interception.

8. Ordre d'appel des greffons interceptant le point de jonction

Les greffons peuvent être appelés les uns après les autres et il est possible de déterminer un ordre de passage préférentiel.

Il suffit d'ajouter l'interface `org.springframework.core.Ordered` à la classe de l'aspect ou d'ajouter l'annotation `@Order` et de coder la méthode `getOrder()` :

```
public int getOrder() {  
    return 0;  
}
```

La classe qui remonte l'ordre le plus haut est exécutée en premier.

9. Injection

Il est possible d'ajouter dynamiquement des méthodes et leurs implémentations à des classes (déclarations inter-types avec AspectJ).

Pour cela, on crée une classe d'aspects et on ajoute la déclaration de la méthode via l'annotation `@DeclareParents`. On ajoute alors un aspect qui indique à quel moment appeler cette méthode.

Par exemple :

```
@Aspect
public class UsageTracking {
    @Before("fr.eni.monService() && this(usageTracked)")
    public void recordUsage(UsageTracked usageTracked) {
        usageTracked.incrementUseCount();
    }
}
```

Ajoute la méthode `recordUsage` pour tous les objets qui implémentent l'interface `service`.

Exemple :

```
UsageTracked usageTracked = (UsageTracked)
context.getBean("monService");
```

10. Le mécanisme des proxies

Spring AOP a recours aux proxies dynamiques du JDK pour les objets qui exposent les interfaces à détourner. Dans le cas contraire, Spring utilise CGLIB.

Les méthodes finales ne peuvent pas être détournées.

Le constructeur de l'objet proxifié est appelé deux fois, une fois pour le proxy et une seconde fois pour l'advice.

Pour forcer l'utilisation d'un proxy CGLIB, il faut passer par la configuration XML :

```
<aop:config proxy-target-class="true">
  <!-- other beans defined here... -->
</aop:config>
```

Si on utilise l'annotation `@AspectJ` il suffit de spécifier dans le fichier XML :

```
<aop:aspectj-autoproxy proxy-target-class="true"/>
```

Les sections `<aop:config/>` des fichiers de configuration XML sont fusionnées en mémoire. L'utilisation des CGLIB est alors commune pour :

- `<tx:annotation-driven/>`
- `<aop:aspectj-autoproxy/>`
- `<aop:config/>`

La décision d'activer les CGLIB doit donc être bien réfléchie car il peut y avoir des effets de bord.

11. La face cachée des proxies en AOP

Il est possible de créer des aspects dans le code qui s'ajoutent à ceux déclarés via la configuration `<aop:config>` ou `<aop:aspectj-autoproxy>` complétée par les annotations dédiées.

On peut utiliser des API Java simples :

```

package fr.eni.editions;
import java.lang.reflect.Method;
import org.springframework.aop.AfterReturningAdvice;
import org.springframework.aop.aspectj.annotation.AspectJProxyFactory;
interface Pojo {
    public void foo();
    public void bar();
class SimplePojo implements Pojo {
    public void foo() {
        System.out.println("foo");
        this.bar();
    }
    public void bar() {
        System.out.println("bar");
    }
class UsageTracker implements AfterReturningAdvice {
    public void afterReturning(Object returnValue, Method method,
        Object[] args, Object target) throws Throwable {
        System.out.println("after "+method.getName());
    }
public class Main2 {
    public static void main(String[] args) {
        AspectJProxyFactory factory = new AspectJProxyFactory(new
SimplePojo2());
        factory.addAspect(SecurityManager.class);
// factory.addAspect(new UsageTracker());
        Pojo pojo = (Pojo) factory.getProxy();
    }

```

Cette façon de programmer met plus facilement en évidence certains effets de bord liés à l'AOP et les proxies.

En temps normal, si nous avons un objet classique non aspecté comme celui-ci et que l'on appelle une méthode sur une référence de cet objet, on appelle directement le code de la méthode :

```

interface Pojo4 {
    public void foo();
    public void bar();
class MonSimplePojo implements Pojo4 {
    public void foo() {
        System.out.println("foo");
        this.bar();
    }
    public void bar() {
        System.out.println("bar");
    }
public class Main3 {
    public static void main(String[] args) {
        Pojo4 pojo = new MonSimplePojo();
        // this is a direct method call on the pojo reference
        pojo.foo();
    }

```

On obtient :

```

foo
bar

```

Mais nous ajoutons un aspect comme celui-ci :

```

interface Pojo3 {
    public void foo();
    public void bar();
class SimplePojo3 implements Pojo3 {

```

```

    public void foo() {
        System.out.println("foo");
        this.bar();
    }
    public void bar() {
        System.out.println("bar");
    }
class RetryAdvice implements AfterReturningAdvice {
    public void afterReturning(Object returnValue, Method method,
        Object[] args, Object target) throws Throwable {
        System.out.println("after "+method.getName());
    }
}
public class MainAvecAspect {
    public static void main(String[] args) {
        ProxyFactory factory = new ProxyFactory(new
SimplePojo3());
        Class<?> intf=Pojo3.class;
        factory.addInterface(intf);
        Advice advice=new RetryAdvice();
        factory.addAdvice(advice);
        factory.setExposeProxy(true);
        Pojo3 pojo = (Pojo3) factory.getProxy();
        pojo.foo();
        pojo.bar();
    }
}

```

Nous aurons à l'exécution :

```

foo
bar
after foo
bar
after bar

```

Si nous sommes dans une classe aspectée pour laquelle une méthode est interceptée, si cette méthode appelle une méthode sur elle-même, et que cette méthode appelée est elle-même interceptée, par défaut l'interception n'aura pas lieu sur la méthode appelée.

Il faut refactoriser le code pour éviter que les méthodes s'appellent entre elles en passant par exemple par une classe de services intermédiaire.

Les API bas niveau pour Spring AOP

Voici un exemple d'utilisation des API de bas niveau. Nous ne les montrerons pas toutes car elles sont très bien présentées dans la documentation Spring et nous ne nous en servons que très rarement. Nous les mentionnons car elles permettent d'expérimenter les aspects très simplement.

1. L'interface PointCut

L'interface `PointCut` correspond à un point de coupe, qui permet grâce à une expression régulière d'identifier les méthodes qui seront interceptées.

```
class MonPointcut implements PointCut {
    public ClassFilter getClassFilter() {
        return null;
    }
    public MethodMatcher getMethodMatcher() {
        return null;
    }
}
```

Il y a deux méthodes dans l'interface :

Classe	Méthode	Utilité
<code>ClassFilter</code>	<code>getClassFilter();</code>	Filtre sur la classe.
<code>MethodMatcher</code>	<code>getMethodMatcher();</code>	Matcher sur la classe.

2. L'interface ClassFilter

L'interface `ClassFilter` ne contient qu'une méthode qui correspond à celle qui sera appelée lors de l'interception.

```
class MonClassFilter implements ClassFilter {
    public boolean matches(Class<?> clazz) {
        return false;
    }
}
```

3. L'interface MethodMatcher

L'interface `MethodMatcher` permet une évaluation statique à la compilation ou dynamiquement à l'exécution des critères de sélection des méthodes à intercepter.

```
class MonMethodMatcher implements MethodMatcher {

    public boolean matches(Method method, Class<?> targetClass) {
        return false;
    }
    public boolean isRuntime() {
        return false;
    }
    public boolean matches(Method method, Class<?> targetClass,
Object[] args) {
        return false;
    }
}
```

Points clés

- La programmation orienté aspect est souvent présente dans les socles logiciels.
- Il est possible de coder simplement des aspects.
- Si on retrouve une action sur un groupe de classe, il faut voir si on peut utiliser un aspect.
- Les aspects sont déroutants en debug, il faut bien les comprendre pour pouvoir corriger du code qui les utilise.

Introduction

Les préoccupations centrées sur les tests sont fondamentales dans les développements d'aujourd'hui. Le TDD (*Test-Driven Development* ou en français développement piloté par les tests), qui consiste, lors de l'écriture de nos programmes, à écrire les tests unitaires avant d'écrire le code source d'un logiciel, est de plus en plus utilisé.

Spring propose des API qui nous simplifient la mise en œuvre des tests unitaires (TU) et des tests d'intégration (TI). Il faut tester tout ce qui vaut le coup d'être testé. Si vous rencontrez des problèmes lors de la rédaction des TU et TI, c'est qu'il faut retravailler (refactoriser) votre code. Une bonne pratique consiste à écrire le test en même temps que la classe testée et même parfois à coder les tests avant l'implémentation des méthodes testées de façon à ce que l'architecture de l'application soit compatible avec les tests. Les tests unitaires sont des tests rapides qui permettent de valider des actions de bas niveau. Les tests d'intégration utilisent des jeux d'essais pour valider du code de plus haut niveau ; ils sont moins rapides. Il existe aussi des tests qui simulent un utilisateur. Ces tests sont faits avec des outils comme Selenium.

Spring nous aide en mettant à disposition un ensemble d'API spécialisées pour les tests. Nous trouverons des API pour simuler les contextes d'exécution de nos applications. Nous aurons aussi des API pour modifier la configuration des objets Spring en mémoire. Pour tester avec des données, Spring nous aidera en nous donnant accès, de façon très simple, à des jeux d'essais utilisant une base de données en mémoire.

Nous configurerons les tests avec les fichiers XML et les annotations en gardant à l'esprit que ces tests devront avoir un haut degré de maintenabilité.

Les Mock Objects

Nous pouvons utiliser des simulacres d'objets nommés mocks pour ne pas avoir à invoquer des objets réels qui demanderaient un contexte d'exécution trop étendu.

Spring fournit un ensemble de mocks très complet. Ils sont plus simples à utiliser que les mocks EasyMock et Mockito.

Type de mock	Utilisation
Environnement	Classes liées à l'environnement d'exécution.
JNDI	Simule des ressources JNDI comme une datasource.
API des Servlets	Simule une servlet, utile avec Spring MVC.
API des Portlets	Simule les portlets Spring MVC.
Support	Outils aidant pour l'inspection des objets.

1. Mocks spécialisés pour "environnement"

On simule les classes d'environnements.

Classe	Mock
Environnement	MockEnvironment
@PropertySource	MockPropertySource

Ces mocks permettent de simuler un environnement et un `PropertySource`.

2. Support

a. Utilités générales

La classe `ReflectionTestUtils` du package `org.springframework.test.util` apporte des aides pour l'inspection et la manipulation des objets. Tous les membres des classes deviennent accessibles, même les membres « `private` ».

Par exemple, pour une classe `Vehicule` :

```
public class Vehicule {  
    [accesseurs]  
    private long id;  
    private String modele;  
}
```

Nous pouvons faire ce que nous voulons :

```
final Vehicule person = new Vehicule();  
ReflectionTestUtils.setField(person, "id", new Long(99),  
    long.class);  
assertEquals("id", 99L, person.getId());  
ReflectionTestUtils.setField(person, "modele", null,  
    String.class);  
assertNull("modele", person.getModele());  
try {  
    ReflectionTestUtils.setField(person, "id", null, long.class);  
    fail("Devrait lever une exception !");  
} catch (IllegalArgumentException aExp) {  
    assertEquals(aExp.getMessage(),  
        "id must not be null");  
}
```



```

        .contains("IllegalArgumentException"));
    }
    ReflectionTestUtils.invokeSetterMethod(person, "id", new
    Long(99), long.class);
    assertEquals("id", 99L, person.getId());

    ReflectionTestUtils.invokeSetterMethod(person, "setId", new
    Long(1), long.class);
    assertEquals("id", 1L, person.getId());
    try {
        ReflectionTestUtils.invokeSetterMethod(person, "id", null,
        long.class);
        fail("Devrait lever une exception !");
    } catch (IllegalArgumentException aExp) {
        assert (aExp.getMessage()
            .contains("IllegalArgumentException"));
    }
}

```

Il est ainsi possible d'intervenir sur des variables ou des méthodes qui ne sont normalement pas disponibles.

b. Spring MVC

La classe `ModelAndViewAssert` du package `org.springframework.test.web` apporte des aides pour tester les objets de type Spring MVC `ModelAndView`. Pour tester un contrôleur Spring MVC, on utilise `ModelAndViewAssert` combiné avec `MockHttpServletRequest`, `MockHttpSession` et ainsi de suite. Le package `org.springframework.mock.web` est basé sur l'API Servlet 3 depuis Spring 4.0. Nous verrons dans le chapitre sur Spring MVC de nombreux exemples d'utilisation de ces mocks.

3. Tests d'intégration

a. Vue d'ensemble

Il faut souvent tester les comportements d'un ensemble d'objets pour vérifier, par exemple, l'interaction entre les couches du socle ou les règles de gestion mettant en scène des jeux de données. L'idéal est de faire ces tests dans un sous-ensemble technique de l'environnement global d'exécution. Ce type de tests est regroupé dans les tests d'intégration. Spring permet de faire ces tests sans lancer le serveur d'application ou l'application au complet. C'est un de ses points forts.

L'idée des tests d'intégration est de fournir les éléments pour tester les différentes couches logicielles en fournissant un environnement d'exécution autonome. La bibliothèque de base pour les aides au codage des tests est dans le module `spring-test` qui est dans le package `org.springframework.test`.

Ces tests sont plus lents que les tests unitaires, mais plus rapides que les tests Selenium (les tests Selenium simulent une session d'un utilisateur en imitant son comportement sur l'application).

Les TI sont identifiés via l'aide d'annotations spécifiques afin de gérer, entre autres, le cache du contexte pour l'IoC, la gestion des transactions, la gestion des jeux d'essais en base.

b. Mise en cache du contexte de test

Le chargement du contexte serait relativement long dans le cas d'une suite de tests si on le rechargeait pour chaque test. Spring permet de charger un contexte et de l'utiliser une batterie de tests. Dans le cas pour lequel un test altérerait le contexte, il est possible de demander à Spring de recharger le contexte initial afin d'avoir toujours un environnement propre et stable.

Nous spécifierons, au chargement d'une batterie de tests, la liste des fichiers de configuration à charger. En général, nous utiliserons un fichier spécifique qui complètera le fichier de configuration principal de l'application. Nous disposerons alors du contexte de l'application principale surchargé pour prendre en

compte tous les éléments qui divergent entre l'environnement d'utilisation et l'environnement de test.

Nous chercherons juste à émuler la base pour la partie back et le serveur web pour la partie front. Nous utiliserons aussi parfois, pour des programmes qui exploitent des fichiers, des extraits de ces fichiers afin de ne tester que les cas fonctionnels passants et non passants.

c. Tests back et front

La partie back transforme des données physiques provenant de base de données, ou des fichiers ou des flux, en données exploitables par une partie front qui traite ou affiche les données.

Les tests des parties back

Pour un test d'intégration back, il faut émuler les sources d'informations physiques provenant de bases de données, de fichiers ou de flux.

Bases de données

Pour les bases de données, nous pouvons utiliser les classes Spring ou un framework tiers comme DBUnit ou Liquibase.



Nous ne montrerons que les exemples en SQL intégrés au framework de Spring mais nous vous invitons à approfondir le sujet en étudiant les possibilités des autres frameworks.

Librairies de tests Spring

Pour les projets simples, n'utilisant pas de librairie d'ORM comme Hibernate et JPA et pour lequel le modèle de données est petit, nous pouvons utiliser le package `org.springframework.test.jdbc` qui contient la classe `JdbcTestUtils`, qui implémente des méthodes statiques utilitaires :

<code>countRowsInTable(..)</code>	Compte le nombre de lignes dans une table.
<code>countRowsInTableWhere(..)</code>	Compte le nombre de lignes d'une table avec une clause WHERE.
<code>deleteFromTables(..)</code>	Vide une table.
<code>deleteFromTableWhere(..)</code>	Supprime les lignes d'une table avec une clause WHERE.
<code>dropTables(..)</code>	Supprime les tables spécifiées.

4. Annotations

a. @ContextConfiguration

L'annotation principale est `@ContextConfiguration`. Elle permet de charger le contexte de tests de multiples façons. Nous spécifions l'emplacement des fichiers de configuration ainsi que les classes annotées `@Configuration` qui portent la configuration :

```
@ContextConfiguration("/test-config.xml")
public class XmlApplicationContextTests {
    // Contenu de la classe...
    @ContextConfiguration(classes = TestConfig.class)
    public class ConfigClassApplicationContextTests {
        // Contenu de la classe...
```

Il est également possible de spécifier la classe d'initialisation `ApplicationContextInitializer`:

```
@ContextConfiguration(initializers =
    CustomContextInitializer.class)
```

```
public class ContextInitializerTests {
// Contenu de la classe...
public interface ApplicationContextInitializer<C extends
ConfigurableApplicationContext>
```

Cette interface est utilisée en callback du chargement du contexte afin de charger les propriétés dans le contexte, notamment pour prendre en compte les paramètres `context-param` et `init-param` pour les applications web.

b. @WebAppConfiguration

Cette annotation sur la classe spécifie que l'on veut un contexte web (`WebApplicationContext`).

Le répertoire de base de l'application web est "file: src / main / webapp".

```
@ContextConfiguration
@WebAppConfiguration
public class WebAppTests {
// Contenu de la classe...
```

Le chemin peut être surchargé via `classpath:` et `file:`.

```
@ContextConfiguration
@WebAppConfiguration("classpath:test-web-resources")
public class WebAppTests {
// Contenu de la classe...
```

c. @ContextHierarchy

Cette annotation sur la classe définit la hiérarchie des `ApplicationContext`. Elle doit être déclarée conjointement à une liste d'une ou plusieurs annotations `@ContextConfiguration` qui définissent les contextes individuels à considérer.

```
@ContextHierarchy({
@ContextConfiguration("/parent-config.xml"),
@ContextConfiguration("/enfant-config.xml")
})
public class ContextHierarchieTests {
// Contenu de la classe...
```

Elle peut s'utiliser en complément de l'annotation qui définit le type de contexte pour la configuration des tests :

```
@WebAppConfiguration
@ContextHierarchy({
@ContextConfiguration(classes = AppConfig.class),
@ContextConfiguration(classes = WebConfig.class)
})
public class WebIntegrationTests {
// Contenu de la classe...
```

Il est possible de surcharger une branche des contextes hiérarchiques en spécifiant le nœud à surcharger le nom dans l'attribut `name` de l'annotation `@ContextConfiguration`. Par exemple, le code suivant permet de clarifier la relation entre deux contextes hiérarchiques pour une application Spring MVC :

```
@RunWith(SpringJUnit4ClassRunner.class)
@WebAppConfiguration(value = "src/main/webapp")
@ContextHierarchy({
@ContextConfiguration(name = "parent", locations =
"classpath:spring-config.xml"),
@ContextConfiguration(name = "child", locations =
```

```
    "classpath:spring-mvc.xml")
})
```

d. @ActiveProfiles

Si nous utilisons des profils qui indiquent, par exemple, dans quel environnement nous souhaitons tester, nous pouvons utiliser l'annotation `@ActiveProfiles` sur une classe, ce qui indique que cette classe n'est à prendre en considération que si nous sommes dans ce ou ces profils.

```
@ContextConfiguration
@ActiveProfiles("dev")
public class DeveloperTests {
    // Contenu de la classe...
}

@ContextConfiguration
@ActiveProfiles({"dev", "integration"})
public class DeveloperIntegrationTests {
    // Contenu de la classe...
}
```

e. @TestPropertySource

Cette annotation sur la classe définit l'emplacement des fichiers de ressources et des propriétés `inlines` à ajouter. Une propriété `inline` est une propriété définie à la volée, directement dans l'annotation.

```
@ContextConfiguration
@TestPropertySource("/test.properties")
public class MyIntegrationTests {
    // Contenu de la classe...
}

@ContextConfiguration
@TestPropertySource(properties = { "timezone = GMT", "port: 9999" })
public class MyIntegrationTests {
    // Contenu de la classe...
}
```

f. @DirtiesContext

Cette annotation contrôle le rechargement du contexte quand le test le détériore. Par défaut, nous ne rechargeons qu'après avoir fait tous les tests. Le paramètre `classMode = ClassMode.AFTER_EACH_TEST_METHOD` indique que l'on veut nettoyer le contexte après chaque test. Si l'on met l'annotation sur une méthode de test, on indique que l'on veut recharger le contexte après le test spécifié.

```
@DirtiesContext
public class ContextDirtyingTests {
    @DirtiesContext(classMode = ClassMode.AFTER_EACH_TEST_METHOD)
    public class ContextDirtyingTests {
        @DirtiesContext
        @Test
        public void testProcessWhichDirtiesAppCtx() {
            // ...
        }
    }
}
```

Dans le cas où le test est dans une hiérarchie de contexte, nous pouvons lui demander de recharger tout le contexte.

```
@ContextHierarchy({
    @ContextConfiguration("/parent-config.xml"),
    @ContextConfiguration("/child-config.xml")
})
public class BaseTests {
    // Contenu de la classe...
    public class ExtendedTests extends BaseTests {
        @Test
        @DirtiesContext(hierarchyMode = HierarchyMode.CURRENT_LEVEL)
        public void test() { [...]}
    }
}
```

g. @TestExecutionListeners

Cette annotation utilisée conjointement avec l'annotation `@ContextConfiguration`, indique quels `TestExecutionListeners` seront utilisés.

```
@ContextConfiguration
@TestExecutionListeners({CustomTestExecutionListener.class,
AnotherTestExecutionListener.class})
public class CustomTestExecutionListenerTests {
    // Contenu de la classe...
```

h. @TransactionConfiguration

C'est une des annotations les plus complexes. Elle précise les conditions d'utilisation des tests qui se font au sein d'une transaction. La transaction permet de faire des opérations et de ne les valider que si elles réussissent toutes ; en cas de défaillance sur l'une d'entre elles, toutes les opérations sont annulées. Dans certains cas, on demandera explicitement l'annulation des modifications à la fin d'une séquence de tests ; dans d'autres cas plus complexes, on devra dérouler la transaction jusqu'au bout et réinitialiser le jeu d'essais.

i. @Transactional

Il existe aussi un type de transaction étendue : la transaction « répartie » qui met en jeu un ensemble de transactions, une transaction globale englobant un ensemble de transactions avec une validation globale de toutes les modifications en cas de succès de toutes les opérations et inversement. Ce type de transaction se produit si nous avons par exemple plusieurs bases de données en jeu pour un seul ensemble de modifications. Nous parlons alors de commit multi-phases.

L'annotation `@Transactional` indique que la méthode marquée participe à la transaction courante.

Dans le cas pour lequel il y aurait plusieurs `TransactionManager` dans l'application, nous indiquons quel `TransactionManager` utiliser pour gérer les transactions.

```
@ContextConfiguration
@TransactionConfiguration(transactionManager = "txMgr",
defaultRollback = false)
public class CustomConfiguredTransactionalTests {
    // Contenu de la classe...
```

j. @Rollback

L'annotation `@Rollback` indique si la transaction pour la méthode de test annotée doit être annulée une fois la méthode de test terminée.

Une valeur à `true` indique que la transaction doit être annulée. Cette valeur par défaut peut être surchargée en indiquant `false` comme dans l'exemple suivant :

```
@Rollback(false)
@Test
public void testProcessWithoutRollback() {
    // ...
```

k. @BeforeTransaction

La méthode portant l'annotation `@BeforeTransaction` est exécutée avant le démarrage de la transaction. Elle peut servir à initialiser le jeu d'essais.

```
@BeforeTransaction
public void beforeTransaction() {
```

l. @AfterTransaction

La méthode portant l'annotation `@AfterTransaction` est exécutée après la fin de la transaction. Elle peut servir à nettoyer le jeu d'essais.

```
@AfterTransaction
public void afterTransaction() {
```

m. `@Sql`, `@SqlConfig` et `@SqlGroup`

L'annotation `@Sql` indique à Spring qu'il doit exécuter les scripts SQL avant de lancer la méthode de tests. Elle est utilisée pour annoter une classe de test ou méthode de test pour configurer des scripts SQL à exécuter en base de données au cours des tests d'intégration.

```
SQL ({ "/test-schema.sql" , "/test-user-data.sql" })
public void usertest {
    // exécuter du code qui repose sur les données de schéma
    // et de test
```

L'annotation `@SqlConfig` indique comment parser les fichiers SQL.

```
@Test
@Sql(
    scripts = "/test-user-data.sql",
    config = @SqlConfig(commentPrefix = "`", separator = "@@")
    public void userTest {
        // Code relatif aux tests de données
```

L'annotation `@SqlGroup` permet d'avoir plusieurs annotations `@Sql`.

Elle peut être omise si on utilise Java 8 car les annotations y sont « répétables », c'est-à-dire que nous pouvons utiliser plusieurs fois la même annotation mais avec des paramètres différents. Par exemple, nous pouvons utiliser l'annotation `@SqlGroup` dans laquelle nous pouvons avoir un ensemble d'annotations `@Sql` :

```
@Test
@SqlGroup({
    @Sql(scripts = "/test-schema.sql", config =
    @SqlConfig(commentPrefix = "`")),
    @Sql("/test-user-data.sql")
})
public void userTest {
    // Code relatif aux tests sur le schéma et les données
```

5. Annotations standards

Les annotations suivantes sont supportées avec la sémantique standard :

- `@Autowired`
- `@Qualifier`
- `@Resource` (javax.annotation) si JSR-250 est présent
- `@Inject` (javax.inject) si JSR-330 est présent
- `@Named` (javax.inject) si JSR-330 est présent
- `@PersistenceContext` (javax.persistence) si APP est présent
- `@PersistenceUnit` (javax.persistence) si APP est présent
- `@Required`
- `@Transactional`

 Attention : les beans pour lesquels est configuré un traitement à la construction et destruction sont complexes à gérer au niveau des TU et TI.

- Le `@PreDestroy` n'est jamais appelé.
- Le `@PostConstruct` est appelé avant le `@Before`.

Les frameworks de test JUnit et testNG

Spring a intégré les deux frameworks les plus complets aujourd'hui afin d'en simplifier l'utilisation. La construction d'un environnement pour lancer nos tests devient un jeu d'enfant et il est possible de ne charger en mémoire que le strict minimum pour pouvoir faire nos tests.

1. Utilisation avec JUnit

La librairie JUnit est très bien documentée sur ce site : <http://junit.org/>

a. Spring JUnit Runner

Spring a intégré JUnit après la version 4.9 avec la classe `SpringJUnit4ClassRunner.class` qui s'utilise en conjonction avec l'annotation `@RunWith`.

```
@RunWith(SpringJUnit4ClassRunner.class)
@TestExecutionListeners({})
public class SimpleTest {
    @Test
    public void testMethod() {
        [...]
    }
}
```

b. @IfProfileValue

L'annotation fonctionne comme `@ActiveProfiles` et peut être mise sur la classe ou sur une méthode.

```
@IfProfileValue(name="java.vendor", value="Oracle Corporation")
@Test
public void testProcessavecJvmOracle() {

    @IfProfileValue ( name = "test-groups" , values = { "unit-tests" ,
        "integration-tests" })
    @Test
    public void
    testProcessWhichRunsForUnitOrIntegrationTestGroups() {
        // une certaine logique qui devrait fonctionner uniquement
        // pour les tests unitaires et les tests d'intégration
    }
}
```

c. @ProfileValueSourceConfiguration

Cette annotation sur une classe spécifie le type de `ProfileValueSource`. Elle est à utiliser pour récupérer les valeurs de profil configurées par l'annotation `@IfProfileValue` :

```
@ProfileValueSourceConfiguration(CustomProfileValueSource.class)
public class CustomProfileValueSourceTests {
```

le `SystemProfileValueSource`.

Par défaut, nous utiliserons

d. @Timed

L'annotation `@Timed` indique que la méthode de test annotée doit se terminer dans un délai spécifié (en millisecondes). Si le temps d'exécution du test dépasse la durée du temps spécifiée, le test échoue.

On ne tient compte que de la durée du test. Elle peut être utilisée en conjonction de l'annotation `@Repeat` vue plus loin.

```
@Timed(millis=1000)
public void testProcessWithOneSecondTimeout() {
    // Code qui ne doit pas durer plus d'une seconde.
}
```

➤ JUnit dispose d'une annotation similaire : `@Test(timeout=...)` mais lance le test dans un Thread différent qu'il interrompt si le timeout est dépassé. Spring laisse le traitement aller jusqu'au bout et invalide le test en cas de dépassement.

e. @Repeat

Indique que la méthode de test annotée doit être exécutée de manière répétée en indiquant le nombre d'itérations en paramètre.

```
@Repeat(10)
@Test
public void testProcessRepeatedly() {
    // ...
}
```

f. Méta-annotations de support pour les tests

Avec Spring Framework 4.0, il est possible d'utiliser les méta-annotations combinées aux annotations de tests pour faire des super annotations regroupant plusieurs annotations.

Liste des méta-annotations :

- @ContextConfiguration
- @ContextHierarchy
- @ActiveProfiles
- @TestPropertySource
- @DirtiesContext
- @WebAppConfiguration
- @TestExecutionListeners
- @Transactional
- @BeforeTransaction
- @AfterTransaction
- @TransactionConfiguration
- @Rollback
- @Sql
- @SqlConfig
- @SqlGroup
- @Repeat
- @Timed
- @IfProfileValue
- @ProfileValueSourceConfiguration

Par exemple, si nous découvrons que nous répétons la configuration suivante dans notre suite de test JUnit :

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration({"app-config.xml", "/test-data-access-
config.xml"})
@ActiveProfiles("dev")
@Transactional
public class OrderRepositoryTests { }

@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration({"app-config.xml", "/test-data-access-
config.xml"})
@ActiveProfiles("dev")
@Transactional
public class UserRepositoryTests { }
```

Nous pouvons réduire la duplication ci-dessus en introduisant une annotation composée customisée :

Nous pouvons utiliser notre annotation

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@ContextConfiguration({"app-config.xml", "/test-data-access-
config.xml"})
@ActiveProfiles("dev")
@Transactional
public @interface TransactionalDevTest { }
```

customisée TransactionalDevTest pour simplifier la configuration des classes de tests individuels comme suit :

```
@RunWith(SpringJUnit4ClassRunner.class)
@TransactionalDevTest
public class OrderRepositoryTests { }

@RunWith(SpringJUnit4ClassRunner.class)
@TransactionalDevTest
public class UserRepositoryTests { }
```

2. Les arcanes du framework Spring TestContext

Le cœur du framework se compose de deux classes et de trois interfaces.

a. Classes et interfaces du framework de tests

Classe TestContextManager

Une par test JUnit. Elle manage un TestContext, met à jour l'état du test dans le contexte, signale au TestExecutionListener les événements enregistrés tels que les before et after ainsi que l'IoC, transaction...

Classe TestContext

Porte le contexte du test courant, et gère la mise en cache des contextes.

Interface TestExecutionListener

Gère les événements enregistrés tels que les before et after ainsi que l'IoC, transaction sur

les tests.

Interface ContextLoader

Charge le contexte d'application pour le test. Le `SmartContextLoader` peut charger les annotations. Peut aussi charger un contexte web.

Interface SmartContextLoader

C'est une extension de la l'interface `ContextLoader` qui la remplace depuis la version 3.1 de Spring. Elle permet de traiter des ressources localisées, des classes annotées, des initialiseurs de contextes.

Spring fournit un ensemble d'implémentation de ces interfaces.

Les chargeurs de contextes sont au nombre de deux :

- **DelegatingSmartContextLoader** : délègue à un `AnnotationConfigContextLoader` ou un `GenericXmlContextLoader` en fonction de la configuration déclarée.
- **WebDelegatingSmartContextLoader** : délègue à un `AnnotationConfigWebContextLoader` ou un `GenericXmlWebContextLoader` en fonction de la configuration déclarée.

Un contexte web `ContextLoader` ne sera utilisé que si l'annotation `@WebAppConfiguration` est présente sur la classe de test.

AnnotationConfigContextLoader

Charge un `ApplicationContext` standard pour une classe annotée.

AnnotationConfigWebContextLoader

Charge un `WebApplicationContext` standard pour une classe annotée.

GenericXmlContextLoader

Charge un `ApplicationContext` à partir d'une ressource XML.

GenericXmlWebContextLoader

Charge un `WebApplicationContext` à partir d'une ressource XML.

GenericPropertiesContextLoader

Charge un `ApplicationContext` à partir d'un fichier des propriétés Java.

b. Configuration du TestExecutionListener par les annotations

Les sections suivantes expliquent comment configurer le framework de `TestContext` avec les annotations et fournissent des exemples.

Spring fournit les implémentations `TestExecutionListener` suivantes qui sont enregistrées par défaut, exactement dans cet ordre :

<code>ServletTestExecutionListener</code>	Configure un mock de Servlet pour un <code>WebApplicationContext</code> .
<code>DependencyInjectionTestExecutionListener</code>	Fournit l'injection de dépendance pour le test.
<code>DirtyContextTestExecutionListener</code>	Gère l'annotation <code>@DirtyContext</code> .
<code>TransactionalTestExecutionListener</code>	Fournit l'exécution du test avec la sémantique transactionnelle d'annulation par défaut.
<code>SqlScriptsTestExecutionListener</code>	Exécute les scripts SQL configurés via l'annotation SQL.

c. Les TestExecutionListeners

Les listeners d'exécution des tests `TestExecutionListeners` sont implémentés sous la forme d'une liste de listeners qui sont appelés séquentiellement lors d'un test. Il est possible de personnaliser cette liste afin de maîtriser les différentes phases du test.

L'enregistrement personnalisé des `TestExecutionListeners` via l'annotation `@TestExecutionListeners` est adapté pour les listeners personnalisés qui sont utilisés dans des scénarios de test limités. Cependant, il peut devenir fastidieux d'utiliser un listener personnalisé dans une suite de tests. Pour résoudre ce problème, Spring Framework 4.1 prend en charge la découverte automatique des implémentations `TestExecutionListeners` via le mécanisme du `SpringFactoriesLoader`.

La liste des listeners par défaut est dans la liste `org.springframework.test.context.TestExecutionListener` dans le fichier de propriétés META-INF /spring.factories. Des frameworks tiers peuvent apporter leur propre `TestExecutionListener` à la liste des listeners par défaut de la même manière via leur propre fichier de propriétés META-INF /spring.factories.

```
# Default TestExecutionListeners for the Spring TestContext
Framework
#
org.springframework.test.context.TestExecutionListener = \
org.springframework.test.context.web.ServletTestExecutionListener,\
org.springframework.test.context.support.DependencyInjectionTest
ExecutionListener,\
org.springframework.test.context.support DirtiesContextTest
ExecutionListener,\
org.springframework.test.context.transaction.TransactionalTest
ExecutionListener,\
org.springframework.test.context.jdbc.SqlScriptsTestExecution
Listener
```

Ordonner les TestExecutionListeners

Lors de la découverte des listeners, Spring trie ceux-ci via `AnnotationAwareOrderComparator` qui implémente l'interface `Ordered` avec la valeur voulue.

Fusion des TestExecutionListeners

Si un `TestExecutionListener` customisé est enregistré, alors les listeners par défaut ne sont plus appelés. Il faut, dans ce cas, déclarer explicitement les listeners requis pour les tests, par exemple :

```
@ContextConfiguration
@TestExecutionListeners({
    MyCustomTestExecutionListener.class,
    ServletTestExecutionListener.class,
    DependencyInjectionTestExecutionListener.class,
    DirtiesContextTestExecutionListener.class,
    TransactionalTestExecutionListener.class,
    SqlScriptsTestExecutionListener.class
})
public class MonTest {
    // Contenu de la classe...
```

En cas de changement de version de Spring, il faut vérifier que la cohérence de la liste des listeners par défaut est toujours valide.

Pour pallier ce problème, Spring permet de fusionner la liste des listeners déclarée avec la liste des listeners par défaut via l'attribut `mergeMode` de

l'annotation `@TestExecutionListeners` qui doit alors être positionné à `MergeMode.MERGE_WITH_DEFAULTS`.

Par exemple, si dans la classe de l'exemple précédent avec l'ordre du listener `MyCustomTestExecutionListener` positionné à 500 qui est inférieur à l'ordre du listener `ServletTestExecutionListener`, à 1000, le `MyCustomTestExecutionListener` peut alors être automatiquement fusionné avec la liste des valeurs par défaut de la `ServletTestExecutionListener`, et l'exemple précédent peut être remplacé par le suivant.

```
@ContextConfiguration
@TestExecutionListeners(
    listeners = MyCustomTestExecutionListener.class,
    mergeMode = MergeMode.MERGE_WITH_DEFAULTS,
    public class MonTest {
        // Contenu de la classe...
```

Accès au contexte depuis le test

Chaque contexte `TestContext` gère son contexte de test. Une référence sur le contexte est accessible si la classe de test implémente l'interface `ApplicationContextAware`.

Les classes abstraites `AbstractJUnit4SpringContextTests` et `AbstractTestNGSpringContextTests` implémentent cette interface.

Une autre façon de procéder consiste à injecter le contexte dans la classe de test :

Pour un contexte simple :

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration
public class MonTest {
    @Autowired
    private ApplicationContext applicationContext;
    // Contenu de la classe...
```

Pour un contexte web :

L'injection est pourvue par le

```
@RunWith(SpringJUnit4ClassRunner.class)
@WebAppConfiguration
@ContextConfiguration
public class MyWebAppTest {
    @Autowired
    private WebApplicationContext wac;
    // Contenu de la classe...
```

listener `DependencyInjectionTestExecutionListener` qui est enregistré par défaut.

Les sections suivantes expliquent comment configurer un `ApplicationContext` via des fichiers de configuration XML, classes annotées (typiquement `ConfigurationClasses`), ou initialiseurs de contexte à l'aide de l'annotation `Spring ContextConfiguration`.

Alternativement, vous pouvez mettre en œuvre et configurer votre propre `SmartContextLoader` customisé pour des usages avancés.

Le chargement des fichiers XML de configuration se fait via un tableau de noms de fichiers. Le chemin des fichiers peut se baser sur `classpath:`, `file:` et `http:`.

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(locations={"/app-config.xml", "/test-config.xml"})
public class MonTest {
    // Contenu de la classe...
```

En général, nous pointons sur un fichier de configuration qui contient une inclusion du fichier principal de configuration de l'application, additionné des surcharges de beans et de configurations spécifiques aux tests.

Le `"locations="` est optionnel :

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration({"app-config.xml", "test-config.xml"})
public class MonTest {
    // Contenu de la classe...
```

Si nous ne spécifions pas les attributs `locations` et `value`, Spring essaie de déduire l'emplacement du fichier de configuration de tests. Les

chargeurs `GenericXmlContextLoader` et `GenericXmlWebContextLoader` se basent sur le nom du test avec le package, et cherche dans le classpath un fichier de configuration correspondant au nom de la classe suffixé par `-context.xml`.

Par exemple pour `com.example.MonTest`, le chargeur `GenericXmlContextLoader` charge le contexte de l'application depuis `"classpath:com/example/MonTest-context.xml"`.

```
package com.example;
@RunWith(SpringJUnit4ClassRunner.class)
// ApplicationContext sera chargé depuis :
// "classpath:com/example/MonTest-context.xml"
@ContextConfiguration
public class MonTest { [...]
```

Spring détecte grâce aux

chargeurs `AnnotationConfigContextLoader` et `AnnotationConfigWebContextLoader` les classes de configuration dans les classes internes statiques incluses dans la classe de test :

```
@RunWith(SpringJUnit4ClassRunner.class)
// ApplicationContext sera chargé depuis la classe interne
//
@ContextConfiguration
public class OrderServiceTest {
    @Configuration
    static class Config {
        @Bean
        public OrderService orderService() {
            OrderService orderService = new OrderServiceImpl();
            [...]
            return orderService;
        }
        @Autowired
        private OrderService orderService;
        @Test
        public void testOrderService() {
            [...]
        }
    }
}
```

d. Mélange XML et classes annotées

Il est possible de pointer vers une ressource de test (fichier ou classe annotée) qui inclut en son sein une configuration mixte. Pour des préoccupations liées à la maintenance, il faut veiller à être cohérent et constant dans les règles d'écriture de code dans votre application.

3. Configuration des contextes de tests

Les contextes de tests sont configurables à travers des annotations et des fichiers.

a. Configuration de contexte avec des initialiseurs de contexte

Pour utiliser des initialiseurs de contexte pour votre test, il faut annoter la classe avec l'annotation `@ContextConfiguration` et fournir à l'attribut `initializers` un tableau de classes implémentant `ApplicationContextInitializer`. Ces dernières sont alors utilisées pour initialiser le `ConfigurableApplicationContext` chargé pour le test.

L'ordre d'invocation des initialiseurs peut être fixé via l'utilisation de l'interface `Ordered`, l'annotation `@Order` ou l'annotation `@Priority`.

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(
    classes = TestConfig.class,
    initializers = TestAppCtxInitialiseur.class)
public class MonTest {
    // Contenu de la classe...
```

L'absence des déclarations des fichiers de configuration ou classe de configuration annotée n'empêche pas la déclaration des initialiseurs :

```
@RunWith(SpringJUnit4ClassRunner.class)
// ApplicationContext sera initialisée par le EntireAppInitializer
```

```
@ContextConfiguration(initializers = EntireAppInitializer.class)
public class MonTest {
// Contenu de la classe...
```

b. L'héritage dans la configuration des contextes

Nous pouvons désactiver l'héritage de configuration en mettant à false les attributs `inheritLocations` et `inheritInitializers` de l'annotation `@ContextConfiguration`. Ceci a pour effet qu'en cas d'héritage sur la classe de test, la classe héritée écrase la configuration de la classe dont elle hérite au lieu de la compléter au niveau de la localisation des fichiers et de la liste des initialiseurs.

Par exemple :

```
@RunWith(SpringJUnit4ClassRunner.class)
// ApplicationContext chargé depuis "/base-config.xml"
// à la racine du classpath
@ContextConfiguration("/base-config.xml")
public class BaseTest {
// Contenu de la classe...
// ApplicationContext chargé depuis "/base-config.xml"
// et "/extended-config.xml" à la racine du classpath
@ContextConfiguration("/extended-config.xml")
public class ExtendedTest extends BaseTest {
// Contenu de la classe...
```

La configuration de classe `ExtendedTest` est composée dans un premier temps par les valeurs de configuration de la classe `BaseTest` qui sont complétées par les valeurs de configuration de la classe `ExtendedTest`.

Si une des valeurs est en double, c'est la valeur de la configuration de la fille qui gagne.

Cela fonctionne de la même façon pour les configurations basées sur des classes annotées :

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = BaseConfig.class)
public class BaseTest {
// Contenu de la classe...
@ContextConfiguration(classes = ExtendedConfig.class)
public class ExtendedTest extends BaseTest {
// Contenu de la classe...
```

Le même principe s'applique pour les initialiseurs, au détail près que l'ordre est pris en compte :

```
@RunWith(SpringJUnit4ClassRunner.class)
// ApplicationContext initialisé par BaseInitializer
@ContextConfiguration(initializers = BaseInitializer.class)
public class BaseTest {
// Contenu de la classe...
// ApplicationContext initialisé par BaseInitializer
// et ExtendedInitializer
@ContextConfiguration(initializers = ExtendedInitializer.class)
public class ExtendedTest extends BaseTest {
// Contenu de la classe...
```

c. Prise en compte des profils d'environnement

Depuis Spring 3.1, il est possible de spécifier des profils d'environnement. Le profil sert alors de critère pour appliquer ou non un élément de la configuration du contexte Spring. L'annotation `@ActiveProfiles` permet de lister les profils pour lesquels il faut lancer le test. Il faut le chargeur `SmartContextLoader` pour prendre en compte les classes annotées. Le chargeur `ContextLoader` n'est pas suffisant.

Par exemple :

Fichier `app-config.xml` :

```
<!-- app-config.xml -->
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:jdbc="http://www.springframework.org/schema/jdbc"
xmlns:jee="http://www.springframework.org/schema/jee"
xsi:schemaLocation="...">
<bean id="transferService"
class="com.banque.service.internal.DefaultTransferService">
<constructor-arg ref="CompteDAO"/>
<constructor-arg ref="politiqueTarifaire"/>
</bean>
<bean id="CompteDAO"
class="fr.eni.DAO.internal.JdbcAccountRepository">
<constructor-arg ref="dataSource"/>
</bean>
<bean id="politiqueTarifaire"
class="fr.eni.service.internal.SansFrais"/>
<beans profile="dev">
<jdbc:embedded-database id="dataSource">
<jdbc:scriptlocation="classpath:com/banque/config/sql/schema.sql"/>
<jdbc:scriptlocation="classpath:com/banque/config/sql/test-data.sql"/>
</jdbc:embedded-database>
</beans>
<beans profile="production">
<jee:jndi-lookup id="dataSource" jndi-name="java:comp/env/jdbc/datasource"/>
</beans>
<beans profile="default">
<jdbc:embedded-database id="dataSource">
<jdbc:scriptlocation="classpath:com/banque/config/sql/schema.sql"/>
</jdbc:embedded-database>
</beans>
</beans>
```

Code java :

La source de données est configurée différemment en fonction des environnements :

```

package fr.eni.banque.service;
@RunWith(SpringJUnit4ClassRunner.class)
// ApplicationContext chargé depuis "classpath:/app-config.xml"
@ContextConfiguration("/app-config.xml")
@ActiveProfiles("dev")
public class TransferServiceTest {
    @Autowired
    private TransferService transferService;
    @Test
    public void testTransferService() {
        // test du transferService
    }
}

```

Production : base de données via le JNDI.

- Dev : base de données embarquée avec initialisation des valeurs.
- Default : base de données embarquée sans initialisation des valeurs.

Certains tests ne se font que dans des environnements spécifiques. Dans l'exemple précédent, le fait de spécifier `@ActiveProfiles("dev")` indique que les tests présents dans la classe `TransferServiceTest` ne se font que dans l'environnement de développement.

La configuration peut être différente en fonction des environnements. Par exemple, la source de données (`datasource`) peut être personnalisée en fonction de l'environnement dans lequel s'exécutent les tests. La base de données ciblée par les tests est en effet propre à chaque environnement.

Il y a la même logique avec les classes annotées. Si nous convertissons sous forme annotée la configuration précédente en utilisant quatre classes pour la configuration (pour HSQL), nous aurons :

```

@Configuration
@Profile("dev")
public class DonneesSeulesConfig {
    @Bean
    public DataSource dataSource() {
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.HSQL)
            .addScript("classpath:com/banque/config/sql/schema.sql")
            .addScript("classpath:com/banque/config/sql/test-data.sql")
            .build();
    }
}

@Configuration
@Profile("production")
public class DonneesJndiConfig {
    @Bean
    public DataSource dataSource() throws Exception {
        Context ctx = new InitialContext();
        return (DataSource) ctx.lookup("java:comp/env/jdbc/datasource");
    }
}

@Configuration
@Profile("default")
public class DonneesStandardsConfig {
    @Bean
    public DataSource dataSource() {
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.HSQL)
            .addScript("classpath:com/banque/config/sql/schema.sql")
            .build();
    }
}

@Configuration
public class TransferServiceConfig {
    @Autowired DataSource dataSource;
    @Bean
    public TransferService transferService() {
        return new DefaultTransferService(accountRepository(),
            feePolicy());
    }
    @Bean
    public AccountRepository accountRepository() {
        return new JdbcAccountRepository(dataSource);
    }
    @Bean
    public PolitiqueTarifaire() {
        return new SansFrais();
    }
}

package com.banque.service;
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = {
    TransferServiceConfig.class,
    DonneesStandardsConfig.class,
    DonneesJndiConfig.class,
    DefaultDataConfig.class})
@ActiveProfiles("dev")
public class TransferServiceTest {
    @Autowired
    private TransferService transferService;
    @Test
    public void testTransferService() {
        // test unitaire du service
    }
}

```

Nous obtenons les variantes suivantes :

Pour simplifier le code, nous utiliserons en général le principe de l'héritage :

Nous pouvons aussi nous servir de paramètres externes pour activer ou désactiver certains tests. Il est en effet facile de sélectionner les tests à activer en fonction de paramètres externes tels que :

TransferServiceConfig

Acquiert un dataSource via l'injection de

	dépendance à l'aide d'Autowired.
DonneesStandardsConfig	Définit un dataSource pour une base de données intégrée appropriée pour les tests de développeur.
DonneesJndiConfig	Définit un dataSource qui est récupéré à partir de JNDI dans un environnement de production.
DonneesStandardsConfig	Définit un dataSource pour une base de données par défaut intégrée dans le cas où aucun profil n'est actif.

```
package com.banque.service;
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = {
    TransferServiceConfig.class,
    DonneesStandardsConfig.class,
    DonneesJndiConfig.class,
    DonneesStandardsConfig.class})
@ActiveProfiles("dev")
public abstract class AbstractIntegrationTest {
    package com.banque.service;
    // "dev" profile hérite de la superclasse
    public class TransferServiceTest extends AbstractIntegrationTest {
        @Autowired
        private TransferService transferService;
        @Test
        public void testTransferService() {
            // Test unitaire du transferService
        }
    }
    package com.banque.service;
    // "dev" profile surchargé par "production"
    @ActiveProfiles(profiles = "production", inheritProfiles = false)
    public class ProductionTransferServiceTest extends
    AbstractIntegrationTest {
        // le test
    }
}
```

Le système d'exploitation.

- L'emplacement des tests, si par exemple on veut détecter que le test s'exécute sur un serveur de build d'intégration continue.
- La présence de certaines variables d'environnement.
- La présence d'annotations personnalisées de niveau de la classe.
- etc.

Nous utiliserons des `ActiveProfilesResolver` personnalisés et nous enregistrons les sélecteurs de profil via l'attribut `resolver` de l'annotation `@ActiveProfiles`.

Par exemple nous pouvons implémenter un sélecteur personnalisé `OperatingSystemActiveProfilesResolver` pour filtrer sur le système d'exploitation :

```
package com.banque.service;
// "dev" profile surchargé par un custom resolver
@ActiveProfiles(
    resolver = OperatingSystemActiveProfilesResolver.class,
    inheritProfiles = false)
public class TransferServiceTest extends AbstractIntegrationTest
{
    // test body
    package com.banque.service.test;
    public class OperatingSystemActiveProfilesResolver implements
    ActiveProfilesResolver {
        @Override
        String[] resolve(Class<?> testClass) {
            String profile = ...;
            return new String[] {profile};
        }
    }
}
```

d. Configuration de contexte avec des fichiers de propriétés de test

Depuis Spring 3.1, il existe la possibilité d'utiliser une hiérarchie des fichiers `.properties`. Depuis Spring 4.1, nous avons la même chose mais pour les tests en plus.

Contrairement à l'annotation `@PropertySource` utilisée sur des classes annotées avec `@Configuration`, l'annotation `@TestPropertySource` peut être déclarée sur une classe de test afin de déclarer des emplacements de ressources pour les fichiers de propriétés de test ou les propriétés "inline". Ces fichiers de propriétés de test seront ajoutés au set de l'environnement pour l'ensemble des `PropertySources` pour le contexte `ApplicationContext` chargés pour le test d'intégration annoté.

L'annotation `@TestPropertySource` n'est utilisable que via le `SmartContextLoader`.

e. Déclarer un fichier de propriétés pour les tests

Les fichiers de propriétés de test peuvent être configurés via les attributs `locations` de l'annotation `@TestPropertySource`.

L'annotation supporte les fichiers `.properties` et `.xml` avec un chemin relatif au classpath ou au système de fichiers :

- "classpath: /com/example/test.properties"
- "file: ///path/to/file.xml"

Chaque ressource est interprétée comme un fichier de ressources Spring.

Un path relatif sera interprété comme un chemin dans le package de la classe alors qu'un fichier commençant par un « / » sera pris comme dans le répertoire absolu correspondant. Un chemin au format URL sera interprété en fonction du préfixe `classpath:`, `file:`, `http:`.

Les jokers ne sont pas autorisés dans le nom du fichier.

```
@ContextConfiguration
@TestPropertySource ("/test.properties")
public class MyIntegrationTests {
// corps de la classe...
```

Les propriétés inline sont déclarées sous la forme de paires clé-valeur et peuvent être configurées via l'attribut `properties` de

l'annotation `@TestPropertySource`.

Toutes les paires clé-valeur seront ajoutées à l'environnement englobant. La syntaxe de prise en charge pour les paires clé-valeur est la même que la syntaxe définie pour les entrées dans un fichier de propriétés Java :

- "Clé = valeur"
- "Clé: valeur"
- "Valeur de la clé"

```
@ContextConfiguration
TestPropertySource ( properties= {"fuseau horaire GMT =", "port:
4242"})
public class MesTestsDIntegrationTests {
// corps de la classe ...
```

f. Détection du fichier de propriétés par défaut

Si l'annotation `@TestPropertySource` est vide (sans valeur pour les attributs `locations` ou `properties`), un fichier de propriété portant le nom de la classe sera cherché dans le répertoire correspondant au package de la classe.

S'il n'est pas trouvé, Spring lève une exception `IllegalStateException`.

Priorités :

- Propriétés inline,
- Fichiers de propriétés de tests,
- Propriétés de l'environnement du système d'exploitation,
- Propriétés système Java,
- Propriétés via l'annotation `@PropertySource`,
- Propriétés par programmation.

4. Hiérarchie des contextes de tests

Il est possible d'avoir une hiérarchie de configuration de contexte pour les tests.

a. Héritage et surcharge des propriétés de test

L'annotation `@TestPropertySource` a les attributs `inheritLocations` et `inheritProperties`.

- `inheritLocations` indique de quelle classe on hérite.
- `inheritProperties` indique le mode d'héritage. S'il est à `true`, les valeurs sont ajoutées ou écrasent les valeurs de la classe dont on hérite. S'il est à `false`, les propriétés masquent et remplacent les propriétés de la classe d'origine.

```
@TestPropertySource ("Base.properties")
@ContextConfiguration
public class BaseTest {
// ...
@TestPropertySource("extended.properties")
@ContextConfiguration
public class ExtendedTest extends BaseTest {
// ...
```

Dans l'exemple suivant,

l'`ApplicationContext` pour `BaseTest` sera chargé en utilisant seulement la propriété inline `key1`. En revanche, le `ApplicationContext` pour `ExtendedTest` sera chargé en utilisant les propriétés inline `Key1` et `Key2`.

```
TestPropertySource (Propriétés = "key1 = value1")
ContextConfiguration
public class BaseTest {
// ...
TestPropertySource (Propriétés = "key2 = value2")
ContextConfiguration
public class ExtendedTest extends BaseTest {
```



```
// ...
```

b. Chargement d'un `WebApplicationContext`

Spring 3.2 a introduit le support de l'annotation `@WebApplicationContext` qui permet d'avoir un contexte de test spécifique pour les tests Web. Il suffit d'annoter une classe de test avec l'annotation `@WebApplicationContext` pour bénéficier du framework de test `TestContext`.

Le framework fournit un mock `MockServletContext` qui pointera sur le répertoire web par défaut "src/main/webapp". Il est possible de changer le chemin par défaut en ajoutant un argument à l'annotation `@WebAppConfiguration`.

Le `WebApplicationContext` fonctionne comme le `ApplicationContext`. Il est donc possible de le configurer via un fichier de configuration XML, une annotation `@Configuration` via l'annotation `@ContextConfiguration`. Il est également possible d'utiliser les autres annotations comme `@TestExecutionListeners`, `@TransactionConfiguration`, `@ActiveProfiles`, etc.

c. Conventions

Par convention, Spring cherche le fichier associé au test à partir du nom et du package dans lequel se trouve le test.

L'exemple suivant montre que Spring recherche le fichier `WacTests-context.xml` dans le répertoire du package associé à la classe.

```
@RunWith(SpringJUnit4ClassRunner.class)
@WebAppConfiguration
@ContextConfiguration
public class WacTests {
//...
```

L'exemple suivant montre comment déclarer explicitement un chemin du fichier de ressources avec

l'annotation `@WebAppConfiguration` et un emplacement de ressources XML avec `@ContextConfiguration`. La chose importante à noter ici est les différentes sémantiques pour les chemins avec ces deux annotations. Par défaut, les chemins de ressources `@WebAppConfiguration` sont basés sur un système de fichiers, alors que l'emplacement de ressources est basé sur le classpath avec l'annotation `@ContextConfiguration`.

```
@RunWith(SpringJUnit4ClassRunner.class)
// Ressource du système de fichiers
@WebAppConfiguration("webapp")
// classpath resource
@ContextConfiguration("/spring/test-servlet-config.xml")
public class WacTests {
//...
```

d. La sémantique de ressources explicite

Dans ce troisième exemple, nous voyons que nous pouvons remplacer la sémantique de ressources par défaut pour les deux annotations en spécifiant un préfixe de ressources de Spring.

```
@RunWith(SpringJUnit4ClassRunner.class)
// classpath resource
@WebAppConfiguration("classpath:test-web-resources")
// Ressource du système de fichiers
@ContextConfiguration("file:src/main/webapp/WEB-INF/servlet-config.xml")
public class WacTests {
//...
```

Afin d'avoir un support de test web complet, Spring 3.2 introduit un

`listenerServletTestExecutionListener` qui est activé par défaut.

Lors de l'utilisation d'un contexte `WebApplicationContext`, ce `TestExecutionListener` met en place l'état des thread-local via le `RequestContextHolder` de Spring Web avant chaque méthode de test et crée un `MockHttpServletRequest`, `MockHttpServletResponse` et `ServletWebRequest` basés sur le chemin de la ressource de base configuré via `WebAppConfiguration`. `ServletTestExecutionListener` s'assure également que le `MockHttpServletResponse` et le `ServletWebRequest` peuvent être injectés dans l'instance de test, et une fois que le test est terminé, il nettoie l'état des thread-local.

Une fois que vous avez un `WebApplicationContext` chargé pour votre test, vous pourriez découvrir que vous avez besoin d'interagir avec les mocks Webs - par exemple, pour configurer votre jeu d'essai ou pour effectuer des assertions après l'appel de votre composant web. L'exemple suivant montre quels mocks peuvent être autowired dans votre instance de test. Notez que le `WebApplicationContext` et le `MockServletContext` sont tous deux mis en cache dans la suite de tests, alors que les autres mocks qui sont gérés par des méthodes de test le sont par le `ServletTestExecutionListener`.

e. Injecter des objets mockés

Nous pouvons ajouter des objets mockés présents dans le package `org.springframework.mock.web` comme dans l'exemple suivant :

```
@WebAppConfiguration
@ContextConfiguration
public class WacTests {
    @Autowired
    WebApplicationContext wac; // cached
    @Autowired
    MockServletContext servletContext; // cached
    @Autowired
    MockHttpSession session;
    @Autowired
    MockHttpServletRequest request;
    @Autowired
    MockHttpServletResponse response;
    @Autowired
```

```
ServletWebRequest webRequest;  
//...
```

f. Cachabilité du contexte de test

Le framework de test `TestContext` charge le contexte une seule fois pour une même configuration de tests. Pour se repérer, Spring génère une clé à partir de la configuration de tests avec les éléments suivants :

- locations (de `@ContextConfiguration`),
- classes (de `@ContextConfiguration`),
- contextInitializerClasses (de `@ContextConfiguration`),
- contextLoader (de `@ContextConfiguration`),
- parent (de `@ContextHierarchy`),
- activeProfiles (de `@ActiveProfiles`),
- propertySourceLocations (de `@TestPropertySource`),
- propertySourceProperties (de `@TestPropertySource`),
- resourceBasePath (de `@WebAppConfiguration`).

Il charge alors le contexte dans un contexte statique et le stocke dans un cache statique avec cette clé.

Par exemple, si la classe `TestClassA` spécifie `{"app-config.xml", "test-config.xml"}` pour les attributs locations (ou value) et les emplacements de `@ContextConfiguration`, le framework `TestContext` charge le contexte correspondant et le stocke dans un cache statique sous une clé qui est basée uniquement sur ces emplacements. Donc, si la classe `TestClassB` définit également `{"app-config.xml", "test-config.xml"}` pour ses emplacements (explicitement ou implicitement par héritage), mais ne définit pas `@WebAppConfiguration`, et ne définit pas non plus un autre `ContextLoader` ou des profils actifs différents ou des initialiseurs de contextes différents ou différents fichiers de propriété de test ou un contexte parent différent, alors le même `ApplicationContext` sera partagé par les deux classes de test. Cela signifie que le coût d'installation pour charger un contexte d'application n'est engagé qu'une seule fois (par suite de tests), et l'exécution du test suivant est beaucoup plus rapide.

➤ Afin de vérifier la cohérence vis-à-vis du nombre de contextes créés, il est possible d'afficher les statistiques du cache en positionnant à `DEBUG` le niveau de log `de.org.springframework.test.context.cache logging category`.

En cas de test sur des altérations volontaires du contexte, il est possible de marquer la méthode de test avec l'annotation `@DirtiesContext` afin de forcer le rechargement du contexte après l'exécution de la méthode. Il est également possible d'annoter la classe ce qui a pour effet de recharger le contexte après chaque appel de méthode, mais cette possibilité est à utiliser avec parcimonie car le chargement du contexte prend du temps.

Le support de cette annotation est fourni par le listener `DirtiesContextTestExecutionListener` qui est activé par défaut.

g. Les hiérarchies des contextes

Nous utiliserons, pour les tests simples, un environnement mono-contexte. Cependant, dès que nous commencerons à faire des tests un peu complexes, nous aurons une hiérarchie de contextes afin d'éviter les éléments redondants dans des tests différents. La bonne idée consiste à réutiliser un maximum de choses entre des tests.

Par exemple, si vous développez une application web Spring MVC, vous aurez généralement un contexte racine `WebApplicationContext` chargé via le `ContextLoaderListener` et un contexte enfant `WebApplicationContext` chargé via le `DispatcherServlet`. Il en résulte une hiérarchie de contextes parent-enfant où les composants partagés et la configuration de l'infrastructure sont déclarés dans le contexte racine et consommés dans le contexte de l'enfant par des composants spécifiques sur le Web. Un autre cas d'utilisation se présente pour les applications de traitement par lots (batches) pour lesquels vous avez souvent un contexte parent qui fournit la configuration de l'infrastructure de lot partagé et un contexte enfant pour la configuration d'un travail d'un lot spécifique.

Depuis Spring Framework 3.2.2, il est possible d'écrire des tests d'intégration qui utilisent les hiérarchies de contextes en déclarant une configuration de contexte par l'annotation `@ContextHierarchy`. Si une hiérarchie de contextes est déclarée sur plusieurs classes dans une hiérarchie de classes de tests, nous pourrions fusionner ou remplacer la configuration de contextes pour un niveau nommé spécifique dans la hiérarchie de contextes. Pour que la fusion de configurations pour un niveau donné dans la hiérarchie du type de ressources de configuration soit opérationnelle, il faudra que les fichiers de configuration XML ou classes annotées soient cohérents. Nous aurons ainsi différents niveaux dans une hiérarchie de frameworks, et celle-ci sera configurée en utilisant différents types de ressources.

Les exemples à base de JUnit suivants démontrent des scénarios de configurations communes pour les tests d'intégration qui nécessitent l'utilisation de hiérarchies de contextes.

Le `ControllerIntegrationTests` représente un scénario typique de tests d'intégration pour une application web Spring MVC. Il déclare une hiérarchie de frameworks composée de deux niveaux, l'un pour la racine `WebApplicationContext` (chargé à l'aide de la classe `TestAppConfig` annotée par `@Configuration`) et l'autre pour le servlet répartiteur `WebApplicationContext` (chargé à l'aide de la classe `WebConfig` annotée par `@Configuration`). Le `WebApplicationContext` qui est injecté dans l'instance de test est celui du contexte enfant (c'est-à-dire le contexte le plus bas dans la hiérarchie).

```
@RunWith(SpringJUnit4ClassRunner.class)  
@WebAppConfiguration  
@ContextHierarchy({  
    @ContextConfiguration(classes = TestAppConfig.class),  
    @ContextConfiguration(classes = WebConfig.class)  
})  
public class ControllerIntegrationTests {  
    @Autowired  
    private WebApplicationContext wac;  
    // ...
```

Les classes de test suivantes définissent une hiérarchie de contextes dans une hiérarchie de classes de test. La classe `AbstractWebTests` déclare la configuration pour une

racine `WebApplicationContext` dans une application web Spring. Notez cependant que `AbstractWebTests` ne déclare pas `@ContextHierarchy`. Les sous-classes de `AbstractWebTests` peuvent éventuellement participer à une hiérarchie de contextes, il suffit de suivre la sémantique standard pour `@ContextConfiguration`. La classe `SoapWebServiceTests` et `RestWebServiceTests` permet à la fois d'étendre `AbstractWebTests` et de définir une hiérarchie contextuelle via `@ContextHierarchy`. Le résultat fait que les trois contextes d'application seront chargés (un pour chaque déclaration de `@ContextConfiguration`). Le contexte d'application chargé en fonction de la configuration dans `AbstractWebTests` est fixé par le contexte de parent pour chacun des contextes chargés pour les sous-classes concrètes.

```
@RunWith(SpringJUnit4ClassRunner.class)
@WebAppConfiguration
@ContextConfiguration("file:src/main/webapp/WEB-INF/applicationContext.xml")
public abstract class AbstractWebTests {
    @ContextHierarchy({ @ContextConfiguration("/spring/soap-ws-config.xml")
    public class SoapWebServiceTests extends AbstractWebTests {}
    @ContextHierarchy({ @ContextConfiguration("/spring/rest-ws-config.xml")
    public class RestWebServiceTests extends AbstractWebTests {}
```

Les classes suivantes illustrent l'utilisation des niveaux hiérarchiques nommés afin de fusionner la configuration des niveaux spécifiques dans une hiérarchie de contexte. `BaseTests` définit deux niveaux de la hiérarchie, parent et

enfant. `ExtendedTests` étend `BaseTests` et charge le framework `TestContext` afin de fusionner la configuration de contexte pour le niveau de la hiérarchie enfant, tout simplement en faisant en sorte que les noms déclarés via l'attribut `name` du `ContextConfiguration` soient tous « enfants ».

Le résultat est que trois contextes applicatifs sont chargés :

- un pour `"/app-config.xml"`,
- un pour `"/user-config.xml"`,
- un pour `"/user-config.xml"`,

en plus de celui de `"/order-config.xml"`.

Comme dans l'exemple précédent, le contexte d'application chargé de `"/app-config.xml"` sera défini comme le contexte parent pour les contextes chargés de `"/user-config.xml"` et `"/user-config.xml"`, `"/order-config.xml"`.

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextHierarchy({
    @ContextConfiguration(name = "parent", locations = "/app-config.xml"),
    @ContextConfiguration(name = "child", locations = "/user-config.xml")
})
public class BaseTests {
    @ContextHierarchy({
    @ContextConfiguration(name = "child", locations = "/order-config.xml")
```

Contrairement à l'exemple précédent, cet exemple montre comment remplacer la configuration pour un niveau nommé donné dans une hiérarchie de contexte en mettant les

flags `inheritLocations` de `ContextConfiguration` à `false`. Par conséquent, le contexte d'application pour `ExtendedTests` ne sera chargé qu'à partir de `"/test-user-config.xml"` et aura son parent réglé sur le framework chargé de `"/app-config.xml"`.

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextHierarchy({
    @ContextConfiguration(name = "parent", locations = "/app-config.xml"),
    @ContextConfiguration(name = "enfant", locations = "/user-config.xml")
})
public class BaseTests {
    @ContextHierarchy({
    @ContextConfiguration(
        name = "enfant",
        locations = "/test-user-config.xml",
        inheritLocations = false
    ))
    public class ExtendedTests extends BaseTests {}
```

h. L'injection de dépendance dans les tests

Lorsque vous utilisez le `DependencyInjectionTestExecutionListener`, qui est configuré par défaut, les dépendances de vos instances de test sont injectées à partir de beans dans le contexte de l'application que vous avez configuré avec le `ContextConfiguration`.

Le framework `TestContext` ne se soucie pas de la façon dont une instance de test est instanciée. Ainsi, l'utilisation de `@Autowired` ou `@Inject` pour les constructeurs n'a aucun effet pour les classes de test. Les champs injectés par l'annotation `@Autowired` sont identifiés par leur type. Afin de les différencier, il faut utiliser l'annotation `@Qualifier`.

À partir de Spring 3, il est possible d'utiliser l'annotation `@Inject` en conjonction avec l'annotation `@Named`. Alternativement, si votre classe de test a accès à son `ApplicationContext`, vous pouvez effectuer une recherche explicite en utilisant (par exemple) un appel à `applicationContext.getBean("monDAO")`.

Si vous ne voulez pas que l'injection de dépendance soit appliquée à vos instances de test, il suffit de ne pas annoter des champs ou des méthodes setter avec `@Autowired` ou `@Inject`. Alternativement, vous pouvez désactiver l'injection de dépendance tout en configurant explicitement votre classe avec `@TestExecutionListeners` et en omettant `DependencyInjectionTestExecutionListener.class` de la liste des listeners.

Par exemple :

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration("dao-config.xml")
public class HibernateTitreDAOtests {
    @Autowired
    private HibernateTitreDAOtitreDAO;
    @Test
    public void findById() {
```

Alternativement, vous pouvez utiliser `@Autowired` pour configurer la classe pour l'injection :

Les listes de code précédentes utilisent le même fichier de contexte XML (`dao-config.xml`) référencé par

```

        Titre titre = titreDAO.findById(new Long(10));
        assertNotNull(titre);
    }
}

```

```

@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration("dao-config.xml")
public class HibernateTitreDAOTests {
    private HibernateTitreDAO titreDAO;

    @Autowired
    public void setTitreDAO(HibernateTitreDAO titreDAO) {
        this.titreDAO = titreDAO;
    }

    @Test
    public void findById() {
        Titre titre = titreDAO.findById(new Long(10));
        assertNotNull(titre);
    }
}

```

l'annotation `@ContextConfiguration` :

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">
    <!-- this bean will be injected into the HibernateTitre
    HibernateTitreDAO DaoTests class -->
    <bean id="titreDAO" class="fr.eni.dao.hibernate.
    HibernateTitreDAO">
        <property name="sessionFactory" ref="sessionFactory"/>
    </bean>
    <bean id="sessionFactory"
    class="org.springframework.orm.hibernate3.LocalSessionFactoryBean">
    <!-- configuration elided for brevity -->
    </bean>
</beans>

```

5. Le scope session lors d'un test de requête

Il est possible d'utiliser les scopes pour les beans lors des tests.

a. Le scope session lors d'un test de requête

Les beans de scope session sont disponibles dans Spring depuis longtemps mais restent fastidieux à tester. Depuis Spring 3.2, il existe une façon simplifiée de faire ces tests :

- Veillez à ce qu'un `WebApplicationContext` soit chargé pour votre test en annotant votre classe de test avec `@WebAppConfiguration`.
- Injectez le mock de la requête ou une session dans votre instance de test et préparez le jeu d'essais.
- Appelez votre composant web que vous avez récupéré du `WebApplicationContext` configuré (c'est-à-dire, via l'injection de dépendance).
- Effectuez les assertions depuis les mocks.

Configuration des beans Request-scope

Par exemple, le `loginAction` est instancié en utilisant des expressions Spel qui récupèrent le nom de l'utilisateur et mot de passe à partir de la requête HTTP actuelle. Dans notre test, nous allons configurer ces paramètres de la demande par l'intermédiaire du mock géré par le framework `TestContext`.

```

<beans>
    <bean id="userService"
        class="com.example.SimpleUserService"
        c:loginAction-ref="loginAction" />
    <bean id="loginAction" class="com.example.LoginAction"
        c:username="{request.getParameter('user')}"
        c:password="{request.getParameter('pswd')}"
        scope="request">
        <aop:scoped-proxy />
    </bean>
</beans>

```

b. Test de beans de scope request

Dans `RequestScopedBeanTests`, nous injectons la méthode `loginUser()` qui est invoquée sur notre `userService` ; nous sommes assurés que le service de l'utilisateur a accès au bean de scope request `loginAction` pour le mock. Nous pouvons alors renseigner les données connues pour le nom d'utilisateur et le mot de passe.

```

@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration
@WebAppConfiguration
public class RequestScopedBeanTests {
    @Autowired UserService userService;
    @Autowired MockHttpServletRequest request;
}

```

```

@Test
public void requestScope() {
    request.setParameter("user", "enigma");
    request.setParameter("pswd", "$pring");
    LoginResults results = userService.loginUser();
    // assert results
}

```

c. Configuration d'un bean de scope session

Le bean `userService` a une dépendance sur un bean de scope session `UserPreferences`. Notez que le bean `UserPreferences` est instancié en utilisant une expression Spel qui récupère le thème de la session HTTP en cours. Dans notre test, nous aurons besoin de configurer un thème au mock de la session géré par le cadre `TestContext`.

```

<beans>
<bean id = "userService"
    classe = "com.example.SimpleUserService"
    c: UserPreferences-ref = "PréférencesUtilisateur" />
<bean id = "userPreferences"
    classe = "com.example.UserPreferences"
    c:theme = "#{session.getAttribute( 'theme ')}"
    scope = "session">
<Aop: scope-proxy />
</ bean>
</ Beans>

```

Bean de scope session

Le bean `userService` a une dépendance sur un bean de scope session `UserPreferences`. Notez que le bean `UserPreferences` est instancié en utilisant une expression Spel qui récupère le thème de la session HTTP en cours. Dans notre test, nous aurons besoin de configurer un thème au mock de la session géré par le cadre `TestContext`.

```

@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration
@WebAppConfiguration
public class SessionScopedBeanTests {
    @Autowired UserService userService;
    @Autowired MockHttpSession session;
    @Test
    public void sessionScope() throws Exception {
        session.setAttribute("theme", "Terminator");
        Results results = userService.processUserPreferences();
        // assert results
    }
}

```

6. Les transactions

Les transactions sont importantes pour les tests. On souhaite en effet soit conserver la base de données en l'état, soit garder les modifications une fois les tests terminés.

a. La gestion des transactions

Dans le framework `TestContext`, les transactions sont gérées déclarativement par le `TransactionalTestExecutionListener` qui est configuré par défaut, même si vous ne déclarez pas explicitement `TestExecutionListeners` sur votre classe de test.

Pour activer le support pour les transactions, vous devez configurer un `beanPlatformTransactionManager` dans le `ApplicationContext` qui est chargé par la sémantique du `@ContextConfiguration`. En outre, vous devez déclarer l'annotation `Spring@Transactional` au niveau de la classe ou d'une méthode pour vos tests.

b. Les transactions managées par le test

Les transactions managées par les tests sont des opérations qui sont gérées de façon déclarative via le `TransactionalTestExecutionListener` ou par programmation via `TestTransaction` (voir ci-dessous). Ces opérations ne doivent pas être confondues avec les transactions gérées directement par Spring dans le `ApplicationContext` chargé pour les tests ou avec les transactions applicatives gérées par programmation dans le code de l'application qui est invoquée par des tests.

Les transactions gérées par Spring ou par l'application participent à des transactions pilotées par les tests. Au niveau de la propagation des transactions, il faut être vigilant si les transactions ne sont pas en mode `REQUIRED` ou `SUPPORTS`.

Comme nous l'avons déjà évoqué, Spring peut annuler des transactions pour retrouver la base de données dans son état d'origine, mais il ne peut le faire que pour des transactions imbriquées. Si une partie du code se trouve dans une transaction isolée non gérée par les tests, la mise à jour se fera normalement ce qui altérera le jeu d'essais.

c. Activation et désactivation des transactions

Annoter une méthode de test avec `@Transactional` fait que le test est exécuté dans une transaction qui est automatiquement annulée après l'achèvement du test.

Si une classe de test est annotée avec `@Transactional`, chaque méthode de test au sein de cette hiérarchie de classe sera exécutée dans une transaction. Les méthodes de test qui ne sont pas annotées avec `@Transactional` (au niveau de la classe ou de la méthode) ne seront pas exécutées dans une transaction. En outre, des tests qui sont annotés avec `@Transactional` mais qui ont le type de propagation `NOT_SUPPORTED` ne seront pas exécutés dans une transaction.

Notez

que `AbstractTransactionalJUnit4SpringContextTests` et `AbstractTransactionalTestNGSpringContextTests` sont préconfigurés pour un support transactionnel au niveau de la classe.

L'exemple suivant montre un scénario commun pour écrire un test d'intégration pour une base d'Hibernate `UserRepository`.

La base est remise dans son état d'origine après le test via un rollback. Par défaut, la transaction est annulée via un rollback à la fin du test.

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = TestConfig.class)
@Transactional
public class HibernateUserRepositoryTests {
    @Autowired
    HibernateUserRepository repository;
    @Autowired
    SessionFactory sessionFactory;
    @Autowired
    JdbcTemplate jdbcTemplate;
    public void setDataSource(DataSource dataSource) {
        this.jdbcTemplate = new JdbcTemplate(dataSource);
    }
    @Test
    public void createUser() {
        // track initial state in test database:
        final int count = countRowsInTable("user");
        User user = new User(...);
        repository.save(user);
        // Manual flush is required to avoid false positive in test
        sessionFactory.getCurrentSession().flush();
        assertEquals("Number of rows in the 'user' table.", expected,
            countRowsInTable("user"));
    }
    protected int countRowsInTable(String tableName) {
        return JdbcTestUtils.countRowsInTable(this.jdbcTemplate, tableName);
    }
    protected void assertEquals(int expected) {
        assertEquals("Number of rows in the 'user' table.", expected,
            countRowsInTable("user"));
    }
}
```

d. Comportement du commit et du rollback d'une transaction

Par défaut, les opérations de test seront automatiquement annulées après l'achèvement du test. Toutefois, ce comportement peut être ajusté via l'annotation `@TransactionConfiguration` sur la classe ou l'annotation `@Rollback` sur la méthode.

Gestion des transactions programmatique

Depuis Spring Framework 4.1, il est possible d'interagir avec les transactions de test gérées par programmation via les méthodes statiques dans `TestTransaction`.

Par exemple, le `TestTransaction` peut être utilisé dans les méthodes de tests, avant les méthodes, et après les méthodes pour commencer ou terminer la transaction de test courante ou pour demander à la transaction de test courante de faire une annulation ou une validation.

Le support pour le `TestTransaction` est automatiquement disponible lorsque `TransactionalTestExecutionListener` est activé.

L'exemple suivant illustre certaines des caractéristiques de `TestTransaction`.

```
Code:
@ContextConfiguration(classes = TestConfig.class)
public class ProgrammaticTransactionManagementTests extends
AbstractTransactionalJUnit4SpringContextTests {
    @Test
    public void transactionalTest() {
        // assert initial state in test database:
        assertEquals(2);
        deleteFromTables("user");
        // changes to the database will be committed!
        TestTransaction.flagForCommit();
        TestTransaction.end();
        assertFalse(TestTransaction.isActive());
        assertEquals(0);
        TestTransaction.start();
        [...]
    }
    protected void assertEquals(int expected) {
        assertEquals("Number of rows in the 'user' table.", expected,
            countRowsInTable("user"));
    }
}
```

e. Exécution de code en dehors d'une transaction

Occasionnellement, vous devez exécuter certains codes avant ou après une méthode de test transactionnelle mais en dehors du contexte transactionnel -par exemple, pour vérifier l'état d'une base de données initiale avant l'exécution de votre test ou pour vérifier un commit prévu.

Le `TransactionalTestExecutionListener` soutient les annotations `@BeforeTransaction` et `@AfterTransaction` exactement pour ces scénarios.

Il suffit d'annoter une méthode de type `public void` dans votre classe de test avec une de ces annotations, et le `TransactionalTestExecutionListener` garantira que votre méthode de transaction avant ou après la méthode de la transaction sera bien exécutée au moment opportun.

f. Configuration d'un gestionnaire de transactions

Le `TransactionalTestExecutionListener` attend qu'un `beanPlatformTransactionManager` soit défini dans le `ApplicationContext` pour le test. Dans le cas où il y a plusieurs instances de `PlatformTransactionManager` au sein

du `ApplicationContext`, `TransactionConfiguration` prend en charge la configuration du nom du bean de la `PlatformTransactionManager` qui devrait être utilisé pour conduire les opérations.

Alternativement, nous pouvons le déclarer par l'intermédiaire de `@Transactional` ("myQualifier"), et le `TransactionManagementConfigurer` peut être mis en œuvre par une classe `Configuration`.

g. Démonstration de toutes les annotations liées à la transaction

L'exemple suivant, à base de JUnit, affiche un exemple d'utilisation des transactions dans les tests.

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration
@Transactional(transactionManager="txMgr",
defaultRollback=false)
public class TransactionFictiveTest {
    @BeforeTransaction
    public void verifyInitialDatabaseState() {
        [...]
    }
    @Before
    public void setUpTestDataWithinTransaction() {
        [...]
    }
    @Rollback(true)
    public void modifyDatabaseWithinTransaction() {
        [...]
    }
    @After
    public void tearDownWithinTransaction() {
        [...]
    }
    @AfterTransaction
    public void verifyFinalDatabaseState() {
        [...]
    }
}
```

Avec Hibernate, il faut vider le cache afin d'éviter les faux positifs.

```
@Autowired
private SessionFactory sessionFactory;
@Test public void falsePositive() {
    updateEntityInHibernateSession();
    // Faux positif !
    @Test(expected = GenericJDBCException.class)
    public void updateWithSessionFlush() {
        updateEntityInHibernateSession();
        // Nous vidons le cache pour avoir les données à jour.
        sessionFactory.getCurrentSession().flush();
        // ...
    }
}
```

7. Les scripts SQL

Spring permet de spécifier le format de la base de données et les données à injecter dans la base sous forme de scripts SQL.

a. Exécuter des scripts SQL

Lors de l'écriture des tests d'intégration avec une base de données relationnelle, l'utilisation de scripts SQL est souvent pratique pour modifier le schéma de base de données ou insérer des données de test dans des tables.

Le module `Spring-jdbc` fournit un support pour l'initialisation de manière intégrée ou sur une base de données existante en exécutant les scripts SQL lorsque l'`ApplicationContext` Spring est chargée. Bien qu'il soit très utile pour initialiser une base de données pour tester une fois lorsque le `ApplicationContext` est chargé, il est parfois indispensable de pouvoir modifier la base de données au cours de tests d'intégration.

Exécuter des scripts SQL programmatiquement

Spring fournit les options suivantes pour exécuter des scripts SQL programmatiquement au sein de méthodes des tests d'intégration.

- `org.springframework.jdbc.datasource.init.ScriptUtils`
- `org.springframework.jdbc.datasource.init.ResourceDatabasePopulator`
- `org.springframework.test.context.junit4.AbstractTransactionalJUnit4 SpringContextTests`
- `org.springframework.test.context.testng.AbstractTransactionalTestNGSpringContextTests`

`ScriptUtils` fournit une collection de méthodes utilitaires statiques pour travailler avec des scripts SQL et il est principalement destiné à un usage interne dans le framework. Toutefois, si vous avez besoin d'un contrôle complet de l'analyse et de l'exécution du SQL, `ScriptUtils` répond à vos besoins mieux que d'autres alternatives décrites ci-dessous.

`ResourceDatabasePopulator` fournit une API orientée objet simple pour manipuler la base de données à l'aide de scripts SQL définis dans les ressources externes.

`ResourceDatabasePopulator` offre des options pour configurer l'encodage de caractères, les séparateurs de déclaration, les séparateurs de commentaires, et la manipulation des flags d'erreurs utilisés lors de l'analyse et l'exécution des scripts, et chacune des options de configuration a une valeur par défaut.

Pour exécuter les scripts configurés dans un `ResourceDatabasePopulator`, vous pouvez invoquer soit la méthode `populate(Connection)` pour exécuter le provisionning à partir d'un `java.sql.Connection`, soit la

méthode `execute(dataSource)` pour exécuter le provisionning à partir d'un `javax.sql.DataSource`.

Par exemple pour un schéma de test et des données de test, avec un séparateur de déclaration à "@@", avec exécution d'un script depuis un `DataSource` :

```
@Test
public void databaseTest {
    ResourceDatabasePopulator populator = new
    ResourceDatabasePopulator();
    populator.addScripts(
    new ClassPathResource("test-schema.sql"),
    new ClassPathResource("test-data.sql"));
    populator.setSeparator("@@");
    populator.execute(this.dataSource);
    // execute code that uses the test schema and data
}
```

Notez

que

le `ResourceDatabasePopulator` interne délègue à la classe `ScriptUtils` l'analyse et l'exécution de scripts SQL.

De même, les méthodes `executeSqlScript(..)` utilisent un `ResourceDatabasePopulator` pour exécuter des scripts.

Exécuter des scripts SQL déclarativement avec SQL

Les scripts SQL peuvent également être configurés de manière déclarative dans le framework Spring TestContext. Plus précisément, l'annotation `@Sql` peut être déclarée sur une méthode ou une classe de test pour configurer les chemins de ressources aux scripts SQL qui doivent être exécutés sur une base de données avant ou après une méthode de test d'intégration.

Notez que les déclarations au niveau de la méthode surchargent les déclarations au niveau de la classe et que le support de `@Sql` est fourni par le listener `SqlScriptsTestExecutionListener` qui est activé par défaut.

b. La sémantique du path des ressources

Par exemple, "schema.sql" est traité comme une ressource de chemin de classe qui est relatif au package dans lequel la classe de test est définie. Un chemin commençant par un slash sera traité comme un chemin de classe comme dans cet exemple : `"/org/example/schema.sql"`. Un chemin qui fait référence à une URL (par exemple, un chemin avec le préfixe `classpath:`, `file:`, `http:`, etc.) sera chargé en utilisant le protocole de ressource spécifié.

L'exemple suivant montre comment utiliser SQL au niveau de la classe et de la méthode dans une classe de test d'intégration basée sur JUnit :

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration
@Sql("/test-schema.sql")
public class DatabaseTests {
    @Test
    public void emptySchemaTest {
        [...]
    }
    @Test
    @Sql({"test-schema.sql", "/test-user-data.sql"})
    public void userTest {
        [...]
    }
}
```

c. La détection de script par défaut

Si aucun des scripts SQL n'est spécifié, une tentative sera faite pour détecter un script par défaut en fonction de l'endroit où l'annotation `@Sql` est déclarée.

Spring lève une exception `IllegalStateException` s'il ne trouve pas de fichier.

- déclaration de niveau de classe : si la classe de test annotée est `com.example.MonTest`, le script par défaut correspondant est `"classpath: com / example / MonTest.sql"`.
- déclaration du niveau de la méthode : si la méthode de test annotée est nommée `testMethod()` et est définie dans la classe `com.example.MonTest`, le script par défaut correspondant est `"classpath: com / example / MonTest.testMethod.sql"`.

d. Déclaration de plusieurs @Sql

Si plusieurs ensembles de scripts SQL doivent être configurés pour une classe de test donnée ou pour la méthode d'essai, mais avec une configuration différente de la syntaxe, des règles différentes de traitement des erreurs, ou différentes phases d'exécution, il est possible de déclarer plusieurs instances de SQL.

Avec Java 8, SQL peut être utilisé comme une annotation « répétable ». Dans le cas contraire, l'annotation `@SqlGroup` peut être utilisée comme un récipient explicite pour déclarer plusieurs instances de `@Sql`.

L'exemple suivant illustre l'utilisation de SQL comme annotation reproductible en utilisant Java 8. Dans ce scénario, le script `test schema.sql` utilise une syntaxe différente pour les commentaires d'une seule ligne.

```
@Test
@Sql(scripts = "/test-schema.sql", config =
@SqlConfig(commentPrefix = ""))
@Sql("/test-user-data.sql")
public void userTest {
    [...]
}
```

L'exemple suivant est identique au précédent sauf que les déclarations `@Sql` sont regroupées au sein d'un `@SqlGroup` pour la compatibilité avec Java 6 et Java 7.


```

@Test
@SqlGroup({
    @Sql(scripts = "/test-schema.sql", config =
        @SqlConfig(commentPrefix = "`")),
    @Sql("/test-user-data.sql") })
public void userTest {
    [...]
}

```

e. Phases d'exécution pour les scripts

Par défaut, les scripts SQL sont exécutés avant la méthode de test correspondante. Toutefois, si un ensemble de scripts doit être exécuté après la méthode de test, par exemple, pour nettoyer l'état de base de données, l'attribut `executionPhase` de l'annotation `@Sql` peut être utilisé. Notez que `ISOLATED` et `AFTER_TEST_METHOD` sont statiquement importés de `Sql.TransactionMode` et `Sql.ExecutionPhase` respectivement.

```

@Test
@Sql(
    scripts = "create-test-data.sql",
    config = @SqlConfig(transactionMode = ISOLATED)
    @Sql(
        scripts = "delete-test-data.sql",
        config = @SqlConfig(transactionMode = ISOLATED),
        executionPhase = AFTER_TEST_METHOD
    )
    public void userTest {
        [...]
    }
}

```

f. Script de configuration avec `SqlConfig`

La configuration pour l'analyse de script et la manipulation d'erreur peut être configurée via l'annotation `@SqlConfig`.

Lorsqu'elle est déclarée comme annotation sur la classe de test d'intégration, `@SqlConfig` sert de configuration globale pour tous les scripts SQL au sein de la hiérarchie de classe de test.

Lors de sa déclaration directement via l'attribut `config` de l'annotation `@Sql`, `@SqlConfig` sert de configuration locale pour les scripts SQL déclarés dans le sein de l'annotation `@Sql`. Chaque attribut dans `@SqlConfig` a une valeur par défaut.

En raison des règles définies pour les attributs d'annotation dans la spécification du langage Java, il est malheureusement impossible d'attribuer une valeur de `null` à un attribut de l'annotation. Ainsi, afin de soutenir les remplacements de configuration globale héritée, les attributs de `@SqlConfig` ont une valeur par défaut explicite `""` pour Spring ou `DEFAULT` pour les Enums.

Cette approche permet aux déclarations locales de `@SqlConfig` de surcharger sélectivement les attributs individuels des déclarations globales de `@SqlConfig` en fournissant une valeur autre que `""` ou `DEFAULT`. Les attributs globaux sont hérités pour chaque attribut `@SqlConfig` locale ne fournissant pas une valeur explicite autre que `""` ou `DEFAULT`. La configuration locale explicite remplace donc la configuration globale.

Les options de configuration fournies par `@Sql` et `@SqlConfig` sont équivalentes à celles soutenues par `ScriptUtils` et `ResourceDatabasePopulator` mais sont un sur-ensemble de celles fournies par l'espace de nommage XML `<jdbc: initialiser-base de données />`.

g. La gestion des transactions pour `@Sql`

Par défaut, le `SqlScriptsTestExecutionListener` va déduire la sémantique de transaction souhaitée pour son script configuré via `@Sql`.

Plus précisément, les scripts SQL peuvent être exécutés hors transactions ou dans une transaction existante gérée par Spring comme par exemple par le `TransactionalTestExecutionListener` pour un test annoté avec `@Transactional`, ou dans une opération isolée, en fonction de la valeur configurée de l'attribut `ModeTransaction` de `@SqlConfig` et de la présence d'un `PlatformTransactionManager` dans le test de `ApplicationContext`.

Il doit au moins y avoir un `javax.sql.DataSource` dans le test de `ApplicationContext`.

Si les algorithmes utilisent `SqlScriptsTestExecutionListener` pour détecter un `DataSource` et utilisent `PlatformTransactionManager` pour déduire la sémantique de transaction et que cela ne convient pas à vos besoins, vous pouvez spécifier des noms explicites via les attributs `dataSource` et `transactionManager` de `@SqlConfig`.

De plus, le comportement de propagation de la transaction peut être contrôlé via l'attribut `ModeTransaction` de `@SqlConfig`, par exemple, si son exécution doit être exécutée dans une transaction isolée.

```

@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = TestDatabaseConfig.class)
@Transactional
public class TransactionalSqlScriptsTests {
    protected JdbcTemplate jdbcTemplate;

    @Autowired
    public void setDataSource(DataSource dataSource) {
        this.jdbcTemplate = new JdbcTemplate(dataSource);
    }

    @Test
    @Sql("/test-data.sql")
    public void usersTest() {
        [...]
        assertNumUsers(2);
        [...]
    }

    protected int countRowsInTable(String tableName) {
        return JdbcTestUtils.countRowsInTable(this.jdbcTemplate,
            tableName);
    }
}

```

```
protected void assertNumUsers(int expected) {
    assertEquals("Number of rows in the 'user' table.", expected,
        countRowsInTable("user"));
}
```

8. Classes de support cadres TestContext

a. Classes de support de JUnit

Le package `org.springframework.test.context.junit4` fournit les classes de support suivantes pour les cas de test basés JUnit.

- `AbstractJUnit4SpringContextTests`
- `AbstractTransactionalJUnit4SpringContextTests`

L'`AbstractJUnit4SpringContextTests` est une classe de test de base abstraite qui intègre le framework Spring `TestContext` avec un support de test explicite d'`ApplicationContext` dans un environnement JUnit 4.9+.

Lorsque vous étendez `AbstractJUnit4SpringContextTests`, vous pouvez accéder à une variable d'instance `protected` de l'`applicationContext` qui peut être utilisée pour effectuer des recherches de beans explicites ou pour tester l'état du contexte dans son ensemble.

`AbstractTransactionalJUnit4SpringContextTests` est un résumé transactionnel extension de `AbstractJUnit4SpringContextTests` qui ajoute certaines fonctionnalités de commodité pour l'accès JDBC. Cette classe attend un bean `javax.sql.DataSource` et un `beanPlatformTransactionManager` qui sont à définir dans le `ApplicationContext`.

Lorsque vous étendez `AbstractTransactionalJUnit4SpringContextTests`, vous pouvez accéder à une variable d'instance `protectedJdbcTemplate` qui peut être utilisée pour exécuter des instructions SQL afin d'interroger la base de données. Ces requêtes peuvent être utilisées pour confirmer l'état de base de données à la fois avant et après l'exécution de code de base de données liée à l'application. `AbstractTransactionalJUnit4SpringContextTests` fournit également des méthodes pratiques qui délèguent à des méthodes de `JdbcTestUtils` utilisant le précité `JdbcTemplate`.

En outre, `AbstractTransactionalJUnit4SpringContextTests` fournit `unexecuteSqlScript(..)` pour exécuter des scripts SQL sur la `DataSource` configurée.

b. Spring JUnit Runner

Le framework `TestContext` Spring offre une intégration complète avec JUnit 4.9+ travers un runner personnalisé (testé sur JUnit 4.9 à 4.11).

En annotant les classes de test avec `RunWith(SpringJUnit4ClassRunner.class)`, les développeurs peuvent mettre en œuvre des tests unitaires et d'intégration basés sur le JUnit standard et en même temps profiter des avantages du framework `TestContext` comme le support aux contextes d'application de chargement, l'injection de dépendance des instances de test, l'exécution de méthode de test transactionnel, et ainsi de suite. Le code suivant affiche les exigences minimales pour la configuration d'une classe de test afin de fonctionner avec le Spring personnalisé.

Le runner `TestExecutionListeners` est configuré avec une liste vide afin de désactiver les listeners par défaut, qui nécessiteraient autrement une `ApplicationContext` et d'être configurés par une `ContextConfiguration`.

```
@RunWith(SpringJUnit4ClassRunner.class)
@TestExecutionListeners({})
public class SimpleTest {
    @Test
    public void testMethod() {
        [...]
    }
}
```

c. Classes de support TestNG

Le package `org.springframework.test.context.testng` fournit les classes de support suivantes pour les cas de test basés TestNG.

- `AbstractTestNGSpringContextTests`
- `AbstractTransactionalTestNGSpringContextTests`

`AbstractTestNGSpringContextTests` est une classe de test de base abstraite qui intègre le framework Spring `TestContext` avec un support de test explicite `ApplicationContext` dans un environnement TestNG.

Lorsque vous étendez `AbstractTestNGSpringContextTests`, vous pouvez accéder à une variable d'instance `protectedapplicationContext` qui peut être utilisée pour effectuer des recherches de beans explicites ou pour tester l'état du contexte dans son ensemble. `AbstractTransactionalTestNGSpringContextTests` est une extension abstraite transactionnelle de `AbstractTestNGSpringContextTests` qui ajoute certaines fonctionnalités de commodité pour l'accès JDBC. Cette classe attend un bean `javax.sql.DataSource` et un `beanPlatformTransactionManager` qui doivent être définis dans le `ApplicationContext`.

Lorsque vous étendez `AbstractTransactionalTestNGSpringContextTests`, vous pouvez accéder à une variable d'instance `protectedJdbcTemplate` qui peut être utilisée pour exécuter des instructions SQL, pour interroger la base de données. Ces requêtes peuvent être utilisées pour tester l'état de base de données à la fois avant et après l'exécution de code de base de données liée à l'application, et s'assure que de telles requêtes s'exécutent dans le framework de la même transaction que le code d'application.

`AbstractTransactionalTestNGSpringContextTests` fournit également des méthodes pratiques qui délèguent à des méthodes de `dbcTestUtils` utilisant le précité `JdbcTemplate`.

En outre, `AbstractTransactionalTestNGSpringContextTests` fournit une méthode `executeSqlScript(..)` pour exécuter des scripts SQL sur la `DataSource` configurée.

9. Framework Spring MVC Test

a. Projet autonome

Avant inclusion dans Spring Framework 3.2, le framework de test Spring MVC existait déjà comme un projet distinct sur GitHub où il a grandi et évolué à travers l'utilisation réelle, la rétroaction et la contribution d'un grand nombre.

Le projet autonome de framework de test Spring MVC est toujours disponible sur GitHub et peut être utilisé en conjonction avec Spring Framework 3.1.x. Les applications mises à niveau vers Spring 3.2 ou une version ultérieure doivent remplacer les dépendances de Spring-test-mvc avec une dépendance sur spring-test.

Le module Spring Test utilise un package `org.springframework.test.web` différent, mais à part cela il est presque identique, à deux exceptions près :

- Le support de fonctionnalités nouvelles dans Spring 3.2 (par exemple, les requêtes web asynchrones).
- La possibilité d'utiliser `MockMvc`.

Dans Spring 3.2 et versions supérieures, il faut utiliser le framework `TestContext`, qui offre des prestations de mise en cache pour la configuration chargée.

Le framework Spring MVC Test fournit un support de première classe pour JUnit pour le client de test et le code côté serveur Spring MVC via une API « fluente ».

Typiquement, il charge la configuration Spring réelle à travers le framework `TestContext` et utilise toujours le `DispatcherServlet` pour traiter les demandes se rapprochant ainsi des tests d'intégration complète sans nécessiter le démarrage d'un conteneur de servlet.

Les tests du côté client sont basés sur `RestTemplate` et permettent des tests de code qui reposent sur le `RestTemplate` sans nécessiter le démarrage d'un serveur pour répondre aux requêtes.

b. Tests côté serveur

Avant Spring Framework 3.2, la manière la plus courante de tester un contrôleur Spring MVC était d'écrire un test unitaire qui instancie le contrôleur, l'injecte avec des dépendances sur des mocks ou des stubs, puis appelle ses méthodes directement, en utilisant un `MockHttpServletRequest` et un `MockHttpServletResponse` si nécessaire.

Bien que ce soit assez facile à faire, les contrôleurs ont de nombreuses annotations, et beaucoup restent non testées. Le mapping des `requests`, le `Binding` des données, la conversion de type et la validation ne sont que quelques exemples de ce qui n'est pas testé. En outre, il existe d'autres types de méthodes annotées tels que `@InitBinder`, `@ModelAttribute` et `@ExceptionHandler` qui sont invoqués dans une partie du traitement de la `request`.

L'idée derrière Spring MVC test est d'être en mesure de réécrire ces tests de contrôleur en effectuant des demandes réelles et générer des réponses, comme s'ils se déroulaient à l'exécution, le long du chemin invoquant contrôleurs à travers la `DispatcherServlet` Spring MVC. Les contrôleurs peuvent encore être injectés avec des dépendances sur des mocks et peuvent donc rester concentrés sur la couche Web. Spring MVC test repose sur les implémentations familières de mocks de l'API Servlet disponible dans le module `spring-test`. Ceci permet l'exécution des `requests` et la production de réponses sans qu'il soit nécessaire d'avoir un conteneur de servlet pour l'exécution. Pour la plupart des traitements, tout devrait fonctionner comme il le ferait à l'exécution, à l'exception du rendu des JSP, qui ne sont pas disponibles en dehors d'un conteneur de Servlet.

En outre, si vous êtes familier avec la façon dont le mock `MockHttpServletResponse` fonctionne, vous saurez que les `forwards` et les `redirections` ne sont pas réellement exécutés.

Au lieu de cela, les URL `"forwarded"` et `"redirected"` sont enregistrées et peuvent être invoquées dans les tests. Cela signifie que si vous utilisez des JSP, vous pouvez vérifier la page JSP à laquelle la demande a été transmise. Tous les autres moyens de rendu, y compris les méthodes `@ResponseBody` et les types `View` (autre JSP) comme `Freemarker`, `Velocity`, `thymeleaf`, et d'autres moteurs de templating pour rendre du HTML, JSON, XML, et ainsi de suite devraient fonctionner comme prévu, et la réponse contiendra le contenu généré.

Voici un exemple de test d'une demande d'information sur un compte au format JSON :

<pre>import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*; import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*; @RunWith(SpringJUnit4ClassRunner.class) @WebAppConfiguration @ContextConfiguration("test-servlet-context.xml") public class ExampleTests { @Autowired private WebApplicationContext wac; private MockMvc mockMvc; @Before public void setup () { this.mockMvc = MockMvcBuilders.webAppContextSetup (this.wac).build (); } @Test public void getAccount () throws Exception { this.mockMvc.perform(get("/"accounts/1").accept(MediaType.parseMediaType("application/json;charset=UTF-8"))) .andExpect (statut (). IsOk ()) .andExpect (content (). contentType ("application / json")) .andExpect (jsonPath ("\$.name") .value ("Lee")); } }</pre>	Le test repose sur soutien du <code>WebApplicationContext</code> du framework <code>TestContext</code>. Il charge la configuration de Spring à partir d'un fichier de configuration XML situé dans le même package que la classe de test (prend également en charge <code>JavaConfig</code>) et injecte le <code>WebApplicationContext</code> créé dans le test, ce qui fait qu'une instance de mock <code>MockMvc</code> peut être créée avec elle. Le <code>MockMvc</code> est ensuite utilisé pour effectuer une demande de <code>"/accounts/1"</code> afin de vérifier que l'état de la réponse obtenue est bien de 200, que le type de contenu de la réponse est bien <code>"application/json"</code>, et que le contenu de la réponse à une propriété JSON appelée <code>"nom"</code> et la valeur <code>"Lee"</code>.
--	---

Le contenu JSON est inspecté avec l'aide du projet `JsonPath` `Jayway`.

Il y a beaucoup d'autres options pour vérifier le résultat de la demande effectuée qui seront discutées plus tard.

L'API « fluente » dans l'exemple ci-dessus nécessite quelques importations statiques comme `MockMvcRequestBuilders.*`, `MockMvcResultMatchers.*` et `MockMvcBuilders.*`.

Le but de la configuration de test du côté serveur est de créer une instance de `MockMvc` qui peut être utilisé pour effectuer des `requests`.

Il y a deux options principales :

- Configurer la configuration à Spring MVC à travers le framework `TestContext`, qui charge la configuration de Spring et injecte un `WebApplicationContext` dans le test à utiliser pour créer un `MockMvc` :

```
@RunWith(SpringJUnit4ClassRunner.class)
@WebAppConfiguration
@ContextConfiguration("my-servlet-context.xml")
public class MyWebTests {
    @Autowired
    private WebApplicationContext wac;
    private MockMvc mockMvc;
    public void setup () {
        this.mockMvc = MockMvcBuilders.webAppContextSetup
        (this.wac).build ();
    }
    // ...
}
```

Enregistrer une instance de contrôleur sans charger toute la configuration de Spring :

```
public class MyWebTests {
    private MockMvc mockMvc;
    @Before
    public void setup() {
        this.mockMvc = MockMvcBuilders.standaloneSetup(new
        AccountController()).build();
    }
    // ...
}
```

Le `webAppContextSetup` charge la configuration réelle de Spring MVC résultant dans un test d'intégration plus complet. `TestContext` met en cache la configuration ce qui aide à garder les tests rapides même si des tests sont ajoutés.

En outre, vous pouvez injecter des services mockés dans les contrôleurs

via la configuration de Spring, afin de rester concentré sur les tests de la couche Web.

Voici un exemple de déclaration d'un service mocké avec Mockito :

```
<bean id="accountService" class="org.mockito.Mockito" factory-
method="mock">
<constructor-arg value="org.example.AccountService"/>
</bean>
```

Ensuite, vous pouvez injecter le service mocké dans le test pour vérifier les attentes :

```
@RunWith(SpringJUnit4ClassRunner.class)
@WebAppConfiguration
@ContextConfiguration("test-servlet-context.xml")
public class AccountTests {
    @Autowired
    private WebApplicationContext wac;
    private MockMvc mockMvc;
    @Autowired
    private AccountService accountService;
    // ...
}
```

Le `standaloneSetup` d'autre part ressemble plus à un test unitaire. Il teste un contrôleur à la fois. Le contrôleur peut être injecté avec des dépendances mocks manuellement, et il ne comporte pas de configuration Spring de chargement. Ces tests sont plus concentrés sur la forme et facilitent la détection du contrôleur testé ou le contrôle d'une configuration spécifique Spring MVC, etc.

Le `standaloneSetup` est aussi un moyen très pratique pour écrire des tests « ad hoc » afin de vérifier certains comportements ou de déboguer un problème. Tout comme avec l'intégration vs les tests unitaires, il n'y a pas de bonne ou de mauvaise façon de procéder. L'utilisation du `standaloneSetup` ne signifie pas la nécessité pour certains tests complémentaires d'utiliser un `webAppContextSetup` pour vérifier la configuration Spring MVC.

Alternativement, vous pouvez décider d'écrire tous les tests avec `webAppContextSetup` et de toujours tester la configuration actuelle de Spring MVC.

Exécution des requests

Pour effectuer des `requests`, utilisez la méthode HTTP appropriée et les méthodes de style builder correspondant aux propriétés de `MockHttpServletRequest`. Par exemple :

```
mockMvc.perform(post("/hotels/{id}",
42).accept(MediaType.APPLICATION_JSON));
```

En plus de toutes les méthodes HTTP, vous pouvez également effectuer les fichiers requestés d'upload, ce qui crée une instance interne de `MockMultipartHttpServletRequest` :

```
mockMvc.perform(fileUpload("/doc").file("a1",
"ABC".getBytes("UTF-8")));
```

Les paramètres de chaîne de requête peuvent être spécifiés dans le modèle d'URI :

```
mockMvc.perform(get("/hotels?foo={foo}", "bar"));
```

Ou en ajoutant des paramètres de la requête de Servlet :

```
mockMvc.perform(get("/hotels").param("foo", "bar"));
```

Si le code de l'application repose sur les paramètres des requêtes Servlet et ne vérifie pas la chaîne de requête, comme c'est souvent le cas, la façon d'ajouter les paramètres n'a pas d'importance. Gardez à l'esprit que les paramètres fournis dans le modèle URI seront déjà décodés tandis

que les paramètres fournis par la méthode `param(...)` devront être décodés. Dans la plupart des cas, il est préférable de laisser intacts, dans l'URI de la requête, le path du contexte ainsi que celui de la servlet. Si vous devez tester avec l'URI de la `request` complète, assurez-vous de régler le `contextPath` et le `servletPath` en conséquence de sorte que le mapping des requests fonctionne :

```
mockMvc.perform(get("/app/main/hotels/{id}").contextPath("/app").
servletPath("/main"))
```

En regardant l'exemple ci-dessus, il serait lourd de régler le `contextPath` et le `servletPath` à chaque requête effectuée.

Voilà pourquoi vous pouvez définir les propriétés par défaut des `requests` lors de la construction du `MockMvc` :

```
public class MyWebTests {
    private MockMvc mockMvc;

    @Before
    public void setup() {
        mockMvc = standaloneSetup(new AccountController())
            .defaultRequest(get("/")
                .contextPath("/app").servletPath("/main")
                .accept(MediaType.APPLICATION_JSON).build());
    }
}
```

Les propriétés ci-dessus seront applicables à chaque demande effectuée par le `MockMvc`.

c. Définir les attentes

Les attentes peuvent être définies en ajoutant un ou plusieurs `.andExpect(...)` après les appels pour effectuer la requête :

```
mockMvc.perform(get("/accounts/1").andExpect(status().isOk()));
```

Le package `MockMvcResultMatchers.*` définit un certain nombre de membres statiques, dont certains types permettent, avec d'autres méthodes, de vérifier le résultat de la request effectuée.

Les assertions tombent dans deux catégories générales :

- Vérifier les propriétés de la réponse, à savoir le statut de réponse, les en-têtes, et le contenu, qui sont les choses les plus importantes à tester.
- Aller au-delà de la réponse, et permettre l'inspection de constructions spécifiques à Spring MVC, par exemple la méthode du contrôleur qui a traité la demande, la levée et le traitement d'une exception, le contenu du modèle, le point de vue sélectionné ou encore les attributs de flash ajoutés. Il est également possible de vérifier les constructions de servlets spécifiques telles que la demande de session et d'attributs. Le test suivant affirme que la liaison/validation a échoué :

```
mockMvc.perform(post("/persons"))
    .andExpect(status().isOk())
    .andExpect(model().attributeHasErrors("person"));
```

Plusieurs fois, lors de l'écriture des tests, il est utile de mettre à jour la base, via un `flush`, le résultat de la request effectuée. Cela peut être fait comme suit, où `print()` est

une importation statique de `MockMvcResultHandlers` :

```
.andDo(print())
.andExpect(status().isOk())
.andExpect(model().attributeHasErrors("person"));
```

Tant que le traitement de la demande provoque une exception non gérée, la méthode `print()` permet d'afficher toutes les données de résultat disponibles sur `System.out`.

Dans certains cas, vous voudrez peut-être obtenir un accès direct au résultat afin de vérifier quelque chose qui ne peut être vérifié autrement. Cela peut être fait en ajoutant `.andReturn()` à la fin après toutes les attentes :

```
MvcResult mvcResult =
mockMvc.perform(post("/persons").andExpect(status().isOk()).andReturn());
```

Lorsque tous les tests répètent les mêmes assertions, vous pouvez définir les assertions communes une fois lors de la construction

du `MockMvc` :

```
standaloneSetup(new SimpleController())
    .alwaysExpect(status().isOk())
    .alwaysExpect(content().contentType("application/json;charset=UTF-8"))
    .build()
```

Notez que l'attente est toujours appliquée et ne peut être annulée sans créer une instance `MockMvc` séparée.

Lorsque le contenu de la réponse JSON contient des liens hypermédias créés avec Spring HATEOAS, les liens qui en résultent peuvent être vérifiés :

```
mockMvc.perform(get("/people").accept(MediaType.APPLICATION_JSON)
)
    .andExpect(jsonPath("$.links[?(@.rel == 'self')].href").value("http://localhost:8080/people"));
```

Lorsque le contenu de la réponse XML contient des liens hypermédias créés avec Spring HATEOAS, les liens qui en résultent peuvent être vérifiés :

```
Map<String, String> ns = Collections.singletonMap("ns",
"http://www.w3.org/2005/Atom");
mockMvc.perform(get("/handle").accept(MediaType.APPLICATION_XML)
    .andExpect(xpath("/person/ns:link[@rel='self']/@href",
ns).string("http://localhost:8080/people"));
```

d. Filtrer les inscriptions

Lorsque vous configurez un `MockMvc`, vous pouvez enregistrer une ou plusieurs instances de `Filter`:

```
mockMvc = standaloneSetup(new PersonController()).addFilters(new
CharacterEncodingFilter()).build();//EC:00200
```

Les filtres enregistrés seront invoqués par le `MockFilterChain` de Spring test et le dernier filtre sera

délégué à la `DispatcherServlet`.

Les TU et TI du framework Spring contiennent une multitude d'exemples de tests.

e. Tests REST côté client

Les tests côté client utilisent le `RestTemplate`. L'objectif est de définir les `requests` attendues et d'apporter "stub" des réponses :

```
RestTemplate restTemplate = new RestTemplate();
MockRestServiceServer mockServer =
MockRestServiceServer.createServer(restTemplate);
mockServer.expect(requestTo("/greeting")).andRespond(withSuccess(
"Hello world", MediaType.TEXT_PLAIN));
// use RestTemplate ...
mockServer.verify();
```

Dans l'exemple précédent, la classe centrale pour les tests de REST côté

client `MockRestServiceServer` configure le `RestTemplate` avec une `ClientHttpRequestFactory` personnalisée qui vérifie les `requests` réelles par rapport aux attentes et fournit en retour des « stub » pour les réponses.

Dans ce cas, nous nous attendons à une seule demande de `/greeting` et une réponse 200 avec un contenu `text/plain`.

Une fois que les demandes et les réponses attendues de dérivation ont été définies, le `RestTemplate` peut être utilisé dans le code côté client comme d'habitude. À la fin du test, `mockServer.verify()` peut être utilisé pour vérifier que toutes les `requests` ont été servies.

10. Autres ressources

Consultez les ressources suivantes pour plus d'informations sur le test :

JUnit : un framework de tests orienté programmeur pour Java. Utilisé par le framework Spring dans sa suite de tests.

TestNG : un framework de tests inspiré par JUnit avec le support supplémentaire pour les annotations, les groupes de test, les tests pilotés par les données, les tests distribués, etc.

MockObjects.com : site web dédié pour mocker des objets, une technique pour améliorer la conception de code dans le développement piloté par les tests.

"Mock Objects" : article dans Wikipedia.

EasyMock : bibliothèque Java qui fournit des Mock Objects pour les interfaces (et des objets à travers l'extension de classe) en les générant à la volée en utilisant le mécanisme de proxy de Java. Utilisé par le framework Spring dans sa suite de tests.

JMock : bibliothèque qui supporte le développement piloté par les tests de code Java avec des mocks.

Mockito : bibliothèque Java de mock basée sur l'espionnage de motif de test.

DbUnit : extension de JUnit (utilisable aussi avec Ant et Maven) ciblée pour des projets axés base de données. Entre autres choses, cela place votre base de données dans un état connu entre chaque test.

The Grinder : framework de test de charge Java.

Points clés

- Les tests sont faciles à implémenter avec Spring.
- Il faut faire tous les tests nécessaires.
- Spring permet de tester dans un environnement très léger.
- Les tests Spring sont très rapides.

Description de la problématique

Un ORM (*Object Relational Mapping*) est un outil qui permet de passer d'un modèle logique objet vers un modèle physique relationnel. Il en existe plusieurs comme Hibernate, MyBatis mais le plus utilisé est Hibernate.

JPA est une API Java qui utilise en back une implémentation d'un ORM, nous utilisons JPA avec Hibernate en partie arrière. Hibernate apporte des fonctionnalités supplémentaires car il est en avance de phase sur JPA. Les annotations et API Hibernate ne sont utilisées que si les fonctionnalités dont on a besoin ne sont pas disponibles par JPA.

L'ORM fait le lien entre la base de données et les objets Java de plus bas niveau que l'on regroupe dans la couche dite « couche de domaine métier ».

Pour les cas simples, l'ORM gère le SQL pour les échanges avec la base de données mais il est possible de personnaliser le SQL pour les cas complexes. On s'abstrait alors totalement du type de la base de données. Il est par exemple possible de tester sur une base de données mémoire comme H2 et d'avoir une base de Production DB2, Oracle ou Sybase sans toucher au code Java.

Les ORM modernes proposent en plus des fonctionnalités supplémentaires comme la journalisation des données, la fourniture de statistiques sur l'usage qui est fait des données ou sur les performances. La mise en place d'un cache simple ou partagé permet également d'avoir des gains importants en performance si l'application est bien pensée.

Faire la correspondance entre un modèle objet qui, par définition, supporte l'héritage et le polymorphisme en un modèle relationnel plat basé sur des tables n'est pas simple. Les exemples du livre, pour une préoccupation de simplification, n'utiliseront que le mapping le plus simple : une table correspondant à un objet Java de domaine.

L'ORM permet également de rendre transparentes les relations entre les objets. Ces associations d'objets sont omnidirectionnelles ou bidirectionnelles.

Type des relations entre les classes :

- One-to-one : un objet unique pointant vers un autre objet unique.
- Many-to-one : un ensemble d'objets pointant vers un objet unique.
- One-to-many : un objet pointant vers un ensemble d'objets.
- Many-to-many : un ensemble d'objets pointant vers un autre ensemble d'objets.

Nous utilisons l'attribut `mappedBy` pour identifier la `foreign key`.

L'ORM fait la correspondance entre les types Java et les types des colonnes des tables de la base de données. L'ORM a aussi pour but de répercuter les contraintes sur les tables dans les objets du domaine et réciproquement.

Historiquement, le premier ORM Java fut TopLink, mais son utilisation était payante. Afin d'harmoniser les développements client/serveur Java, Sun (aujourd'hui Oracle) proposa une architecture J2EE reposant sur des EJB avec différentes façons de faire le mapping entre les objets Java et les bases de données. L'utilisation des EJB 1.0 puis des EJB 2.0 était laborieuse. Hibernate fut alors créé afin de simplifier les choses. Java intégra ensuite les concepts Hibernate et TopLink dans la version entreprise (JEE) de Java en passant aux EJB3 qui sont plus simples à mettre en œuvre et en créant la première version de l'API JPA 1.0. TopLink deviendra ensuite EclipseLink et JPA continua d'évoluer en parallèle d'Hibernate. Vous choisirez souvent Hibernate comme implémentation pour JPA.

Comme pour Spring, il est possible de configurer JPA et Hibernate via des fichiers XML et des annotations. On utilise aujourd'hui le plus possible les annotations en ne gardant qu'une petite partie de la configuration dans des fichiers `.xml` et `.properties`.

Modèle en couche :

- Présentation

- Métier
- DAO (Repository)
- Base de données

JPA 2 apporte des nouveautés qui n'étaient disponibles auparavant que dans Hibernate, comme la gestion de collections d'éléments avec l'annotation `@ElementCollection`, la suppression des orphelins (orphan removal), l'utilisation des requêtes typées, un cache de niveau 2 (L2), les Bean Validation (JSR-303) et l'API Criteria.

Mise en œuvre

1. Configuration d'une entité de la couche domaine

Par exemple, pour configurer une entité simple de la couche domaine :

```
@Entity
@Table(name = "t_book")
public class Livre implements Serializable {
    @Id
    @Column(name = "id")
    private String auteur;
    private int nbPage;
    private String titre;
    // getters et setters
    [...]
```

Il n'y a que des API et des annotations JPA car ce cas est simple :

Élément	Signification	
@Entity	Indique qu'il s'agit d'une entité POJO.	
@Table(name = "t_book")	Indique le nom de la table public class Livre implements Serializable. Une entité doit être sérialisable.	
@Id	Indique que le champ sera une clé primaire.	
@Column(name = "id")	Indique le nom de la colonne qui contiendra la clé primaire.	
private String auteur	Donnée métier	L'auteur du livre
private int nbPage	Donnée métier	Le nombre de pages
private String titre	Donnée métier	Le titre

Il n'y a pas d'annotation sur les données métier car il s'agit d'un type qu'Hibernate mappe sans difficulté au niveau du type et du nom de la colonne. Dans le cas contraire, nous aurions ajouté une annotation @Column et personnalisé la colonne.

Il est aussi possible d'avoir des variables de classe n'ayant pas de correspondance avec une colonne. On annote alors la variable avec l'annotation @Transient.

Les classes de la couche domaine sont accédées via la couche Repository.

Mapping many-to-one et one-to-many

```
@Entity
public class Book implements Serializable {
    @Id
    private String title;
    private int nbPage;
    @OneToMany
    private Set<Chapter> chapters =
        new HashSet<Chapter>();
    // getters et setters
}
```

Par défaut, les relations entre objets sont du type lazy-loading. Cela signifie que les données contenues dans les collections ne sont remontées de la base que si on y accède. Il faut donc éviter d'utiliser `instanceOf` sur des objets Entity car on a souvent un proxy à la place de l'objet. Les Entity ne doivent pas être de type « final » afin de pouvoir être modifiées par la JPA ou via la programmation orientée

aspect (AOP).

Types de données

Il est possible de créer ses propres types mais cela dépasse le cadre de cette introduction.

🔴 Nous pouvons utiliser l'annotation `@EntityListeners` sur l'entité pour détecter des changements. Cela va à l'encontre du modèle en couche car cela ajoute du code dans l'entité qui ne contient normalement que du code de validation.

2. Accès à l'objet du domaine

Accès à l'objet du domaine depuis un objet de la couche Repository

```
@ApplicationScoped
public class PersonRepository {
    @PersistenceContext
    private EntityManager em;
    public Person createOrUpdateBook(Book book) {
        return em.merge(book);
    }
    public List<Book> findAll() {
        return em.createNamedQuery("findAllBooks").getResultList();
    }
    public Person findBook(String titre) {
        return em.find(Book.class, titre);
    }
}
```

La couche Repository contient l'EntityManager `em` qui est un espace mémoire qui contient les données en cours de modification. L'EntityManager gère la fusion entre les objets de son cache et les objets extérieurs que l'on veut intégrer au cache pour une future mise à jour. Il gère également les transactions. On peut le considérer comme un super DAO car c'est lui qui fait les accès à la base de données. Il fournit également des proxies sur les objets liés (relation ou collection) qui sont en Lazy Loading, et retourne les "vrais" objets quand on accède aux entités.

Elle permet de gérer en base directement des collections de types primitifs de base comme les String, les integer, etc.

```
@Entity
public class Index {
    @ElementCollection
    private Collection<String> words;
```

Sans Hibernate, il fallait d'abord effacer les éléments enfants pour ensuite effacer l'élément principal (parent).

Séquence :

1. Chercher l'élément à effacer.
2. Trouver les enfants de cet élément.
3. Effacer tous ses enfants.
4. Effacer l'élément principal.

Avec Hibernate, il suffisait de spécifier explicitement l'annotation `CascadeType.DELETE_ORPHAN` qui s'utilisait conjointement avec `CascadeType.ALL` JPA.

Maintenant avec JPA 2.0, on utilise l'option `orphanRemoval = true` dans l'annotation de la relation :

```
@OneToMany(mappedBy="foo", orphanRemoval=true)
```

3. Requêtes typées

Auparavant, il fallait systématiquement caster le type de retour des requêtes.

```
Query q1 = em.createQuery("SELECT c FROM Country c");  
List results = query.getResultList();
```

Nous pouvons maintenant utiliser la généricité pour spécifier le type sous cette forme :

```
TypedQuery<Country> q2 =  
    em.createQuery("SELECT c FROM Country c", Country.class);  
List<Country> countries = query.getResultList();
```

4. Cache de niveaux 1 et 2

Le cache de niveau 1, utilisé par l'EntityManager, contient les transactions en cours. À la fin de la transaction, le cache est renouvelé. Pour sauvegarder en bases on fusionne les entités et on vide le cache par une action de `flush`. L'EntityManager garde en mémoire une copie de l'état de la base de données qui correspond à son dernier accès. Il existe plusieurs modes par rapport au verrouillage des données en cours de modification. En mode `optimistic-lock`, on utilise un champ annoté `@version` pour aider JPA à vérifier si la donnée a été modifiée (par un autre EntityManager JPA) depuis le dernier accès. En cas de problème, JPA lève l'exception `OptimisticLockException`.

Cela ne fonctionne que si la base de données n'est modifiée que depuis JPA. Si des triggers ou des procédures stockées modifient les données, le cache est alors faux, il doit être invalidé manuellement. La nouvelle annotation `@Cacheable` active le cache de niveau 2. Ce cache est relativement difficile à configurer. On utilise des implémentations différentes si on a un serveur ou un cluster. En effet, en cas de cluster, il faut un cache partagé entre les membres du cluster.

Les principaux caches de niveau 2 sont Ehcache, Hazelcast et Infinispan.

5. Bean Validation (JSR-303)

La validation Hibernate était utilisée avant que cette API ne soit disponible. La validation Bean Validation n'est pas spécifique à JPA. Certaines validations ne sont pas prises en charge par la JSR-303 mais sont disponibles dans la validation Hibernate. On utilise donc souvent ces deux types de validation conjointement, en préférant toujours celles de JPA en cas de redondances.

Par exemple :

```
import java.util.Date;  
import javax.validation.constraints.NotNull;  
import javax.validation.constraints.Past;  
import javax.validation.constraints.Size;  
public class Voiture {  
    private String marque;  
    private Date dateImmat;  
    @NotNull  
    @Size(max=50)  
    public String getMarque() {  
        return marque;  
    }  
    public void setMarque(String marque) {  
        this.marque = marque;  
    }  
    @Past  
    public Date getDateImmat() {
```

```

    return dateImmat;
}
public void setDateImmat(Date dateImmat) {
    this.dateImmat = dateImmat;
}

```

Nous vérifions que les champs ne sont pas null, que la marque fait au plus 50 caractères et que la date d'immatriculation est dans le passé.

Avant de faire une sauvegarde de la donnée, on fait valider les données et on obtient en retour une liste d'erreurs. Si l'on tente de sauvegarder sans valider, JPA lève une `exceptionConstraintViolationException` et invalide la transaction.

On peut explicitement appeler la validation avant de demander la sauvegarde :

```

ValidatorFactory factory =
Validation.buildDefaultValidatorFactory();
Validator validator = factory.getValidator();
Set<ConstraintViolation<Book>> constraintViolations =
validator.validate(book);
for (ConstraintViolation<Book> constraintViolation :
    constraintViolations) {
    log.debug(constraintViolation.getPropertyPath() + " - " +
constraintViolation.getMessage());
}

```

Cela permet de ne pas invalider la transaction et de récupérer les erreurs pour les afficher si besoin.

Annotations standards :

Annotation	Signification
@Size(min=2, max=240)	Taille
@AssertTrue / @AssertFalse	Vérifier une condition
@Null / @NotNull	Test sur valeur nulle
@Max / @Min	Le min ou le max
@Future / @Past	Par rapport à un repère temporel
@Digits(integer=6, fraction=2)	Format des nombres décimaux
@Pattern(regexp="(\\d+\\s)?\\d+-\\d+")	Expression régulière téléphone "(555)123-4567"

Annotations validateur Hibernate :

Annotation	Signification
@Email	Email
@URL	URL
@CreditCardNumber	Carte de crédit
@Range	Plage

6. L'API Criteria

L'API Criteria de JPA est un complément à JPQL (le langage de requêtage JPA). Son avantage est son typage fort qui élimine beaucoup d'erreurs, ces dernières étant détectées à la compilation plutôt qu'à l'exécution. Elle est utilisée pour des requêtes complexes et permet, dans certains cas, de guider JPA pour la construction de la requête SQL pour la base de données.

Elle permet aussi de construire ses requêtes dynamiquement en Java.

7. Accès à la base de données

Nous accédons à la base de données via une connexion unique ou via une DataSource. La gestion des connexions est souvent déléguée à Spring, ce qui permet d'avoir des connexions de typologies différentes en fonction des environnements.

8. Le fichier persistence.xml

Le fichier persistence.xml est un des fichiers principaux de la configuration de JPA. Il est généralement dans src/main/resources/META-INF/. Il se configure dans Maven via le « resource filtering ».

Exemple de persistence.xml :

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.0"
  xmlns="http://java.sun.com/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://java.sun.com/xml/ns/persistence
    http://java.sun.com/xml/ns/persistence/persistence_2_0.xsd">
  <persistence-unit name="primary">
    <jta-data-source>
java:jboss/datasources/CompanyFight</jta-data-source>
    <properties>
      <property name="hibernate.hbm2ddl.auto"
value="update" />
      <property name="hibernate.dialect"
        value="
org.hibernate.dialect.PostgreSQLDialect"/>
      <property name="hibernate.show_sql" value="true"/>
      <property name="hibernate.generate_statistics"
value="false"/>
    </properties>
  </persistence-unit>
</persistence>
```

9. Tests JPA

Les tests sont approfondis dans le chapitre sur les tests. Voici juste un aperçu.

Les tests JPA sont faciles à coder avec Spring. Ils permettent d'anticiper une grande partie des problèmes qui ne seraient vus que trop tard.

En méthode Agile, vous construirez souvent une application qui fonctionne de bout en bout dans laquelle vous intégrerez progressivement les fonctionnalités. Cela signifie que le modèle de donnée est complexifié au fur et à mesure avec souvent plusieurs programmeurs. Il est alors indispensable de vérifier que son code n'apporte pas de régression au code existant. Il faut toujours modifier les tests existants quand on apporte une nouvelle relation afin de tester le comportement des objets liés.

Ces tests améliorent donc le code, vérifient que les règles de gestion sont bien prises en compte, aident à éviter les régressions et permettent une correction rapide en cas d'automatisation des builds (voir chapitre consacré à l'intégration continue).

La couverture de tests est plus importante dans le cas d'une utilisation d'un générateur de code comme jHipster ou SpringFuse car la génération systématique de test est alors possible. Nous utiliserons en général JUnit ou TestNG pour coder les tests.

JPA, les stubs et les mocks

Spring est basé sur les interfaces. Spring trouve pour nous la meilleure classe qui implémente l'interface de l'objet sur lequel on souhaite travailler.

C'est particulièrement pratique pour les tests car on peut fabriquer des objets qui implémentent une

interface mais qui sont vidés de leur substance. On peut, par exemple, faire un DAO qui expose les méthodes mais qui renvoie des jeux d'essais au lieu d'accéder à une vraie base de données. On appelle ce type d'objet un "stub".

C'est très pratique mais perfectible. Il faut en effet créer l'objet en respectant les contrats d'une interface même si on n'utilise pas toutes les méthodes de l'interface.

Un outil basé sur une librairie qui générerait des objets à la volée qui respecte les interfaces et avec lequel on ne programmerait que les interfaces à tester serait plus productif. Cet outil existe, on l'appelle un générateur de mocks.

Les mocks sont plus simples et plus rapides à utiliser que les stubs. On utilise en général les librairies Mockito (<http://code.google.com/p/mockito/>) et EasyMock (<http://easymock.org/>). Spring fournit également un ensemble très complet de mocks déjà faits, comme nous le verrons dans le chapitre sur les tests.

Exemple de simulation d'un EntityManager par un mock pour faire des tests :

```
@Test
public void testCarPersist {
    carRepository carRepository =
        new carRepository();
    EntityManager em =
        EasyMock.createMock(EntityManager.class);
    carRepository.setEntityManager(em) ;
    Car car = new Car("Ford T");
    em.persist(car) ;
    EasyMock.replay(em);
    personRepository.save(car);
    EasyMock.verify(em);
}
```

Pour tester la couche domaine, on utilisera le conteneur Spring. Dans le cas où l'on ferait du Spring et du JPA dans un serveur JEE, on utilise alors Arquillian.

Exemple de test d'intégration avec Spring :

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(locations=
{"classpath*:META-INF/spring/application-context-test.xml"})
public class IntegrationTest {
    @Inject
    private CarService carService;
    @Test
    public void createCar() {
        try {
            carService.findCar ("test_car");
            fail("Car already exists in the database.");
        } catch (ObjectRetrievalFailureException orfe) {
            // User should not already exist in the database.
        }
        Car car = new Car();
        car.setLogin("test_car");
        car.setFirstName("Name");
        carService.createCar(car);
        Car carFoundInDatabase = carService.findCar("test_car");
        assertEquals("Name", carFoundInDatabase.getFirstName());
    }
}
```

Points clés

- Les objets de la couche métier sont des objets de type Entity gérés par un EntityManager.
- Les objets sont parfois disponibles sous forme de proxies.
- JPA propose l'utilisation de plusieurs niveaux de cache.
- Nous pouvons avoir des types personnalisés.
- La gestion des accès concurrents est réglable.
- Nous améliorerons la maintenabilité de l'application en respectant le principe des couches applicatives.

Spring MVC

Spring MVC permet de créer facilement une application web dans un conteneur léger comme Tomcat ou Jetty. Il est très simple d'emploi et permet d'avoir une très bonne couverture de tests. Il est possible d'utiliser une configuration basée sur des fichiers XML ou sur les annotations. Comme pour les autres types d'applications Spring, les configurations annotées sont à préférer car elles sont plus facilement maintenables. Nous présentons dans ce chapitre les deux types de configurations car les anciens projets utilisent encore la version basée sur les fichiers XML.

1. Fonctionnement global

a. Configuration simple XML

Pour la configuration XML, nous utilisons comme point de départ une application web classique. Nous configurerons ensuite le fichier web.xml pour que l'application utilise Spring MVC.

Extrait du fichier web.xml :

```
<servlet>
<servlet-name>hello-dispatcher</servlet-name>
<servlet-class>
    org.springframework.web.servlet.DispatcherServlet
</servlet-class>
<init-param>
<param-name>contextConfigLocation</param-name>
<param-value>/WEB-INF/spring-mvc-config.xml</param-value>
</init-param>
<load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
<servlet-name>hello-dispatcher</servlet-name>
<url-pattern>/</url-pattern>
</servlet-mapping>
<listener>
<listener-class>
    org.springframework.web.context.ContextLoaderListener
</listener-class>
</listener>
<context-param>
<param-name>contextConfigLocation</param-name>
<param-value>/WEB-INF/spring-core-config.xml</param-value>
</context-param>
```

Nous utilisons la

classe `org.springframework.web.servlet.DispatcherServlet` pour la servlet en indiquant l'emplacement du fichier de configuration dans le paramètre `contextConfigLocation` :

```
contextConfigLocation=/WEB-INF/spring-mvc-config.xml
```

et nous demandons à ce que la servlet soit chargée au démarrage :

```
load-on-startup=1
```

Nous paramétrons le listener `ContextLoaderListener` pour qu'il charge le fichier de configuration de la partie back :

```
<listener>
<listener-class>
    org.springframework.web.context.ContextLoaderListener
</listener-class>
</listener>
```

et nous indiquons
l'emplacement du
fichier de
configuration back
dans

le `contextConfigLocation`.

Nous créons alors la configuration dans le fichier `spring-mvc-config.xml` :

```
<context:component-scan base-package="fr.eni.spring.mvc.xml.web"/>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="viewClass" value="org.springframework.web.servlet.view.JstlView"/>
<property name="prefix" value="/WEB-INF/views/jsp/" />
<property name="suffix" value=".jsp" />
</bean>
<mvc:resources mapping="/resources/**" location="/resources/" />
<mvc:annotation-driven />
```

Nous indiquons où
sont les beans web
annotés dans

le `context:component-scan`.

Nous créons un bean pour le résolveur de vue par défaut, `InternalResourceViewResolver`, avec comme propriétés :

- `viewClass=org.springframework.web.servlet.view.JstlView`
- `prefix="/WEB-INF/views/jsp/"`
- `suffix=".jsp"`

Nous indiquons ensuite où sont les fichiers de ressources :

```
<mvc:resources mapping="/resources/**" location="/resources/" />
```

et nous indiquons que les beans Spring MVC utilisent les

annotations :

```
<mvc:annotation-driven />
```

On peut alors créer le contrôleur avec une classe

annotée `@Controller` :

```
@Controller
public class BonjourController {
    private final Logger logger =
        LoggerFactory.getLogger(BonjourController.class);
    private final BonjourService bonjourService;
    @Autowired
    public BonjourController(BonjourService bonjourService) {
        this.bonjourService = bonjourService;
    }
    @RequestMapping(value = "/", method = RequestMethod.GET)
    public String index(Map<String, Object> model) {
        model.put("titre", bonjourService.getTitre(""));
        model.put("message", bonjourService.getMessage());
        return "index";
    }
    @RequestMapping(value = "/bonjour/{nom:.+}", method =
        RequestMethod.GET)
    public ModelAndView bonjour(@PathVariable("nom") String name) {
        ModelAndView model = new ModelAndView();
        model.setViewName("index");
        model.addObject("titre", bonjourService.getTitre(name));
        model.addObject("message", bonjourService.getMessage());
        return model;
    }
}
```

Le contrôleur utilise un service qui est injecté.

Nous indiquons via

l'annotation `@RequestMapping` le routage des méthodes en fonction de l'URL :

- Méthode appelée s'il n'y a pas de paramètre.

```
@RequestMapping(value = "/", method = RequestMethod.GET)
```

La méthode renvoie une page : `index`.

- Méthode appelée s'il y a une action `bonjour` avec le paramètre `nom`.

```
@RequestMapping(value = "/bonjour/{nom:.+}", method =
    RequestMethod.GET)
```

La méthode renvoie un `ModelAndView`.

Nous créons ensuite le service `BonjourService` appelé par le contrôleur :

```
@Service
public class BonjourService {
    public String getMessage() {
        return "Exemple application Spring MVC version XML";
    }
    public String getTitre(String nom) {
        logger.debug("getTitre() is executed! $nom: {}", nom);
        if(StringUtils.isEmpty(nom)){
            return "Bonjour à tous";
        }else{
            return "Bonjour " + nom;
        }
    }
}
```

Il s'agit d'un service standard.

b. Configuration par annotation

La version avec annotation permet de ne pas avoir de fichier web.xml.

Nous utilisons une classe pour la partie back et une classe pour la partie web.

Pour la partie back :

```
@Configuration
@ComponentScan({ "fr.eni.mvc.annot.service" })
public class SpringArriereConfig {
```

Pour la partie web :

La partie web utilise les annotations suivantes pour la configuration :

```
@EnableWebMvc
@Configuration
@ComponentScan({ "fr.eni.mvc.annot.web" })
public class SpringWebConfig extends WebMvcConfigurerAdapter {
    @Override
    public void addResourceHandlers(ResourceHandlerRegistry
registry) {
        registry.addResourceHandler("/resources/**").addResourceLocations
("/resources/");
    }

    @Bean
    public InternalResourceViewResolver viewResolver() {
        InternalResourceViewResolver viewResolver = new
InternalResourceViewResolver();
        viewResolver.setViewClass(JstlView.class);
        viewResolver.setPrefix("/WEB-INF/views/jsp/");
        viewResolver.setSuffix(".jsp");
        return viewResolver;
    }
}
```

@EnableWebMvc : indique qu'il s'agit d'une configuration Spring MVC.

- @Configuration : bean de configuration.
- @ComponentScan({ "fr.eni.mvc.annot.web" }) : les packages à scanner pour les classes annotées.

Les autres classes Service et Controller sont identiques à celles de la configuration XML.

2. Éléments complexes du contrôleur

a. Fonctionnement global du contrôleur

Le contrôleur interprète les données provenant du client pour les transformer sous la forme d'un modèle correspondant à une vue pour l'utilisateur.

Nous utilisons l'annotation @Controller pour identifier les contrôleurs.

Le Request Mapping peut se faire depuis les méthodes ou depuis la classe pour un premier niveau, il est suivi de la méthode pour un second niveau.

est équivalent à :

```
@Controller
@RequestMapping("/class-mapping/*")
public class ClasslevelMappingController {
    @RequestMapping("/pathc")
    public @ResponseBody String parLePath() {
        return "Mappé par le path!!";
    }
}
```

Le contrôleur le plus simple indique pour une URL, ici

```
@Controller
public class ParMappingController {
    @RequestMapping("/mapping/pathm")
    public @ResponseBody String parLePath() {
        return "Mappé par le path!";
    }
}
```

"/mapping/pathm" quelle méthode correspond, ici parLePath(), qui effectue un traitement et indique la page qui sera retournée.

Pour plus de concision, on se base dans les exemples sur un routage uniquement dans la méthode utilisée par les contrôleurs mais le principe reste le même si on partageait le routage entre la classe et la méthode.

Le test unitaire par path est un peu plus complexe car il faut créer un mock MockMvc et l'initialiser :

```
@RunWith(SpringJUnit4ClassRunner.class)
public class ParMappingControllerTests extends
AbstractContextControllerTests {
    private MockMvc mockMvc;
    @Before
    public void setup() throws Exception {
        this.mockMvc =
webAppContextSetup(this.wac).alwaysExpect(status().isOk())
.build();
    }
    @Test
    public void byPathc() throws Exception {
        this.mockMvc.perform(get("/mapping/pathc")).andExpect(content()
.string("Mappé par path!"));
    }
}
```

➤ On réutilise le mock mockMvc pour les autres tests du Request Mapping.

b. Exemple de la classe Simple

Nous commencerons par le cas le plus simple. Il s'agit d'afficher une page qui répond sur une URL de type /premier :

Code dans la page :

Nous ajoutons un href pour pointer vers la servlet :

```
<a id="simpleLink" class="textLink" href="<c:url value="/premier"
/>">GET /premier</a>
```

Dans le contrôleur :

simple :

Ce contrôleur est

```
@RequestMapping("/premier")
public @ResponseBody String simple() {
    logger.debug("Réponse");
    return "ENI vous souhaite le bonjour!";
}
```

Tests unitaires :

Le TU charge la page, vérifie le statut et le type de contenu.

```
@Test
public void simple() throws Exception {
    standaloneSetup(new PremierControleur()).build()
        .perform(get("/premier"))
        .andExpect(status().isOk())
        .andExpect(content().contentType("text/plain;charset=ISO-
8859-1"));
    .andExpect(content().string("ENI vous souhaite le bonjour!"));
}
```

c. Simple revisité

La méthode simple2() précise qu'elle n'accepte que les requêtes de type GET.

Code dans la page :

```
<a id="simpleRevisited" class="textLink" href="<c:url
value="/premier/revisite" />">GET /premier/revisite</a>
```

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping(value="/premier/revisite",
method=RequestMethod.GET, headers="Accept=text/plain")
public @ResponseBody String simple2() {
    return "ENI vous souhaite bon appétit!";
}
```

```
@Test
public void simple2() throws Exception {
    standaloneSetup(new PremierControleur()).build()
        .perform(get("/premier/revisite").accept(MediaType.TEXT_PLAIN))
        .andExpect(status().isOk())
        .andExpect(content().contentType("text/plain"))
        .andExpect(content().string("ENI vous souhaite bon appétit!"));
}
```

d. Par le path

Si la méthode est annotée avec @ResponseBody, le type de retour est écrit dans le corps de réponse HTTP.

La valeur de retour sera convertie au type d'argument de la méthode déclarée à l'aide deHttpMessageConverter.

Code dans la page :

```
<a id="byPath" class="textLink" href="<c:url  
value="/mapping/path" />">Par le path</a>
```

Dans le contrôleur

:

Tests unitaires :

```
@RequestMapping("/mapping/path")  
public @ResponseBody String byPath() {  
    return "Mappé par path!";  
}
```

Dans le contrôleur

:

```
@Test  
public void byPath() throws Exception {  
    this.mockMvc.perform(get("/mapping/path")).andExpect(content()  
        .string("Mappé par path!"));  
}
```

```
@RequestMapping(value="/mapping/path/*",  
method=RequestMethod.GET)  
public @ResponseBody String byPathPattern(HttpServletRequest  
request) {  
    return "Mappé par le path pattern ('" + request.getRequestURI()  
        + "')";  
}
```

e. Par un motif sur le path

L'annotation `@RequestMapping` traite aussi les chemins de type Ant (par exemple, `/myPath/*.do`).

On peut mixer les modèles d'URI avec variables.

Code dans la page :

```
<a id="byPathPattern" class="textLink" href="<c:url  
value="/mapping/path/wildcard" />">Par un motif sur le path</a>
```

Tests unitaires :

```
@Test  
public void byPathPattern() throws Exception {  
    this.mockMvc.perform(get("/mapping/path/wildcard"))  
        .andExpect(content().string("Mappé par le path pattern  
('/mapping/path/wildcard')"));  
}
```

f. Par le path et une méthode

Nous pouvons utiliser l'URL pour pointer sur une méthode via un path.

Code dans la page :

```
<a id="byMethod" class="textLink" href="<c:url  
value="/mapping/method" />">Par le path et une méthode</a>
```

Dans le contrôleur

:

Tests unitaires :

```
@RequestMapping(value="/mapping/method",  
method=RequestMethod.GET)  
public @ResponseBody String byMethod() {  
    return "Mappé par path + méthode";  
}
```

```
@Test  
public void byMethod() throws Exception {  
    this.mockMvc.perform(get("/mapping/method"))  
        .andExpect(content().string("Mappé par path + méthode"));  
}
```

g. Mappé par path + méthode + présence de paramètres de query

Nous pouvons passer des paramètres dans l'URL.

Code dans la page :

```
<a id="byParameter" class="textLink" href="<c:url  
value="/mapping/parameter?foo=bar" />">Mappé par path + méthode +  
présence de paramètres de query!</a>
```

Dans le contrôleur

:

Tests unitaires :

```
@RequestMapping(value="/mapping/parameter",  
method=RequestMethod.GET, params="foo")  
public @ResponseBody String byParameter() {
```

```
return "Mappé par path + méthode + présence de paramètres de query!";
```

```
@Test
public void byParameter() throws Exception {
    this.mockMvc.perform(get("/mapping/parameter?foo=bar"))
        .andExpect(content().string("Mappé par path + méthode + présence de paramètres de query!"));
}
```

h. Mappé par path + méthode + présence d'un header

Nous pouvons passer des paramètres dans l'URL et spécifier la méthode à appeler :

Code dans la page :

```
<a id="byHeader" href="<c:url value="/mapping/header" />">Par la présence d'un header</a>
```

Dans le contrôleur :

```
@RequestMapping(value="/mapping/header",
method=RequestMethod.GET, headers="FooHeader=foo")
public @ResponseBody String byHeader() {
    return "Mappé par path + méthode + présence d'un header!";
}
```

Tests unitaires :

```
@Test
public void byHeader() throws Exception {
    this.mockMvc.perform(get("/mapping/header").header("FooHeader", "foo"))
        .andExpect(content().string("Mappé par path + méthode + présence d'un header!"));
}
```

i. Par l'absence d'un header

Nous pouvons appeler une méthode si nous détectons l'absence du header :

Code dans la page :

```
<a id="byHeaderNegation" class="textLink" href="<c:url value="/mapping/header" />">Par l'absence d'un header</a>
```

Dans le contrôleur :

```
@RequestMapping(value="/mapping/header",
method=RequestMethod.GET, headers="!FooHeader")
public @ResponseBody String byHeaderNegation() {
    return "Mappé par path + méthode + absence d'un header!";
}
```

Tests unitaires :

```
@Test
public void byHeaderNegation() throws Exception {
    this.mockMvc.perform(get("/mapping/header"))
        .andExpect(content().string("Mappé par path + méthode + absence d'un header!"));
}
```

j. Par consommation

Nous pouvons avoir une méthode de contrôleur qui consomme une ressource, par exemple un JSON :

Code dans la page :

```
<form id="byConsumes" class="readJsonForm" action="<c:url value="/mapping/consumes" />" method="post">
<input id="byConsumesSubmit" type="submit" value="Par consommation" />
</form>
```

Dans le contrôleur :

```
@RequestMapping(value="/mapping/consumes",
method=RequestMethod.POST,
consumes=MediaType.APPLICATION_JSON_VALUE)
public @ResponseBody String byConsumes(@RequestBody JavaBean javaBean) {
    return "Mappé par path + méthode + consommable par un type de média (javaBean ' + javaBean + ' )";
}
```

Tests unitaires :

```
@Test
public void byConsumes() throws Exception {
```

```

        this.mockMvc.perform(
            post("/mapping/consumes")
                .contentType(MediaType.APPLICATION_JSON)
                .content("{ \"foo\": \"bar\", \"fruit\": \"apple\" }")
        ).andExpect(content().string(contains("Mappé par path + méthode + consommable par un type de média (javaBean)")));

```

k. Par production via Accept=application/json

Nous pouvons produire une ressource de type JSON :

Code dans la page :

```

<a id="byProducesAcceptJson" class="writeJsonLink" href="#>Par production via
Accept=application/json</a>

```

Dans le contrôleur :

```

@RequestMapping(value="/mapping/produces",
    method=RequestMethod.GET,
    produces=MediaType.APPLICATION_JSON_VALUE)
public @ResponseBody JavaBean byProducesJson() {
    return new JavaBean();
}

```

Tests unitaires :

```

@Test
public void byProducesAcceptJson() throws Exception {
    this.mockMvc.perform(get("/mapping/produces").accept(
        MediaType.APPLICATION_JSON))
        .andExpect(jsonPath("$.foo").value("bar"))
        .andExpect(jsonPath("$.fruit").value("apple"));
}

```

l. Par production via Accept=application/xml

Nous pouvons passer des paramètres dans l'URL au format XML ou JSON :

Dans la page pour XML :

```

<a id="byProducesAcceptXml" class="writeXmlLink" href="#>Par production via
Accept=application/xml</a>

```

Dans le contrôleur :

```

@RequestMapping(value="/mapping/produces",
    method=RequestMethod.GET,
    produces=MediaType.APPLICATION_XML_VALUE)
public @ResponseBody JavaBean byProducesXml() {
    return new JavaBean();
}

```

Code dans la page pour JSON :

Tests unitaires :

```

@Test
public void byProducesAcceptXml() throws Exception {
    this.mockMvc.perform(get("/mapping/produces").accept(
        MediaType.APPLICATION_XML))
        .andExpect(xpath("/javaBean/foo").string("bar"))
        .andExpect(xpath("/javaBean/fruit").string("apple"));
}

```

Code dans la page XML (variante) :

Tests unitaires :

```

<a id="byProducesJsonExt" class="writeJsonLink" href="#>By produces via ".json"</a>

```

```

@Test
public void byProducesJsonExtension() throws Exception {
    this.mockMvc.perform(get("/mapping/produces.json"))
        .andExpect(jsonPath("$.foo").value("bar"))
        .andExpect(jsonPath("$.fruit").value("apple"));
}

```

```

<a id="byProducesXmlExt" class="writeXmlLink" href="#>By produces via ".xml"</a>

```

```

@Test
public void byProducesXmlExtension() throws Exception {
    this.mockMvc.perform(get("/mapping/produces.xml"))
        .andExpect(xpath("/javaBean/foo").string("bar"))
        .andExpect(xpath("/javaBean/fruit").string("apple"));
}

```

m. Paramètres de query

Nous pouvons utiliser des paramètres de query isolés :

Code dans la page :

```
<a id="param" class="textLink" href="<c:url  
value="/data/param?foo=bar" />">Paramètres de query</a>
```

Dans le contrôleur :

```
@RequestMapping(value="param", method=RequestMethod.GET)  
public @ResponseBody String withParam(@RequestParam String foo) {  
    return "La valeur du paramètre de query 'foo' [" + foo + "];"
```

Tests unitaires :

```
@Test  
public void param() throws Exception {  
    this.mockMvc.perform(get("/data/param?foo=bar"))  
        .andExpect(content().string("La valeur du paramètre de query  
'foo' [bar]"));
```

n. Groupes de paramètres de query

Nous pouvons utiliser des paramètres de query multiples :

Code dans la page :

```
<a id="group" class="textLink" href="<c:URL  
value="/data/group?param1=foo&param2=bar&param3=baz" />">Groupe  
de paramètres de query</a>
```

Dans le contrôleur :

```
@RequestMapping(value="group", method=RequestMethod.GET)  
public @ResponseBody String withParamGroup(JavaBean bean) {  
    return "Le groupe de paramètres " + bean;
```

Tests unitaires :

```
@Test  
public void group() throws Exception {  
    this.mockMvc.perform(get("/data/group?param1=  
foo&param2=bar&param3=baz"))  
        .andExpect(content().string(startsWith(  
            "Le groupe de paramètres  
com.eni.spring.mvc.exemple.data.JavaBean@")));
```

o. Variable du path

Nous pouvons utiliser le path pour spécifier une variable et sa valeur :

Code dans la page :

```
<a id="var" class="textLink" href="<c:url value="/data/path/foo"  
>">Variable du path</a>
```

Dans le contrôleur :

```
@RequestMapping(value="path/{var}", method=RequestMethod.GET)  
public @ResponseBody String withPathVariable(@PathVariable String  
var) {  
    return "La valeur de la variable 'var' du path [" + var + "];"
```

Tests unitaires :

```
@Test  
public void pathVar() throws Exception {  
    this.mockMvc.perform(get("/data/path/foo"))  
        .andExpect(content().string("La valeur de la variable 'var' du  
path [foo]"));
```

p. Variable Matrix

Nous pouvons utiliser une grille ou matrice pour passer des paramètres :

 Les variables Matrix sont délicates à utiliser mais économi-ques au niveau de la taille de l'URL.

Code dans la page :

```
<a id="matrixVar" class="textLink" href="<c:url
value="/data/matrixvars;foo=bar/simple" />">Variable Matrix</a>
```

Dans le contrôleur :

```
@RequestMapping(value="{path}/simple", method=RequestMethod.GET)
public @ResponseBody String withMatrixVariable(@PathVariable
String path, @MatrixVariable String foo) {
return "La variable Matrix 'foo=[" + foo + "]" du segment de path
["+path+"]";
```

Tests unitaires :

```
@Test
public void matrixVar() throws Exception {
this.mockMvc.perform(get("/data/matrixvars;foo=bar/simple"))
.andDo(print())
.andExpect(content().string("La variable Matrix 'foo=[bar]' du
segment de path [matrixvars]"));
```

q. Variables Matrix multiples

Nous pouvons utiliser une grille ou matrice pour passer des paramètres multiples :

Code dans la page :

```
<a id="matrixVarMultiple" class="textLink" href="<c:url
value="/data/matrixvars;foo=bar1/multiple;foo=bar2" />">Variables
Matrix multiples</a>
```

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping(value="{path1}/{path2}",
method=RequestMethod.GET)
public @ResponseBody String withMatrixVariablesMultiple (
@PathVariable String path1, @MatrixVariable(value="foo",
pathVar="path1") String foo1,
@PathVariable String path2, @MatrixVariable(value="foo",
pathVar="path2") String foo2) {
return "La variable Matrix foo=[" + foo1 + "] du
segment de path [" + path1
+ "]" La variable Matrix foo=[" + foo2 + "]
du segment de path [" + path2 + "]";
```

```
@Test
public void matrixVarMultiple() throws Exception {
this.mockMvc.perform(get("/data/matrixvars;foo=bar1/multiple;foo=
bar2"))
.andExpect(content().string("La variable Matrix
foo=[bar1] du segment de path [matrixvars] La variable Matrix
foo=[bar2] du segment de path [multiple]"));
```

r. Header

Nous pouvons détecter la méthode à appeler en se basant sur le header :

Code dans la page :

```
<a id="header" class="textLink" href="<c:url value="/data/header"
/>">Header</a>
```

Dans le contrôleur :

```
@RequestMapping(value="header", method=RequestMethod.GET)
public @ResponseBody String withHeader(@RequestHeader String
Accept) {
return "Obtention du 'Accept' du header '" + Accept + "'";
```

s. Obtention du cookie

Nous pouvons utiliser les cookies.

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping(value="cookie", method=RequestMethod.GET)
public @ResponseBody String withCookie(@CookieValue String
openid_provider) {
return "Obtention du cookie 'openid_provider' [" +
openid_provider + "]";
```

t. Corps de la req

```
@Test
public void header() throws Exception {
    this.mockMvc.perform(get("/data/header").accept(MediaType.ALL))
        .andExpect(content().string("Obtention du 'Accept' du header '*/*'"));
}
```

Nous pouvons utiliser le corps de la request pour le mapping.

Code dans la

page :

```
<form id="requestBody" class="textForm" action="<c:url
value="/data/body" />" method="post">
<input id="requestBodySubmit" type="submit" value="Corps de la
requête" />
</form>
```

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping(value="body", method=RequestMethod.POST)
public @ResponseBody String withBody(@RequestBody String body) {
    return "Corps de la requête postée : [" + body + "];";
}
```

```
@Test
public void requestBody() throws Exception {
    this.mockMvc.perform(
        post("/data/body")
            .contentType(MediaType.TEXT_PLAIN)
            .content("foo".getBytes()))
        .andExpect(content().string("Corps de la requête postée :
[foo]"));
}
```

u. En-tête et corps de la requête

Nous pouvons également utiliser l'en-tête et le corps de la request.

Code dans la page :

```
<form id="requestBodyAndHeaders" class="textForm" action="<c:url
value="/data/entity" />" method="post">
<input id="requestBodyAndHeadersSubmit" type="submit"
value="En-tête et corps de la requête" />
</form>
```

Dans le contrôleur :

Tests unitaires :

Code dans la page :

```
@RequestMapping(value="entity", method=RequestMethod.POST)
public @ResponseBody String withEntity(HttpEntity<String> entity)
{
    return "Corps de la requête postée [" + entity.getBody() + "];
headers = " + entity.getHeaders();
}
```

```
@Test
public void requestBodyAndHeaders() throws Exception {
    this.mockMvc.perform(
        post("/data/entity")
            .contentType(MediaType.TEXT_PLAIN)
            .content("foo".getBytes()))
        .andExpect(content().string(
            "Corps de la requête postée [foo]; headers =
{Content-Type=[text/plain], Content-Length=[3]}"));
}
```

```
<html>
<head><title>First JSP</title></head>
<%@ page import="java.sql.*" %>
<body>
<h2>Hello World!</h2>
<%
    out.println("Hello Hervé");
%>
</body>
</html>
```

v. Arguments de la requête

Il est possible d'utiliser des arguments pour la request.

Code dans la page :

```
<a id="request" class="textLink" href="<c:url
```

Dans le contrôleur :

```
value="/data/standard/request" />">Les arguments de la  
requête</a>
```

Tests unitaires :

```
// request related  
@RequestMapping(value="/data/standard/request",  
method=RequestMethod.GET)  
public @ResponseBody String  
standardRequestArgs(HttpServletRequest request, Principal user,  
Locale locale) {  
    StringBuilder buffer = new StringBuilder();  
    buffer.append("request = ").append(request).append(", ");  
    buffer.append("userPrincipal = ").append(user).append(", ");  
    buffer.append("requestLocale = ").append(locale);  
    return buffer.toString();  
}
```

```
@Test  
public void request() throws Exception {  
    this.mockMvc.perform(get("/data/standard/request"))  
        .andExpect(content().string(startsWith(  
            "request = "  
org.springframework.mock.web.MockHttpServletRequest@"))));  
}
```

w. Reader sur la requête

Nous pouvons mettre un `reader` sur la `request`.

Code dans la page :

```
<form id="requestReader" class="textForm" action="<c:url  
value="/data/standard/request/reader" />" method="post">  
<input id="requestReaderSubmit" type="submit" value="Reader sur  
la requête" />  
</form>
```

Dans le contrôleur

:

Tests unitaires :

```
@RequestMapping(value="/data/standard/request/reader",  
method=RequestMethod.POST)  
public @ResponseBody String requestReader(Reader  
requestBodyReader) throws IOException {  
    return "Lecture des caractères du corps de la requête = " +  
        FileCopyUtils.copyToString(requestBodyReader);  
}
```

```
@Test  
public void requestReader() throws Exception {  
    this.mockMvc.perform(  
        post("/data/standard/request/reader")  
            .contentType(MediaType.TEXT_PLAIN)  
            .content("foo".getBytes()))  
        .andExpect(content().string("Lecture des caractères du corps de la  
requête = foo"));
```

x. InputStream de la requête

Nous pouvons aussi récupérer un `InputStream` sur la `request`.

Code dans la page :

```
<form id="requestIs" class="textForm" action="<c:url  
value="/data/standard/request/is" />" method="post">  
<input id="requestIsSubmit" type="submit" value="L'InputStream de  
la requête" />  
</form>
```

Dans le contrôleur

:

Tests unitaires :

```
@RequestMapping(value="/data/standard/request/is",  
method=RequestMethod.POST)  
public @ResponseBody String requestReader(InputStream  
requestBodyIs) throws IOException {  
    return "Valeurs binaires du corps de la requête = " + new  
        String(FileCopyUtils.copyToByteArray(requestBodyIs));  
}
```

```
@Test  
public void requestIs() throws Exception {  
    this.mockMvc.perform(  
        post("/data/standard/request/is")
```

```

        .contentType(MediaType.TEXT_PLAIN)
        .content("foo".getBytes()))
.andExpect(content().string("Valeurs binaires du corps de la
requête = foo"));

```

y. Arguments de la réponse

Nous pouvons travailler avec les arguments de la réponse.

Code dans la page :

```

<a id="response" class="textLink" href="<c:url
value="/data/standard/response" />">Les arguments de la
réponse</a>

```

Dans le contrôleur :

Tests unitaires :

```

// response related
@RequestMapping("/data/standard/response")
public @ResponseBody String response(HttpServletResponse
response) {
    return "response = " + response;
}

```

```

@Test
public void response() throws Exception {
    this.mockMvc.perform(get("/data/standard/response"))
        .andExpect(content().string(startsWith(
            "response =
org.springframework.mock.web.MockHttpServletResponse@"))));
}

```

z. Writer de la réponse

Nous avons accès au `Writer` de la `Response`.

Code dans la page :

```

<a id="writer" class="textLink" href="<c:url
value="/data/standard/response/writer" />">Writer de la
réponse</a>

```

Dans le contrôleur :

Tests unitaires :

```

@RequestMapping("/data/standard/response/writer")
public void availableStandardResponseArguments(Writer
responseWriter) throws IOException {
    responseWriter.write("Ecriture de caractères dans la réponse en
utilisant le Writer");
}

```

```

@Test
public void writer() throws Exception {
    this.mockMvc.perform(get("/data/standard/response/writer"))
        .andExpect(content().string("Ecriture de caractères dans la
réponse en utilisant le Writer"));
}

```

aa. OutputStream de la réponse

Nous avons aussi accès à l'`OutputStream` de la `Response`.

Code dans la page :

```

<a id="os" class="textLink" href="<c:url
value="/data/standard/response/os" />">OutputStream de la
réponse</a>

```

Dans le contrôleur :

Tests unitaires :

```

@RequestMapping("/data/standard/response/os")
public void availableStandardResponseArguments(OutputStream os)
throws IOException {
    os.write("Ecriture en binaire dans la réponse en utilisant
OutputStream".getBytes());
}

```

```

@Test
public void os() throws Exception {
    this.mockMvc.perform(get("/data/standard/response/os"))
        .andExpect(content().string("Ecriture en binaire dans la réponse
en utilisant OutputStream"));
}

```

ab. Session

Il est possible de récupérer les éléments de la session comme le SESSIONID avec un contrôleur de type `@RequestMapping("/data/standard/session")` :

Code dans la page :

```
<a id="session" class="textLink" href="<c:URL
value="/data/standard/session" />">Session</a>
```

Dans le contrôleur :

```
// HttpSession
@RequestMapping("/data/standard/session")
public @ResponseBody String session(HttpSession session) {
    StringBuilder buffer = new StringBuilder();
    buffer.append("session=").append(session);
    return buffer.toString();
}
```

Tests unitaires :

```
@Test
public void session() throws Exception {
    this.mockMvc.perform(get("/data/standard/session"))
        .andExpect(content().string(startsWith(
            "session=org.springframework.mock.web.MockHttpSession@")));
}
```

ac. Handler personnalisé

Nous avons personnalisé le handler de méthode pour effectuer des opérations avant de l'invoquer.

Code dans la page :

```
<a id="customArg" class="textLink" href="<c:URL
value="/data/custom" />">Customisé</a>
```

Dans le contrôleur :

```
@ModelAttribute
void beforeInvokingHandlerMethod(HttpServletRequest request) {
    request.setAttribute("foo", "bar");
}
@RequestMapping(value="/data/custom", method=RequestMethod.GET)
public @ResponseBody String custom(@ModelAttribute("foo")
String foo) {
    return "Obtention de la valeur de l'attribut 'foo'=[" + foo +
    "]";
}
```

Le résolveur d'arguments personnalisé :

Tests unitaires :

```
public class CustomArgumentResolver implements
HandlerMethodArgumentResolver {
    public boolean supportsParameter(MethodParameter parameter) {
        return
parameter.getParameterAnnotation(RequestAttribute.class) != null;
    }
    public Object resolveArgument(MethodParameter parameter,
ModelAndViewContainer mavContainer, NativeWebRequest webRequest,
WebDataBinderFactory binderFactory) throws Exception {
        RequestAttribute attr =
parameter.getParameterAnnotation(RequestAttribute.class);
        return webRequest.getAttribute(attr.value(),
WebRequest.SCOPE_REQUEST);
    }

    @Target(ElementType.PARAMETER)
    @Retention(RetentionPolicy.RUNTIME)
    @Documented
    public @interface RequestAttribute {
        String value();
    }
}
```

```
@Before
public void setup() throws Exception {
    this.mockMvc = standaloneSetup(new CustomArgumentController())
        .setCustomArgumentResolvers(new
CustomArgumentResolver()).build();
}
@Test
public void param() throws Exception {
    this.mockMvc.perform(get("/data/custom"))
        .andExpect(content().string("Obtention de la valeur de l'attribut
'foo'=[bar]"));
}
```

ad. Utiliser l'annotation `@ResponseBody` seule

L'annotation `@ResponseBody` indique que la valeur retournée par la méthode du contrôleur ira dans le corps de la réponse.

Code dans la page :

```
<a id="responseBody" class="textLink" href="<c:url
value="/response/annotation" />">@ResponseBody</a>
```

Dans le contrôleur
:

Tests unitaires :

```
@RequestMapping("/annotation")
public @ResponseBody String responseBody() {
    return "La String ResponseBody";
}
```

```
@Test
public void responseBody() throws Exception {
    this.mockMvc.perform(get("/response/annotation"))
        .andExpect(status().isOk())
        .andExpect(content().string("La String ResponseBody"));
}
```

ae. Utiliser l'annotation `@ResponseBody` avec UTF-8 requis

Nous pouvons spécifier le charset. Nous essaierons, dans la mesure du possible, d'utiliser l'UTF-8, de façon à ce qu'il n'y ait pas de problèmes avec les caractères accentués.

Code dans la page :

```
<a id="responseCharsetAccept" class="utf8TextLink" href="<c:url
value="/response/charset/accept" />">@ResponseBody (UTF-8 charset
requested)</a>
```

Dans le contrôleur
:

Tests unitaires :

```
@RequestMapping("/charset/accept")
public @ResponseBody String responseAcceptHeaderCharset() {
    return "\u3053\u3093\u306b\u3061\u306f\u4e16\u754c\u00ff01
(\\\"Hello world!\\\" en japonais)";
}
```

```
@Test
public void responseCharsetAccept() throws Exception {
    this.mockMvc.perform(
        get("/response/charset/accept")
        .accept(new MediaType("text", "plain",
            Charset.forName("UTF-8"))))
        .andExpect(status().isOk())
        .andExpect(content().string(
            "\u3053\u3093\u306b\u3061\u306f\u4e16\u754c\u00ff01
(\\\"Hello world!\\\" en japonais)"));
}
```

af. Utiliser l'annotation `@ResponseBody` avec UTF-8 produit

Nous vérifions que le charset est supporté.

Code dans la page :

```
<a id="responseCharsetProduce" class="textLink" href="<c:url
value="/response/charset/produce" />">@ResponseBody (UTF-8
charset produced)</a>
```

Dans le contrôleur
:

Tests unitaires :

```
@RequestMapping(value="/charset/produce",
produces="text/plain;charset=UTF-8")
public @ResponseBody String responseProducesConditionCharset() {
    return "\u3053\u3093\u306b\u3061\u306f\u4e16\u754c\u00ff01
(\\\"Hello world!\\\" en japonais)";
}
```

```
@Test
public void responseCharsetProduce() throws Exception {
    this.mockMvc.perform(get("/response/charset/produce"))
        .andExpect(status().isOk())
        .andExpect(content().string(
            "\u3053\u3093\u306b\u3061\u306f\u4e16\u754c\u00ff01
(\\\"Hello world!\\\" en japonais)"));
}
```

ag. Réponse avec un statut personnalisé

Nous pouvons retourner un code de `status` spécifique.

Code dans la page :

```
<a id="responseEntityStatus" class="textLink" href="<c:url
value="/response/entity/status" />">ResponseEntity (custom
status)</a>
```

Dans le contrôleur
:

Tests unitaires :

```
@RequestMapping("/entity/status")
public ResponseEntity<String> responseEntityStatusCode() {
    return new ResponseEntity<String>("La String ResponseBody avec
le code status customisé (403 Forbidden)",
    HttpStatus.FORBIDDEN);
```

```
@Test
public void responseEntityStatus() throws Exception {
    this.mockMvc.perform(get("/response/entity/status"))
        .andExpect(status().isForbidden())
        .andExpect(content().string(
"La String ResponseBody avec le code status customisé (403
Forbidden)"));
```

ah. Réponse avec un en-tête personnalisé

Nous pouvons retourner un header personnalisé.

Code dans la page :

```
<a id="responseEntityHeaders" class="textLink" href="<c:url
value="/response/entity/headers" />">ResponseEntity (custom
headers)</a>
```

Dans le contrôleur
:

Tests unitaires :

```
@RequestMapping("/entity/headers")
public ResponseEntity<String> responseEntityCustomHeaders() {
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.TEXT_PLAIN);
    return new ResponseEntity<String>("La String du ResponseBody
avec le header customisé Content-Type=text/plain",
    headers, HttpStatus.OK);
```

```
@Test
public void responseEntityHeaders() throws Exception {
    this.mockMvc.perform(get("/response/entity/headers"))
        .andExpect(status().isOk())
        .andExpect(content().string(
"La String du ResponseBody avec le header customisé Content-
Type=text/plain"));
```

ai. Lire une chaîne de caractères

Nous pouvons lire une chaîne de caractères, caractère par caractère.

Code dans la page :

```
<form id="readString" class="textForm" action="<c:url
value="/convertisseurdemessages/string" />" method="post">
<input id="readStringSubmit" type="submit" value="Lire une
chaîne de caractères" />
</form>
```

Dans le contrôleur
:

Tests unitaires :

```
// String => Message
@RequestMapping(value="/string", method=RequestMethod.POST)
public @ResponseBody String readString(@RequestBody String
string) {
    return "Lire une chaîne de caractères '" + string + "'";
```

```
@Test
public void readString() throws Exception {
    this.mockMvc.perform(post(URI,
"string").content("foo".getBytes()))
        .andExpect(content().string("Lire une chaîne de caractères 'foo'"));
```

aj. Écrire une chaîne de caractères

Nous pouvons écrire une chaîne de caractères directement. Il faudra éviter de générer du balisage HTML par ce biais.

Code dans la page :

```
<a id="writeString" class="textLink" href="<c:url
value="/convertisseurdemessages/string" />">Écrire une chaîne
de caractères</a>
```

Dans le contrôleur
:
Tests unitaires :

```
@RequestMapping(value="/string", method=RequestMethod.GET)
public @ResponseBody String writeString() {
    return "Écrire une chaîne de caractères";
}
```

```
@Test
public void writeString() throws Exception {
    this.mockMvc.perform(get(URI, "string"))
        .andExpect(content().string("Écrire une chaîne de caractères"));
}
```

ak. Lire les données encodées dans l'URL

Nous pouvons lire des données encodées dans l'URL.

Code dans la page :

```
<form id="readForm" action="<c:url
value="/convertisseurdemessages/form" />" method="post">
<input id="readFormSubmit" type="submit" value="Lire depuis un
Data" />
</form>
```

Dans le contrôleur
:
Tests unitaires :

```
// Depuis une donnée encodée (application/x-www-form-urlencoded)
@RequestMapping(value="/form", method=RequestMethod.POST)
public @ResponseBody String readForm(@ModelAttribute JavaBean
bean) {
    return "Lecture x-www-form-urlencoded: " + bean;
}
```

```
@Test
public void readForm() throws Exception {
    this.mockMvc.perform(
        post(URI, "form")
            .contentType(MediaType.APPLICATION_FORM_URLENCODED)
            .param("foo", "bar")
            .param("fruit", "apple"))
        .andExpect(content().string("Lecture x-www-form-
-urlencoded: JavaBean {foo=[bar], fruit=[apple]}"));
}
```

al. Retourner une MultiValueMap

Nous pouvons retourner des données dans une MultiValueMap.

Code dans la page :

```
<a id="writeForm" href="<c:url
value="/convertisseurdemessages/form" />">Ecrire depuis un
Data</a>
```

Dans le contrôleur
:
Tests unitaires :

```
@RequestMapping(value="/form", method=RequestMethod.GET)
public @ResponseBody MultiValueMap<String, String> writeForm() {
    MultiValueMap<String, String> map = new
    LinkedMultiValueMap<String, String>();
    map.add("foo", "bar");
    map.add("fruit", "apple");
    return map;
}
```

```
@Test
public void writeForm() throws Exception {
    this.mockMvc.perform(get(URI, "form"))
        .andExpect(content()
            .contentType(MediaType.APPLICATION_FORM_URLENCODED))
        .andExpect(content()
            .string("foo=bar&fruit=apple"));
}
```

am. Lire une structure XML

Nous pouvons récupérer les membres d'un objet sous la forme d'une structure XML.

Code dans la page :

```
<form id="readXml" class="readXmlForm" action="<c:url
value="/convertisseurdemessages/xml" />" method="post">
<input id="readXmlSubmit" type="submit" value="Lire une structure XML" />
</form>
```

Dans le contrôleur
:

```
// Jaxb => RootElement
// (requires JAXB2 on the classpath - useful for serving clients
that expect to work with XML)
@RequestMapping(value="/xml", method=RequestMethod.POST)
public @ResponseBody String readXml(@RequestBody JavaBean bean) {
return "Lecture depuis XML: " + bean;
```

```
private static String XML =
    "<?xml version='1.0' encoding='UTF-8' "
    standalone='yes'?>" +
    "<javaBean><foo>bar</foo><fruit>apple</fruit></javaBean>";
```

an. Écrire dans une structure XML via Accept=application/xml

Nous pouvons retourner une structure XML qui contient les membres d'un objet.

Code dans la page :

```
<a id="writeXmlAccept" class="writeXmlLink" href="<c:url
value="/convertisseurdemessages/xml" />">Ecrire dans une structure XML via Accept=application/xml</a>
```

Dans le contrôleur
:
Tests unitaires :

```
@RequestMapping(value="/xml", method=RequestMethod.GET)
public @ResponseBody JavaBean writeXml() {
return new JavaBean("bar", "apple");
```

Code dans la
page :

```
@Test
public void readXml() throws Exception {
this.mockMvc.perform(
    post(URI, "xml")
        .contentType(MediaType.APPLICATION_XML)
        .content(XML.getBytes()))
    .andExpect(content().string("Lecture depuis XML:
JavaBean {foo=[bar], fruit=[apple]}"));
```

```
<a id="writeXmlExt" class="writeXmlLink" href="<c:URL
value="/convertisseurdemessages/xml.xml" />">Write XML via
".xml"</a>
```

ao. Lire une structure JSON

Nous pouvons récupérer les membres d'un objet sous la forme d'une structure JSON.

Code dans la page :

```
<form id="readJson" class="readJsonForm" action="<c:url
value="/convertisseurdemessages/json" />" method="post">
<input id="readJsonSubmit" type="submit" value="Lire une structure JSON" />
</form>
```

Dans le contrôleur
:

Tests unitaires :

Code dans la
page :

```
// Jackson => Message
//(requires Jackson on the classpath - particularly useful for
serving JavaScript clients that expect to work with JSON)
@RequestMapping(value="/json", method=RequestMethod.POST)
public @ResponseBody String readJson(@Valid @RequestBody JavaBean
bean) {
return "Lire depuis une structure JSON: " + bean;
```

```
@Test
public void writeXml() throws Exception {
this.mockMvc.perform(get(URI,
"xml").accept(MediaType.APPLICATION_XML))
    .andExpect(content().xml(XML));
```

```
<form id="readJsonInvalid" class="readJsonForm invalid"
action="<c:url value="/convertisseurdemessages/json" />"
method="post">
<input id="readInvalidJsonSubmit" type="submit" value="Lire une
structure JSON invalide (400 response code)" />
</form>
```

ap. Écrire dans une structure JSON via Accept=application/json

Nous pouvons retourner une structure JSON qui contient les membres d'un objet.

Code dans la page :

```
<a id="writeJsonAccept" class="writeJsonLink" href="<c:url
value="/convertisseurdemessages/json" />">Ecrire dans une
structure JSON via Accept=application/json</a>
```

Dans le contrôleur :

```
@RequestMapping(value="/json", method=RequestMethod.GET)
public @ResponseBody JavaBean writeJson() {
    return new JavaBean("bar", "apple");
}
```

Tests unitaires :

```
@Test
public void readJson() throws Exception {
    this.mockMvc.perform(
        post(URI, "json")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{ \"foo\": \"bar\", \"fruit\": \"apple\" }")
    ).getBytes())
    .andExpect(content().string("Lire depuis un JSON: JavaBean
{foo=[bar], fruit=[apple]}"));
}
```

```
@Test
public void writeJson() throws Exception {
    this.mockMvc.perform(get(URI,
"json").accept(MediaType.APPLICATION_JSON))
        .andExpect(jsonPath("$.foo").value("bar"))
        .andExpect(jsonPath("$.fruit").value("apple"));
}
```

```
@Test
public void writeJson2() throws Exception {
    this.mockMvc.perform(get(URI,
"json").accept(MediaType.APPLICATION_JSON))
        .andExpect(print())
        .andExpect(jsonPath("$.foo").value("bar"))
        .andExpect(jsonPath("$.fruit").value("apple"));
}
```

aq. Lire un Atom

Nous pouvons récupérer un Atom (<http://www.w3.org/wiki/Atom>). Un Atom est un élément qui a cette forme :

```
<atom:title type="texte">
...du texte...
</atom:title>
```

Code dans la page :

```
<form id="readAtom" action="<c:url
value="/convertisseurdemessages/atom" />" method="post">
<input id="readAtomSubmit" type="submit" value="Lire un Atom" />
</form>
```

Dans le contrôleur :

Tests unitaires :

```
// AtomFeed => Message
// (requires Rome on the classpath - useful for serving Atom feeds)
@RequestMapping(value="/atom", method=RequestMethod.POST)
public @ResponseBody String readFeed(@RequestBody Feed feed) {
    return "Lecture " + feed.getTitre();
}

private static String ATOM_XML = "<?xml version=\"1.0\"
encoding=\"UTF-8\"?>" +
    "<feed xmlns=\"http://www.w3.org/2005/Atom\"><title>Mon
Atom feed</title></feed>";
```

ar. Écrire un Aton

```
@Test
public void readAtom() throws Exception {
    this.mockMvc.perform(
        post(URI, "atom")
            .contentType(MediaType.APPLICATION_ATOM_XML)
            .content(ATOM_XML.getBytes()))
        .andExpect(content().string("Lecture Mon Atom feed"));
```

Nous pouvons retourner un Atom.

Code dans la page :

Dans le contrôleur :

```
<a id="writeAtom" href="<c:url
value="/convertisseurdemessages/atom" />">Ecrire un Atom</a>
```

Tests unitaires :

```
@RequestMapping(value="/atom", method=RequestMethod.GET)
public @ResponseBody Feed writeFeed() {
    Feed feed = new Feed();
    feed.setFeedType("atom_1.0");
    feed.setTitle("Mon Atom feed");
    return feed;
```

```
@Test
public void writeAtom() throws Exception {
    this.mockMvc.perform(get(URI,
"atom").accept(MediaType.APPLICATION_ATOM_XML))
        .andExpect(content().xml(ATOM_XML));
```

as. Lire un flux RSS

Nous pouvons lire un flux RSS.

Code dans la page :

```
<form id="readRss" action="<c:url
value="/convertisseurdemessages/rss" />" method="post">
<input id="readRssSubmit" type="submit" value="Lire un flux RSS" />
</form>
```

Dans le contrôleur :

Tests unitaires :

```
// RssChannel => Message
//(requires Rome on the classpath - useful for serving RSS feeds)
@RequestMapping(value="/rss", method=RequestMethod.POST)
public @ResponseBody String readChannel(@RequestBody Channel
channel) {
    return "Lecture " + channel.getTitre();
```

```
@Test
public void readRss() throws Exception {
    String rss = "<?xml version=\"1.0\" encoding=\"UTF-8\"?><rss
version=\"2.0\">" +
        "<channel><title>Mon RSS feed</title></channel></rss>";
    this.mockMvc.perform(
        post(URI, "rss")
            .contentType(MediaType.valueOf("application/rss+xml"))
            .content(rss.getBytes()))
        .andExpect(content().string("Lecture Mon RSS feed"));
```

at. Écrire un flux RSS

Nous pouvons retourner un flux RSS.

Code dans la page :

```
<a id="writeRss" href="<c:url
value="/convertisseurdemessages/rss" />">Ecrire un flux RSS</a>
```

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping(value="/rss", method=RequestMethod.GET)
public @ResponseBody Channel writeChannel() {
    Channel channel = new Channel();
    channel.setFeedType("rss_2.0");
    channel.setTitle("My RSS feed");
    channel.setDescription("Description");
    channel.setLink("http://localhost:8080/mvc-showcase/rss");
    return channel;
```

```
@Test
```

```
public void writeRss() throws Exception {
    String rss = "<?xml version=\"1.0\" encoding=\"UTF-8\"?>" +
        "<rss version=\"2.0\"><channel><title>My RSS"
        feed</title><link>http://localhost:8080/mvc-showcase/rss</link><description>Description</description></channel>"
        </rss>";
    this.mockMvc.perform(get(URI,
        "rss").accept(MediaType.valueOf("application/rss+xml")))
        .andExpect(content().xml(rss));
}
```

au. HTML généré par un template JSP

Il est possible de construire un template HTTP pour les JSP.

Code dans la page :

```
<a href="<c:url value="/views/html" />">HTML généré par un
template JSP</a>
```

Dans le contrôleur :

```
@RequestMapping(value="html", method=RequestMethod.GET)
public String prepare(Model model) {
    model.addAttribute("foo", "bar");
    model.addAttribute("fruit", "apple");
    return "views/html";
}
```

Tests unitaires :

```
@Test
public void htmlView() throws Exception {
    this.mockMvc.perform(get("/views/html"))
        .andExpect(view().nom(containsString("views/html")))
        .andExpect(model().attribute("foo", "bar"))
        .andExpect(model().attribute("fruit", "apple"))
        .andExpect(model().size(2));
}
```

av. Mapping à partir d'un modèle

Nous pouvons utiliser `DefaultRequestToViewNameTranslator`.

C'est très pratique car cela permet de faire un mapping automatique à partir d'un modèle.

Code dans la page :

```
<a href="<c:url value="/views/viewName"
/>">DefaultRequestToViewNameTranslator convention</a>
```

Dans le contrôleur :

```
@RequestMapping(value="/viewName", method=RequestMethod.GET)
public void usingRequestToViewNameTranslator(Model model) {
    model.addAttribute("foo", "bar");
    model.addAttribute("fruit", "apple");
}
```

Tests unitaires :

```
@Test
public void viewName() throws Exception {
    this.mockMvc.perform(get("/views/viewName"))
        .andExpect(view().nom(containsString("views/viewName")))
        .andExpect(model().attribute("foo", "bar"))
        .andExpect(model().attribute("fruit", "apple"))
        .andExpect(model().size(2));
}
```

aw. Utiliser des variables dans un template de vue

Nous pouvons utiliser des variables dans un template de vue avec l'utilisation de l'annotation `@PathVariable`.

Code dans la page :

```
<a href="<c:url value="/views/pathVariables/bar/apple"
/>">Utiliser des variables dans un template de vue </a>
```

Dans le contrôleur :

```
@RequestMapping(value="pathVariables/{foo}/{fruit}",
method=RequestMethod.GET)
public String pathVars(@PathVariable String foo, @PathVariable
String fruit) {
    // No need to add @PathVariables "foo" and "fruit" to the
    model
    // They will be merged in the model before rendering
    return "views/html";
}
```

Tests unitaires :

```
@Test
public void uriTemplate() throws Exception {
    this.mockMvc.perform(get("/views/pathVariables/bar/apple"))
        .andExpect(view().nom(containsString("views/html"))));
}
```

ax. Data binding avec des variables d'URI

Nous pouvons aussi binder automatiquement les variables d'URI.

Code dans la page :

```
<a href="<c:url value="/views/dataBinding/bar/apple" />">Data
binding avec des variables d'URI</a>
```

Dans le contrôleur :

```
@RequestMapping(value="dataBinding/{foo}/{fruit}",
method=RequestMethod.GET)
public String dataBinding(@Valid JavaBean javaBean, Model model)
{
    // JavaBean "foo" and "fruit" properties populated from URI
    variables
    return "views/dataBinding";
}
```

ay. Types primitifs

Nous pouvons mapper simplement les types primitifs tels que les dates.

Code dans la page :

```
<a id="primitive" class="textLink" href="<c:url
value="/convert/primitive?value=3" />">Primitif</a>
```

Classe utilisée pour montrer les différents types primitifs :

```
public class JavaBean {
    private Integer primitive;
    @DateTimeFormat(iso=ISO.DATE)
    private Date date;
    @MaskFormat("(###) ###-####")
    private String masked;
    // list will auto-grow as its dereferenced e.g. list[0]=value
    private List<Integer> list;
    // annotation type conversion rule will be applied to each
    list element
    @DateTimeFormat(iso=ISO.DATE)
    private List<Date> formattedList;
    // map will auto-grow as its dereferenced e.g. map[key]=value
    private Map<Integer, String> map;
    // nested will be set when it is referenced e.g.
    nested.foo=value
    private NestedBean nested;
    public Integer getPrimitive() {
        return primitive;
    }
    public void setPrimitive(Integer primitive) {
        this.primitive = primitive;
    }
    public Date getDate() {
        return date;
    }
    public void setDate(Date date) {
        this.date = date;
    }
    public String getMasked() {
        return masked;
    }
    public void setMasked(String masked) {
        this.masked = masked;
    }
    public List<Integer> getList() {
        return list;
    }
    public void setList(List<Integer> list) {
        this.list = list;
    }
    public List<Date> getFormattedList() {
        return formattedList;
    }
    public void setFormattedList(List<Date> formattedList) {

```

Tests unitaires :

```

        this.formattedList = formattedList;
    }
    public Map<Integer, String> getMap() {
        return map;
    }
    public void setMap(Map<Integer, String> map) {
        this.map = map;
    }
    public NestedBean getNested() {
        return nested;
    }
    public void setNested(NestedBean nested) {
        this.nested = nested;
    }
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("JavaBean");
        if (primitive != null) {
            sb.append(" primitive=").append(primitive);
        }
        if (date != null) {
            sb.append(" date=").append(date);
        }
        if (masked != null) {
            sb.append(" masked=").append(masked);
        }
        if (list != null) {
            sb.append(" list=").append(list);
        }
        if (formattedList != null) {
            sb.append(" formattedList=").append(formattedList);
        }
        if (map != null) {
            sb.append(" map=").append(map);
        }
        if (nested != null) {
            sb.append(" nested=").append(nested);
        }
        return sb.toString();
    }
}

@Target(value={ElementType.FIELD, ElementType.METHOD,
ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface MaskFormat {
    String value();
}

public class MaskFormatAnnotationFormatterFactory implements
AnnotationFormatterFactory<MaskFormat> {
    public Set<Class<?>> getFieldTypes() {
        Set<Class<?>> fieldTypes = new HashSet<Class<?>>(1, 1);
        fieldTypes.add(String.class);
        return fieldTypes;
    }
    public Parser<?> getParser(MaskFormat annotation, Class<?>
fieldType) {
        return new MaskFormatter(annotation.value());
    }
    public Printer<?> getPrinter(MaskFormat annotation, Class<?>
fieldType) {
        return new MaskFormatter(annotation.value());
    }
}

    private static class MaskFormatter implements
Formatter<String> {
        private javax.swing.text.MaskFormatter delegate;
        public MaskFormatter(String mask) {
            try {
                this.delegate = new
javax.swing.text.MaskFormatter(mask);
            } catch (ParseException e) {
                throw new IllegalArgumentException("Mask could not be
parsed " + mask, e);
            }
        }
        public String print(String object, Locale locale) {
            try {
                return delegate.valueToString(object);
            } catch (ParseException e) {
                throw new IllegalArgumentException("Unable to print
using mask " + delegate.getMask(), e);
            }
        }
    }

```

```

    }
    }
    public String parse(String text, Locale locale) throws
ParseException {
        return (String) delegate.stringToValue(text);
    }
}

public final class SocialSecurityNumber {
    private final String value;

    public SocialSecurityNumber(String value) {
        this.value = value;
    }

    @MaskFormat("###-##-####")
    public String getValue() {
        return value;
    }

    public static SocialSecurityNumber valueOf(@MaskFormat("###-
##-####") String value) {
        return new SocialSecurityNumber(value);
    }
}

@RequestMapping("primitive")
public @ResponseBody String primitive(@RequestParam Integer
value) {
    return "Conversion du type primitif " + value;
}

```

```

@Before
public void setup() throws Exception {
    FormattingConversionService cs = new
DefaultFormattingConversionService();
    cs.addFormatterForFieldAnnotation(new
MaskFormatAnnotationFormatterFactory());
    this.mockMvc = standaloneSetup(new ConvertController())
        .setConversionService(cs)
        .alwaysExpect(status().isOk())
        .build();

    private String getTimezone(int year, int month, int day)
    Calendar calendar = Calendar.getInstance();
    calendar.set(Calendar.YEAR, year);
    calendar.set(Calendar.MONTH, month);
    calendar.set(Calendar.DAY_OF_MONTH, day);
    Date date = calendar.getTime();
    TimeZone timezone = TimeZone.getDefault();
    boolean inDaylight = timezone.inDaylightTime(date);
    return timezone.getDisplayName(inDaylight, TimeZone.SHORT);
}

```

```

@Test
public void primitive() throws Exception {
    this.mockMvc.perform(get("/convert/primitive").param("value",
"3"))
        .andExpect(content().string("Conversion du type primitif 3"));
}

```

az. Date

Nous pouvons décoder les dates directement :

Code dans la page :

```

<a id="date" class="textLink" href="<c:URL
value="/convert/date/2010-07-04" />">Date</a>

```

Dans le contrôleur
:

Tests unitaires :

```

// requires Joda-Time on the classpath
@RequestMapping("date/{value}")
public @ResponseBody String date(@PathVariable
@DateTimeFormat(iso=ISO.DATE) Date value) {
    return "Date convertie " + value;
}

```

```

@Test
public void date() throws Exception {
    String timezone = getTimezone(2010, 7, 4);
    this.mockMvc.perform(get("/convert/date/2010-07-04"))
        .andExpect(content().string("Date convertie Sun Jul 04
00:00:00 " + timezone + " 2010"));
}

```

ba. Conversion des collections

Les collections sont souvent difficiles à passer. Spring permet de les convertir automatiquement.

Code dans la page :

```
<a id="collection" class="textLink" href="<c:url
value="/convert/collection?values=1&values=2&values=3&values=
4&values=5" />">Collection 1 (multi-value parameter)</a>
```

Dans le contrôleur :

```
@RequestMapping("collection")
public @ResponseBody String collection(@RequestParam
Collection<Integer> values) {
    return "Collection convertie " + values;
```

Tests unitaires :

```
@Test
public void collection() throws Exception {
    this.mockMvc.perform(get("/convert/collection?
values=1&values=2&values=3&values=4&values=5"))
        .andExpect(content().string("Collection convertie [1, 2,
3, 4, 5]"));

<a id="collection2" class="textLink" href="<c:url
value="/convert/collection?values=1,2,3,4,5" />">Collection 2
(single comma-delimited parameter value)</a>
```

```
@Test
public void collection2() throws Exception {
    this.mockMvc.perform(get("/convert/collection?values=1,2,3,4,5"))
        .andExpect(content().string("Collection convertie [1, 2, 3, 4, 5]"));
```

bb. Utiliser des collections formatées

Nous pouvons utiliser des collections formatées.

Code dans la page :

```
<a id="formattedCollection" class="textLink" href="<c:url
value="/convert/formattedCollection?values=2010-07-04,2011-07-04"
/>">@Formatted Collection</a>
```

Dans le contrôleur :

```
@RequestMapping("formattedCollection")
public @ResponseBody String formattedCollection(@RequestParam
@DateTimeFormat(iso=ISO.DATE) Collection<Date> values) {
    return "Collection convertie formatée " + values;
```

bc. Utiliser des objets personnalisés

Nous avons aussi la possibilité d'utiliser des objets personnalisés.

Code dans la page :

```
<a id="valueObject" class="textLink" href="<c:url
value="/convert/value?value=123456789" />">Objet personnalisé</a>
```

Dans le contrôleur :

```
@RequestMapping("value")
public @ResponseBody String valueObject(@RequestParam
SocialSecurityNumber value) {
    return "Valeur de l'objet converti " + value;
```

Tests unitaires :

```
@Test
public void valueOf() throws Exception {
    this.mockMvc.perform(get("/convert/value?value=123456789"))
        .andExpect(content().string(startsWith(
            "Valeur de l'objet converti
com.eni.spring.mvc.exemple.convert.SocialSecurityNumber"))));
```

bd. Utiliser un convertisseur personnalisé

Nous avons aussi les `Converters` pour nous aider à changer le format des données.

Code dans la page :

```
<a id="customConverter" class="textLink" href="<c:url
value="/convert/custom?value=123-45-6789" />">Custom
Converter</a>
```

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping("custom")
public @ResponseBody String customConverter(@RequestParam
@MaskFormat("###-##-####") String value) {
    return "Converti '"+ value +" avec un convertisseur customisé";
}
```

Code dans la page :

Tests unitaires :

```
@Test
public void custom() throws Exception {
    this.mockMvc.perform(get("/convert/custom?value=123-45-6789"))
        .andExpect(content().string("Converti '123456789' avec un
convertisseur customisé"));
}
```

Code dans la page :

Tests unitaires :

Code dans la page :

```
<a id="primitiveProp" class="textLink" href="<c:url
value="/convert/bean?primitive=3" />">Primitive</a>
```

Tests unitaires :

Code dans la page :

```
@Test
public void beanPrimitive() throws Exception {
    this.mockMvc.perform(get("/convert/bean?primitive=3"))
        .andExpect(content().string("Converti JavaBean
primitive=3"));
}
```

Tests unitaires :

Code dans la page :

Tests unitaires :

```
<a id="dateProp" class="textLink" href="<c:url
value="/convert/bean?date=2010-07-04" />">Date</a>
```

Code dans la page :

Tests unitaires :

```
@Test
public void beanDate() throws Exception {
    String timezone = getTimezone(2010, 7, 4);
    this.mockMvc.perform(get("/convert/bean?date=2010-07-04"))
        .andExpect(content().string("Converti JavaBean date=Sun
Jul 04 00:00:00 " + timezone + " 2010"));
}
```

Code dans la page :

Objet pour les exemples :

Tests unitaires :

```
<a id="maskedProp" class="textLink" href="<c:url
value="/convert/bean?masked=(205) 333-3333" />">Masked</a>
```

```
@Test
public void beanMasked() throws Exception {
    this.mockMvc.perform(get("/convert/bean?masked=(205) 333-
3333"))
        .andExpect(content().string("Converti JavaBean
masked=2053333333"));
}
```

```
<a id="listProp" class="textLink" href="<c:url
value="/convert/bean?list[0]=1&list[1]=2&list[2]=3" />">List
Elements</a>
```

```
@Test
public void beanCollection() throws Exception {
    this.mockMvc.perform(get("/convert/bean?list[0]=1&list[1]=
2&list[2]=3"))
        .andExpect(content().string("Converti JavaBean list=
[1, 2, 3]"));
}
```

```
<a id="formattedListProp" class="textLink" href="<c:url
value="/convert/bean?formattedList[0]=2010-07-
04&formattedList[1]=2011-07-04" />">@Formatted List Elements</a>
```

```
@Test
public void beanFormattedCollection() throws Exception {
    String timezone2010 = getTimezone(2010, 7, 4);
    String timezone2011 = getTimezone(2011, 7, 4);
    this.mockMvc.perform(get("/convert/bean?formattedList[0]=
2010-07-04&formattedList[1]=2011-07-04"))
        .andExpect(content().string(
```

```

        "Converti JavaBean formattedList=[Sun Jul 04
00:00:00 " + timezone2010
        + " 2010, Mon Jul 04 00:00:00 " + timezone2011
+ " 2011]"));

<a id="mapProp" class="textLink" href="<c:url value="/convert/bean?
map[0]=apple&map[1]=pear" />">Map Elements</a>

```

```

@Test
public void beanMap() throws Exception {
    this.mockMvc.perform(get("/convert/bean?map[0]=apple&map[1]=pear"
))
        .andExpect(content().string("Converti JavaBean
map={0=apple, 1=pear}"));

```

```

<a id="nestedProp" class="textLink" href="<c:url
value="/convert/bean?nested.foo=bar&nested.list[0].foo=baz&nested
.map[key].list[0].foo=bip" />">Nested</a>

```

```

@Test
public void beanNested() throws Exception {
    this.mockMvc.perform(get("/convert/bean?
nested.foo=bar&nested.list[0].foo=baz&nested.map[key]
.list[0].foo=bip"))
        .andExpect(content().string(
            "Converti JavaBean nested=NestedBean
foo=bar list=[NestedBean foo=baz] map={key=NestedBean
list=[NestedBean foo=bip]}"));

```

```

<a id="validateNoErrors" class="textLink" href="<c:url
value="/validate?number=3&date=2029-07-04" />">Validate, no
errors</a>

```

```

public class JavaBean {

    @NotNull
    @Max(5)
    private Integer number;

    @NotNull
    @Future
    @DateTimeFormat(iso=ISO.DATE)
    private Date date;

    public Integer getNumber() {
        return number;
    }

    public void setNumber(Integer number) {
        this.number = number;
    }

    public Date getDate() {
        return date;
    }

    public void setDate(Date date) {
        this.date = date;
    }
}

```

```

@Before
public void setup() throws Exception {
    this.mockMvc = standaloneSetup(new
ValidationController()).alwaysExpect(status().isOk()).build();

@Test
public void validateSuccess() throws Exception {
    this.mockMvc.perform(get("/validate?number=3&date=2029-07-
04"))
        .andExpect(content().string("Pas d'erreur"));

```

be. Validation

Nous pouvons utiliser des validateurs. L'idéal est de valider à tous les niveaux afin d'éviter les problèmes, c'est-à-dire dans la page, dans le contrôleur, dans les services et dans les DAO.

Code dans la page :

```

<a id="validateErrors" class="textLink" href="<c:url
value="/validate?number=3&date=2010-07-01" />">Validate,

```

Il faut ajouter une gestion d'exception de type Business.

```
errors</a>
```

Dans le contrôleur
:

```
@SuppressWarnings("serial")
public class BusinessException extends Exception {
```

Tests unitaires :

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler
    public @ResponseBody String
    handleBusinessException(BusinessException ex) {
        return "Handled BusinessException";
    }

    // enforcement of constraints on the JavaBean arg require a JSR-
    303 provider on the classpath
    @RequestMapping("/validate")
    public @ResponseBody String validate(@Valid JavaBean bean,
    BindingResult result) {
        if (result.hasErrors()) {
            return "L'Object a des erreurs de validation";
        } else {
            return "Pas d'erreur";
        }
    }
}
```

```
@Test
public void validateErrors() throws Exception {
    this.mockMvc.perform(get("/validate?number=3&date=2010-07-
    01"))
    .andExpect(content().string("L'Object a des erreurs de
    validation"));
}
```

bf. @ExceptionHandler dans un contrôleur

Nous pouvons avoir un gestionnaire d'exception intégré au contrôleur grâce à l'utilisation de l'annotation @ExceptionHandler.

Code dans la page :

```
<a id="exception" class="textLink" href="<c:URL
value="/exception" />">@ExceptionHandler dans un contrôleur</a>
```

Dans le contrôleur
:

```
@RequestMapping("/exception")
public @ResponseBody String exception() {
    throw new IllegalStateException("Sorry!");
}
```

bg. @ExceptionHandler global

Nous pouvons avoir un gestionnaire d'exception intégré au contrôleur.

Nous aurons recours à l'annotation @ExceptionHandler.

Code dans la page :

```
<a id="globalException" class="textLink" href="<c:URL
value="/global-exception" />">@ExceptionHandler global</a>
```

Dans le contrôleur
:

```
@RequestMapping("/global-exception")
public @ResponseBody String businessException() throws
BusinessException {
    throw new BusinessException();
}
```

bh. Templates de String pour les URI

Nous pouvons utiliser des templates d'URI.

Code dans la page :

```
<a href="<c:url value="/redirect/uriTemplate" />">String de
Template d'URI</a>
```

Tests unitaires :

```
private final ConversionService conversionService;
@Inject
```

```
public RedirectController(ConversionService conversionService) {
    this.conversionService = conversionService;

    @RequestMapping(value="/uriTemplate", method=RequestMethod.GET)
    public String uriTemplate(RedirectAttributes redirectAttrs) {
        redirectAttrs.addAttribute("account", "a123"); // Used as URI
        // template variable
        redirectAttrs.addAttribute("date", new LocalDate(2011, 12,
        31)); // Appended as a query parameter
        return "redirect:/redirect/{account}";
    }
}
```

```
@Before
public void setup() throws Exception {
    this.mockMvc = standaloneSetup(new RedirectController(new
    DefaultFormattingConversionService()))
        .alwaysExpect(status().isMovedTemporarily()).build();

    @Test
    public void uriTemplate() throws Exception {
        this.mockMvc.perform(get("/redirect/uriTemplate"))
            .andExpect(redirectedUrl("/redirect/a123?date=12%2F31%2F11"));
    }
}
```

bi. UriComponentsBuilder

Nous avons à notre disposition le `UriComponentsBuilder` qui génère pour nous l'URI.

Code dans la page :

```
<a href="<c:url value="/redirect/uriComponentsBuilder"
/>">UriComponentsBuilder</a>
```

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping(value="/uriComponentsBuilder",
method=RequestMethod.GET)
public String uriComponentsBuilder() {
    String date = this.conversionService.convert(new
    LocalDate(2011, 12, 31), String.class);
    UriComponents redirectUri =
    UriComponentsBuilder.fromPath("/redirect/{account}")
        .queryParams("date", date)
        .build().expand("a123").encode();
    return "redirect:" + redirectUri.toUriString();
}
```

```
@Test
public void uriComponentsBuilder() throws Exception {
    this.mockMvc.perform(get("/redirect/uriComponentsBuilder"))
        .andExpect(redirectedUrl("/redirect/a123?date=12/31/11"));

    @RequestMapping(value="/{account}", method=RequestMethod.GET)
    public String show(@PathVariable String account,
    @RequestParam(required=false) LocalDate date) {
        return "redirect/redirectResults";
    }
}
```

bj. GET /async/callable/response-body

Nous pouvons fonctionner en mode asynchrone pour la partie qui correspond au corps de la requête.

Code dans la page :

```
<a id="callableResponseBodyLink" class="textLink"
href="<c:url value="/async/callable/response-body" />">GET
/async/callable/response-body</a>
```

Dans le contrôleur :

Tests unitaires :

```
public class JavaBean {
    private String foo;
    private String fruit;
    public JavaBean(String foo, String fruit) {
        this.foo = foo;
        this.fruit = fruit;
    }
    public String getFoo() {
        return foo;
    }
    public void setFoo(String foo) {
        this.foo = foo;
    }
    public String getFruit() {
        return fruit;
    }
}
```

```

        return fruit;
    }
    public void setFruit(String fruit) {
        this.fruit = fruit;
    }

    public class TimeoutCallableProcessingInterceptor extends
    CallableProcessingInterceptorAdapter {
        @Override
        public <T> Object handleTimeout(NativeWebRequest request,
        Callable<T> task) throws Exception {
            throw new IllegalStateException "[" +
            task.getClass().getName() + "] timed out");
        }
    }

```

```

@RequestMapping("/response-body")
public @ResponseBody Callable<String> callable() {
    return new Callable<String>() {
        @Override
        public String call() throws Exception {
            Thread.sleep(2000);
            return "Résultat Callable";
        }
    };
}

```

bk. GET /async/c

Pour un rendu asynchrone, nous construirons le lien href de la façon suivante :

Code dans la page :

```

@Test
public void responseBody() throws Exception {
    MvcResult mvcResult =
    this.mockMvc.perform(get("/async/callable/response-body"))
        .andExpect(request().asyncStarted())
        .andExpect(request().asyncResult("Résultat Callable"))
        .andReturn();
    this.mockMvc.perform(asyncDispatch(mvcResult))
        .andExpect(status().isOk())
        .andExpect(content().contentType("text/plain;charset=ISO-
6859-1"))
        .andExpect(content().string("Résultat Callable"));
}

```

Dans le contrôleur :

Tests unitaires :

```

<a id="callableViewLink" class="textLink"
    href="<c:url value="/async/callable/view" />">GET
/async/callable/view</a>

```

```

@RequestMapping("/view")
public Callable<String> callableWithView(final Model model) {
    return new Callable<String>() {
        @Override
        public String call() throws Exception {
            Thread.sleep(2000);
            model.addAttribute("foo", "bar");
            model.addAttribute("fruit", "apple");
            return "views/html";
        }
    };
}

```

```

@Test
public void view() throws Exception {
    MvcResult mvcResult =
    this.mockMvc.perform(get("/async/callable/view"))
        .andExpect(request().asyncStarted())
        .andExpect(request().asyncResult("views/html"))
        .andReturn();
    this.mockMvc.perform(asyncDispatch(mvcResult))
        .andExpect(status().isOk())
        .andExpect(forwardedUrl("/WEB-INF/views/views/html.jsp"))
        .andExpect(model().attribute("foo", "bar"))
        .andExpect(model().attribute("fruit", "apple"));
}

```

bl. GET /async/callable/exception

La gestion des exceptions en mode asynchrone est particulière. Nous pourrions procéder de la façon suivante :

Code dans la page :

```

<a id="callableExceptionLink" class="textLink"
    href="<c:url value="/async/callable/exception" />">GET
/async/callable/exception</a>

```

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping("/exception")
public @ResponseBody Callable<String> callableWithException(
    final @RequestParam(required=false, defaultValue="true")
    boolean handled) {
    return new Callable<String>() {
        @Override
        public String call() throws Exception {
            Thread.sleep(2000);
            if (handled) {
                // see handleException method further below
                throw new IllegalStateException("Erreur sur
Callable");
            }
            else {
                throw new IllegalArgumentException("Erreur sur Callable");
            }
        }
    };
}
```

```
@Test
public void exception() throws Exception {
    MvcResult mvcResult =
this.mockMvc.perform(get("/async/callable/exception"))
    .andExpect(request().asyncStarted())
    .andExpect(request().asyncResult(instanceOf(IllegalStateException
.class)))
    .andReturn();
    this.mockMvc.perform(asyncDispatch(mvcResult))
    .andExpect(status().isOk())
    .andExpect(content().contentType("text/plain;charset=ISO-
8859-1"))
    .andExpect(content().string("Handled exception: Erreur sur
Callable"));
}
```

```
<a id="callableUnhandledExceptionLink" class="textLink"
    href="<c:url value="/async/callable/exception?handled=false"
/>">GET /async/callable/exception?handled=false</a>
(500 Error expected)
```

bm. GET /async/callable/custom-timeout-handling

En mode asynchrone, nous devons gérer le timeout pour gérer les cas de réponses trop tardives.

Code dans la page :

Dans le contrôleur :

```
<a id="callableCustomTimeoutLink" class="textLink"
    href="<c:url value="/async/callable/custom-timeout-handling"
/>">GET /async/callable/custom-timeout-handling</a>
```

```
@RequestMapping("/custom-timeout-handling")
public @ResponseBody WebAsyncTask<String>
callableWithCustomTimeoutHandling() {
    Callable<String> callable = new Callable<String>() {
        @Override
        public String call() throws Exception {
            Thread.sleep(2000);
            return "Callable result";
        }
    };
    return new WebAsyncTask<String>(1000, callable);
}
```

```
@ExceptionHandler
@ResponseBody
public String handleException(IllegalStateException ex) {
    return "Handled exception: " + ex.getMessage();
}
```

bn. GET /async/deferred-result/response-body

Afin d'obtenir le résultat de la requête en mode asynchrone, nous procéderons de cette façon :

Code dans la page :

Tests unitaires :

```
<a id="deferredResultSuccessLink" class="textLink"
    href="<c:url value="/async/deferred-result/response-body"
/>">GET /async/deferred-result/response-body</a>
```

```
private final Queue<DeferredResult<String>> responseBodyQueue =
new ConcurrentLinkedQueue<DeferredResult<String>>();
private final Queue<DeferredResult<ModelAndView>> mavQueue = new
ConcurrentLinkedQueue<DeferredResult<ModelAndView>>();
private final Queue<DeferredResult<String>> exceptionQueue = new
ConcurrentLinkedQueue<DeferredResult<String>>();

@RequestMapping("/deferred-result/response-body")
public @ResponseBody DeferredResult<String> deferredResult() {
    DeferredResult<String> result = new DeferredResult<String>();
    this.responseBodyQueue.add(result);
    return result;
}
```

```
@Test
public void responseBody() throws Exception {
    MvcResult mvcResult =
this.mockMvc.perform(get("/async/deferred-result/response-body"))
    .andExpect(status().isOk())
    .andExpect(request().asyncStarted())
    .andExpect(request().asyncResult("Résultat différé"))
    .andReturn();
    this.mockMvc.perform(asyncDispatch(mvcResult))
    .andExpect(status().isOk())
    .andExpect(content().contentType("text/plain;charset=ISO-
8859-1"))
    .andExpect(content().string("Résultat différé"));
}
```

bo. GET /async/deferred-result/model-and-view

Pour la gestion d'un Model and View en mode asynchrone, il est préférable d'avoir ce type d'implémentation :

Code dans la page :

```
<a id="deferredResultModelAndViewLink" class="textLink"
href="<c:url value="/async/deferred-result/model-and-view"
/>">GET /async/deferred-result/model-and-view</a>
```

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping("/deferred-result/model-and-view")
public DeferredResult<ModelAndView> deferredResultWithView() {
    DeferredResult<ModelAndView> result = new
DeferredResult<ModelAndView>();
    this.mavQueue.add(result);
    return result;
}
```

```
@Test
public void view() throws Exception {
    MvcResult mvcResult =
this.mockMvc.perform(get("/async/deferred-result/model-and-view"))
    .andExpect(status().isOk())
    .andExpect(request().asyncStarted())
    .andExpect(request().asyncResult(instanceOf(ModelAndView
.class)))
    .andReturn();
    this.mockMvc.perform(asyncDispatch(mvcResult))
    .andExpect(status().isOk())
    .andExpect(forwardedUrl("/WEB-INF/views/views/html.jsp"))
    .andExpect(model().attributeExists("javaBean"));
}
```

bp. GET /async/deferred-result/exception

Pour gérer les exceptions lors de la phase de réception du contenu asynchrone, nous aurons du code comme celui-ci :

Code dans la page :

```
<a id="deferredResultErrorLink" class="textLink"
href="<c:url value="/async/deferred-result/exception" />">GET
/async/deferred-result/exception</a>
```

Dans le contrôleur :

Tests unitaires :

```
@RequestMapping("/deferred-result/exception")
public @ResponseBody DeferredResult<String>
deferredResultWithException() {
    DeferredResult<String> result = new DeferredResult<String>();
    this.exceptionQueue.add(result);
    return result;
}
```

```

@Test
public void exception() throws Exception {
    MvcResult mvcResult =
this.mockMvc.perform(get("/async/deferred-result/exception"))
    .andExpect(status().isOk())
    .andExpect(request().asyncStarted())
    .andExpect(request().asyncResult(instanceOf(IllegalStateException
.class)))
    .andReturn();
    this.mockMvc.perform(asyncDispatch(mvcResult))
    .andExpect(status().isOk())
    .andExpect(content().contentType("text/plain;charset=ISO-
8859-1"))
    .andExpect(content().string("Handled exception: Erreur
DeferredResult"));
}

```

bq. GET /async/deferred-result/timeout-value

De la même façon, la gestion du timeout est importante :

Code dans la page :

```

<a id="deferredResultTimeoutValueLink" class="textLink"
href="<c:url value="/async/deferred-result/timeout-value" />">GET
/async/deferred-result/timeout-value</a>

```

Dans le contrôleur :

```

@RequestMapping("/deferred-result/timeout-value")
public @ResponseBody DeferredResult<String>
deferredResultWithTimeoutValue() {
    // Provide a default result in case of timeout and override
the timeout value
    // set in src/main/webapp/WEB-INF/spring/appServlet/servlet-
context.xml
    return new DeferredResult<String>(1000L, "Résultat différé
après timeout");
}

@Scheduled(fixedRate=2000)
public void processQueues() {
    for (DeferredResult<String> result : this.responseBodyQueue) {
        result.setResult("Résultat différé");
        this.responseBodyQueue.remove(result);
    }
    for (DeferredResult<String> result : this.exceptionQueue) {
        result.setErrorResult(new IllegalStateException("Erreur
DeferredResult"));
        this.exceptionQueue.remove(result);
    }
    for (DeferredResult<ModelAndView> result : this.mavQueue) {
        result.setResult(new ModelAndView("views/html", "javaBean",
new JavaBean("bar", "apple")));
        this.mavQueue.remove(result);
    }
}

```

```

@ExceptionHandler
@ResponseBody
public String handleException(IllegalStateException ex) {
    return "Handled exception: " + ex.getMessage();
}

```


Client REST

Le client REST bénéficie des facilités offertes par le template RestTemplate

1. Utilisation du RestTemplate

Un client REST est assez simple avec Spring.

```
@Log
public class ClientRest {
    public static void main(String[] args) {
        ApplicationContext ctx = new ClassPathXmlApplicationContext(
            new String[] { "applicationContext.xml" });
        RestTemplate restTemplate = new RestTemplate();
        Utilisateur utilisateur= restTemplate.getForObject(
            "http://localhost:8080/chap09-sr2/utilisateurs/2",
            Utilisateur.class);

        log.info(utilisateur.toString());
    }
}
```

2. Le bean de domaine avec l'annotation REST pour les champs manquants

Il est possible d'ignorer les champs manquants.

```
@JsonIgnoreProperties(ignoreUnknown = true)
@Getter @Setter @ToString
@NoArgsConstructor
@AllArgsConstructor
public class Utilisateur {
    private long id;
    private String prenom;
    private String nom;
}
```

Spring Security

1. Introduction à Spring Security

Spring Security s'articule autour de quatre concepts :

Principal	L'utilisateur, le système ou l'appareil qui fait une action.
Authentification	Vérification de la validité des droits du Principal.
Autorisation	Vérification que l'action est autorisée pour le Principal.
Ressource sécurisée	La ressource pour laquelle il faut vérifier les droits d'accès.

Il existe plusieurs mécanismes d'authentification basé sur :

- un mécanisme ultra basique,
- un digest,
- un formulaire,
- un accès X-509 (LDAP par exemple),
- des cookies,
- SSO (*Single Sign-On*).

Quand cela sera possible, il est conseillé d'utiliser le SSO pour ne pas avoir à s'identifier sur l'application.

Pour les applications très sensibles, il est possible d'utiliser un lecteur de carte à puce ou d'utiliser un système à token (système basé sur un nombre affiché sur un boîtier ou une calculatrice qui change toutes les x secondes).

À cette authentification peuvent être associés des droits qui sont persistés :

- en dur dans un programme pendant le cycle de développement,
- dans un fichier .properties,
- dans une base de données (SQL, NoSQL via un DAO classique),
- dans une base LDAP,
- dans un système externe dédié.

Pour déterminer si une action est possible, il faut tout d'abord vérifier qui fait la demande en utilisant le mécanisme d'authentification. Nous pouvons définir qu'un utilisateur qui n'est pas encore identifié est considéré comme un invité (GUEST). Le système le plus simple peut alors considérer un utilisateur standard comme membre (MEMBER) s'il s'agit d'un utilisateur n'ayant pas des droits d'administration. Un utilisateur administrateur a, quant à lui, un rôle administrateur (ADMIN). Ces trois rôles : GUEST, MEMBER, ADMIN sont les rôles de base dans Spring Security.

Nous verrons qu'il est aussi prévu de pouvoir définir d'autres rôles qui viendront compléter ces rôles simples. Spring Security est indépendant des mécanismes standards utilisés dans les serveurs d'applications. De cette façon, il devient portable d'un serveur à un autre. Il reste toutefois possible de connecter Spring Security à un type de fournisseur de sécurité (*security provider*). Nous perdons la portabilité mais nous unifions alors le mécanisme par rapport au serveur.

Spring Security est très souple et permet de personnaliser tous les éléments liés à l'Authentification et les droits associés.

Il est possible d'utiliser Spring Security sur les différentes couches applicatives :

- Filtre de servlet,

- Spring AOP pour une approche par aspects,
- Sur les DAO et les services,
- ...

Les mécanismes d'authentification et d'autorisation sont découplés pour simplifier la maintenance. L'authentification fournit le contexte de sécurité. L'autorisation récupère les attributs de la ressource sécurisée, récupère les informations sur ce qui est connecté (personne, appareil, application) à partir du contexte de sécurité et détermine si l'accès est valide ou non.

2. Spring Security dans un environnement web

Nous déclarons un filtre de servlet et nous indiquons ce qui sera filtré :

Code dans la page :

```
<filter>
<filter-name>springSecurityFilterChain</filter-name>
<filter-class>org.springframework.web.filter.DelegatingFilterProxy
</filter-class>
</filter>
<filter-mapping>
<filter-name>springSecurityFilterChain</filter-name>
<URL-pattern>/*</URL-pattern>
</filter-mapping>
```

Il ne reste alors plus qu'à définir la configuration Spring avec le namespace Security.

```
<intercept-url pattern="/admin*" access="ROLE_USER" />
</http>
```

Les URL sont interceptées dans l'ordre de leur déclaration.

a. Authentification par Spring

Nous utilisons le `authentication-manager` avec le `authentication-provider` et le `user-service` :

```
<authentication-provider>
<user-service>
<user nom="admin" password="admin" authorities="ROLE_USER" />
</user-service>
</authentication-provider>
</authentication-manager>
```

b. Authentification par page de login personnalisée

Nous spécifions la page qui porte la mire de login avec le `form-login`.

```
<http auto-config="true">
<intercept-url pattern="/admin*" access="ROLE_USER" />
<form-login
  login-page="/login"
  default-target-url="/welcome"
  authentication-failure-url="/login?error"
  username-parameter="username"
  password-parameter="password" />
<logout logout-success-url="/login?logout" />
<!-- enable csrf protection -->
<csrf/>
```

```
</http>
```

c. Authentification par base de données

Nous ajoutons un dataSource classique :

```
<bean id="dataSource"
    class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="org.h2.Driver" />
    <property name="url" value="jdbc:h2:~/testsecu" />
    <property name="username" value="sa" />
    <property name="password" value="sa" />
</bean>
```

Nous utilisons un authentication-provider :

```
<authentication-manager>
<authentication-provider>
<jdbc-user-service data-source-ref="dataSource"
    users-by-username-query=
        "select username,password, enabled from users where
username=?"
    authorities-by-username-query=
        "select username, role from user_roles where
username =? " />
</authentication-provider>
</authentication-manager>
```

Nous créons une page 403 :

```
<html>
<body>
<h1>HTTP Status 403 - Access is denied</h1>
<c:choose>
<c:when test="${empty username}">
<h2>You do not have permission to access this page!</h2>
</c:when>
<c:otherwise>
<h2>Username : ${username} <br/>You do not have permission to
access this page!</h2>
</c:otherwise>
</c:choose>
</body>
</html>
```

Nous créons une page accessible aux utilisateurs identifiés :

```
<html>
<body>
<h1>Title : ${title}</h1>
<h1>Message : ${message}</h1>
<sec:authorize access="hasRole('ROLE_USER')">
<!-- For login user -->
<c:URL value="/j_spring_security_logout" var="logoutUrl" />
<form action="${logoutUrl}" method="post" id="logoutForm">
<input type="hidden" name="${_csrf.parameterName}"
    value="${_csrf.token}" />
</form>
<script>
    function formSubmit() {
        document.getElementById("logoutForm").submit();
    }
</script>
<c:if test="${pageContext.request.userPrincipal.name != null}">
<h2>
```

User : \${pageContext.request.userPrincipal.name} | Logout

</h2>

</c:if>

</sec:authorize>

</body>

</html>

Points clés

- Spring MVC est très puissant et permet de faire beaucoup de choses.
- Nous pouvons tester la majorité de notre code.
- Le mode MVC standardise la façon de faire l'application.

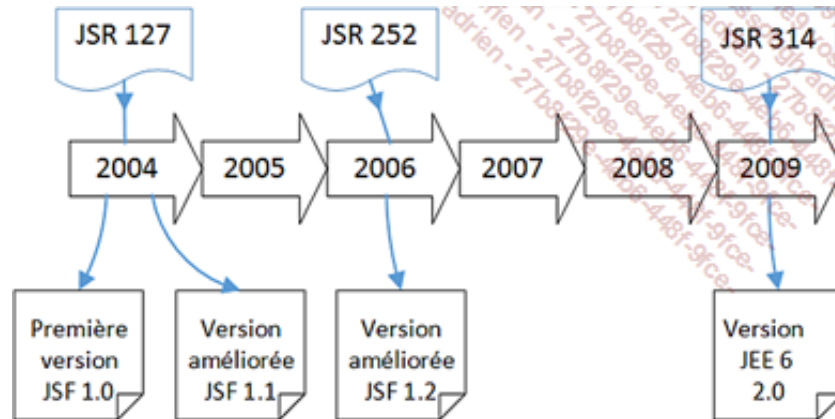
Introduction

Nous n'aborderons pas JSF2 en détail, nous ne rappellerons que les éléments principaux pour comprendre comment l'intégrer avec Spring. JSF2 est en effet un ensemble assez complexe qui demanderait un livre entier pour en décrire le fonctionnement dans les détails. Nous verrons d'abord l'implémentation standard Majorra. Les exemples téléchargeables du livre montreront deux implémentations plus sophistiquées RichFaces et PrimeFaces.

Majorra

Majorra est l'implémentation standard. JSF2 remplace les JSP pour avoir une approche composant. L'idée principale est d'exposer directement des beans "managés" dans les vues. De la même façon, la notion de contrôleur est simplifiée. L'objectif est de masquer la complexité mais cela reste cependant assez délicat à mettre en œuvre.

JSF a évolué de la version 1.0 à la version 2.0 :



Architecture

La configuration se base sur le fichier web.xml et le fichier faces-config.xml.

Les dernières versions des frameworks permettent de se passer de ces fichiers via les annotations comme nous le verrons dans un exemple.

Sous une vision simpliste :

- La requête arrive au contrôleur.
- Le contrôleur valide les données.
- S'il y a des erreurs, la page en cours est retournée avec les erreurs.
- Sinon il cherche la vue correspondant à la réponse.
- La vue interroge les beans managés présents dans la page ; ces beans sont soit des données métier, soit des services.
- La couche métier et/ou DAO renvoie les données à la page.
- La page est ensuite rendue.

Il est possible de court-circuiter des étapes lors des appels AJAX. Il faut considérer qu'il y a un contexte sur le serveur. Ce contexte est en relation avec la page. Il contient la structure de la page au niveau des composants avec les données métier (services et data). La page modifie le contexte puis le retourne. Le contexte retourné peut être valide ou invalide du point de vue de la validité des données modifiées. Les données ne peuvent être persistées que si le modèle est valide.



Il faudra éviter d'avoir des rollbacks venant d'une interruption lors de la mise à jour en base car cela corrompt le contexte.

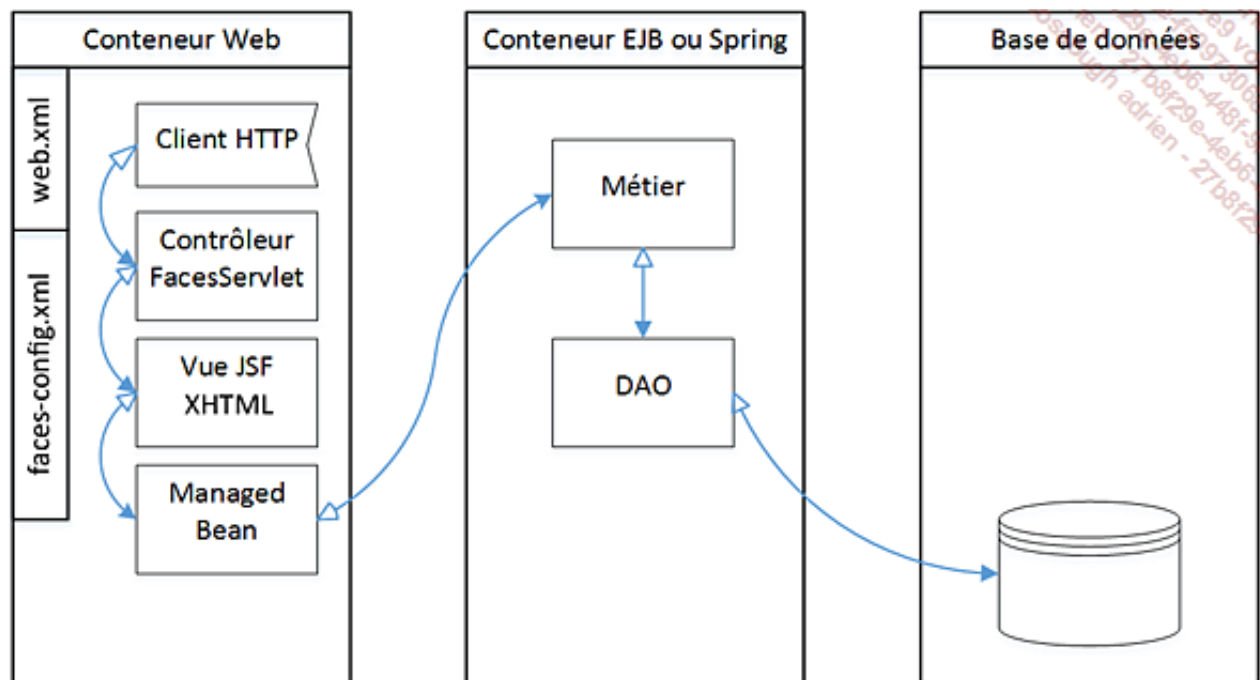
Le contexte est de type FacesContext et contient toutes les informations nécessaires au cycle de vie de la requête.

On y trouve :

- les composants,
- les données de la vue,
- les erreurs.

Les pages sont en XHTML et peuvent être issues d'une composition de pages. Elles sont, en général, composées à partir d'un modèle de page dans lequel on insère des composants sous la forme de morceau de pages. Au niveau de la transaction pour les données, il est possible d'utiliser une conversation courte (ModelInView) ou longue en utilisant un contexte étendu.

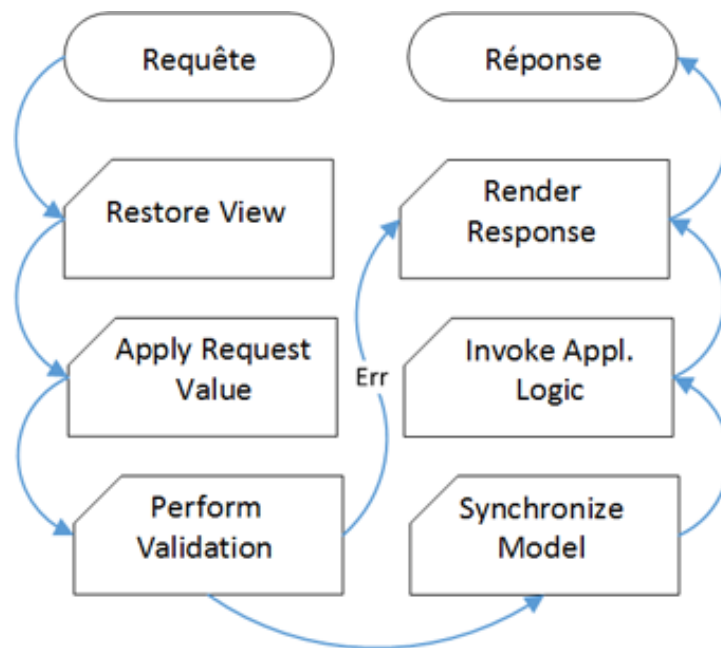
L'architecture :



Cycle de vie

Le cycle de vie décrit les étapes faites par le contrôleur. Il est possible de tracer dans quelle étape on se trouve pour un traitement donné en activant la log.

Cycle de vie :



Les termes sont en anglais car ils apparaissent tels quels dans la configuration et les logs.

1. Requête

Il s'agit d'une requête HTTP. La requête contient les données nécessaires à JSF pour gérer son cycle de vie. On y retrouve des éléments sur le contexte associé à la requête.

2. Restore View ou Reconstruct Component Tree

À partir de la session, le serveur retrouve le contexte associé à la session et recompose l'arborescence des composants de la page.

3. Apply Request Value

On extrait de la requête HTTP les valeurs des données qui correspondent aux composants. Nous utilisons des convertisseurs pour adapter le format des données qui sont passées sous forme de chaînes de caractères dans la requête. Nous aurons donc des convertisseurs à créer.

4. Perform Validation

Il s'agit d'une phase cruciale : nous utilisons les validateurs enregistrés dans les composants pour valider les données de la page en cours. Il est possible avec JSF2 de valider les données dans les pages avant de soumettre le formulaire. Il faut tout de même valider les données côté serveur pour éviter les problèmes.

La validation des données peut souvent être déléguée d'une part au service pour les questions métiers et d'autre part à l'ORM de la couche domaine pour une validation directe sur les beans du modèle. La validation peut induire un décalage entre le contexte correspondant à la page et le contexte JSF serveur qui lui est en

phase avec le contexte de l'ORM. Il faut alors veiller que toutes les données soient validées avant de demander une mise à jour.

Il est possible de valider partiellement les données pour les données présentes sur plusieurs vues. On ne valide alors que les données de la vue que l'on affiche.

En cas d'erreur, on rend tout de suite la page courante via l'étape "Render Response" avec les erreurs dans le contexte.

S'il n'y a pas d'erreur, on continue le cycle.

5. Synchronize Model ou Update Model Values

Comme il n'y a pas d'erreur, le contexte JSF est mis à jour avec les données "extraites" de la vue. C'est un progrès considérable, le binding est automatique et utilise les convertisseurs de types au besoin pour passer du format affiché dans la page au format de la variable qui correspond à la donnée en base.

6. Invoke Application Logic

Durant cette étape, on dispose de données valides pour faire des traitements. Nous traitons les données sous forme d'événements. C'est durant cette étape que l'on détermine quelle sera la page qui sera affichée pour la réponse.

7. Render Response

À partir du nom de la page, nous chargeons la page et on positionne les champs. Nous provoquons ensuite l'affichage de la page correspondante.

8. Réponse

La page est retournée sous la forme d'un flux HTML. Certaines parties de la page peuvent aussi être rendues via un appel AJAX.

9. Fichier web.xml

Le fichier web.xml est configuré pour que la servlet FacesServlet soit utilisée pour traiter les requêtes.

```
<servlet>
  <servlet-name>Faces Servlet</servlet-name>
  <servlet-class>javax.faces.webapp.FacesServlet</servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>Faces Servlet</servlet-name>
  <url-pattern>*.jsf</url-pattern>
</servlet-mapping>
```

10. Dépendances

Afin de passer d'un projet utilisant les JSP à un projet utilisant JSF, nous ajoutons une dépendance vers jsf-api et jsf-impl dans le fichier pom.xml :

```
<dependency>
  <groupId>com.sun.faces</groupId>
  <artifactId>jsf-api</artifactId>
```

```

    <version>${jsf.version}</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>com.sun.faces</groupId>
    <artifactId>jsf-impl</artifactId>
    <version>${jsf.version}</version>
    <scope>runtime</scope>
</dependency>

```

 Nous spécifions le scope runtime. Cela indique que la dépendance est requise pour la compilation mais qu'elle sera fournie par l'environnement. Nous aurions utilisé provided si la librairie n'avait pas été requise pour la compilation.

11. Fichier faces-config.xml

Ce fichier décrit les informations relatives à JSF comme par exemple :

- la liste des beans managés,
- des énumérations,
- des paramètres d'initialisation,
- des maps des lists et des propriétés,
- des messages d'erreur personnalisés,
- des validateurs et convertisseurs personnalisés,
- des règles de navigation,
- un rendu personnalisé (thème),
- des composants personnalisés,
- etc.

Exemple de fichier initial faces-config.xml :

```

<?xml version="1.0" encoding="UTF-8"?>
<faces-config
  xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-facesconfig_2_2.xsd"
  version="2.2">
</faces-config>

```

Pour le moment, ce fichier est vide mais nous pourrions spécifier des éléments complémentaires pour JSF dans ce fichier.

Exemple de fichier faces-config.xml avec la navigation :

```

<?xml version="1.0"? encoding="UTF-8"?>
<faces-config ...>
  <navigation-rule>
    <from-view-id>/start-page-1.xhtml</from-view-id>
    <navigation-case>
      <from-outcome>too-short</from-outcome>
      <to-view-id>/error-message.xhtml</to-view-id>
    </navigation-case>
    <navigation-case>
      <from-outcome>page1</from-outcome>
      <to-view-id>/result-page-1.xhtml</to-view-id>
    </navigation-case>
  </navigation-rule>
</faces-config>

```

```
</navigation-case>
[...]
```

```
<navigation-case>
</navigation-rule>
</faces-config>
```

Un exemple plus complet est disponible dans les exemples téléchargeables.

12. Bean managé sans Spring

JSF permet de manager des beans. Ces beans servent comme interface entre les données dans les pages et les données côté back.

```
import java.util.Date;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.RequestScoped;
@ManagedBean(name="utilisateur")
@RequestScoped
public class Utilisateur {
    private long id;
    private String nom;
    private String prenomnom;
    private Date dateNaissance;
    public String save(){
        // Traitement
        System.out.println(nom);
        System.out.println(dateNaissance);
        return "success";
    }
}
[Getters et Setters]
```

13. Exemple de vue JSF

La vue JSF est, en général, un fichier XHTML et elle est souvent composée de fragments.

Nous utiliserons un ensemble de templates qui fixeront la disposition des fragments de la vue. Ce template sera accompagné d'un ensemble de fragments réutilisables qui permettront de fabriquer une vue sur mesure.

- webapp/WEB-INF/templates/template.xhtml
- webapp/mavue.xhtml

La vue déclare un composant visuel sous la forme d'un fragment :

```
<ui:composition>
    <ui:define name="content">
    </ui:define>
</ui:composition>
```

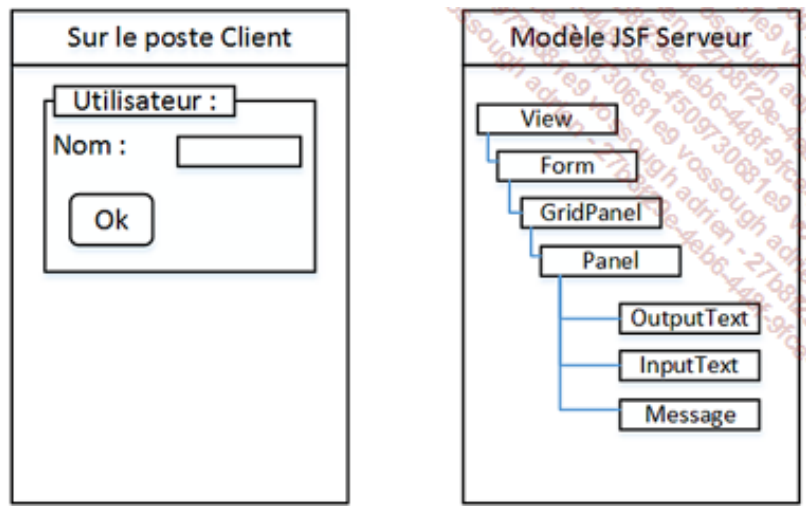
Le template inclut alors le composant ou le fragment :

```
<h:panelGroup id="content" layout="block">
    <ui:insert name="content">Main Content</ui:insert>
</h:panelGroup>
```

Il faut garder à l'esprit que les fragments seront injectés dans la page. Il ne s'agit pas d'une inclusion classique. Le fragment repéré sera mis à l'emplacement défini par une balise de placement `<ui:insert />`.

14. Aperçu d'un composant JSF

Le composant ressemble à un DOM HTML mais avec une partie dynamique.



Les éléments de la partie front (dans le navigateur web) sont liés aux données des beans managés.

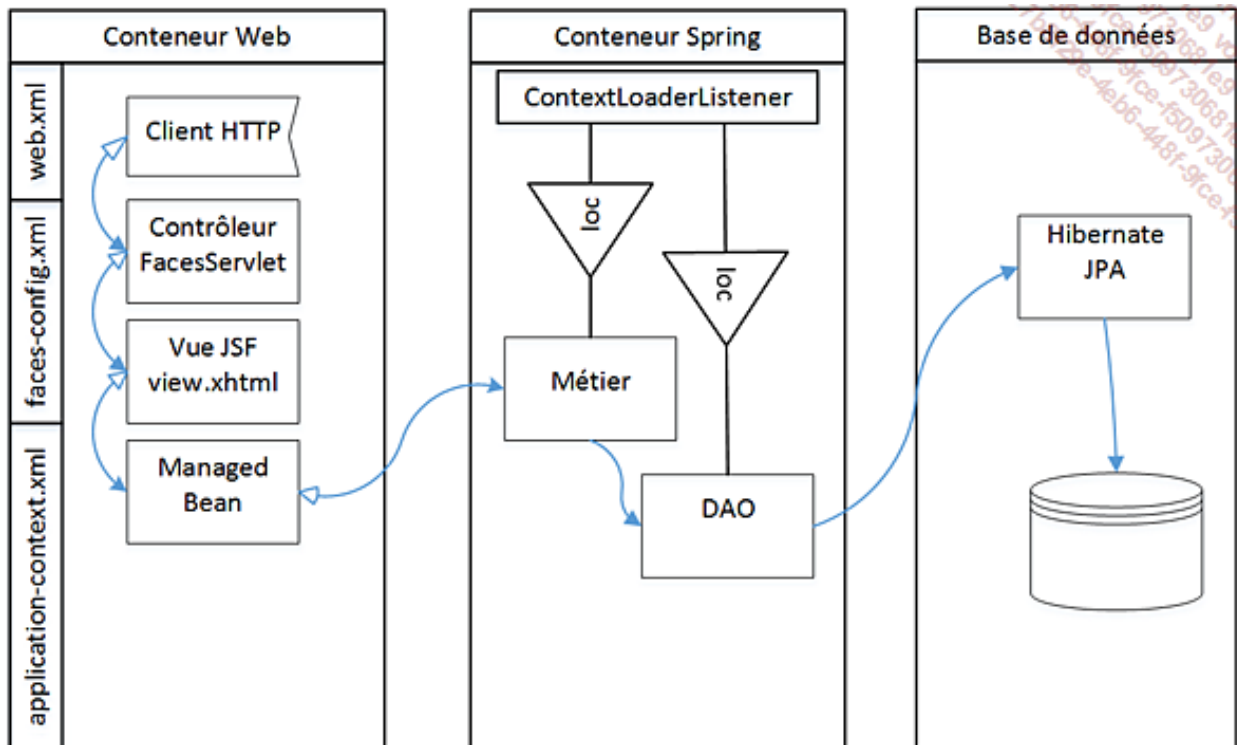
Intégration Spring

Le bean managé par JSF devient un bean Spring.

```
@Component("messageBackingBean")
@Scope("request")
public class MessageBackingBean {
    private Message message = new Message();
    private List<Message> messages;
    @Autowired
    private MessageDao messageDao;

    public String getMessage() {
        return "Bonjour !";
    }
    public Message getMessage() {
        return message;
    }
    public void saveMessage() {
        messageDao.save(message);
        message = new Message();
        invalidateMessages();
    }
    private void invalidateMessages() {
        messages = null;
    }
    public List<Message> getMessages() {
        if (messages == null) {
            messages = messageDao.list();
        }
        return messages;
    }
}
```

1. Architecture



2. Dépendances

Il n'y a pas de dépendances supplémentaires pour utiliser JSF2.

3. Fichier web.xml

On indique dans ce fichier les éléments à détourner pour Spring.

On charge le contexte via le `ContextLoaderListener`.

```
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/spring/application-context.xml</param-value>
</context-param>
<context-param>
  <param-name>javax.faces.FACELETS_SKIP_COMMENTS</param-name>
  <param-value>true</param-value>
</context-param>
<context-param>
  <param-name>
    javax.faces.INTERPRET_EMPTY_STRING_SUBMITTED_VALUES_AS_NULL
  </param-name>
  <param-value>true</param-value>
</context-param>
<context-param>
  <param-name>javax.faces.PROJECT_STAGE</param-name>
  <param-value>Development</param-value>
</context-param>
<context-param>
  <param-name>javax.faces.PARTIAL_STATE_SAVING</param-name>
  <param-value>false</param-value>
</context-param>
<listener>
  <listener-class>
    org.springframework.web.context.ContextLoaderListener
  </listener-class>
</listener>
```

4. Fichier faces-config.xml

Nous spécifions que nous utilisons le langage de Spring pour les expressions dans la page en indiquant `SpringBeanFacesELResolver` dans le paramètre `el-resolver` :

```
<faces-config
  xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee/web-
facesconfig_2_2.xsd"
  version="2.2">

  <application>
    <el-resolver>
      org.springframework.web.jsf.el.SpringBeanFacesELResolver
    </el-resolver>
    <locale-config>
    </locale-config>
    <resource-bundle>
      <base-name>messages</base-name>
      <var>msg</var>
    </resource-bundle>
  </application>
</faces-config>
```

5. Couches basses (back)

Nous gardons les données viables le temps d'un cycle de vie via

leOpenEntityManagerInViewFilter.

```
<filter>
  <filter-name>hibernateFilter</filter-name>
  <filter-class>
    org.springframework.orm.jpa.support.OpenEntityManagerInViewFilter
  </filter-class>
</filter>
<filter-mapping>
  <filter-name>hibernateFilter</filter-name>
  <servlet-name>Faces Servlet</servlet-name>
</filter-mapping>
```



Il serait possible d'avoir une conversation à la place d'une durée limitée à un échange.

Points clés

- JSF2 est très simple à utiliser et doit dans la mesure du possible remplacer les JSP pour les nouveaux projets.
- Spring s'interface très facilement avec JSF2.
- Nous pourrions utiliser JSF2 pour les applications Spring MVC.

Étude de cas : les applications générées avec JHipster

Pour illustrer une utilisation avancée de Spring, nous regarderons quelques éléments de configuration d'un projet JHipster. Le projet correspondra à l'exemple qui est proposé sur le site de présentation de cet outil open source.

JHipster est un générateur d'application interactif. Il est basé sur Spring Boot et AngularJS. Nous regarderons dans ce chapitre la partie back de l'application car le côté front AngularJS sort du cadre de cet ouvrage.

Il est basé sur Yeoman, Maven, Spring et AngularJS.

La particularité principale de ce générateur est d'être en open source et que l'on peut assez facilement modifier les templates de génération pour générer sa propre application. Il est très en vogue et les frameworks qu'il utilise en profitent pour s'améliorer en le prenant comme vecteur d'amélioration.

Nous ne verrons que quelques parties car cet outil orienté DEVOPS nécessiterait plusieurs ouvrages pour être expliqué dans sa globalité.

Afin que le cas soit reproductible, nous avons choisi de suivre le tutoriel en gardant les valeurs par défaut.

Structure du projet

1. La partie front

La partie front utilise AngularJS. Elle est « responsive », c'est-à-dire que l'affichage se réorganise si nous changeons la taille de la zone d'affichage du navigateur web. Il s'agit d'une « Single Page Application », c'est-à-dire qu'il n'y a qu'une page qui se met à jour par AJAX. Il utilise HTML5 Boilerplate (<https://html5boilerplate.com/>) et Twitter Bootstrap (<http://getbootstrap.com>). Des tests sont générés pour Karma et PhantomJS.

Pour le développement de la partie front, JHipster utilise Yeoman, Grunt, Bower et AngularJS.

Ce générateur utilise en fait tous les meilleurs frameworks clients utilisables actuellement.

2. Les grandes lignes de la partie back

Nous présentons dans ce paragraphe les grandes lignes de la partie back. La partie Front AngularJS n'est pas présentée car elle sort du cadre de cet ouvrage. Le lecteur pourra s'appuyer sur le livre "AngularJS - Développez aujourd'hui les applications web de demain" de Pierre-Alexandre Gury et Sébastien Ollivier, paru aux Éditions ENI.

La partie back utilise Maven, Spring, Spring MVC REST et Spring data JPA. Elle est très centrée sur l'utilisation de Spring.

a. Spring Boot

Comme nous l'avons déjà évoqué, Spring Boot (<http://projects.spring.io/spring-boot/>) n'est pas encore pleinement utilisé en production, mais permet de faire des applications de démonstration assez facilement.

Le générateur JHipster profite de cette possibilité et il est très facile de changer la configuration pour être dans une stack Spring classique.

Spring Boot permet donc de créer rapidement une application fonctionnelle. La configuration est simplifiée.

b. Système de gestion des dépendances

Il est possible de générer du Maven ou du Gradle. Nous ne verrons que les aspects de la génération de projets basés sur Maven. JHipster utilise les profils pour personnaliser les éléments externes comme la base de données pour avoir des valeurs spécifiques en développement et en production.

c. Utilisation de Spring Security

L'intégration de Spring Security est très bien faite, ce qui nous permettra d'étudier des exemples concrets.

d. Spring MVC REST + Jackson

Nous profiterons de cette utilisation pour introduire ce sujet que nous n'avions pas traité auparavant. Nous utiliserons swagger pour autodocumenter les services REST directement depuis le browser.

e. Support optionnel de WebSocket avec Spring WebSocket

Nous n'étudierons pas cette possibilité car elle sort du cadre de ce livre.

Se référer au livre "Les API JavaScript du HTML5 - Intégrez la puissance du HTML5 dans vos applications Web" de Luc Van Lancker, paru aux Éditions ENI pour ce sujet.

f. Spring Data JPA + Bean Validation

Les exemples illustreront l'utilisation de cette partie de Spring. La validation nous montrera une intégration dans Spring MVC.

g. Évolution des bases de données avec Liquibase

Nous avons étudié l'utilisation du SQL intégré pour les tests unitaires. Liquibase est une alternative qui est très intéressante du point de vue des cycles d'évolutions du schéma de la base de données.

h. Support Elasticsearch

Nous n'étudierons pas cette possibilité car elle sort du cadre de ce livre.

i. Bases SQL et NoSQL

JHipster supporte les bases SQL et NoSQL. Il est alors possible d'utiliser Cassandra et mongoDB pour les bases noSQL.

Nous nous limiterons aux bases SQL.

j. Packaging multiplateforme

Avec le même packaging, il est possible de faire fonctionner l'application en développement ou en production en ciblant différents types de bases, typiquement H2 en développement, et MySQL ou Oracle en production.

k. Monitoring et métriques

JHipster a intégré des outils de monitoring dans ses applications générées.

JHipster utilise le système Logback que nous avons abordé dans le chapitre sur les éléments externes à Spring.

l. Autres avantages non abordés

Nous pouvons utiliser un cache local ehcache ou distribué avec hazelcast ainsi que des clusters de serveur d'application légers mais nous ne détaillerons pas ce point qui sort du cadre de ce livre.

JHipster permet aussi d'optimiser les ressources statiques (gzip filter, HTTP cache headers). Il utilise HikariCP pour le pooling des connexions afin d'optimiser les performances.

Spring Boot

Les principaux avantages de Spring Boot sont :

- Création d'applications Spring autonomes.
- Serveur HTTP/servlet embarqué qui permet de se passer de Jetty et de Tomcat.
- Fichier pom.xml simplifié.
- Configuration automatique de Spring, dans la mesure du possible.
- Applications pouvant être directement déployées en production avec des métriques et une configuration externalisée.
- Pas de code généré ni de nécessité d'utiliser une configuration basée sur le XML.

1. Dépendances Maven

L'application utilise les dépendances Maven suivantes :

Bibliothèque	Version
HikariCP	2.3.7
assertj-core	3.1.0
awaitility	1.4.0
commons-io	2.4
commons-lang	2.6
gatling	2.1.6
gatling-maven-plugin	2.1.6
Hibernate	4.3.6.Final
Java	1.8
javax.inject	1
joda-time-hibernate	1.3
json-path	0.9.1
Liquibase	3.3.2
liquibase-slf4j	1.2.1
liquibase-hibernate4	3.5
mapstruct	1.0.0.CR1
metrics-spark-reporter	1.2
metrics-spring	3.0.4
sonar-maven-plugin	2.3
spring-security	4.0.1.RELEASE
springfox	2.0.3
usertype-core	4.0.0.GA

2. Configuration de l'application

L'application est basée sur une configuration entièrement en Java.

Toute la configuration se trouve dans le package `com.mycompany.myapp.config` et ses sous-packages.

a. La classe Application qui contient le main

On indique avec les annotations que nous souhaitons que Spring trouve de lui-même les beans via l'annotation `@ComponentScan`.

Étant donné que la classe se trouve dans le package racine de l'application, Spring scannerait tous les packages sauf pour les classes `MetricFilterAutoConfiguration.class` et `MetricRepositoryAutoConfiguration.class` grâce

au paramètre `exclude` qui indique les classes à exclure.

```
@ComponentScan
@EnableAutoConfiguration(exclude =
{MetricFilterAutoConfiguration.class, MetricRepositoryAutoConfiguration.class})
public class Application {
[...]
}
```

Le lancement se passe en deux temps. Le `main` invoque l'application, et

le `@PostConstruct` avec la méthode `{public void initApplication()}` initialise l'application en fonction des profils actifs :

- Default
- Prod
- Fast
- Cloud

Le système basé sur Liquidbase se lance et met à disposition la base de données.

b. La classe serveur http/servlet

La classe `WebConfigurer` contient le lanceur Spring Boot de l'application.

```
/**
 * Configuration of web application with Servlet 3.0 APIs.
 */
@Configuration
@AutoConfigureAfter(CacheConfiguration.class)
public class WebConfigurer implements ServletContextInitializer,
EmbeddedServletContainerCustomizer {

    private final Logger log =
LoggerFactory.getLogger(WebConfigurer.class);

    @Inject
    private Environment env;

    @Autowired(required = false)
    private MetricRegistry metricRegistry;

    @Override
    public void onStartup(ServletContext servletContext) throws
ServletException {
        log.info("Web application configuration, using profiles:
{}\"", Arrays.toString(env.getActiveProfiles()));
        EnumSet<DispatcherType> disps =
EnumSet.of(DispatcherType.REQUEST, DispatcherType.FORWARD,
DispatcherType.ASYNC);
        if (!env.acceptsProfiles(Constants.SPRING_PROFILE_FAST)) {
            initMetrics(servletContext, disps);
        }
        if (env.acceptsProfiles(Constants.SPRING_PROFILE_PRODUCTION)) {
            initCachingHttpHeadersFilter(servletContext, disps);
            initStaticResourcesProductionFilter(servletContext, disps);
            initGzipFilter(servletContext, disps);
        }
        if (env.acceptsProfiles(Constants.SPRING_PROFILE_DEVELOPMENT)) {
            initH2Console(servletContext);
        }
        log.info("Web application fully configured");
    }

    /**
     * Set up Mime types.
     */
    @Override
    public void customize(ConfigurableEmbeddedServletContainer
container) {
        MimeMappings mappings = new
MimeMappings(MimeMappings.DEFAULT);
        // IE issue, see https://github.com/jhipster/generator-
jhipster/pull/711
        mappings.add("html", "text/html;charset=utf-8");
        // CloudFoundry issue, see
https://github.com/cloudfoundry/gorouter/issues/64
        mappings.add("json", "text/html;charset=utf-8");
    }
}
```

La


```

        container.setMimeMappings(mappings);
    }

    /**
     * Initializes the GZip filter.
     */
    private void initGzipFilter(ServletContext servletContext,
EnumSet<DispatcherType> disps) {
        log.debug("Registering GZip Filter");
        FilterRegistration.Dynamic compressingFilter =
servletContext.addFilter("gzipFilter", new GZipServletFilter());
        Map<String, String> parameters = new HashMap<>();
        compressingFilter.setInitParameters(parameters);
        compressingFilter.addMappingForUrlPatterns(disps, true,
"*.css");
        compressingFilter.addMappingForUrlPatterns(disps, true,
"*.json");
        compressingFilter.addMappingForUrlPatterns(disps, true,
"*.html");
        compressingFilter.addMappingForUrlPatterns(disps, true,
"*.js");
        compressingFilter.addMappingForUrlPatterns(disps, true,
"*.svg");
        compressingFilter.addMappingForUrlPatterns(disps, true,
"*.ttf");
        compressingFilter.addMappingForUrlPatterns(disps, true,
"/api/*");
        compressingFilter.addMappingForUrlPatterns(disps, true,
"/metrics/*");
        compressingFilter.setAsyncSupported(true);
    }

    /**
     * Initializes the static resources production Filter.
     */
    private void
initStaticResourcesProductionFilter(ServletContext servletContext,
EnumSet<DispatcherType> disps) {

        log.debug("Registering static resources production Filter");
        FilterRegistration.Dynamic
staticResourcesProductionFilter =
servletContext.addFilter("staticResourcesProductionFilter",
new StaticResourcesProductionFilter());

        staticResourcesProductionFilter.addMappingForUrlPatterns(disps,
true, "/");
        staticResourcesProductionFilter.addMappingForUrlPatterns(disps,
true, "/index.html");
        staticResourcesProductionFilter.addMappingForUrlPatterns(disps,
true, "/assets/*");
        staticResourcesProductionFilter.addMappingForUrlPatterns(disps,
true, "/scripts/*");
        staticResourcesProductionFilter.setAsyncSupported(true);
    }

    /**
     * Initializes the caching HTTP Headers Filter.
     */
    private void initCachingHttpHeadersFilter(ServletContext
servletContext,
EnumSet<DispatcherType> disps) {
        log.debug("Registering Caching HTTP Headers Filter");
        FilterRegistration.Dynamic cachingHttpHeadersFilter =
servletContext.addFilter("cachingHttpHeadersFilter",
new CachingHttpHeadersFilter(env));

        cachingHttpHeadersFilter.addMappingForUrlPatterns(disps,
true, "/assets/*");
        cachingHttpHeadersFilter.addMappingForUrlPatterns(disps,
true, "/scripts/*");
        cachingHttpHeadersFilter.setAsyncSupported(true);
    }

    /**
     * Initializes Metrics.
     */
    private void initMetrics(ServletContext servletContext,
EnumSet<DispatcherType> disps) {
        log.debug("Initializing Metrics registries");
        servletContext.setAttribute(InstrumentedFilter.REGISTRY_ATTRIBUTE,
metricRegistry);
        servletContext.setAttribute(MetricsServlet.METRICS_REGISTRY,

```

```

        metricRegistry);

        log.debug("Registering Metrics Filter");
        FilterRegistration.Dynamic metricsFilter =
servletContext.addFilter("webappMetricsFilter",
        new InstrumentedFilter());

        metricsFilter.addMappingForUrlPatterns(disps, true, "/*");
        metricsFilter.setAsyncSupported(true);

        log.debug("Registering Metrics Servlet");
        ServletRegistration.Dynamic metricsAdminServlet =
        servletContext.addServlet("metricsServlet", new
MetricsServlet());

        metricsAdminServlet.addMapping("/metrics/metrics/*");
        metricsAdminServlet.setAsyncSupported(true);
        metricsAdminServlet.setLoadOnStartup(2);
    }

    /**
     * Initializes H2 console
     */
    private void initH2Console(ServletContext servletContext) {
        log.debug("Initialize H2 console");
        ServletRegistration.Dynamic h2ConsoleServlet =
servletContext.addServlet("H2Console", new
org.h2.server.web.WebServlet());
        h2ConsoleServlet.addMapping("/console/*");
        h2ConsoleServlet.setInitParameter("-properties",
"src/main/resources");
        h2ConsoleServlet.setLoadOnStartup(1);
    }
}

```

classe `WebConfigurer` implémente l'interface `Spring Boot ServletContextInitializer` et l'interface `EmbeddedServletContainerCustomizer`.

La méthode `onStartup` tient compte de l'environnement pour initialiser à la demande des parties spécifiques de l'application en fonction des environnements :

SPRING_PROFILE_FAST

- `initMetrics`

SPRING_PROFILE_PRODUCTION

- `initCachingHttpHeadersFilter`
- `initStaticResourcesProductionFilter`
- `initGzipFilter(servletContext)`



On voit que les flux ne sont compressés que dans l'environnement de production via l'utilisation de la méthode `initGzipFilter(servletContext)` ; dans les autres cas, on a alors le comportement par défaut.

SPRING_PROFILE_DEVELOPMENT

- `initH2Console`

La méthode `customize` contourne des bugs en patchant le serveur web via le `MimeMappings`.

Nous voyons ensuite le filtre qui permet de zipper les flux : `initGzipFilter`.

Nous mappons ensuite les parties statiques et dynamiques.

Pour les profils pour lesquels les métriques sont disponibles, les métriques sont activées pour l'ensemble des URL « / ».

L'application initialise alors la console h2 pour les profils qui l'utilisent.

Cette classe d'initialisation est un bon modèle de référence. Elle permet en effet, en quelques lignes, de définir un serveur HTTP/servlet, de configurer des modules en fonction des environnements, d'ajouter des filtres de servlets pour zipper les flux, etc.

c. La classe de configuration de la sécurité des URL

La classe `SecurityConfiguration` assure la configuration de la sécurité.

```

package com.mycompany.myapp.config;

import com.mycompany.myapp.security.*;
import com.mycompany.myapp.web.filter.CsrfCookieGeneratorFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.env.Environment;

import
org.springframework.data.repository.query.spi.EvaluationContext
Extension;
import org.springframework.data.repository.query.spi.EvaluationContext
ExtensionSupport;
import org.springframework.security.access.expression.SecurityExpression
Root;
import
org.springframework.security.config.annotation.authentication.
builders.AuthenticationManagerBuilder;
import org.springframework.security.config.annotation.method.
configuration.EnableGlobalMethodSecurity;
import
org.springframework.security.config.annotation.method.configuration.
GlobalMethodSecurityConfiguration;
import org.springframework.security.config.annotation.web.builders.
HttpSecurity;
import org.springframework.security.config.annotation.web.builders.
WebSecurity;
import
org.springframework.security.config.annotation.web.configuration.
EnableWebSecurity;
import
org.springframework.security.config.annotation.web.configuration.
WebSecurityConfigurerAdapter;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import
org.springframework.security.data.repository.query.Security
EvaluationContextExtension;

import
org.springframework.security.web.authentication.RememberMeServices;
import org.springframework.security.web.csrf.CsrfFilter;

import javax.inject.Inject;

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true, securedEnabled
= true)
public class SecurityConfiguration extends
WebSecurityConfigurerAdapter {

    @Inject
    private Environment env;

    @Inject
    private AjaxAuthenticationSuccessHandler
ajaxAuthenticationSuccessHandler;

    @Inject
    private AjaxAuthenticationFailureHandler
ajaxAuthenticationFailureHandler;

    @Inject
    private AjaxLogoutSuccessHandler ajaxLogoutSuccessHandler;

    @Inject
    private Http401UnauthorizedEntryPoint
authenticationEntryPoint;

    @Inject
    private UserDetailsService userDetailsService;

    @Inject
    private RememberMeServices rememberMeServices;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

Les annotations
suivantes
indiquent que
nous avons une
classe de
configuration.

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity

Cette classe
configure un
ensemble de
handlers :

```

@Inject
public void configureGlobal(AuthenticationManagerBuilder
auth) throws Exception {
    auth
        .userDetailsService(userDetailsService)
        .passwordEncoder(passwordEncoder());
}

@Override
public void configure(WebSecurity web) throws Exception {
    web.ignoring()
        .antMatchers("/scripts/**/*.js,html")
        .antMatchers("/bower_components/**")
        .antMatchers("/i18n/**")
        .antMatchers("/assets/**")
        .antMatchers("/swagger-ui.html")
        .antMatchers("/test/**")
        .antMatchers("/console/**");
}

@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .csrf()
        .ignoringAntMatchers("/websocket/**")
        .and()
        .addFilterAfter(new CsrfCookieGeneratorFilter(),
CsrfFilter.class)
        .exceptionHandling()
        .authenticationEntryPoint(authenticationEntryPoint)
        .and()
        .rememberMe()
        .rememberMeServices(rememberMeServices)
        .rememberMeParameter("remember-me")
        .key(env.getProperty("jhipster.security.rememberme.key"))
        .and()
        .formLogin()
        .loginProcessingUrl("/api/authentication")
        .successHandler ajaxAuthenticationSuccessHandler
        .failureHandler ajaxAuthenticationFailureHandler
        .usernameParameter("j_username")
        .passwordParameter("j_password")
        .permitAll()
        .and()
        .logout()
        .logoutUrl("/api/logout")
        .logoutSuccessHandler ajaxLogoutSuccessHandler
        .deleteCookies("JSESSIONID")
        .permitAll()
        .and()
        .headers()
        .frameOptions()
        .disable()
        .and()
        .authorizeRequests()
        .antMatchers("/api/register").permitAll()
        .antMatchers("/api/activate").permitAll()
        .antMatchers("/api/authenticate").permitAll()
        .antMatchers("/api/account/reset_password/init")
        .permitAll()
        .antMatchers("/api/account/reset_password/finish")
        .permitAll()
        .antMatchers("/api/logs/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/api/**").authenticated()
        .antMatchers("/webjars/**").permitAll()
        .antMatchers("/metrics/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/health/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/trace/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/dump/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/shutdown/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/beans/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/configprops/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/info/**").hasAuthority
(AuthoritiesConstants.ADMIN)

```

```

        .antMatchers("/autoconfig/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/env/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/trace/**").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/v2/api-docs/**").hasAuthority(AuthoritiesConstants.ADMIN)
        .antMatchers("/configuration/security")
.hasAuthority(AuthoritiesConstants.ADMIN)
        .antMatchers("/configuration/ui").hasAuthority
(AuthoritiesConstants.ADMIN)
        .antMatchers("/swagger-
ui.html").hasAuthority(AuthoritiesConstants.ADMIN)
        .antMatchers("/protected/**").authenticated();

    }

    @Bean
    public SecurityEvaluationContextExtension
securityEvaluationContextExtension() {
        return new SecurityEvaluationContextExtension();
    }
}

```

• AjaxAuthenticationSuccessHandler

- AjaxAuthenticationFailureHandler
- AjaxLogoutSuccessHandler
- Http401UnauthorizedEntryPoint
- UserDetailsService
- RememberMeServices

Cette classe indique également comment les mots de passe seront encryptés.

Elle prépare et configure le service des utilisateurs (userDetailsService).

Nous filtrons ensuite les URL qui ne seront pas soumises à la sécurité.

- /scripts/**/*.{js,html}
- /bower_components/**
- /i18n/**
- /assets/**
- /swagger-ui.html
- /test/**
- /console/**

La méthode `protected void configure(HttpSecurity http)` concentre la configuration :

- La gestion du login et du logout.
- La gestion du « remember me » pour garder les paramètres de connexions entre deux sessions.
- La sécurité sur les URL.

URL	Code associé
/api/register	permitAll()
/api/activate	permitAll()
/api/authenticate	permitAll()
/api/account/reset_password/init	permitAll()
/api/account/reset_password/finish	permitAll()
/api/logs/**	hasAuthority(AuthoritiesConstants.ADMIN)
/api/**	authenticated()
/webjars/**	permitAll()
/metrics/**	hasAuthority(AuthoritiesConstants.ADMIN)
/health/**	hasAuthority(AuthoritiesConstants.ADMIN)

La sécurité sur les URL est très simplement mise en place.

/trace/**	hasAuthority(AuthoritiesConstants.ADMIN)
/dump/**	hasAuthority(AuthoritiesConstants.ADMIN)
/shutdown/**	hasAuthority(AuthoritiesConstants.ADMIN)
/beans/**	hasAuthority(AuthoritiesConstants.ADMIN)
/configprops/**	hasAuthority(AuthoritiesConstants.ADMIN)
/info/**	hasAuthority(AuthoritiesConstants.ADMIN)
/autoconfig/**	hasAuthority(AuthoritiesConstants.ADMIN)
/env/**	hasAuthority(AuthoritiesConstants.ADMIN)
/trace/**	hasAuthority(AuthoritiesConstants.ADMIN)
/v2/api-docs/**	hasAuthority(AuthoritiesConstants.ADMIN)
/configuration/security	hasAuthority(AuthoritiesConstants.ADMIN)
/configuration/ui	hasAuthority(AuthoritiesConstants.ADMIN)
/swagger-ui.html	hasAuthority(AuthoritiesConstants.ADMIN)
/protected/**	authenticated();

d. La classe de configuration de la sécurité SecurityUtils

La classe `SecurityUtils` permet de récupérer la personne connectée, et de savoir si elle est authentifiée et si elle endosse un rôle passé en paramètre.

```
package com.mycompany.myapp.security;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;

/**
 * Utility class for Spring Security.
 */
public final class SecurityUtils {

    private SecurityUtils() {
    }

    /**
     * Get the login of the current user.
     */
    public static String getCurrentLogin() {
        SecurityContext securityContext = SecurityContextHolder.getContext();
        Authentication authentication = securityContext.getAuthentication();
        UserDetails springSecurityUser = null;
        String userName = null;
        if (authentication != null) {
            if (authentication.getPrincipal() instanceof UserDetails) {
                springSecurityUser = (UserDetails) authentication.getPrincipal();
                userName = springSecurityUser.getUsername();
            } else if (authentication.getPrincipal() instanceof String) {
                userName = (String) authentication.getPrincipal();
            }
        }
        return userName;
    }

    /**
     * Check if a user is authenticated.
     *
     * @return true if the user is authenticated, false otherwise
     */
    public static boolean isAuthenticated() {
        SecurityContext securityContext = SecurityContextHolder.getContext();
        Collection<? extends GrantedAuthority> authorities = securityContext.getAuthentication().getAuthorities();
        if (authorities != null) {
            for (GrantedAuthority authority : authorities) {
                if (authority.getAuthority().equals(AuthoritiesConstants.ANONYMOUS)) {
                    return false;
                }
            }
        }
        return true;
    }
}
```

Les rôles sont
identifiés dans la

```

        }
    }
    return true;
}

/**
 * If the current user has a specific security role.
 */
public static boolean isUserInRole(String role) {
    SecurityContext securityContext = SecurityContextHolder.getContext();
    Authentication authentication = securityContext.getAuthentication();
    if(authentication != null) {
        if (authentication.getPrincipal() instanceof UserDetails) {
            UserDetails springSecurityUser = (UserDetails) authentication.getPrincipal();
            return springSecurityUser.getAuthorities().contains(new SimpleGrantedAuthority(role));
        }
    }
    return false;
}
}
}

```

classe AuthoritiesConstants :

Rôle	Constante associée
ADMIN	ROLE_ADMIN
USER	ROLE_USER
ANONYMOUS	ROLE_ANONYMOUS

Spring Data JPA + Bean Validation

La couche domaine est améliorée afin que les web services JSON puissent l'utiliser.

Nous spécifierons dans cette couche les champs à ignorer lors des échanges JSON.

1. La couche domaine

a. La classe Book

JHipster respecte le modèle en couches.

```
/**
 * A Book.
 */
@Entity
@Table(name = "BOOK")
@Cache(usage = CacheConcurrencyStrategy.NONSTRICT_READ_WRITE)
public class Book implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(name = "title")
    private String title;

    @Column(name = "description")
    private String description;

    @Type(type = "org.jadira.usertype.dateandtime.joda.PersistentLocalDate")
    @JsonSerialize(using = CustomLocalDateSerializer.class)
    @JsonDeserialize(using = ISO8601LocalDateDeserializer.class)
    @Column(name = "publication_date")
    private LocalDate publicationDate;

    @Column(name = "price", precision=10, scale=2)
    private BigDecimal price;

    @ManyToOne
    private Author author;

    [accesseurs, equals, hashCode]
}
```

Les annotations sur la classe indiquent que nous avons affaire à une entité JPA :

- @Entity
- @Table(name = "BOOK")
- @Cache(usage = CacheConcurrencyStrategy.NONSTRICT_READ_WRITE)

Certaines annotations sont de Hibernate car les fonctionnalités liées ne sont pas disponibles avec JPA :

```
import org.hibernate.annotations.Cache;
import org.hibernate.annotations.CacheConcurrencyStrategy;
import org.hibernate.annotations.Type;
```

Certaines données dont le format est complexe disposent d'annotations supplémentaires :

```
@Type(type = "org.jadira.usertype.dateandtime.joda.PersistentLocalDate")
```



```
@JsonSerialize(using = CustomLocalDateSerializer.class)
@JsonDeserialize(using = ISO8601LocalDateDeserializer.class)
@Column(name = "publication_date")
private LocalDate publicationDate;
```

Nous utilisons la librairie `com.fasterxml.jackson.databind.annotation` pour le format de sérialisation de la date.

b. La classe User

Cette classe est intéressante car nous y retrouvons des validateurs.

```
@NotNull
@Pattern(regexp = "^[a-z0-9]*$")
@Size(min = 1, max = 50)
@Column(length = 50, unique = true, nullable = false)
private String login;
```

Les liens sont personnalisés au niveau du format JSON :

```
@JsonIgnore
@ManyToMany
@JoinTable(
    name = "JHI_USER_AUTHORITY",
    joinColumns = {@JoinColumn(name = "user_id",
referencedColumnName = "id")},
    inverseJoinColumns = {@JoinColumn(name = "authority_name",
referencedColumnName = "name")})
@Cache(usage = CacheConcurrencyStrategy.NONSTRICT_READ_WRITE)
private Set<Authority> authorities = new HashSet<>();
```

Nous avons besoin de cet artifice `@JsonIgnore` afin de ne pas avoir l'attribut dans la sérialisation JSON.

Spring MVC REST + Jackson

Les bibliothèques Spring MVC REST avec le support de Jackson permettent à la couche Repository d'exposer très simplement la couche domaine.

Il suffit de déclarer les interfaces et Spring déduit les implémentations.

Nous pouvons toujours ajouter nos propres implémentations pour les Repositories complexes.

1. La classe BookRepository

La classe `BookRepository` contient le repository JPA pour cette entité `Book`.

On voit qu'elle est vide car Spring fournit directement toutes les API que nous utilisons par la suite dans le code. Si nous voulons avoir des méthodes en plus, il suffit de les déclarer pour que Spring Data ajoute le code pour nous.

```
/**
 * Spring Data JPA repository for the Book entity.
 */
public interface BookRepository extends JpaRepository<Book, Long> {

}
```

Il s'agit d'un cas très simple.

2. La classe UserRepository

La classe `UserRepository` contient le repository JPA pour cette entité `User`.

On voit qu'elle n'est pas vide. Spring fournit directement toutes les API que nous utilisons par la suite dans le code et nous demandons explicitement des API supplémentaires en ajoutant seulement les signatures des méthodes.

```
/**
 * Spring Data JPA repository for the User entity.
 */
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findOneByActivationKey(String activationKey);

    List<User> findAllByActivatedIsFalseAndCreatedDateBefore(DateTime dateTime);

    Optional<User> findOneByResetKey(String resetKey);

    Optional<User> findOneByEmail(String email);

    Optional<User> findOneByLogin(String login);

    @Override
    void delete(User t);

}
```

Ce cas est un peu plus complexe.

3. La classe CustomAuditEventRepository

Dans cette classe la méthode `find()` est personnalisée :

```

@Override
public List<AuditEvent> find(String principal, Date after) {
    Iterable<PersistentAuditEvent> persistentAuditEvents;
    if (principal == null && after == null) {
        persistentAuditEvents = persistenceAuditEventRepository.findAll();
    } else if (after == null) {
        persistentAuditEvents =
persistenceAuditEventRepository.findByPrincipal(principal);
    } else {
        persistentAuditEvents =
persistenceAuditEventRepository.findByPrincipalAndAuditEventDateAfter
(principal, new LocalDateTime(after));
    }
    return auditEventConverter.convertToAuditEvent(persistentAuditEvents);
}

```

4. La couche Service

Cette couche ne contient que les traitements métiers complexes. En général, on considérera un traitement comme complexe s'il fait intervenir plusieurs objets de la couche modèle ou s'il y a des actions particulières à faire qui sortent du cadre strict de la couche Repository.

Dans JHipster nous ne trouvons que les classes relatives à la gestion de l'utilisateur, des mails et des événements d'audit dans cette couche.

5. La couche Resource

Cette couche contient les contrôleurs Spring MVC. Les méthodes travaillent en général directement avec la couche Repository et avec la couche intermédiaire Service pour les entités complexes. Prenons l'exemple de la classe `BookResource`.

La majorité des traitements est concentrée dans cette classe pour exposer les données à travers des web services JSON pour l'exploitation de ces données par AngularJS.

a. Contrôleur REST

```

/**
 * REST controller for managing Book.
 */

```

L'annotation `@RestController` indique que nous avons un contrôleur REST.

```

@RestController
@RequestMapping("/api")
public class BookResource {

    private final Logger log = LoggerFactory.getLogger(BookResource.class);

    @Inject
    private BookRepository bookRepository;
}

```

b. Création ou mise à jour d'un livre

On indique que le contrôleur retourne l'entité book au format JSON :

```

/**
 * POST /books -> Create a new book.
 */
@RequestMapping(value = "/books",
    method = RequestMethod.POST,

```

```

        produces = MediaType.APPLICATION_JSON_VALUE)
    @Timed
    public ResponseEntity<Book> create(@RequestBody Book book)
    throws URISyntaxException {
        log.debug("REST request to save Book : {}", book);
        if (book.getId() != null) {
            return ResponseEntity.badRequest().header("Failure",
"A new book cannot already have an ID").body(null);
        }
        Book result = bookRepository.save(book);
        return ResponseEntity.created(new URI("/api/books/"
+ book.getId())).body(result);
    }

```

On indique que le contrôleur crée l'entité book à partir de l'entité au format JSON :

```

/**
 * PUT /books -> Updates an existing book.
 */
@RequestMapping(value = "/books",
    method = RequestMethod.PUT,
    produces = MediaType.APPLICATION_JSON_VALUE)
@Timed
public ResponseEntity<Book> update(@RequestBody Book book) throws URISyntaxException {
    log.debug("REST request to update Book : {}", book);
    if (book.getId() == null) {
        return create(book);
    }
    Book result = bookRepository.save(book);
    return ResponseEntity.ok().body(result);
}

```

c. Obtenir la liste de livres

On indique que le contrôleur retourne une liste d'entités books :

```

/**
 * GET /books -> get all the books.
 */
@RequestMapping(value = "/books",
    method = RequestMethod.GET,
    produces = MediaType.APPLICATION_JSON_VALUE)
@Timed
public List<Book> getAll() {
    log.debug("REST request to get all Books");
    return bookRepository.findAll();
}

```

d. Obtenir le détail d'un livre

Le contrôleur retourne un book en fonction de son id :

```

/**
 * GET /books/:id -> get the "id" book.
 */
@RequestMapping(value = "/books/{id}",
    method = RequestMethod.GET,
    produces = MediaType.APPLICATION_JSON_VALUE)
@Timed
public ResponseEntity<Book> get(@PathVariable Long id) {
    log.debug("REST request to get Book : {}", id);
    return Optional.ofNullable(bookRepository.findOne(id))
        .map(book -> new ResponseEntity<>(
            book,
            HttpStatus.OK))
        .orElse(new ResponseEntity<>(HttpStatus.NOT_FOUND));
}

```

```
}
```

e. Supprimer un livre

Le contrôleur supprime un book en fonction de son id :

```
/**
 * DELETE /books/:id -> delete the "id" book.
 */
@RequestMapping(value = "/books/{id}",
    method = RequestMethod.DELETE,
    produces = MediaType.APPLICATION_JSON_VALUE)
@Timed
public void delete(@PathVariable Long id) {
    log.debug("REST request to delete Book : {}", id);
    bookRepository.delete(id);
}
}
```

Les méthodes notées avec `@Timed` sont monitorées.

Configuration de Logback

Le fichier de configuration de Logback peut être repris pour nos projets :

```
<?xml version="1.0" encoding="UTF-8"?>

<configuration scan="true">
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <charset>utf-8</charset>
      <Pattern>[%p] %c - %m%n</Pattern>
    </encoder>
  </appender>

  <!-- The FILE and ASYNC appenders are here as examples
  for a production configuration -->
  <!--
  <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
    <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
      <fileNamePattern>logFile.%d{yyyy-MM-dd}.log</fileNamePattern>
      <maxHistory>90</maxHistory>
    </rollingPolicy>
    <encoder>
      <charset>utf-8</charset>
      <Pattern>%d %-5level [%thread] %logger{0}: %msg%n</Pattern>
    </encoder>
  </appender>

  <appender name="ASYNC" class="ch.qos.logback.classic.AsyncAppender">
    <queueSize>512</queueSize>
    <appender-ref ref="FILE"/>
  </appender>
-->

  <logger name="com.mycompany.myapp" level="${logback.loglevel}"/>
  <logger name="javax.activation" level="WARN"/>
  <logger name="javax.mail" level="WARN"/>
  <logger name="javax.xml.bind" level="WARN"/>
  <logger name="ch.qos.logback" level="WARN"/>
  <logger name="com.codahale.metrics" level="WARN"/>
  <logger name="com.ryantenney" level="WARN"/>
  <logger name="com.sun.xml.internal.bind" level="WARN"/>
  <logger name="com.zaxxer" level="WARN"/>
  <logger name="io.undertow" level="WARN"/>
  <logger name="net.sf.ehcache" level="WARN"/>
  <logger name="org.apache" level="WARN"/>
  <logger name="org.apache.catalina.startup.DigesterFactory" level="OFF"/>
  <logger name="org.bson" level="WARN"/>
  <logger name="org.hibernate.validator" level="WARN"/>
  <logger name="org.hibernate" level="WARN"/>
  <logger name="org.hibernate.ejb.HibernatePersistence" level="OFF"/>
  <logger name="org.springframework" level="WARN"/>
  <logger name="org.springframework.web" level="WARN"/>
  <logger name="org.springframework.security" level="WARN"/>
  <logger name="org.springframework.cache" level="WARN"/>
  <logger name="org.thymeleaf" level="WARN"/>
  <logger name="org.xnio" level="WARN"/>
  <logger name="springfox" level="WARN"/>
  <logger name="liquibase" level="WARN"/>
  <logger name="sun.rmi.transport" level="WARN"/>

  <contextListener class="ch.qos.logback.classic.jul.LevelChangePropagator">
    <resetJUL>true</resetJUL>
  </contextListener>

  <root level="${logback.loglevel}">
    <appender-ref ref="CONSOLE"/>
  </root>
```

```
</configuration>
```

Il y a un niveau de log global pour notre code et il est possible d'activer la log pour les frameworks tiers.

La classe PaginationUtil

La classe `PaginationUtil` permet de gérer la pagination :

```
package com.mycompany.myapp.web.rest.util;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.http.HttpHeaders;

import java.net.URI;
import java.net.URISyntaxException;

/**
 * Utility class for handling pagination.
 *
 * <p>
 * Pagination uses the same principles as the
 * <a href="https://developer.github.com/v3/#pagination">Github API</api>,
 * and follow <a href="http://tools.ietf.org/html/rfc5988">RFC 5988
 * (Link header)</a>.
 * </p>
 */
public class PaginationUtil {

    public static final int DEFAULT_OFFSET = 1;

    public static final int MIN_OFFSET = 1;

    public static final int DEFAULT_LIMIT = 20;

    public static final int MAX_LIMIT = 100;

    public static Pageable generatePageRequest(Integer offset, Integer limit) {
        if (offset == null || offset < MIN_OFFSET) {
            offset = DEFAULT_OFFSET;
        }
        if (limit == null || limit > MAX_LIMIT) {
            limit = DEFAULT_LIMIT;
        }
        return new PageRequest(offset - 1, limit);
    }

    public static HttpHeaders generatePaginationHttpHeaders(Page<?> page,
String baseUrl, Integer offset, Integer limit)
        throws URISyntaxException {

        if (offset == null || offset < MIN_OFFSET) {
            offset = DEFAULT_OFFSET;
        }
        if (limit == null || limit > MAX_LIMIT) {
            limit = DEFAULT_LIMIT;
        }
        HttpHeaders headers = new HttpHeaders();
        headers.add("X-Total-Count", "" + page.getTotalElements());
        String link = "";
        if (offset < page.getTotalPages()) {
            link = "<" + (new URI(baseUrl + "?page=" + (offset + 1)
+ "&per_page=" + limit)).toString()
+ ">; rel=\"next\"";
        }
        if (offset > 1) {
            link += "<" + (new URI(baseUrl + "?page=" + (offset - 1)
+ "&per_page=" + limit)).toString()
+ ">; rel=\"prev\"";
        }
        link += "<" + (new URI(baseUrl + "?page=" + page.getTotalPages()
+ "&per_page=" + limit)).toString()
+ ">";
    }
}
```



```
+ "&per_page=" + limit)).toString()
    + "> rel=\"last\"," +
    "<" + (new URI(baseUrl + "?page=" + 1 + "&per_page="
+ limit)).toString()
    + "> rel=\"first\"";
    headers.add(HttpHeaders.LINK, link);
    return headers;
}
}
```

Elle est très pratique pour réduire le volume des données à échanger avec la partie cliente AngularJS de l'application.

Points clés

- JHipster génère une application Spring propre qui peut servir de modèle.
- JHipster permet de tester et de comparer différentes options de générations.
- Spring est cohérent et il est assez facile de comprendre de nouvelles parties du framework.